مصرف قطر المركزي
Qatar Central Bank
State of Qatar • دولـــة قطـــر

Qatar Open Finance Standards

Version: 1.0-rc1

> ⚠ **Status: Release Candidate**
>
> This site provides the standards at **v1.0-rc1**. This version is a Release Candidate, and subject to approval by Qatar Central Bank, in readiness for adoption by ecosystem participants.

Table of Contents

- Overview
- License
  - Attribution Notice
- Contribution Agreement
- Versioning
  - Versioned Artefacts
  - Version Labels
  - Major, Minor and Errata Versioning
  - Requirement IDs Across Versions
  - Changes Between Versions
  - Version Review Process
- Overview
- Overview
- Common Rules and Guidelines
- User Payment Account Selection
- Payment Amount & Currency
- Creditor Identification
- Debtor Reference
  - Debtor Reference Scenarios
- Creditor Reference
  - Creditor Reference Scenarios
- Purpose of Payment
- Customer IP Address
- Adding a Trusted Payee (Beneficiary)
- Use of ISO Country and Currency Codes
  - ISO 3166-1 Country Codes
  - Open Finance User-Assigned Country Codes
  - ISO 4217 Currency Codes
- Is Single Authorization flag
- Authorization Expiration Date Time
- Risk Information Block
  - TPP Obligations

    - LFI Obligations
    - Additional TPP Obligations
- Consent Staging
- Check Is Single Authorization flag
- Multi-User Authorization Flow
- Check Authorization Time Window
- LFI Manual Consent Handling
- Hand-off to LFI
- Payment Account Selection at LFI
- Hand-off back to the TPP
- Payment Status Update
    - Payment Status Model
    - Open Finance Payment States & Description
    - Payment Rail Status Mapping
- Payment Status Information
    - Description
    - Overall Process Flow
    - Status Information Rules & Guidelines
    - General
- Confirmation to User
- Payment Notifications
    - Multi-User Authorization Flow Notifications
    - Customer Notifications - Beneficiary and TPP Identification
- Exception Handling
    - Exception Handling User Journey
    - Summary authorization screen with greyed out button
    - Known Scenarios
- Supplementary Information
    - Summary authorization screen with warning on bottom
    - Known Scenarios
- LFI Exception & Warning Journey Scenarios
    - Introduction
    - Exception Handling Scenarios and Guidelines

    - Warning / Supplemental Information Scenarios
- Supported Accounts & Payment Rails
- TPP Obligations towards Customer Verification
- Authentication and Authorization | Overview
- Authentication and Authorization | Redirection
- Authentication and Authorization | Decoupled
- Authentication by LFI | Overview
- Authentication by LFI | Redirection
- Authentication by LFI | Decoupled Redirection
- Effective Use of Redirection Screens
- Error Scenarios During Redirection
- Strong Customer Authentication | Introduction
- Strong Customer Authentication | Principles
- Strong Customer Authentication | Controls
- Industry Standards
- Consent Setup | Overview
- Consent Setup | Codification
- Consent Types
- Consent States
- Consent Setup | Rules and Guidelines
- Consent Setup | Implementation
- Consent Management Interfaces | Overview
- The Role of Consent Management Interfaces (CMIs)
- Principles Behind Effective Consent Management Interfaces (CMIs)
- Naming of Consent Management Interfaces (CMIs)
- LFI Consent Management Interface | Overview
- LFI Consent Management Interface (CMI) Examples
- LFI Consent Management Interface | Consent Revocation Journey
- LFI Consent Management Interface | Consent Management Interfaces (CMIs) when customer-facing service provider and TPP are different entities
- TPP Consent Management Interface | Overview
- TPP Consent Management Interfaces (CMIs) examples
- TPP Consent Management Interface | Consent Revocation Journey

- Consent Pause Journey
- Reactivating a Paused Consent Journey
- Consent Update Journey
- TPP Consent Management Interface | Consent Management Interfaces (CMIs) when customer-facing service provider and TPP are different entities
- User Experience Principles | Introduction
- User Experience Principles | Customer Journey
- User Experience Principles
- Design to Maximize Transparency to Users
- Design to Maximize User Trust
- Maximize the User's Sense of Control
- User in Control
- Protection for Vulnerable Users
- Useful Elements in the User Journey
- Unhelpful Elements in the User Journey
- Other Rules for User Journeys
- API Security | Overview
- API Security | Security Profile - FAPI
    - 1. Introduction
    - 2. Security Profile
    - 3. Consent Authorization Sequence Diagram
    - 4. Design Considerations
- Account Information | Description
    - Data Sharing Segments
    - Account Information Sharing - Generic User Story
    - User Journey
- Bank Data Sharing Consent Set-up & Authorization
    - Customer Experience
    - Bank Data Sharing Consent Rules & Guidelines
    - Journey Variations
- Data Sharing Requests
    - Rules & Guidelines
    - Beneficiaries Data Sharing

- Account Information | Consent Updates
  - Consent Update
- Permissions and Data Clusters
  - Permissions
  - Data Clusters
  - Data Cluster Structure & Language
  - Relevance of Data Clusters by Product Type
- LFI as a TPP Example
  - User provides consent to share data from one LFI to another
  - User views consolidated transaction history
- Single Instant Payment | Description
  - Debtor and Creditor Segments
  - Example User Story
  - User Journey
- Single Instant Payment | Customer Experience
  - Standard Journey
  - Payment-linked Account Data Sharing
- What are Multi-Payments
  - Multi-Payment Types Set-up
  - Debtor and Creditor Segments
  - User Journey
- TPP Consent Staging Rules & Guidelines - Basic Consent Parameters
  - Multi-Payments Consent
  - PAR authorization_details
  - PersonalIdentifiableInformation
- TPP Consent Staging Rules & Guidelines - Consent Staging
  - Consent Staging
  - Hand-off to LFI
  - Authentication
  - Authorization
  - Hand-off back to the TPP
  - Confirmation to User
- Multi-Payments Common Rules & Guidelines - Period & Payment Schedule
  - Period Type & Start Date

- Balance Check Permission
- Balance Check
  - Balance Check
  - Balance Check Request Processing
- Payment Status Update & Payment Notifications
  - Payment Status Update
  - Payment Notifications
- Multi-Payment (Generic) | Consent Updates
  - Consent Update
- Creditor Requirements
- Variable On-Demand Multi-Payments to Multi-Creditors
  - Multi-Creditors
  - Consent Set-up - Variable On-Demand Payments
  - Consent Set-up - Variable On-Demand Payments to a Defined List of Creditors
- TPP Consent Staging Rules & Guidelines - Variable On-Demand Consent Set-up
  - Consent Cumulative Control Parameters
  - Consent Periodic Control Parameters
  - Consent Expiration DateTime
- Multi-Payments Common Rules & Guidelines
  - Creditor Identification for Variable-defined Beneficiaries
  - Creditor Identification for Variable Beneficiaries
- TPP and User Events to Agree to Initiate Payments
- Payment Initiation — Submission Rules for Variable On-Demand Payments (VariableOnDemand)
  - Payment Initiation
- Processing of Payment Initiation Requests for Variable On-Demand Payments (VariableOnDemand)
  - Processing of Payment Initiation Requests
- Multi-Payment Example Use Cases
  - Variable On-Demand Payments (VariableOnDemand) Example Use Cases
- Constants and Limits | Limits
- Constants and Limits | Constants
- LFI Channel Limits
- Overview

- Retrieving Accounts Information Data | Overview
  - Three roles, three guides
- Retrieving Accounts Information Data | TPP Implementation Guide
  - 1. Onboard the User (where required)
  - 2. Collect and review the data sharing consent parameters
  - 3. Stage the consent
  - 4. Hand off the User to the LFI
  - 5. Wait for the LFI to disclose and authorize
  - 6. Receive the hand-off back from the LFI
  - 7. Submit data sharing requests
  - 8. Confirm to the User
  - 9. Manage the consent over its lifetime
  - 10. Access Beneficiaries data (conditional)
  - 11. Account for journey variations
- Retrieving Accounts Information Data | LFI Implementation Guide
  - 1. Support eligible accounts
  - 2. Receive the hand-off from the TPP
  - 3. Authenticate the User
  - 4. Disclose the consent details
  - 5. Select accounts
  - 6. Present the consent for authorization
  - 7. Hand off back to the TPP
  - 8. Share the requested data
  - 9. Maintain Beneficiaries lists
  - 10. Support consent lifecycle management
  - 11. Account for journey variations
- Retrieving Accounts Information Data | OFP Implementation Guide
  - 1. Validate and confirm the staged consent
  - 2. Broker the hand-off to the LFI
  - 3. Maintain the consent state machine
  - 4. Process data sharing requests
  - 5. Broker the hand-off back to the TPP
  - 6. Support consent lifecycle management

    - 7. Account for journey variations
- Retrieving Accounts Information Data | Handle Journey Variations
    - User consents to non-account-specific data only (e.g. Contact and Party details)
    - LFI acting as a TPP
- Making a Single Instant Payment | Overview
    - Three roles, three guides
- Making a Single Instant Payment | TPP Implementation Guide
    - 1. Meet your standing obligations
    - 2. Collect and review the consent parameters
    - 3. Stage the consent
    - 4. Hand off the User to the LFI
    - 5. Wait for the LFI to authenticate and authorize
    - 6. Receive the hand-off back from the LFI
    - 7. Submit the payment initiation request
    - 8. Track status, confirm, and notify
    - 9. Trusted Payee retrieval (where applicable)
    - 10. Account for journey variations
- Making a Single Instant Payment | LFI Implementation Guide
    - 1. Support eligible accounts and payment rails
    - 2. Receive the hand-off from the TPP
    - 3. Authenticate the User
    - 4. Select the payment account
    - 5. Present the consent for authorization
    - 6. Offer to save a Trusted Payee
    - 7. Populate and apply the Risk Information Block
    - 8. Hand off back to the TPP
    - 9. Execute the payment
    - 10. Update status and notify
    - 11. Handle exceptions and supplementary information
    - 12. Support multi-user authorization
    - 13. Account for journey variations
- Making a Single Instant Payment | OFP Implementation Guide
    - 1. Validate and confirm the staged consent

    - 2. Broker the hand-off to the LFI
    - 3. Check the authorization time window
    - 4. Relay multi-user authorization information
    - 5. Broker the hand-off back to the TPP
    - 6. Enforce idempotency on payment initiation
    - 7. Validate and route the payment initiation request
    - 8. Propagate payment status
    - 9. Respect the limits and timers that govern the OFP
    - 10. Account for journey variations
- Making a Single Instant Payment | Handle Journey Variations
    - User selects an account at the TPP, and the LFI provides supplementary information (duplicate payment)
    - User selects an account at the TPP, and the LFI provides supplementary information
    - User selects an LFI at the TPP, and a Merchant is the user-facing entity
- Making a Variable On-Demand Payment | Overview
    - Three roles, three guides
    - What makes Variable On-Demand different from a single payment
- Making a Variable On-Demand Payment | TPP Implementation Guide
    - 1. Collect and review the basic consent parameters
    - 2. Set the Variable On-Demand control parameters
    - 3. Set up the Multi-Creditors list
    - 4. Offer the Balance Check permission (optional)
    - 5. Stage the consent
    - 6. Hand off the User to the LFI
    - 7. Receive the hand-off back from the LFI
    - 8. Submit payment initiations
    - 9. Track status, confirm, and notify
    - 10. Manage consent updates
    - 11. Account for journey variations
- Making a Variable On-Demand Payment | LFI Implementation Guide
    - 1. Authenticate the User
    - 2. Present the consent for authorization
    - 3. Support multi-user authorization

    - 4. Hand off back to the TPP
    - 5. Provide balance check responses (conditional)
    - 6. Process payment initiation requests
    - 7. Support consent updates
- Making a Variable On-Demand Payment | OFP Implementation Guide
    - 1. Validate and confirm the staged consent
    - 2. Broker the hand-off to the LFI
    - 3. Check the authorization time window and start Control Period tracking
    - 4. Relay multi-user authorization information
    - 5. Broker the hand-off back to the TPP
    - 6. Validate payment initiation requests
    - 7. Propagate payment status
    - 8. Provide balance check responses (conditional)
- Making a Variable On-Demand Payment | Handle Journey Variations
    - The two Creditor scopes
- Overview
- General | Overview
- API Hub Domains and DNS
    - Introduction
    - QCB Open Finance Platform Domains
    - API Hub Service Examples
    - LFI Service Examples
    - Network Diagram
- Keys, CSRs and Certificates
    - Introduction
    - Transport Keys and Certificates
    - Signing Keys and Certificates
    - Encryption Keys and Certificates
    - Creating Certificates
- LFI Integration Guide | Overview
    - Introduction
    - High-Level Consent Authorization Flow
    - The API Hub

    - LFI Integration Guide
- Security | Overview
    - Transport Security
    - Application Layer Authentication
- Application Layer Authentication
    - Introduction
    - Application Layer Authentication Switched Off
    - API Key
    - Client Credentials Grant
    - JWT Auth
    - Service Initiation Token
- JWT Authorization
    - Introduction
    - Requirements for the LFI as the Client (Sending a Token)
    - Requirements for the LFI as the Server (Receiving a Token)
    - JWT Auth Claims Reference
- Ozone Connect | Overview
    - Introduction
    - Request Handling
    - Success Responses
    - Error Responses
- Data and Service Mapping | Overview
- Payment Personal Identifiable Information
    - Introduction
    - Operations
    - Scenarios
- Account Information Mapping | Overview
    - Context
    - Purpose
    - Permission Codes
- Accounts
    - API Operations & Permissions
    - Data Permissions: Basic vs Detail

- Balances
  - API Operations & Permissions
  - API Hub to Standards Response Data Mapping
- Transactions
  - API Operations & Permissions
  - API Hub to Standards Response Data Mapping
  - Data Permissions: Basic vs Detail
- Consent Manager | Overview
  - Introduction
  - Context
  - Error Handling
- Consent Management Interface
  - Introduction
  - Retrieving User Consents
  - Connection History
  - Payment History
  - Status Code Language
- Authorization Server | Overview
  - Introduction
  - Context
  - Success Response
  - Error Handling
- Trust Framework | Overview

Overview

The Framework sets out the legal and licensing terms on which the Qatar Open Finance Standards are published and operated. It applies to the Financial Institutions (FIs) and Third Party Providers (TPPs) participating in the ecosystem, and sits alongside the business and technical rules, the implementation guides, and the API descriptions.

The License and the Contribution Agreement govern the Standards themselves — the terms on which they may be used, and the terms on which contributions to them are accepted. Versioning sets out how releases of the Standards are numbered and what each release commits to; it sits here, alongside the other terms of publication, because it is not itself versioned with the Standards it describes.

License →

The terms on which the QCB Open Finance Standards are published, and the standards they attribute.

Contribution Agreement →

The terms under which contributions to the QCB Open Finance Standards are accepted.

Versioning →

How the Standards are versioned, what each release suffix commits to, and how changes between versions are recorded.

License

Copyright 2026 QCB

This License sets out the terms and conditions which apply to any natural person or legal entity ("Contributor") who accesses or implements the QCB Open Finance Standards, including associated specifications, guidelines, documentation files, and downloadable applications (the "Standards"). You must agree to the terms of the License before accessing or implementing the Standards.

The Qatar Central Bank (QCB) grants to any contributor, developer, implementer, or other interested party a non-exclusive, royalty-free, worldwide copyright license to reproduce, prepare derivative works from, distribute, perform, and display, the QCB Open Finance Standards, solely for the purposes of developing and implementing the Standards, provided that attribution be made to QCB as the source of the material, but that such attribution does not indicate the endorsement by QCB.

The Standards is provided "as is", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement. In no event shall QCB be held liable for any claim, damages, or other liability, whether in an action of contract, tort or for any other common law, equitable or statutory cause of action, arising from, out of or in connection with the Standards or the use or other dealings in the Standards by any Contributor or other third party.

The Standards, any amendments, and any contributions (as defined in the Contribution Agreement) are considered as "Collective Work" as described in Copyright Law governed by Law No. 7 of 2002 on the Protection of Copyright and Neighbouring Rights, which grants protection to innovative literary, artistic, and scientific creations in any form, expression, significance, or purpose.

Attribution Notice

Portions of the QCB Open Finance Standards are based on:

- The UK Open Banking Standard (see UK Open Banking Open Licence)
- The KSA Open Banking Standard (see KSA Open Banking Open License and Contribution Agreement)

- The Brazil Open Insurance Standards (see Brazil Open Insurance Terms of Use)
- The OpenID Foundation's Financial Grade API (FAPI) Profile (see OpenID Foundation Contribution Agreements)
- The CBUAE Open Finance Standards (see CBUAE Open License and Contribution Agreement)

Contribution Agreement

The Contribution Agreement sets out the terms and conditions which apply to any natural person or legal entity (“Contributor”) contributes to the development of the QCB Open Finance Standards. You must agree to the terms of this Contribution Agreement before making a Contribution to the QCB Open Finance Standards. This Contribution Agreement covers any and all Contributions that you submit to the QCB Open Finance Standards, either now or in the future.

As defined in the License, QCB intends to make the Standards available for implementation by any Contributor without a fee. To facilitate this, any Contributor who is providing feedback, comments, suggestions, or other submissions regarding the Standards, or uploads, submits, transmits, or distributes content regarding the Standards (but excluding communication that is conspicuously marked or otherwise designated as “Not a Submission” or “Not a Contribution”) (“Contribution”), continuously represents and warrants that:

1. If the Contributor is a legal entity, its representatives are, and at all times material will be, either its employees (who will provide Contributions within their scope of employment and who are legally obligated to assign to Contributor all intellectual property created within the scope of their employment), or its independent contractors (who are under written obligation to assign to Contributor all rights in any intellectual property created within the scope of their engagement).

2. If the Contributor is an individual, and the employer(s) of the Contributor has rights to intellectual property that are included in the Contribution, the Contributor represents that they are permitted to make Contributions on behalf of that employer, or that the Contributor’s employer has waived such rights in the Contribution, or that the Contributor’s employer has entered into a separate Contribution Agreement with the QCB.

3. Contributor has no obligation to any third party (e.g., an obligation to assign inventions) that would preclude or limit:

    1. participating in any working group or consultation process;

    2. making any Contributions to the Standards; or

    3. granting rights in (or making commitments related to) intellectual property in any Contribution.

4. Contributor, itself, and through its representatives (if Contributor is a legal entity), has the unfettered right to make any Contributions, fully in accordance with this License and this

Agreement. Contributor (whether a legal entity or an individual) represents that it is entitled to grant the licences set out in this Agreement.

5. All Contributions are considered as Collective Work.

6. All Contributions, and other materials shared with QCB and/or any other Contributors for the purpose of developing the Standards (but not including materials shared with Contributors for any purpose other than developing the Standards), will be considered non-confidential information, regardless of any markings to the contrary included thereon or related thereto.

7. Some Contributions may not be subject to copyright. To the extent, however, that a Contribution is or may be subject to copyright, the Contributor hereby grants a perpetual, irrevocable (except in case of breach of this license), non-exclusive, royalty-free, worldwide license in such copyright to QCB, and to other Contributors, to reproduce, prepare derivative works from, distribute, perform, and display, the Contribution and derivative works thereof solely for purposes of developing and implementing the Standards.

8. Contributor represents that Contributions comprised of written submissions submitted by such a Contributor to QCB comply with any copyright attribution requirements relating to third party content.

9. Subject to (8) above, Contributor represents that Contributions are original works of the Contributor. Where a Contribution contains content that is not an original work of the Contributor, the Contribution may be submitted provided that the Contributor:

    1. retains all copyright and license information that subsisted in the content received by, or obtained by the Contributor;

    2. includes in the Contribution a statement that the Contribution includes content of a third party and providing the name of the third party(ies) and details of any applicable licences or restrictions in relation to the third party content; and

    3. adheres to any guidelines, instructions, or requirements issued by the QCB with regards to third party content.

10. Subject to each Contributor's rights in individual Contributions, the copyright in any draft or final version of the Standards will be owned solely by QCB. Each Contributor will execute and deliver such instruments and take such other actions as and when QCB may reasonably request to perfect or protect QCB's copyright in the Standards.

11. Subject to any licensing obligations herein, Contributor retains all rights in and to its Contribution, and there are no other limitations whatsoever on Contributor's ability to exercise any copyright rights in its Contribution or any portion thereof.

12. There is no requirement or expectation by others that Contributors should disclose patents or patent applications, however QCB hereby disclaims any responsibility for identifying the existence, or for evaluating the applicability, of any patents, patent applications, or other rights (including copyrights) claimed to be applicable to the Standards and will take no position on the validity or scope of any such rights.

13. Contributor acknowledges that QCB has no duty to publish or otherwise use or disseminate any Contribution.

14. Contributor shall keep strictly confidential and shall not disclose, or cause or permit to be disclosed, or publish, or cause to be published, to any person or entity, any information acquired during the development process, unless provided with explicit written authorization by QCB to do so.

15. Contributor hereby grants permission to QCB to reference the Contributor's name(s) and email address(es) without the prior written consent of Contributor, but only in association with the Contribution(s) of Contributor and not with respect to any work derived from such Contribution(s).

16. Contributions and Contributor's personal details (e.g. names and email addresses) may, in accordance with applicable data protection laws, be stored on cloud services hosted outside Qatar.

17. Contributor agrees to notify QCB, in writing, of any facts or circumstances of which Contributor later becomes aware that would make the Contributors representations in this Agreement inaccurate in any respect.

18. The laws of Qatar shall govern the construction, validity and operation of this Agreement and the performance of all obligations hereunder.

19. The courts of the Qatar shall have exclusive jurisdiction in relation to any dispute or matter arising under or in connection with this Agreement.

In the course of contributing to the development of the QCB Open Finance Standards, a Contributor may become aware of the Confidential Information of another party. Confidential Information means all confidential or proprietary information relating to the business, assets,

affairs, customers, clients, suppliers, plans, intentions, operations, processes, product information, specifications, guidelines, know-how, technical information, designs, trade secrets, intellectual property rights, software, information, findings, data or analysis, or any other information that is identified by a party as being of a confidential or proprietary nature.

Each party to this Open Licence and Contribution Agreement undertakes that, in the course of participation in, or contribution to, this Open Licence and Contribution Agreement it shall not, at any time, disclose the Confidential Information of another party which that party directly or indirectly discloses, or makes available, before, on, or after the date of this Agreement.

No party shall use any other party's Confidential Information for any purpose other than to exercise its rights or perform its obligations under, or in connection with, this Open Licence and Contribution Agreement.

Each party may disclose the other party's Confidential Information:

a) to its employees, officers, representatives, contractors, subcontractors or advisers who need to know such information for the purposes of exercising the party's rights or carrying out its obligations under or in connection with this Open Licence and Contribution Agreement. Each party shall ensure that its employees, officers, representatives, contractors, subcontractors, or advisers to whom it discloses the other party's Confidential Information complies with the above obligations as to confidentiality;

b) where permitted to do so by the other party; and

c) as may be required by law, a court of competent jurisdiction or any governmental or regulatory authority.

By accessing or implementing the Standards, or by providing any Contribution to the Standards, you agree to be bound by the terms of this License and Contribution Agreement.

Versioning

The Qatar Open Finance Standards implement a versioning mechanism common to most open banking and open finance standards, which are described in detail below.

Versioned Artefacts

Standards are released with a given version label, visible on each page of this website as a banner.

The following artefacts are versioned with a single, consistent version label:

- Business and technical rules.
- Implementation guides.
- API descriptions, created using the OpenAPI Specification.

The artefacts above *may* also be available through other distribution mechanisms, such as separate GitHub repositories.

Version Labels

A version takes the form `v<major>.<minor>` followed by, where required, a qualifier for the release label, as defined below.

Version	Meaning	Stability
`v1.0-draft1`	A draft published for review.	Open for comment. Anything may change before the next release based on industry feedback or revisions to requirements.
`v1.0-rc1`	A release candidate — content is believed complete.	Open for comment. Only defects and clarifications are expected to change.
`v1.0`	The published release.	Normative. Changes only through a new version or an errata.
`v1.0-errata1`	Corrections to a published release.	Normative. Carries no new obligations — see below.

Suffixes sort in the order shown, in that:

- `draft` precedes `rc`,
- `rc` which precedes the release itself
- The release precedes any `errata`.

Drafts and release candidates are numbered in sequence (`draft1`, `draft2` and so on) as each review cycle completes.

Major, Minor and Errata Versioning

The number that advances says what kind of change the release carries:

- **Minor version**: Requirements are added or existing requirements clarified. An implementation conformant with the previous minor version remains conformant except where new requirements apply to it. This applies to all artefacts, including API descriptions that **must** only implement additive, non-breaking changes.
- **Major version**: Requirements retired or changed in a way that breaks an existing conformant implementation. This applies to all artefacts, including API descriptions that may carry changes that break existing implementations.
- **Errata**: Corrections to a published release including typographical error, a broken cross-reference, a mis-stated constant, or an example that contradicts the rule it illustrates. An errata never introduces, removes or alters an obligation.

Requirement IDs Across Versions

> This section **only** applies to Business and Technical Rules

Every normative requirement carries a Requirement ID (for example `COMM-261`, `SIPD-013`, `ACCI-058`) which is shown alongside the rule it identifies.

- An ID is assigned once and **never changes**, however much the wording or location of the requirement changes afterwards.
- An ID is **never reused**. When a requirement is retired, its number is retired with it.
- New requirements take the next available number for their domain.
- Existing requirements are never renumbered to make room.

Requirements **must** always be cited by ID, not by heading or page position. Headings exist for navigation and may be reorganised between versions, whereas the Requirement ID will not be.

Preferred	Not preferred
“Compliant with `COMM-261`.”	“Compliant with the Trusted Payee section.”
“Non-conformance against `COMM-262`.”	“See the Shari’ah compliance page, third heading.”

Changes Between Versions

A full change log will provides between versions and published to the website.

- Business and Technical Rules: Will be cited by Requirement ID, as defined above.
- Implementation Guides: Will be cited by Page and Section.
- API Descriptions: Will cited by API operation.

Version Review Process

Reviewing a given version is described by the Review Process.

Overview

The Standards set out what participants in the Qatar Open Finance Framework must do and how they do it. The Business and Technical Rules are the normative requirements on Licensed Financial Institutions (LFIs) and Third Party Providers (TPPs). The Implementation Guides walk through putting those rules into practice, end to end. The API descriptions define the contract that every implementation is built against.

Business and Technical Rules →

The rules governing participants in the Qatar Open Finance Framework.

Implementation Guides →

Step-by-step guides for building against the Standards.

APIs →

Reference documentation generated from the published OpenAPI descriptions.

Overview

The Business and Technical Rules are the normative requirements that Licensed Financial Institutions (LFIs) and Third Party Providers (TPPs) must meet when participating in the Qatar Open Finance Framework. They cover both Bank Data Sharing and Bank Service Initiation, describing the obligations on each party, the consent a User must give, and the journeys that carry them out.

The rules are organised in three layers. **Common** and **Common Components** apply across every capability — authentication and authorization, consent setup, the consent management interfaces, and the user experience principles. The capability sections then set out what is specific to account information, to single instant payments, and to multi-payments. **Constants and Limits** collects the values the rules refer to throughout.

Common →

Rules and guidelines common to all Bank Data Sharing and Bank Service Initiation capabilities.

Common Components →

Authentication and authorization, consent setup, consent management interfaces, and user experience principles.

Account Information →

How a TPP retrieves a User's data held with an LFI, under an Account Access Consent.

Single Instant Payment →

How a TPP initiates a domestic Single Instant Payment from the User's account, under a Single Use Consent.

Multi-Payment →

A series of payments initiated by a TPP, supporting long-lived payment use cases.

Constants and Limits →

The limits and constant values the rules refer to.

Common Rules and Guidelines

This section contains Rules & Guidelines which are common for all Bank Data Sharing and Bank Service Initiation capabilities described in the QCB Open Finance Standards.

IMPORTANT NOTE:

- COMM-001 LFIs & TPPs MUST comply with the rules and guidelines listed in this section.
- COMM-002 LFIs MUST connect all mandatory APIs to the Open Finance Platform and may provide direct access to TPPs.
  - COMM-003 *Note: The Standard does not support the direct access of LFIs to TPPs.*
- COMM-004 LFIs & TPPs MUST store all requests made to and responses received from the API Hub. These requests and responses MUST be stored for at least 5 years, in their original, unaltered state and retained in accordance with the Open Finance Regulations in Qatar.

Basic terminology

In the context of the Open Finance Standard, the following terms have been used:

- **Debtor:** Term used to describe the originating payer of an Open Finance bank service initiation. In most of the cases, the Debtor is the User of the Open Finance services offered by TPPs.
- **Creditor:** Term used to describe the beneficiary payee of an Open Finance bank service initiation.

User Payment Account Selection

TPPs MUST:

COMM-005 Provide Users at least one of the following options for selecting their payment account to be used for the Consent:

- Select their Payment Account Identification details (e.g. IBAN). This assumes that these have been saved previously and the User has a long-term profile with the TPP, having completed a User onboarding step.

- Select their LFI only (so that they can select their Payment Account later on in the journey after authenticating with the LFI). The LFI **MUST** be identified using the trading name which is familiar to Users.

- COMM-006 TPPs **MUST** use logos and the brand names of the LFIs as they are defined in the Trust Framework Directory.

> **NOTE:**
>
> It is in the competitive space of TPPs to enable additional services for Users that may provide better customer experience. For example, TPPs may use one or more of the following ways to populate the list of the Users' Payment Account Identification details:*

- TPPs *with a long-term relationship with Users may have dual Data Sharing and Service Initiation roles and have access to the Users' list of accounts with specific LFIs, making it easier for the Users to select the payment accounts they want to use for their payment initiations.*

- *Additionally, TPPs with long-term relationship with Users may allow Users to manually enter their Payment Account Identification details (e.g. IBAN) prior to initiating a payment or setting up a long-lived payment consent (for example during User onboarding) and save these in the list of Payment Account Identification details. It is the responsibility of TPPs to ensure that the manually entered account details are validated.*

Payment Amount & Currency

TPPs MUST:

COMM-035 Either allow Users to manually enter the payment amount or pre-populate it for the Users. This is dependent on the use case.

COMM-036 For domestic payments, ensure that Users clearly understand that the payment amount is in **local currency** (i.e. Qatari Riyal - QAR) as used by the local payment systems infrastructure for domestic payments.

COMM-037 COMM-038 Ensure and validate that the lowest allowable amount is 0.01 QAR (see Fawran Implementation Guide v5.4, section 1.4 Payment Validation Service, Minimum Amount Check). The amount of the payment **MUST** be specified with max 2 decimal digits.

COMM-039 Ensure that the maximum allowable amount is equal to Limits and Constants | Max Inter-bank Payment Amount for any payments to Creditor accounts managed by a different account holding entity than the User's LFI.

- COMM-040 **NOT** apply any maximum amount limit for payments to Creditor accounts managed by the same LFI as the User's LFI (i.e. Intra-bank payments).

COMM-041 Display a message to Users informing them that the payment amount cannot exceed the Limits and Constants | Max Inter-bank Payment Amount value for all Inter-bank payments.

Creditor Identification

TPPs MUST:

COMM-042 Either allow Users to manually enter/select their Creditor Identification details or pre-populate them for the Users.

> **NOTE:**
> It is in the competitive space of TPPs to enable additional services for Users that may provide a better customer experience. For example, a TPP with a long-term relationship with a User may have dual Data Sharing and Service Initiation roles and have access to Users' list of Authorized Beneficiaries with a specific LFI, making it easier for Users to select the Creditor Identification details.

COMM-043 Allow the Creditor Identification using the following information:

- IBAN of the Creditor payment account
- Name of the Creditor

COMM-258 Allow the Creditor Identification using an alias of the Creditor payment account, of one of the alias types supported by Fawran Core (see Fawran Implementation Guide v5.4, section 4.13 prxy.001.001.01 and Appendix J: Guidelines, section 14.1 Alias Registration). Users **MUST** be able to use the Creditor QID once Fawran Core supports it as an alias type. Present the IBAN and the alias as separate fields, so that Users can enter whichever one they hold. The alias is an additional way to identify the Creditor, not a replacement for the IBAN.

> **NOTE:**
> Depending on the use case, the Creditor details may not be displayed to Users in full. However, these still need to be part of the payment Consent request sent by the TPP.

COMM-044 Display to Users the full IBAN details of the Credit payment account, if these details have been manually entered/provided by the User.

COMM-045 Display to Users the masked IBAN (only last 4 digits) of the Credit payment account, if these details have been populated/provided by the TPP (e.g. in case of merchant onboarded by TPPs, when initiating a refund masking user account details to merchant).

COMM-259 Apply the same treatment to an alias used for Creditor Identification: display the full alias if it has been manually entered/provided by the User, and the masked alias (only last 4 characters) if it has been populated/provided by the TPP.

COMM-267 Validate a Creditor IBAN entered by the User against the format and check-digit rules for IBANs in Qatar (see QATCH Advanced Link Interface Specifications v2.7, section 3.6 General IBAN Specifications).

COMM-046 COMM-047 Ensure that the format and other validation rules of the Creditor Identification details are correct in case Users manually enter the Creditor Identification details. If incorrect, TPPs **MUST** request Users to review and re-enter the information.

COMM-048 TPPs **MUST** send the Merchant Category Code where the Creditor is a Merchant onboarded by them.

LFIs MUST:

COMM-260 Resolve an alias provided for Creditor Identification to the Creditor payment account, using the alias resolution service of Fawran Core (see Fawran Implementation Guide v5.4, section 2.1.2 Alias Resolution/ Alias Check/Name Enquiry). Where the payment is then directed to Tahweel, use the Creditor account returned by the alias resolution, identified as specified in QATCH Advanced Link Interface Specifications v2.7, section 2.1 Direct Credit. Where the alias cannot be resolved, reject the payment Consent and return an appropriate error to the TPP via the OFP.

COMM-266 Display to Users the account name returned by the alias resolution, so that they can confirm the Creditor before authorizing the payment Consent.

Debtor Reference

A Debtor Reference is additional information for a given User that may be available to help understand the reason for a given payment instruction.

TPPs MUST:

COMM-049 Optionally allow Users to manually enter a Debtor Reference.

- Where using Structured References (as defined below) encourage Users to provide a Free Text Reference value, to provide a comprehensive reference for display in the Users statement or transaction history.

COMM-050 COMM-051 Ensure that the Debtor Reference reflects the source of the information, for example whether the value was set at a given TPP or Merchant.

- The Debtor Reference contains this information to allow Users to clearly identify the source of a given payment instruction.
- Information included in the Debtor Reference must be presented to Users in a meaningful way, designed to be human readable.

COMM-052 Where not dictated by the requirements of a given payments use case, use a Structured Reference that provides additional information to the User additional to that provided by a simple free text format.

The prescribed information to be included in the Structured Reference is as follows, to a maximum of the Max Payment Reference Length, using only characters in the Payment Reference Character Set:

- Free Text Reference, which is provided by the User, to a maximum of 22 characters.
- A dash ("-") as a separator.
- The Bank Name for the Creditor LFI, to a maximum of 8 characters, where the payment is not initiated on behalf of a Merchant.
- A dash ("-") as a separator.
- Optionally, the Merchant Name, to a maximum of 8 characters, as understood by the TPP where the TPP initiates payment on behalf of a Merchant.

- A dash (“-”) as a separator, where the Merchant Name is supplied.
- The TPP Name, either as found in the Trust Framework or using an alternative that is recognizable to the User.

COMM-053 The Free Text Reference, TPP Name, Merchant Name, and Bank Name MUST NOT contain the separator character, which [COMM-054] MUST be removed by TPPs before creating the Debtor Reference value.

COMM-055 Where values need to be truncated or removed due to the Max Payment Reference Length, the following is the order in which the attributes described above should be truncated or removed:

- TPP Name.
- Merchant Name, where supplied.
- Bank Name, where supplied.

COMM-056 The Free Text Reference MUST NOT be truncated. [COMM-057] TPPs MUST limit the Free Text Reference value to 22 characters where they implement a Structured Reference.

If a User declines to enter a Free Text Reference and no default is set by the TPP the space can be released to provide the TPP Name, Bank Name (where supplied) and Merchant Name (where supplied).

COMM-058 TPPs MUST NOT treat the Free Text Reference as reserved space, and

COMM-059 MUST also omit the separator after the Free Text Reference value.

COMM-060 For the avoidance of doubt, TPPs and LFIs MUST NOT treat a Structured Reference as a parsable string that can be used to consistently provide information programmatically. The intended audience for the Structured Reference is the User, to provide additional information when understanding their statement or transaction history in their internet or mobile banking application, or to help assist with reconciliation where specific Debtor Reference values are not required.

COMM-061 Include the Debtor Reference in each Payment Initiation Request. This information will not be transferred via the payment rails, and may be available to the User via existing online channels.

Additional Information

For clarity, the format of the Structure Reference scenarios (Merchant, non-Merchant) for Debtor References are shown in the tables below.

COMM-256 LFIs **MUST** support Debtor References up to the Max Payment Reference Length.

Debtor Reference Scenarios

Payment Initiated On-Behalf of Merchant

Free Text Reference	Separator	Merchant Name	Separator	TPP Name	Total Maximum Characters
Maximum 22	1	Maximum 8	1	Remaining characters	Max Payment Reference Length

Non-Merchant Payment

Free Text Reference	Separator	Creditor Bank Name	Separator	TPP Name	Total Maximum Characters
Maximum 22	1	Maximum 8	1	Remaining characters	Max Payment Reference Length

Creditor Reference

A Creditor Reference is a note for a given Creditor or Creditor LFI that supports reconciliation of a given payment instruction.

TPPs MUST:

COMM-063 Optionally allow Users to manually enter a Creditor Reference where a given use case does not require the value to be pre-populated.

- The Creditor Reference may be populated by the User (Debtor) using information requested by the Creditor or any other information the User wants to provide to assist the Creditor in identifying and reconciling the payment.
- The Creditor Reference will be transferred via the payment rails to the Creditor LFI, when an inter-LFI transaction is required.
- Where using Structured References (as defined below) encourage Users to provide a Free Text Reference value, to provide a comprehensive reference for display in the Creditor statement or transaction history.

COMM-064 Optionally pre-populate the Creditor Reference and prohibit Users from entering a Creditor Reference where a given use cases requires a specific Creditor Reference value.

- TPPs pre-populate the Creditor Reference based on specific use cases or business requirements, for example credit card payments that require a payment to a clearing account, invoice payments, and payments to clearing accounts for building society or credit union accounts.

COMM-065 Where a Creditor Reference value is not pre-populated based on a given use case, ensure that the Creditor Reference reflects the source of the information, for example whether the value was set at a given TPP or Merchant.

- The Creditor Reference contains this information to allow the Creditor to clearly identify the source of a given payment instruction.
- Information included in the Creditor Reference must be presented to the Creditor in a meaningful way, designed to be human readable.

COMM-066 Where not dictated by the requirements of the settlement system at the Creditor, use a Structured Reference that provides additional information to the Creditor above that

provided by a simple free text format.

The prescribed information to be included in the Structured Reference is as follows, to a maximum of the Max Payment Reference Length, using only characters in the Payment Reference Character Set:

- Free Text Reference, which is provided by the User, to a maximum of 22 characters.
- A dash (“-”) as a separator.
- The Bank Name for the Debtor LFI, to a maximum of 8 characters.
- A dash (“-”) as a separator.
- The TPP Name, either as found in the Trust Framework or using an alternative that is recognizable to the User.

COMM-067 The Free Text Reference, TPP Name, Merchant Name, and Bank Name **MUST NOT** contain the separator character, which [COMM-068] **MUST** be removed by TPPs before creating the Creditor Reference value.

Where values need to be truncated or removed due to the Max Payment Reference Length, the following is the order in which the attributes described above should be truncated or removed:

- TPP Name.
- Bank Name.

COMM-069 The Free Text Reference **MUST NOT** be truncated. [COMM-070] TPPs **MUST** limit the Free Text Reference value to 22 characters where they implement a Structured Reference.

If a User declines to enter a Free Text Reference and no default is set by the TPP the space can be released to provide the TPP Name. [COMM-071] TPPs **MUST NOT** treat the Free Text Reference as reserved space, and [COMM-072] **MUST** also omit the separator after the Free Text Reference value.

COMM-073 For the avoidance of doubt, TPPs and LFIs **MUST NOT** treat a Structured Reference as a parsable string, that can be used to consistently provide information programmatically. The intended audience for the Structured Reference is the Creditor, to provide additional information when understanding their statement or transaction history in their internet or mobile banking application, or to help assist with reconciliation where specific Creditor Reference values are not required.

COMM-074 Include the Creditor Reference in the Payment Initiation Request. The Creditor Reference will be used by the LFI to populate the Remittance Information of the payment.

LFIs MUST:

COMM-075 Where acting as the Debtor in a payment instruction, ensure the Creditor Reference is set appropriately in the payment instruction for the target payment rails.

- For Fawran, the Creditor Reference is expected to be set in the `/Document/FIToFICstmrCdtTrf/CdtTrfTxInf/RmtInf/Ustrd` property of the FI to FI Customer Credit Transfer (PACS.008) message (see Fawran Implementation Guide v5.4, section 4.9 pacs.008.001.08).

- LFIs **MUST NOT** send the Creditor Reference in `CdtTrfTxInf/PmtId/EndToEndId` (Fawran) or `CdtTrfTxInf/PmtId/TxId` (Tahweel). These properties carry the payment's unique transaction reference (see COMM-272).

- For Tahweel, the Creditor Reference is expected to be set in the remittance information property specified in QATCH Advanced Link Interface Specifications v2.7, section 2.1 Direct Credit.

- COMM-076 LFIs **MUST** verify the correct element for the payment instruction with their internal teams, to ensure the correct understanding of how the Creditor Reference will be transmitted by the target payment system.

Additional Information

For clarity, the format of the Structure Reference scenario for Creditor References is shown in the tables below.

COMM-257 LFIs **MUST** support Creditor References up to the Max Payment Reference Length.

Creditor Reference Scenarios

All Payments

Free Text Reference	Separator	Debtor Bank Name	Separator	TPP Name	Total Maximum Characters
Maximum 22	1	Maximum 8	1	Remaining characters	Max Payment Reference Length

Purpose of Payment

TPPs MUST:

COMM-007 Support all category purpose codes for domestic payments including their descriptions as defined in Payment Purpose Codes**.** TPPs **MUST** ensure that they are using the correct category purpose code and description as appropriate for their supported use cases.

COMM-008 Either allow Users to select the available purpose of payment from a list **or** pre-populate it for Users. Where Users select from a list, offer the codes that apply to the Debtor: the codes for individuals to retail Users, and the codes for corporates to SME Users (see Fawran Implementation Guide v5.4, Appendix G: External Category Purpose-Code, section 11.1 Individuals and section 11.2 Corporates).

COMM-009 Ensure and validate that the format of the payment purpose is according to the definition stated in Payment Purpose Codes.

LFIs MUST:

COMM-010 Check the payment purpose code is applicable for domestic payments. If the payment purpose code does not meet the specifications defined in Payment Purpose Codes**.** LFIs **MUST** reject the consent or payment initiation request using the code and return the appropriate error code to TPPs.

COMM-011 Apply the same BAU rules as they use today for other digital channels in relation to blocking/rejecting payment consents or initiations for specific category purpose codes. LFIs **MUST** provide the appropriate error code to TPPs in case they reject a consents or payment initiation based on a category purpose code.

COMM-265 Where a payment is directed to Tahweel, convert the category purpose code of the payment into the corresponding Tahweel purpose code (see QATCH Advanced Link Interface Specifications v2.7, section 2.1 Direct Credit). The mapping between the two sets of codes is currently **<TBC with QCB>**.

Customer IP Address

TPPs MUST:

COMM-012 Capture the public IP address assigned to the customer's device or network when the customer logs into the TPP's app or website.

COMM-013 Populate the `x-fapi-customer-ip-address` request header with the customer's IP address when the customer is currently logged into the TPP's app or website and a request is initiated based on a customer action. The presence of this header indicates that the API call is being made in a *customer-present* (attended) context.

TPPs MUST NOT:

COMM-014 Provide a generic, default, masked, or the TPP's own IP address in the `x-fapi-customer-ip-address` request header in place of the customer's actual IP address.

COMM-015 Populate the `x-fapi-customer-ip-address` request header when the request is initiated without the customer being present (i.e. when the customer is not logged into the TPP's app or website).

Adding a Trusted Payee (Beneficiary)

LFIs MUST:

COMM-016 Within the authorization page of the Bank Service Initiation consent journey, provide the User with the option to add the single payee and their Creditor account details to the User's list of Open Finance Trusted Payees (Beneficiaries).

This requirement applies when:

- The payee is of Type 'Individual', 'Merchant', 'Business', 'Charity', 'GovernmentBody', 'Other', (note Type = `PersonalIdentifiableInformation.Initiation.Creditor.CreditorAccount.Type`) and
- The payment type is a Single Instant Payment (SIP), or
- The Creditor is NOT already listed as a Trusted Payee for Open Finance payments.

COMM-017 When the above conditions are met, display a checkbox to the User with the following wording: **"Add {dynamic text} to my list of Trusted Payees."**

COMM-018 Apply the following mappings to determine and display the correct dynamic text:

- {dynamic text} = **Individual** - if `PersonalIdentifiableInformation.Initiation.Creditor.CreditorAccount.Type = Individual`.
- {dynamic text} = **Merchant** - if `PersonalIdentifiableInformation.Initiation.Creditor.CreditorAccount.Type = Merchant`.
- {dynamic text} = **Business** - if `PersonalIdentifiableInformation.Initiation.Creditor.CreditorAccount.Type = Business`.
- {dynamic text} = **Charity** - if `PersonalIdentifiableInformation.Initiation.Creditor.CreditorAccount.Type = Charity`.
- {dynamic text} = **Government Body** - if `PersonalIdentifiableInformation.Initiation.Creditor.CreditorAccount.Type = GovernmentBody`.
- {dynamic text} = **Organization** - if `PersonalIdentifiableInformation.Initiation.Creditor.CreditorAccount.Type`

= `Other`.

COMM-019 If the payee is already listed as an Open Finance Trusted Payee, the option to “Add” **MUST** be suppressed or replaced with a label such as **“Already Trusted”**.

Use of ISO Country and Currency Codes

COMM-021 **LFIs** and **TPPs MUST** use the applicable ISO code standard defined for the relevant data property to ensure consistent interpretation of country and currency information across Open Finance API requests and responses.

COMM-022 ISO 3166-1 country codes and ISO 4217 currency codes serve different purposes and MUST NOT be used interchangeably.

ISO 3166-1 Country Codes

COMM-023 A country code provided for geographic or jurisdiction-based information MUST be either:

- a valid, officially assigned ISO 3166-1 country code; or
- an Open Finance user-assigned country code defined in Section 27.2.

COMM-024 The country code MUST conform to the format defined for the relevant property in the applicable OpenAPI specification.

LFIs and TPPs MUST:

- COMM-025 Check the applicable API operation and property in the OpenAPI specification to determine whether an ISO 3166-1 alpha-2 or alpha-3 country code is required. The required format is defined by the property's schema pattern and description and may differ between properties.
- COMM-026 Provide the country code in the format defined for that specific property and MUST NOT assume that the format used for one property or API operation applies to another.

Geographic or jurisdiction-based information includes, but is not limited to:

- postal addresses;
- country of residence or incorporation;
- country of issue for identity documents; and
- country fields associated with customers, businesses, policies, insured risks, or insured assets.

The following schema patterns indicate the required ISO 3166-1 format:

OpenAPI schema pattern	Required format
`^[A-Z]{2}$`	A valid ISO 3166-1 alpha-2 country code or the applicable two-letter Open Finance user-assigned code
`^[A-Z]{3}$`	A valid ISO 3166-1 alpha-3 country code or the applicable three-letter Open Finance user-assigned code

COMM-027 Where another schema pattern is defined, the country code MUST conform to that pattern and to the property description in the applicable OpenAPI specification.

Conformance with the schema pattern alone does not make a value valid. [COMM-028] Except for the Open Finance user-assigned codes defined in Section 27.2, the value MUST also be an officially assigned ISO 3166-1 country code in the required format.

Examples

Country	ISO 3166-1 alpha-2	ISO 3166-1 alpha-3
Qatar	`QA`	`QAT`
United Kingdom	`GB`	`GBR`
India	`IN`	`IND`

Open Finance User-Assigned Country Codes

ISO 3166-1 does not assign generic country codes for an unknown or undetermined country. ISO 3166-3 provides four-letter codes for specific formerly used country names; however, those codes cannot be provided where an API property requires a two-letter or three-letter value.

For scenarios that cannot be represented using an officially assigned code supported by the applicable property, the Open Finance Standards define the following user-assigned code values.

Scenario	Where the property requires alpha-2	Where the property requires alpha-3
The country is unknown or cannot be determined	ZZ	ZZZ

Scenario	Where the property requires alpha-2	Where the property requires alpha-3
The information relates to a former country that cannot be represented using a more specific code supported by the applicable property	XX	XXX

LFIs and TPPs MUST:

- COMM-029 use ZZ or ZZZ, as determined by the applicable property schema, only where the country is genuinely unknown or cannot reasonably be determined;
- COMM-030 use XX or XXX, as determined by the applicable property schema, only where the information relates to a country that no longer exists and the applicable property cannot represent that former country using a more specific supported code;
- COMM-031 use a valid current ISO 3166-1 country code where the location can be attributed to a current successor country and doing so accurately represents the information being provided;
- COMM-032 not use ZZ, ZZZ, XX, or XXX as default values where the applicable country can be determined; and
- COMM-033 treat these values as Open Finance-defined user-assigned codes. Their meanings apply only within the Open Finance APIs and MUST NOT be assumed to apply in other systems or external schemes.

ISO 4217 Currency Codes

COMM-034 A currency code provided for a financial amount or monetary value MUST use a valid three-letter ISO 4217 currency code. ISO 4217 defines internationally recognized currency codes and specifies a three-letter alphabetic structure.

This includes, but is not limited to:

- premiums;
- payments;
- fees and commissions;
- cover limits;
- balances;

- exchange rates; and
- other monetary amounts.

Examples

Currency	ISO 4217 code
Qatari Riyal	`QAR`
Pound sterling	`GBP`
Indian rupee	`INR`

Is Single Authorization flag

TPPs MUST:

COMM-078 Set Is Single Authorization flag in the Consent to specify that the Consent **MUST** be subject to single User authorization only. This is used when there is a clear requirement that the Consent on the account to be used cannot be subject to multiple User authorizations.

Authorization Expiration Date Time

TPPs MUST:

COMM-079 Have the option to set the appropriate parameter in the Consent specifying the time window during which the Consent **MUST** be authorized by all required authorizers (one or multiple). The Authorization Expiration Date Time that can be specified by a TPP is limited to Limits and Constants | Max Authorization Time Window after the Consent has been submitted (i.e. staged) by the TPP for User authorization.

Risk Information Block
TPP Obligations

Based on the type of payment/use case, TPPs have different obligations with populating the various parts of the Risk Information Block.

TPPs MUST:

COMM-080 Set ALL the available data in the Risk Information Block in order to provide additional information in relation to the Payment Consent and allow LFI to assess their risk. This includes the following information as shown in the table below, based on the type of payment/use case. The information MUST be provided as per the following requirements:

- M- Mandatory
- N - Not Required
- C- Conditional
- O - Optional (if available/applicable)

COMM-081 Use this grid as the definitive source for which fields are to be populated, and how, across all payment scenarios. Each Risk Block field in this grid is marked with one of three population rules.

- An Active Session: Customer present, interactive, and all fields explicitly marked “Active session” in the standard must be fully populated with live values captured during that session.
- From consent set up: Reuse all values flagged as consent-based, captured at the original authorisation event, to anchor the risk context for all future transactions under that consent.
- From payment initiation: Populate fields with the data available at the exact time the payment is executed under a long-lived consent, even if the customer is not present.

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
A. User (Debtor) Indicators *Information related to the User.*					
1. User Name	Active session	From consent set up	C (if onboarded)	C (if onboarded)	C (if onboarded)
B. Authentication (Debtor) Indicators [COMM-082] When no interactive authentication occurs at the time of a payment (e.g. a subsequ under a long-lived consent), TPPs MUST only populate the fields with the authentication data cap time the consent was originally authorized. **TPPs MUST NOT**: • [COMM-083] Populate these fields with values from a later or unrelated session. • [COMM-084] Approximate authentication values if no relevant data exists. [COMM-085] Where the original consent context cannot be reliably retrieved, TPPs SHOULD leav fields null.					
1. Authentication Channel	Active session	From consent set up	C	C	C
3. Possession Factor Used (true/false)	Active session	From consent set up	M	M	M
4. Possession Factor Type	Active session	From consent set up	C	C	C

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
5. Knowledge Factor Used (true/false)	Active session	From consent set up	M	M	M
6. Knowledge Factor Type	Active session	From consent set up	C	C	C
7. Inherence Factor Used (true/false)	Active session	From consent set up	M	M	M
8. Inherence Factor Type	Active session	From consent set up	M	M	M
9. Challenge Outcome	Active session	From consent set up	C (if User prompted for MFA or other challenge)	C (if User prompted for MFA or other challenge)	C (if User prompted for MFA or other challenge)
10. Authentication Flow	Active session	From consent set up	C (if Challenge Outcome is Pass or Fail)	C (if Challenge Outcome is Pass or Fail)	C (if Challenge Outcome is Pass or Fail)
11. Authentication Value	Active session	From consent set up	C (if Challenge Outcome is Pass and	C (if Challenge Outcome is Pass and	C (if Challenge Outcome is Pass and

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
			value supported)	value supported)	value supported)
12. Challenge Date Time	Active session	From consent set up	C (if User prompted for MFA or other challenge)	C (if User prompted for MFA or other challenge)	C (if User prompted for MFA or other challenge)
C. (Debtor) GeoLocation					
1. Latitude	Active session	Null	C (if User consents to sharing location data)	C (if User consents to sharing location data)	C (if User consents to sharing location data)
2. Longitude	Active session	Null	C (if User consents to sharing location data)	C (if User consents to sharing location data)	C (if User consents to sharing location data)
D. (Debtor) Device Information					

[COMM-086] TPPs **MUST** populate Debtor Device Information with telemetry from the device use current session, where the user is present.

[COMM-087] When no device interaction occurs at the time of a payment (e.g. a subsequent pay long-lived consent), TPPs MUST populate the Device Information fields in the Risk Information Blo device metadata captured at the time the consent was originally authorized.

TPPs ~~**MUST NOT**~~:

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
• [COMM-088] Populate these fields with values from a more recent or unrelated session. • [COMM-089] Approximate device data if no relevant data exists. [COMM-090] Where the original device context used for consent setup is unavailable or cannot b SHOULD leave the affected fields null.					
1. Device ID	Active session	From consent set up	C	C	C
2. Alternative Device ID	Active session	From consent set up	O	O	O
3. Device Operating System	Active session	From consent set up	C	C	C
4. Device Operating System Version	Active session	From consent set up	C	C	C
5. Device Binding ID	Active session	From consent set up	C	C	C
6. Last Binding Date Time	Active session	From consent set up	C	C	C

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
7. Binding Duration	Active session	From consent set up	C	C	C
8. Binding Status	Active session	From consent set up	C	C	C
9. Device Type	Active session	From consent set up	C	C	C
10. Device Manufacturer Model	Active session	From consent set up	C	C	C
11. Device Manufacturer	Active session	From consent set up	C	C	C
12. Device Language	Active session	From consent set up	C	C	C
13. Device Local Date Time	Active session	From consent set up	C	C	C
14. Connection Type	Active session	From consent set up	C (with permissions)	C (with permissions)	C (with permissions)

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
15. Screen Information: Pixel Density	Active session	Null	C	C	C
16. Screen Information: Orientation	Active session	Null	C	C	C
17. BatteryStatus: Level	Active session	Null	C	C	C
18. BatteryStatus: Is Charging (true/false)	Active session	Null	C	C	C
19. Touch Support: Supported	Active session	Null	C	C	C
20. Touch Support: Max Touch Points	Active session	Null	C	C	C
21. Motion Sensors: Status	Active session	Null	C	C	C
22. Motion Sensors: Accelerometer	Active session	Null	C	C	C

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
23. Motion Sensors: Gyroscope	Active session	Null	C	C	C
24. Device Environment Context	Active session	From consent set up	C	C	C
E. (Debtor) Biometric Capabilities					
1. Supports Biometric	Active session	Null	M	M	M
2. Biometric Types	Active session	Null	C	C	C
F. (Debtor) App Information					
1. App Version	Active session	From consent set up	C	C	C
2. Package Name	Active session	From consent set up	C	C	C
3. Build Number	Active session	From consent set up	C	C	C
G. (Debtor) Browser Information					

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
1. User Agent	Active session	Null	C	C	C
2. Cookies Enabled (true/false)	Active session	Null	C	C	C
3. Available Fonts	Active session	Null	O	O	O
4. Plugins	Active session	Null	O	O	O
5. Pixel Ratio	Active session	Null	C	C	C
H. (Debtor) User Behavior					
1. Scroll Behavior: Direction	Active session	Null	C	C	C
2. Scroll Behavior: Speed	Active session	Null	C	C	C
3. Scroll Behavior: Frequency	Active session	Null	C	C	C
I. (Debtor) Account Risk Indicators					

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
[COMM-091] TPPs MUST populate all fields in this section with values that reflect the actual cont of consent authorization or payment initiation.					
1. User Onboarding Date Time	Active session	From consent set up	C (if onboarded)	C (if onboarded)	C (if onboarded)
2. Last Account Change Date	Active session	From consent set up	C (if onboarded)	C (if onboarded)	C (if onboarded)
3. Last Password Change Date Suspicious Activity	Active session	From consent set up	C (if onboarded)	C (if onboarded)	C (if onboarded)
4. Transaction History: LastDay	Active session	From payment initiation	C (if onboarded)	C (if onboarded)	C (if onboarded)
5. Transaction History: LastYear	Active session	From payment initiation	C (if onboarded)	C (if onboarded)	C (if onboarded)
J. (Debtor) Supplementary Data [COMM-092] TPPs MUST populate all fields in this section with values that reflect the actual cont of consent authorization or payment initiation.					

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
1. Supplementary Data	Active session	From payment initiation	O	O	O
K. Transaction Indicators					
[COMM-093] TPPs MUST populate all fields in this section with values that reflect the actual cont of consent authorization or payment initiation.					
1. Is Customer Present (true/false)	Active session	Null	M	M	M
2. Is Contract Present (true/false)	Active session	From consent set up or payment initiation	M	M	M
3. Channel Type	Active session	Null	M	M	M
4. Sub Channel Type	Active session	Null	O	O	O
L. (Transaction) Payment Process					
1. Total Duration	Active session	From consent set up	M	M	M
2. Current Session	Active session	From consent	M	M	M

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
Attempts		set up			
3. Current Session Failed Attempts	Active session	From consent set up	M	M	M
4. Last 24 Hour Attempts	Active session	From payment initiation	M	M	M
5. Last 24 Failed Attempts	Active session	From payment initiation	M	M	M
M. (Transaction) Merchant Risk					
1. Delivery Timeframe	Active session	From payment initiation	M	C	C
2. Reorder Items Indicator	Active session	From payment initiation	M	C	C
3. Pre Order Purchase Indicator	Active session	From payment initiation	M	C	C
4. Is Gift Card Purchase (true/false)	Active session	From payment initiation	M	C	C

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
5. Is Delivery Address Matches Billing (true/false)	Active session	From payment initiation	M	C	C
6. Address Match Level	Active session	From payment initiation	M	C	C
N. (Transaction) Supplementary Data					
1. Supplementary Data	Active session	From payment initiation	O	O	O
O. Creditor Indicators [COMM-094] **TPPs MUST** ensure that all Risk Block fields are consistent with the beneficiary cor the Consent. • [COMM-095] Where the consent specifies a fixed or defined beneficiary, the same values M for all subsequent payments. • [COMM-096] Where the consent supports a variable or open beneficiary (e.g., Variable Rec Payments), the relevant Risk Block fields MUST be populated dynamically at each payment i reflect the actual Creditor for that payment.					
1. Beneficiary Account Type (Retail)	Active session	From consent set up or payment initiation	M	M (Retail)	M

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
2. Beneficiary Prepopulated by TPP (true/false)	Active session	From consent set up or payment initiation	M	M	M
3. Merchant Trading Name	Active session	From consent set up or payment initiation	M	N	N
4. Is Verified By TPP (true/false)	Active session	From consent set up or payment initiation	C (if verified by KYC or COP)	N	N
5. Additional Beneficiary Account holder Identifiers such as: • Qatar ID or Passport Number for Retail accounts	Active session	From consent set up or payment initiation	M (Trade License number) M (Account Holder Name)	O (Qatar ID) C (D4) (Account Holder Name)	O (Qatar ID) C (D4) (Account Holder Name)

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
or Trade License number number for Business accounts • Account Holder Name					
P. (Creditor) Merchant Details [COMM-097] **TPPs MUST** ensure that all Risk Block fields reflect the merchant context defined ir • [COMM-098] Where the consent specifies a fixed merchant (e.g., a defined payee relationsl values MUST be retained for all subsequent payments. • [COMM-099] Where the consent supports a variable or open beneficiary, the merchant deta populated dynamically at each payment initiation to reflect the actual merchant involved in t					
1. Merchant ID	Active session	From consent set up or payment initiation	C	C	C
2. Merchant Name	Active session	From consent set up or payment initiation	C	C	C

Risk Information Block Element	Customer Present	Customer Not Present	Payments to Merchants onboarded by TPPs (Pay by Account)	P2P/B2P Payments	Payments-to-self
3. Merchant SIC Code	Active session	From consent set up or payment initiation	C	C	C
4. Merchant Category Code	Active session	From consent set up or payment initiation	C	C	C
Q. (Creditor) Supplementary Data					
1. Supplementary Data	Active session	From payment initiation	O	O	O

Risk Field Definitions

Additional details and descriptions for each factor can be found in the below spreadsheet:

LFI Obligations

LFIs MUST:

COMM-100 Use all the available information of the Risk Information Block provided by the TPP for their risk profile analysis of the Payment Consent or a Payment Initiation in a non-discriminatory way of Open Finance initiated payments compared to payments initiated by other LFI channels such as online or mobile banking.

Additional TPP Obligations

Details Verification for Merchants onboarded by TPPs

TPPs MUST:

COMM-101 Ensure that all Merchants onboarded follow KYC/KYB compliant processes. In addition to all other information the following Merchant details **MUST** be verified:

- Merchant's Trading Name
- Merchant's Bank Account details
- All other Merchant details are included in the Risk Information Block as defined in Common Rules and Guidelines | TPP Obligations for the Payments to Merchants onboarded by TPPs (Pay by Account) type of payments.

> **NOTE**
>
> TPPs will be populating some of this information in the Risk Information Block when submitting Payment Initiation requests.

Capturing of Qatar ID

TPPs MUST:

COMM-102 Capture a user's Qatar ID for all payments where a long-lived consent is required.

COMM-103 Capture a user's Qatar ID for all data sharing use cases where transaction data is shared, regardless of the consent type.

COMM-104 Capture the user's Qatar ID prior to initiating the prescribed user journey for the relevant use case.

- When a user is known to the TPP, the Qatar ID can be captured during onboarding.
- When a user is unknown to the TPP, the Qatar ID can be captured prior to the consent customer journey.

> **NOTE**
>
> TPPs are not required to verify the Qatar ID.

Consent Staging

> **Note**
>
> The TPP stages a Consent by submitting a Pushed Authorization Request (PAR) to the API Hub, with the Consent in the PAR `authorization_details`. See Consent Setup | Codification and the Authorization Endpoints API.

TPPs MUST:

COMM-105 Provide all the Consent parameters to OFP before redirecting Users to LFIs for authentication and proceed with Consent confirmation/authorization.

COMM-106 Setup the asynchronous event notifications (i.e. webhooks) with the OFP during this stage, in case TPPs want to use this method for any Consent or payment status updates.

OFP MUST:

COMM-107 Check the Consent parameters and validate their format and compliance to business rules.

COMM-108 Connect to the LFI and request the LFI to perform additional Consent parameter validations.

COMM-109 Reject the Consent and provide the appropriate error message to TPPs, if any of these validations performed fail.

COMM-110 Allow TPPs to retrieve information about the status of the Consent at any point in time.

LFIs MUST:

COMM-111 Check the Consent parameters requested by the OFP and validate them. If these are not validated, then LFIs **MUST** provide the appropriate error code to the OFP.

Check Is Single Authorization flag

> **Assumption**
>
> These rules assume that each LFI maintains an authorization matrix for each account, defining how many authorizers, and of which type, must approve a Consent or payment from that account. The authorization matrix is held and managed by the LFI under its existing processes; it is not defined by, or exchanged through, the Open Finance Standard. The same assumption applies to the Max Submitter Authorization Time Window (LIMT-003) and to Consents requiring multiple authorizers (COMP-105).

LFIs MUST:

COMM-112 COMM-113 Check if Is Single Authorization flag of the payment Consent is set against the authorization matrix of the account selected for the payment initiation. If the flag is set while the authorization matrix of the selected account requires multiple authorizers then LFIs **MUST** only display accounts that require single Authorization. If none exists, LFIs **MUST** decline the payment consent and **cancel** the payment journey and redirect the User back to the TPP with an appropriate response.

Multi-User Authorization Flow

> **NOTE**
>
> ***The rules in this section are additional rules intended to cover authorization cases for multi-user payment initiation from payment accounts.***

LFIs MUST:

COMM-114 Respond to the payment Consent request staged by the TPP with the OFP by providing an appropriate message to inform the User that the payment Consent to initiate one or more payments via the TPP is received by the LFI but is subject to further authorization. It is in the domain of the LFIs to determine how to do this in alignment with their own businesses' journeys.

COMM-115 Allow Users to use the payment initiation services if it is permitted within their user authority for the account, as defined by the LFI. In cases of shared personal accounts, a User will be able to initiate a payment Consent using a TPP, subject to having credentials with the LFI and the authority to initiate payments via the LFI's channels under their profile permissions.

COMM-116 Allow a User acting with delegated user authority on behalf of a business, to only use the Service Initiation, if this is permitted within the parameters of the delegated user authority and appropriate profile permissions. Payment Consent requests **MUST** be resolved by the LFI in line with the entitlements (i.e. access privileges) held by each specific business user. These entitlements may be set via approvals from authorized signatories of the business entities, and LFIs are assumed to have these processes already.

COMM-117 Inform the OFP about the requirement for multiple authorizations of the staged payment consent. LFIs **MUST** make available to the OFP via the payment Consent further information in relation to multi-authorization process. This information includes:

- The total number of Authorizers required for the payment Consent to be fully authorized
- For each Authorizer:
    - the Type of Authorizer (for example, Financial, Manager, Director or Accountant)
    - the date and time of when the Authorizer actioned the payment Consent
    - the status reflecting the Authorizer's final decision regarding the payment Consent (i.e. Pending, Approved or Rejected)

COMM-118 Be able to notify the OFP about the event of each authorizer in the chain authorizing or rejecting the payment Consent pending further authorization.

COMM-119 Change the state of the payment Consent with the OFP from Awaiting Authorization to Authorized when all Authorizers (one or more) have authorized the payment Consent.

OFP MUST:

COMM-120 Inform the TPP about the requirement for multiple authorizations of the staged payment consent. The OFP **MUST** make available to the TPPs via the payment Consent further information in relation to multi-authorization process. This information includes:

- All the information received from the LFI
- For each Authorizer:
    - the Type of Authorizer (for example, Financial, Manager, Director or Accountant)
    - the date and time of when the Authorizer actioned the payment Consent
    - the status reflecting the Authorizer's final decision regarding the payment Consent (i.e. Pending, Approved or Rejected)

COMM-121 Notify the TPPs about the event of each authorizer in the chain authorizing or rejecting the payment Consent pending further authorization.

Check Authorization Time Window

OFP MUST:

COMM-122 Reject a Consent Awaiting Authorization and move it to the terminal state in the following cases:

- COMM-123 The Limits and Constants | Max Submitter Authorization Time Window has elapsed AND the Consent had not been authorized by any of the required authorizers AND the TPP has not set the Authorization Time Window as per Common Rules and Guidelines | Authorization Expiration Date Time.

- COMM-124 The Authorization Time Window as per Common Rules and Guidelines | Authorization Expiration Date Time, if set by the TPP, has expired and the Consent has not been fully authorized.

LFI Manual Consent Handling

LFIs MUST:

COMM-254 Implement a process to update the status of consents to a revoked or suspended status based on authorized customer instruction.

COMM-255 When an authorized User for a specific SME has their access removed as per existing processes and policies between the LFI and their client, either suspend or revoke the associated long-lived consents for which that User is the authorizing User for consents currently in “Authorized” status.

Hand-off to LFI

TPPs MUST:

COMM-125 Provide messaging to inform Users that they will be redirected to their LFIs to complete the payment Consent authorization.

> **NOTE:**
>
> For guidelines on generic TPP to LFI redirection screens and messages, please refer to section Authentication and Authorization

COMM-126 Trigger the Authentication & Authorization process with the LFI by using the LFI's authorization endpoint.

LFIs MUST:

COMM-127 **NOT** present any additional inbound redirection screens.

COMM-128 Request the details included in the staged Consent from the OFP

OFP MUST:

COMM-129 Provide the Consent details to the LFIs based on the unique Consent identifier.

LFIs MUST:

COMM-130 Initiate the User Authentication process.

Payment Account Selection at LFI

LFIs MUST:

COMM-131 Enable Users, after authentication, to select the payment account for the payment Consent from a list of their available payment accounts, if the payment Consent parameters do not contain this information. The payment accounts displayed to the User **MUST** be eligible for account information or initiating payments via online and mobile banking channels and have no AML or other related restrictions.

COMM-132 Display to Users the interim available balance including all credit lines, arranged overdrafts and any other financing facility available on the account, of each payment account in the list, if the purpose is Service Initiation.

COMM-133 Provide Users with the ability to **cancel** the payment journey, if Users decide to terminate the request. The LFI **MUST** hand-off the Users back to the TPP, providing the necessary error message to the TPP.

Hand-off back to the TPP

LFIs MUST:

COMM-134 Confirm to the OFP that the interaction session is now completed.

OFP MUST:

COMM-135 Initiate the User Redirection process by providing all necessary information to the LFI (including the required Authorization code)

LFIs MUST:

COMM-136 Provide messaging to inform Users that they will be taken back to their TPP.

COMM-137 Redirect Users to the TPP immediately after receiving the required information from the OFP.

x-idempotency-key Checks

TPPs MUST:

COMM-138 When submitting a `POST /payments` request, provide a **unique value** in the mandatory `x-idempotency-key` header parameter.

OFP MUST:

COMM-139 When a `POST /payments` request is received, verify whether the `x-idempotency-key` value:

- has previously been provided by the **same TPP**,
- was received **within the last 24 hours**, and
- is associated with a request where the **payment amount is the same**.

When **all three conditions** above are met:

- COMM-140 the payment initiation request (`POST /payments`) to the LFI **MUST NOT** be triggered,

- COMM-141 if a previous **201 Created** response was returned, the **same 201 response MUST** be returned to the TPP, including the original payment resource details,
- COMM-142 if an error response was previously returned, the **same error response MUST** be returned again to the TPP.

COMM-143 COMM-144 When the `x-idempotency-key` value:

- has previously been provided by the **same TPP**,
- was received **within the last 24 hours**, **but**
- the payment amount in the new request **differs** from the payment amount associated with the original request for that key,

then:

- the payment initiation request (`POST /payments`) to the LFI **MUST NOT** be triggered, and
- a **409 Conflict** error **MUST** be returned to the TPP.

Payment Status Update

TPPs MUST:

COMM-145 Check with the OFP for the status of a payment at various points in time throughout the payment lifecycle. The frequency of these queries to the OFP **MUST** be subject to fair usage policies. This is applicable in cases where TPPs are using the method of querying the payment status from the OFP.

COMM-146 Have the option to be notified by the OFP of the payment request status change when it occurs without requiring to query the OFP. This is only applicable in cases where TPPs selected to use the method of requesting the OFP to notify them asynchronously in the event of changes in the status of a payment.

- COMM-147 Setup the asynchronous event notifications (i.e. webhooks) with the OFP during the payment Consent Staging step as defined in Common Rules and Guidelines | Consent Staging in case they want to use this method for status updates.

OFP MUST:

COMM-148 Request the LFIs to notify it asynchronously on the event of changes in the status of a payment. The setup the asynchronous event notifications with the LFI will take place during the payment Consent staging.

COMM-149 Update the status of each payment initiated by the TPP throughout the payment lifecycle with the appropriate status value based on its progress through payment initiation processing and execution as provided by the LFI.

COMM-150 When a payment under a Variable On-Demand Consent is Rejected, the OFP **MUST** decrement the Consent Control Parameter Values of payments by 1 and its total value by the payment amount.

COMM-151 If the rejected payment was made in the current Control Period, the OFP **MUST** also decrement that Control Period's total number of payments by 1 and its total value by the payment amount. If it was made in an earlier Control Period, the OFP **MUST NOT** change the Control Period totals.

Each decrement **MUST** be applied only once per rejected payment.

A rejection of a Tahweel payment can arrive days later, after its Control Period has ended. In that case, only the Consent Control Parameter Values are decremented.

COMM-152 Respond to payment status query requests from a TPP with the appropriate status codes as listed below in section Common Rules and Guidelines | Payment Status Model based on the status information provided by the LFI.

COMM-153 Return to the TPP, in addition to the payment status, other payment system-specific status codes and payment rejection reasons codes as specified in the QCB Open Finance Standard specifications. For Fawran Core these are defined in Fawran Implementation Guide v5.4, Appendix E: Status Codes and Appendix F: Status Reason Codes; for Tahweel, in QATCH Advanced Link Interface Specifications v2.7, section 2.3 Payment Status Report and section 3.1 Status of Validation.

LFIs MUST:

COMM-154 Update the status of each payment initiated by the OFP throughout the payment lifecycle with the appropriate status value based on its progress through payment initiation processing and execution.

COMM-155 Be able to notify the OFP asynchronously about any changes in the status of the payment request. More specifically, LFIs **MUST** provide asynchronous notification to the OFP by sending the appropriate status codes as listed below in section Common Rules and Guidelines | Payment Status Model,

COMM-156 Return to the OFP in addition to the payment status, other payment system-specific status codes and payment rejection reasons codes as specified in the QCB Open Finance Standard specifications, including the status and reason codes returned by the payment rail (see COMM-153).

COMM-268 Where a payment is directed to Tahweel, provide the OFP with the settlement window information returned by Tahweel (see QATCH Advanced Link Interface Specifications v2.7, section 3.1 Status of Validation), so that the TPP can inform the User when the payment will be settled.

Payment Status Model

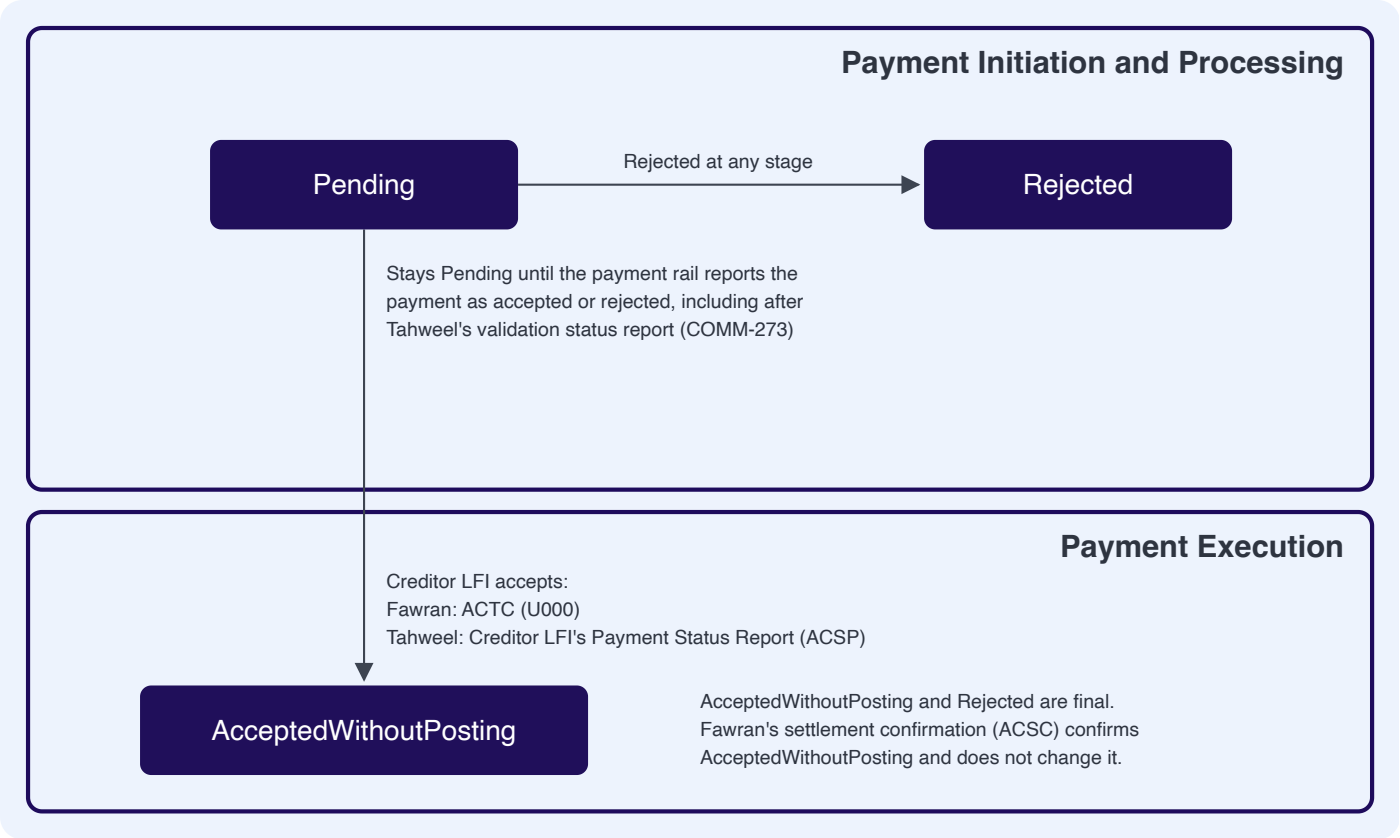
The Open Finance payment status model has three coarse-grained payment states, suitable for communication to TPPs and, by extension, Users, and used within the payment log: Pending, AcceptedWithoutPosting and Rejected. The model includes only the payment states that both Fawran Core and Tahweel can report. How the ISO 20022 payment transaction statuses relate to

these three states, and how a payment moves through them on each payment rail, is described in Payment Status Information.

This model applies to all supported payment types, including Single Immediate and multi-payments.

COMM-157 The Open Finance payment state MUST reflect the highest confirmed lifecycle state received from the payment rail or the LFI.

> **NOTE:**
>
> In relation to Payment execution, once the Debtor LFI has performed all process steps and checks, the payment can be placed in the infrastructure of the chosen payment rails, processed and passed on to the Creditor LFI.*
>
> *On both Fawran Core and Tahweel, the Creditor LFI responds to the payment by accepting or rejecting it, and the payment rail passes that response to the Debtor LFI. Neither payment rail sends the Debtor LFI a confirmation that the Creditor account has been credited.*
>
> *For Fawran Core, the Creditor LFI responds to Fawran Core, which sends the Debtor LFI its acceptance or rejection of the payment, followed, for an accepted payment, by a settlement confirmation (Fawran Implementation Guide v5.4, section 2.2.2 Credit Transfer – Successful Completion, section 2.2.3 Credit Transfer - Rejected by the Receiving Participant and Appendix E: Status Codes). If the Creditor LFI does not respond within Fawran Core's time out period, Fawran Core rejects the payment (section 2.2.6 Credit Transfer – FAWRAN System Times out). For Tahweel, the Creditor LFI's response is its Payment Status Report (QATCH Advanced Link Interface Specifications v2.7, section 2.3 Payment Status Report), which is separate from Tahweel's own validation status report (section 3.1 Status of Validation). Refer to Payment Status Information | Payment Execution Status for more details.*



Open Finance Payment States & Description

Open Finance Payment Status	Open Finance Payment State Description
Pending	The payment has been received by the Debtor LFI and has not yet been accepted or rejected. COMM-274 The LFI MUST report the payment as Pending from the moment the payment resource is created until the payment rail reports it as accepted or rejected. COMM-273 A Tahweel validation status report that accepts the payment (ACTC, QATCH Advanced Link Interface Specifications v2.7, section 3.1 Status of Validation) MUST NOT change the status. It is Tahweel's own acknowledgement, not the Creditor LFI's response.
AcceptedWithoutPosting	The Creditor LFI has accepted the payment. This is the final successful status. Neither payment rail confirms that the Creditor account has been credited.

Open Finance Payment Status	Open Finance Payment State Description
	COMM-160 The LFI MUST set the status to AcceptedWithoutPosting when the payment rail reports the Creditor LFI's acceptance: Fawran Core status ACTC with reason code U000 (Fawran Implementation Guide v5.4, section 2.2.2 Credit Transfer – Successful Completion), or Tahweel status ACSP in the Creditor LFI's Payment Status Report (QATCH Advanced Link Interface Specifications v2.7, section 2.3 Payment Status Report). COMM-275 The payment rail's report that the payment has been settled MUST NOT change the status: for Fawran Core, the settlement confirmation (ACSC, Fawran Implementation Guide v5.4, section 2.2.2); for Tahweel, the update of the payment to settled (ACSC) at Tahweel's daily incremental settlement, reported in its settlement reports (QATCH Incremental Settlement v1.0, sections 1.2 and 1.3). When a TPP has subscribed to receive Payment Status notifications, the final status update that will be received is when the payment state is reported as AcceptedWithoutPosting or Rejected. As settlement breaks may occur after the payment rail has confirmed its final accepted status, COMM-159 TPPs supporting merchants under an acquirer model SHOULD verify actual funds settlement by checking the merchant's account and COMM-158 MUST NOT rely solely on the Open Finance AcceptedWithoutPosting status.
Rejected	The payment has been rejected or cancelled and will not proceed further. COMM-276 The LFI MUST set the status to Rejected when the LFI rejects the payment, when either payment rail reports a rejection, or when the cancellation of a payment directed to Tahweel succeeds (COMM-269). Tahweel reports the outcome of a cancellation in its response to the Request for Cancellation (QATCH DD/DC Cancellation Process v1.1, section 2.1.3).

Open Finance Payment Status	Open Finance Payment State Description
	The status is accompanied by an error code and description identifying the reason for the rejection (see COMM-153 and COMM-156).

Payment Rail Status Mapping

Payment rail status	Payment rail	Open Finance status
Received, being checked	Both	Pending
ACTC: batch accepted, settlement window set (QATCH Advanced Link Interface Specifications v2.7, section 3.1)	Tahweel	Pending
ACTC with reason code U000: accepted by the Creditor LFI (Fawran Implementation Guide v5.4, section 2.2.2)	Fawran Core	AcceptedWithoutPosting
ACSC: settlement confirmation (Fawran Implementation Guide v5.4, section 2.2.2)	Fawran Core	AcceptedWithoutPosting (no change)
ACSP: the Creditor LFI's reply (QATCH Advanced Link Interface Specifications v2.7, section 2.3)	Tahweel	AcceptedWithoutPosting
ACSC: settled at Tahweel's daily incremental settlement, reported in its settlement reports (QATCH Incremental Settlement v1.0, sections 1.2 and 1.3)	Tahweel	AcceptedWithoutPosting (no change)
Response to a Request for Cancellation accepting the cancellation; Tahweel changes the original payment to rejected (QATCH DD/DC Cancellation Process v1.1, section 2.1.3)	Tahweel	Rejected

Payment rail status	Payment rail	Open Finance status
RJCT at any stage, or a Fawran Core Message Reject (admi.002, Fawran Implementation Guide v5.4, section 2.2.4)	Both	Rejected

Open questions, **<TBC with QCB>**:

- The status for Fawran Core ACTC with reason code U002 (Fawran Implementation Guide v5.4, Appendix F: Status Reason Codes).
- What the LFI receives from Tahweel if the Creditor LFI does not respond.

For each rail's messages in full, and how to tell a Tahweel validation status report from the Creditor LFI's Payment Status Report, see Payment Status Information | Payment Execution Status.

Payment Status Information

Description

This section provides additional information and guidelines to TPPs and LFIs in relation to the status of the payment Consent and the payment initiation process. The information covers both short-lived Consents for Single Instant Payments (SIPs) and long-lived Consents for Multi-Payments. More specifically, the additional information covers the following:

- The status at any time after staging the payment Consent and the submission of the payment Initiation request.
- The meaningful status messages and reason codes of why the staged payment Consent or submitted payment Initiation request is stopped in a particular stage of the process.
- The confirmation that the payment Initiation request has been received and executed.
- The enriched list of reasons for failure through the initiation, processing and execution stages. This includes the reasons for failure reported by the underlying payment rail, Fawran Core or Tahweel, after the payment has been submitted to it.

The payment states used throughout this page are those of the Payment Status Model: Pending, AcceptedWithoutPosting and Rejected.

Overall Process Flow

The Open Finance Standard allows LFIs to provide TPPs with the following payment status information:

- The status at any time after the submission of the payment Initiation request, for all supported payment types, including SIPs and Multi-Payments.
- A meaningful status message to the TPP for each processing phase, including the reason for any rejection.
- A confirmation that the payment has been accepted by the Creditor LFI (the status AcceptedWithoutPosting). For Fawran Core, a settlement confirmation follows immediately (Fawran Implementation Guide v5.4, section 2.2.2). For Tahweel, the payment is settled at Tahweel's daily incremental settlement and reported as settled in its settlement reports (QATCH Incremental Settlement v1.0, sections 1.2 and 1.3).
- An enriched and more granular list of status reasons for failure or rejection, as per the ISO 20022 standard, through the payment initiation, processing and execution stages of

payments at the Debtor LFI, the payment rail and the Creditor LFI.

Domestic payments are executed through Fawran Core as the default payment rail, and through Tahweel as the fallback payment rail for payments that Fawran Core cannot process, as described in Supported Accounts & Payment Rails. The two rails report the outcome of a payment differently, so the process for each is described separately.

Process for Fawran Core

Fawran Core is an instant payment rail and operates 24 hours a day, 7 days a week. A payment submitted to Fawran Core reaches its final status when Fawran Core reports the Creditor LFI's acceptance or a rejection. If the Creditor LFI does not respond within Fawran Core's time out period, Fawran Core rejects the payment (Fawran Implementation Guide v5.4, section 2.2.6).

A payment executed via Fawran Core moves from Pending directly to AcceptedWithoutPosting or Rejected, as Fawran Core reports the payment's acceptance only once the Creditor LFI has accepted it.

Process for Tahweel

Tahweel is an ACH batch-clearing payment rail. A payment directed to Tahweel under COMM-204, COMM-206 or COMM-261 is processed in a Tahweel settlement session (LIMT-026). Its final status is reported when the Debtor LFI receives the Creditor LFI's Payment Status Report, or a rejection, through Tahweel.

A payment accepted at validation by Tahweel remains Pending until Tahweel reports that the payment has been accepted or rejected (COMM-273). Where a payment is directed to Tahweel, the LFI provides the OFP with the settlement window information returned by Tahweel, so that the TPP can inform the User when the payment will be settled (COMM-268).

Status Information Rules & Guidelines

Payment Consent Status

Payment Consent Response Status

The TPP stages the payment Consent by submitting a Pushed Authorization Request (PAR) to the API Hub. The Consent is carried in the PAR `authorization_details`, with a Consent ID that

the TPP assigns (see Consent Setup | Codification and the Authorization Endpoints API). Following the PAR:

- If the PAR is validated successfully, the payment Consent is created with the status `AwaitingAuthorization`. The API Hub responds with HTTP 200 and a `request_uri`, which the TPP uses to redirect the User to the LFI (see Multi-Payments | Consent Staging, MPGN-013). The response does not include the Consent ID or the Consent status.
- If the PAR fails validation, the API Hub responds with HTTP 400 and an OAuth error, made up of an `error` code and an `error_description`. The TPP **MUST** rely on this error to identify the issue.

The TPP can read the status of the Consent at any time with `GET /payment-consents/{ConsentId}`, using the Consent ID it assigned (COMM-110). At this stage, the status is `AwaitingAuthorization`.

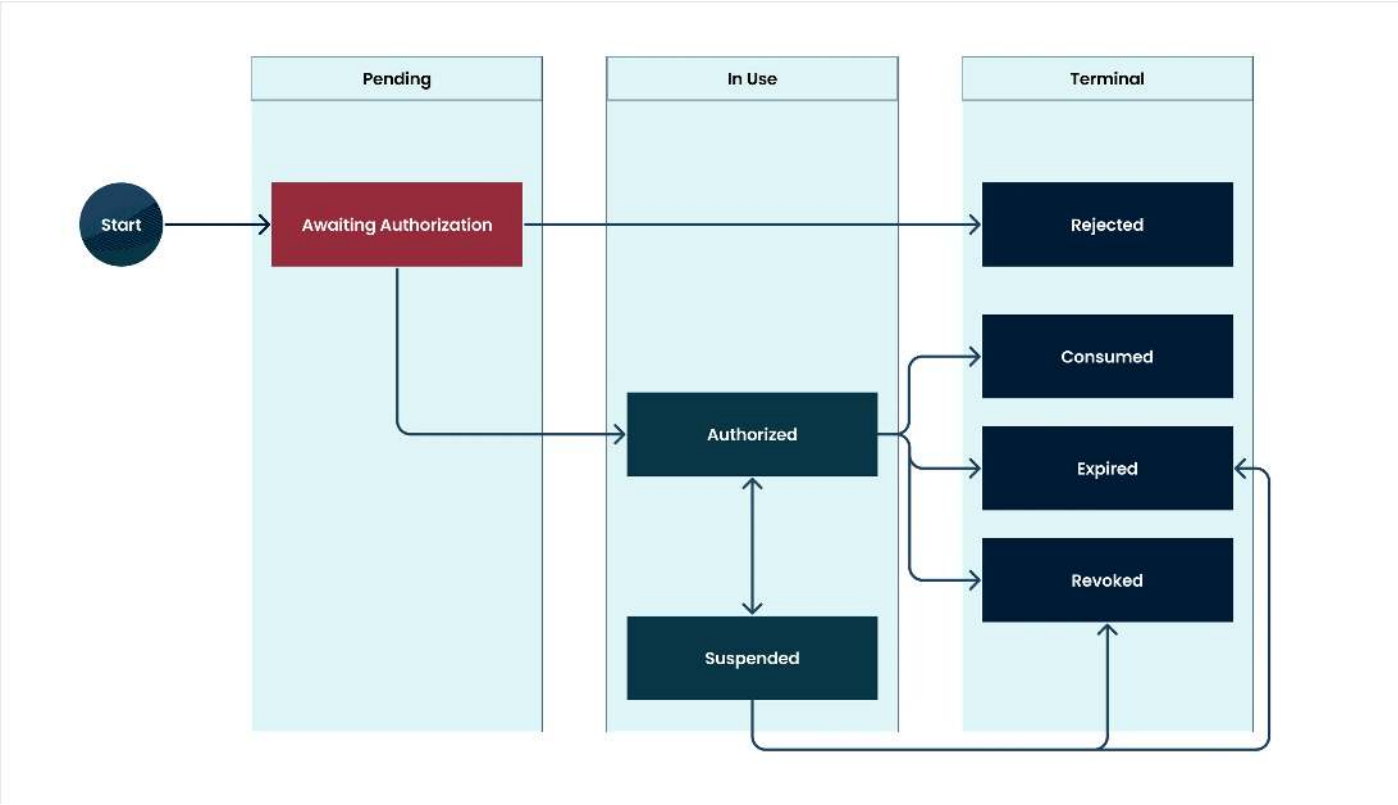


Figure 1: Staged Payment Consent

> **Note**
>
> The Consent may not move to its next natural stage of `Authorized` or `Rejected` by a User action at the authorization stage with the LFI. This may happen when the User is not redirected to the LFI for authentication, the User fails to complete authentication, or for various other reasons. In this case, the API Hub **MUST** move the Consent to `Rejected` after the Max Submitter Authorization Time Window (LIMT-003) has elapsed, and populate the appropriate error reason code(s). For example:

Status Code	Status Reason Code	Status Reason Description
Rejected	**<TBC with QCB>**	Authorization not completed within the Max Submitter Authorization Time Window (LIMT-003)

User Authentication/Authorization Status

If the payment Consent is staged successfully, the User is redirected to the LFI for authentication and authorization of the Consent.

If the User authenticates successfully, then:

- Where there is no further call to action for the User after authentication, as in the Fast Track Journey of Single Instant Payments | Customer Experience, the status of the payment Consent **MUST** move to `Authorized`, as the authentication is also considered to be the User's authorization.
- Where there is a further call to action for the User after authentication, as in the Standard Journey, the User's decision to proceed with the payment (or the long-lived payment Consent) is considered to be the User's authorization, and **MUST** move the status of the payment Consent to `Authorized`. For example:

Status Code	Status Reason Code	Status Reason Description
Authorized	<empty>	<empty>

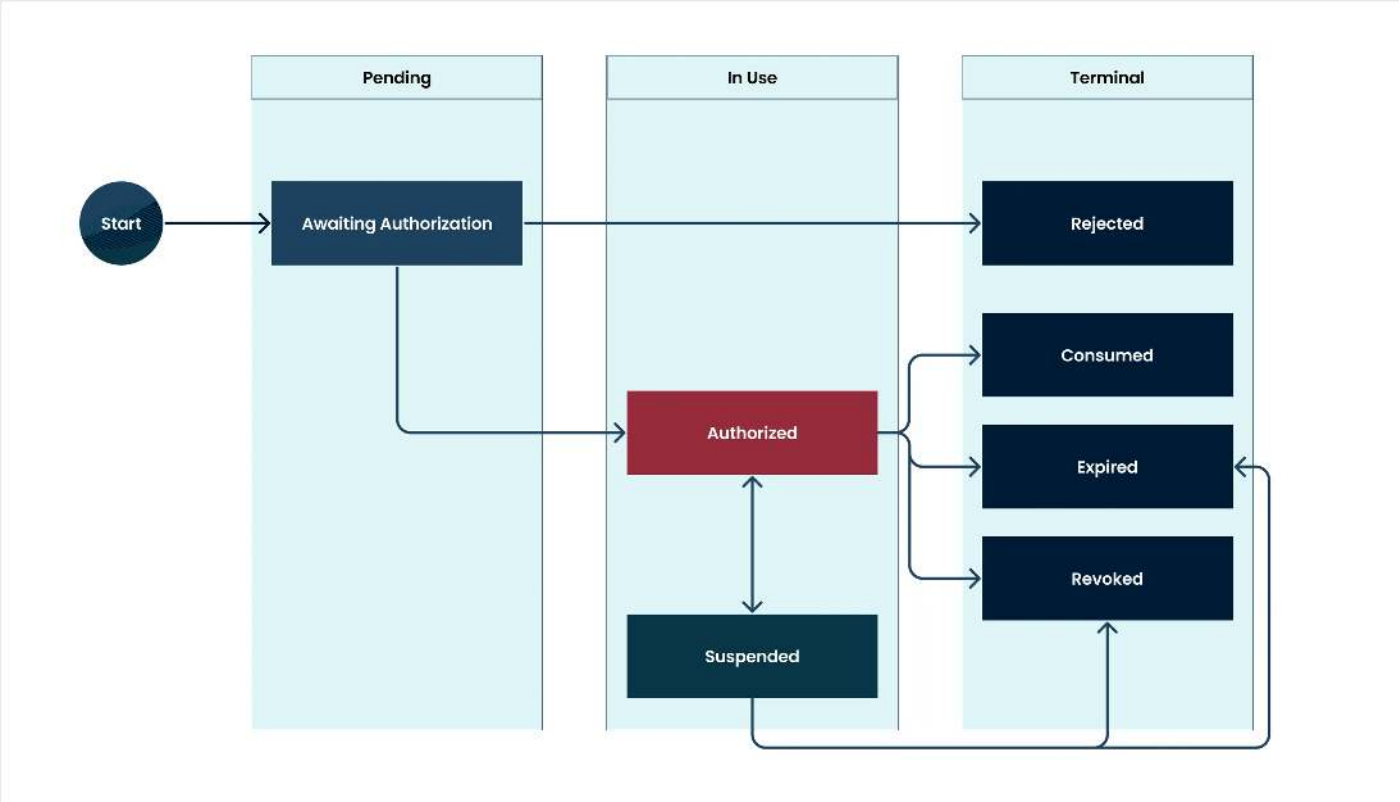


Figure 2: Authenticated Payment Consent

The payment Consent **MUST** be set to `Rejected` where:

- the User's call to action leads to the User cancelling the payment (or the long-lived payment Consent);
- the User fails to complete authentication (for example, after a number of attempts), and the LFI rejects the Consent; or
- further checks performed by the LFI on the staged payment Consent fail, and the LFI rejects the Consent. An example is a Fast Track SIP where the User selected a payment account with insufficient funds for the payment.

Example rejections are shown below. The status reason codes are ISO 20022 examples, for illustration only; Fawran Core and Tahweel do not define them.

Status Code	Status Reason Code	Status Reason Description
Rejected	AM04	Insufficient Funds
Rejected	CN01	Authorization Cancelled
Rejected	**<TBC with QCB>**	Authorization failed (after a number of attempts)

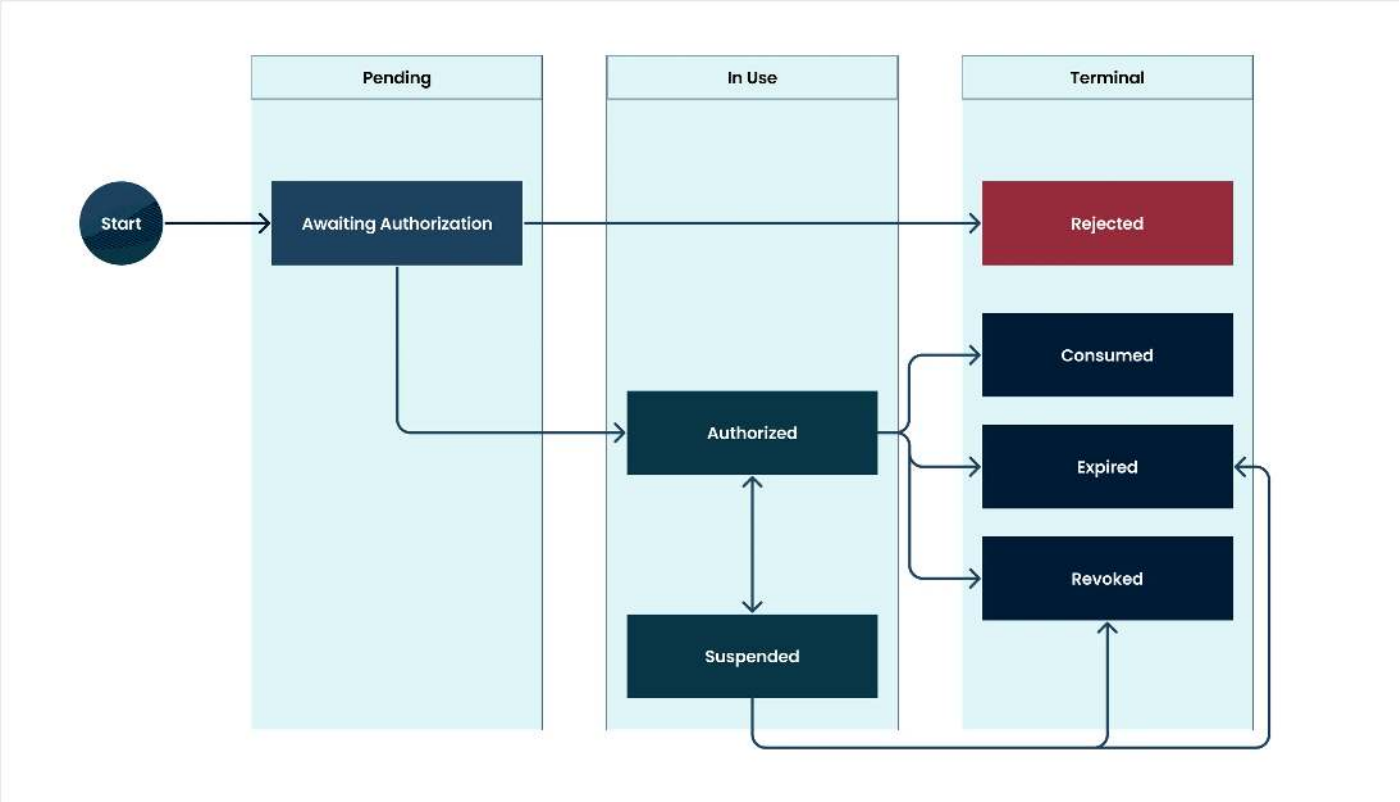


Figure 3: Rejected Payment Consent

A TPP can call the GET status endpoint at this stage to find out whether the payment Consent resource has been updated to `Authorized` or `Rejected`. A TPP that has subscribed to event notifications **MUST** receive a notification from the API Hub of the updated status of the payment Consent resource, using the registered Webhook URL.

Mapping to the ISO 20022 Payment Transaction Status Model

The Open Finance Payment Status Model is a simplified version of the ISO 20022 Payment Transaction Status model. It retains only the statuses that both Fawran Core and Tahweel can report, at the start, middle and end of the payment lifecycle. The ISO 20022 statuses and their definitions are listed in ISO 20022 Payment Transaction Status Model.

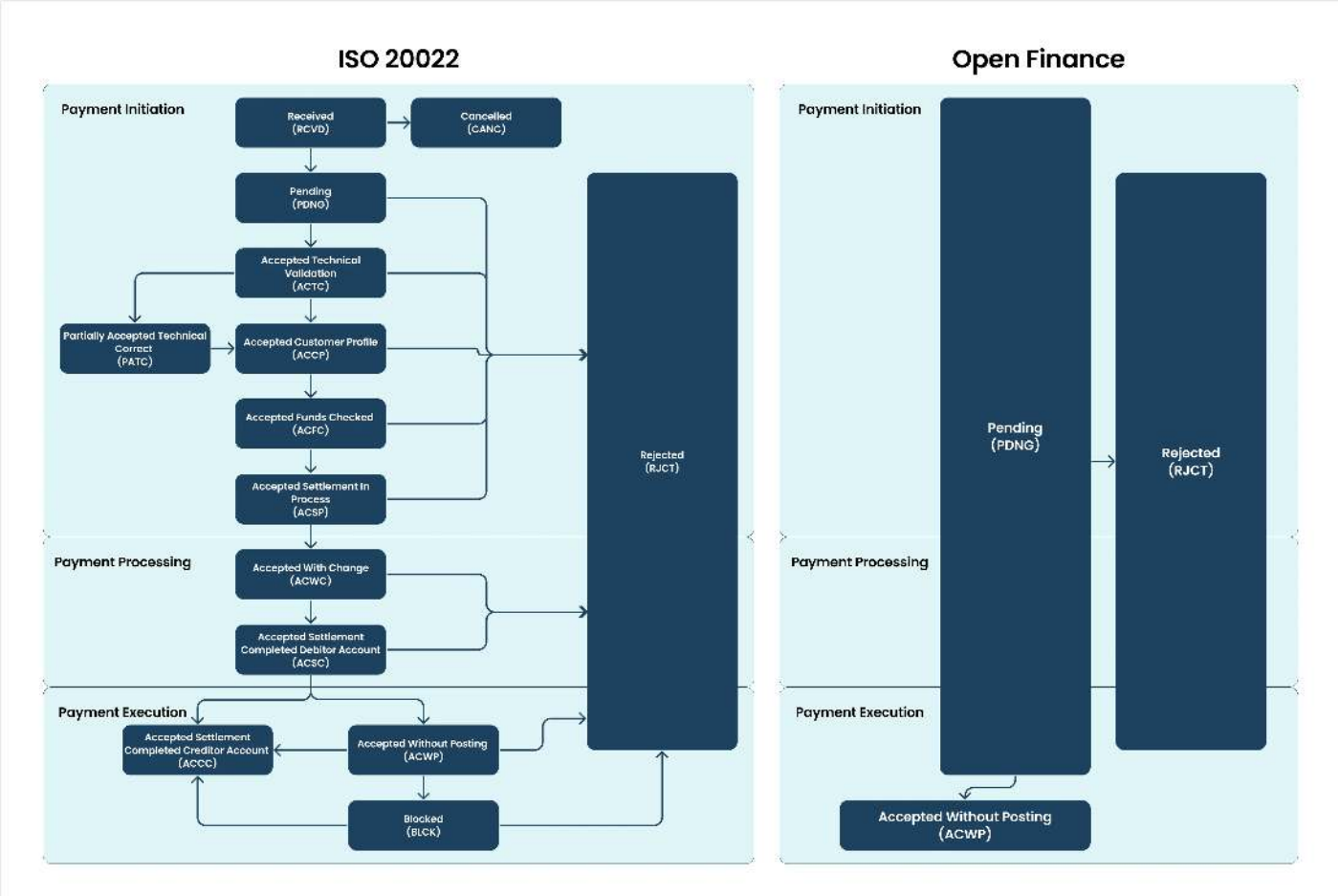


Figure 4: ISO 20022 Payment Transaction Status model versus Open Finance Payment Status model

Open Finance retains the following statuses:

- **Pending (ISO 20022 PDNG):** The initial state of the Payment resource when it is created. It signifies that the payment initiation checks are starting. The payment remains Pending while the Debtor LFI completes its checks and submits the payment to the payment rail, and until the payment rail reports that the payment has been accepted or rejected. An acknowledgement from the payment rail that it has accepted the payment for processing does not change the status (COMM-273).
- **AcceptedWithoutPosting (ISO 20022 ACWP):** The terminal state for a successful payment. The payment has been accepted by the Creditor LFI, as reported by the payment rail, but the credit to the Creditor account has not been confirmed (COMM-160). Neither Fawran Core nor Tahweel sends the Debtor LFI a confirmation that the Creditor account has been credited, so this is the final status for a successful payment on both rails.
- **Rejected (ISO 20022 RJCT):** The rejection status. Rejection can happen at any point during payment initiation, payment processing or payment execution, and is accompanied by an error code identifying the reason for the failure.

The other ISO 20022 statuses map to the Open Finance statuses as follows:

ISO 20022 Status	Open Finance Status
RCVD, PDNG, ACTC, PATC, ACCP, ACFC, ACSP, ACWC, ACSC	Pending
ACWP	AcceptedWithoutPosting
ACCC	Not used. Neither Fawran Core nor Tahweel reports the credit to the Creditor account
BLCK	Not used. Neither Fawran Core nor Tahweel reports it
CANC	Rejected (for example, a Tahweel payment cancelled under COMM-269)
RJCT	Rejected

Payment Initiation Status

Payment Initiation Response Status

Subject to the payment Consent being successfully authorized by the User, the TPP submits the payment Initiation request to the API Hub/LFI.

When the payment Initiation request is received by the API Hub and/or the LFI, the payment Consent resource status **MUST** be set to `Consumed`. This applies only to single payments such as SIPs. For Multi-Payments, the payment Consent remains `Authorized` until it is fully consumed, expires or is revoked.

Status Code	Status Reason Code	Status Reason Description
Consumed	<empty>	<empty>

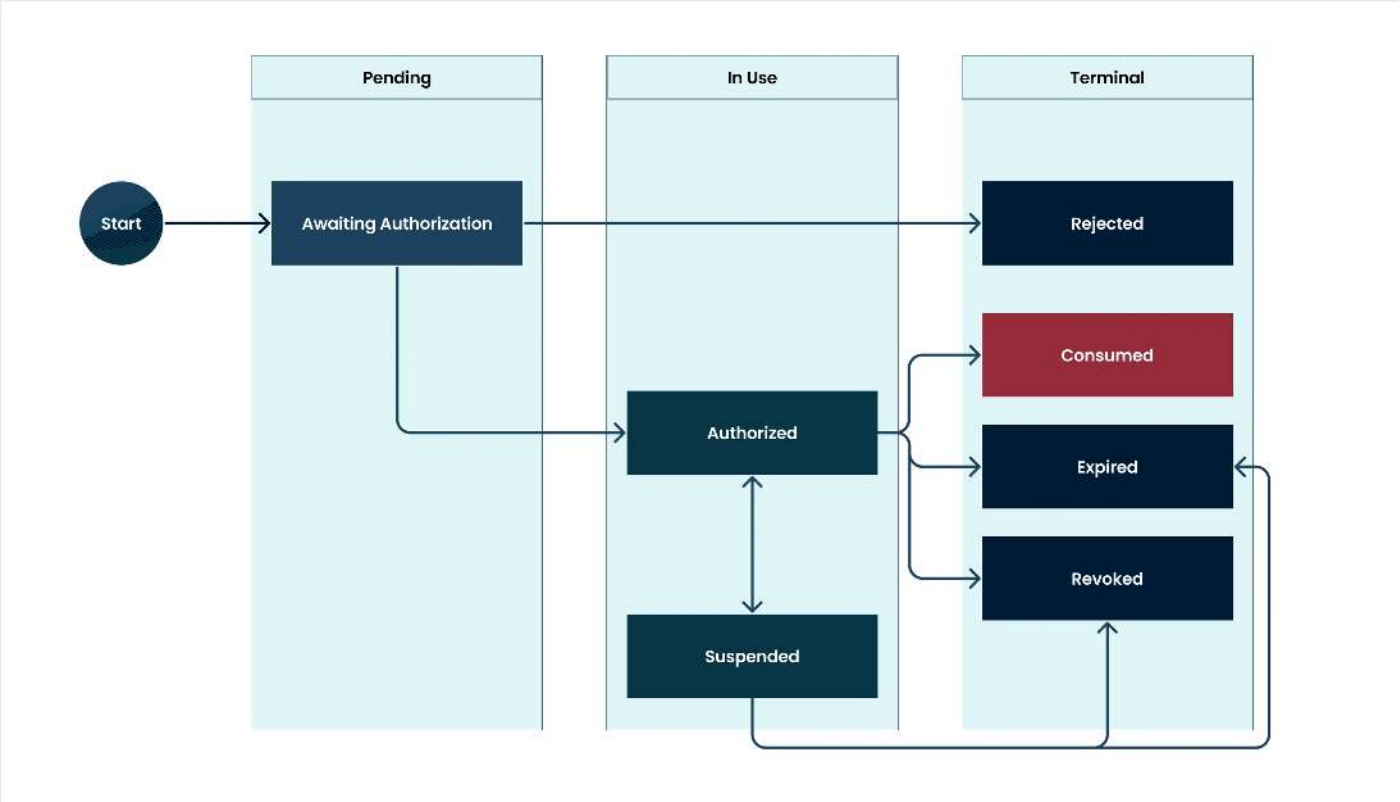


Figure 5: Payment Consent Consumed

If the payment Initiation request has been successfully received and validated by the API Hub and/or the LFI, a new Payment resource is created. The status at this point, as per the ISO 20022 model, would be Received (RCVD). As per the mapping to the Open Finance Payment Status model, the status of the Payment resource **MUST** be set to Pending, the initialization value for the Payment resource status. The API Hub/LFI **MUST** also respond to the TPP that the payment Initiation request has been successful (HTTP 201 status) including the Payment resource status Pending and the Payment resource ID.

Status Code	Status Reason Code	Status Reason Description
Pending	<empty>	<empty>

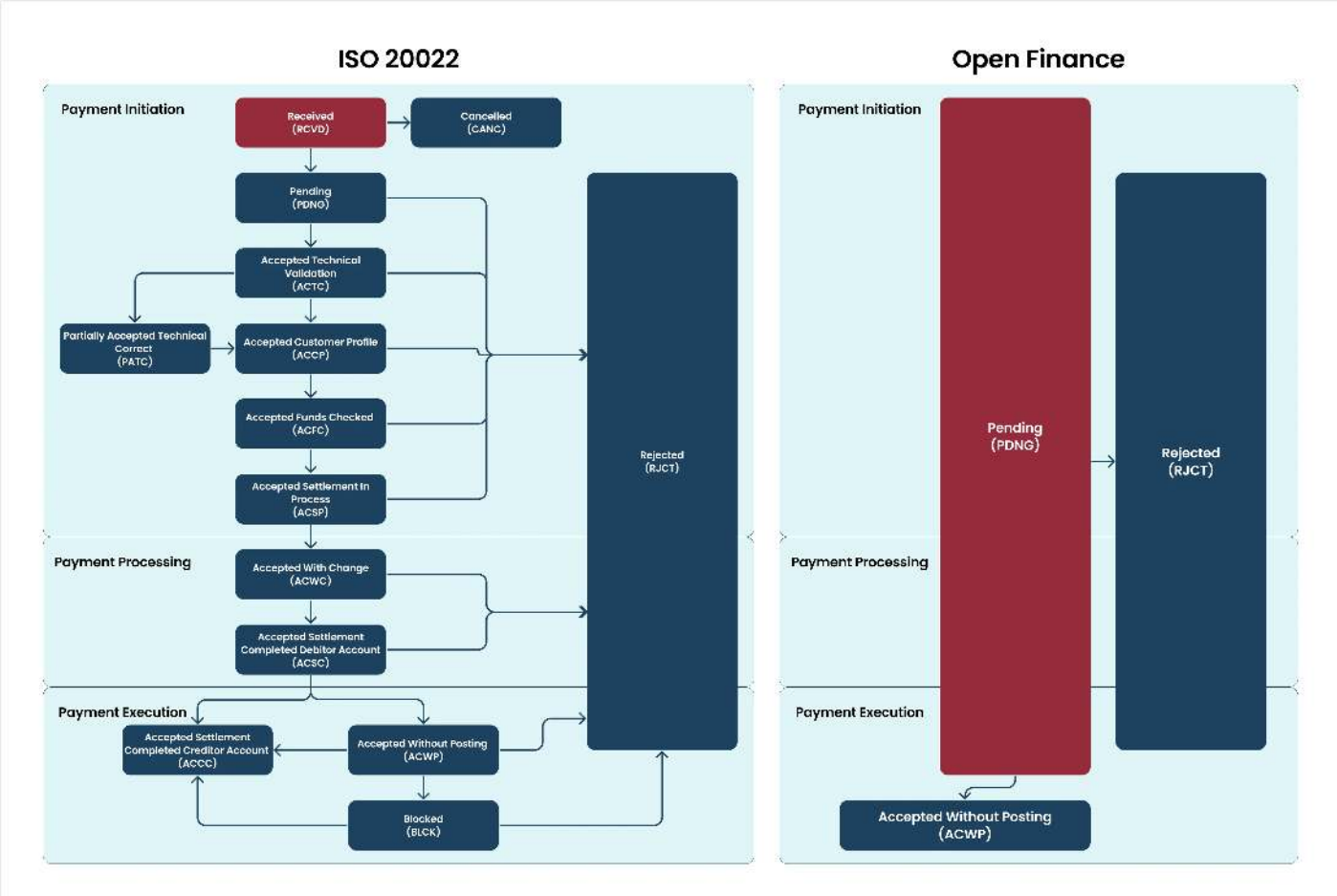


Figure 6: Payment Initiation started

A TPP can call the GET payment Consent status endpoint at this stage to confirm that the payment Consent resource has been updated to `Consumed` (single payments only). A TPP that has subscribed to event notifications **MUST** receive a notification from the API Hub of the updated status of the payment Consent resource, using the registered Webhook URL.

If the payment Initiation request fails to be received and validated by the API Hub or the LFI, an HTTP 4xx series response with status reason code(s) **MUST** be sent back to the TPP. In this case the TPP has to rely on the status reason(s) in the error response to identify the issue, as the Payment resource was not created.

Further Payment Initiation Status

Once the Payment resource has been created with the status Pending, further checks and validations take place at the LFI as part of the payment initiation process. As per the ISO 20022 model, the payment may pass through Pending (PDNG), Accepted Technical Validation (ACTC), Accepted Customer Profile (ACCP) and Accepted Funds Checked (ACFC). As per the mapping to the Open Finance Payment Status model, the Payment resource **MUST** remain Pending throughout.

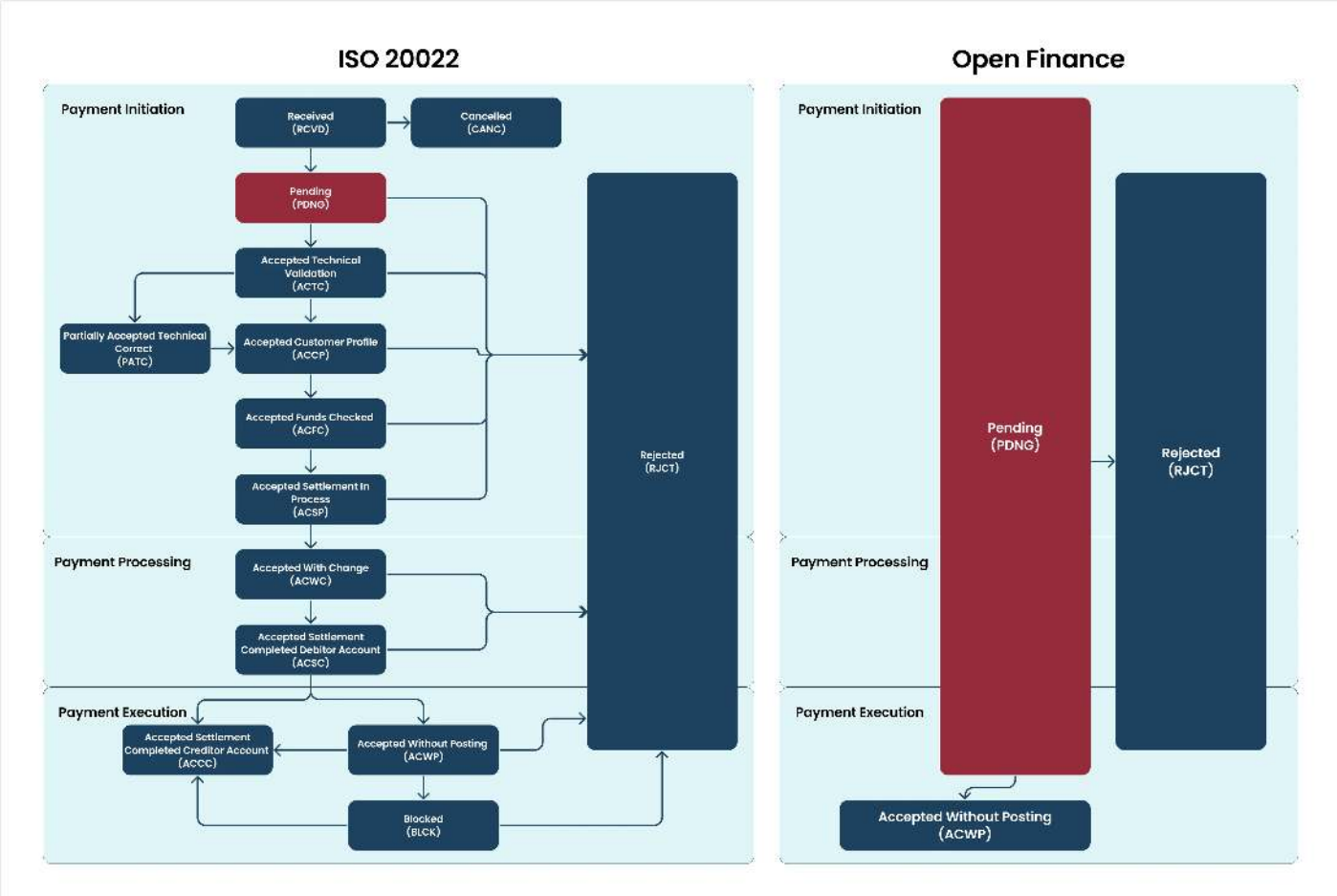


Figure 7: Payment Initiation pending

A payment with a future execution date is submitted to Fawran Core on its execution date (COMM-264), and the Payment resource **MUST** remain Pending until then. Where the User cancels such a payment at their LFI before it is submitted, the LFI **MUST** update the Payment resource status to Rejected and provide the appropriate error code.

Status Code	Status Reason Code	Status Reason Description
Rejected	**<TBC with QCB>**	Cancelled by User

> **Note**
>
> In several situations (such as SIPs) the progress from the payment initiation phase to the payment processing phase happens very quickly, depending on each LFI's implementation, so there is limited value in returning the granular ISO 20022 status. The Open Finance Payment Status model returns Pending to represent all of the intermediate statuses before the payment rail reports that the payment has been accepted or rejected.

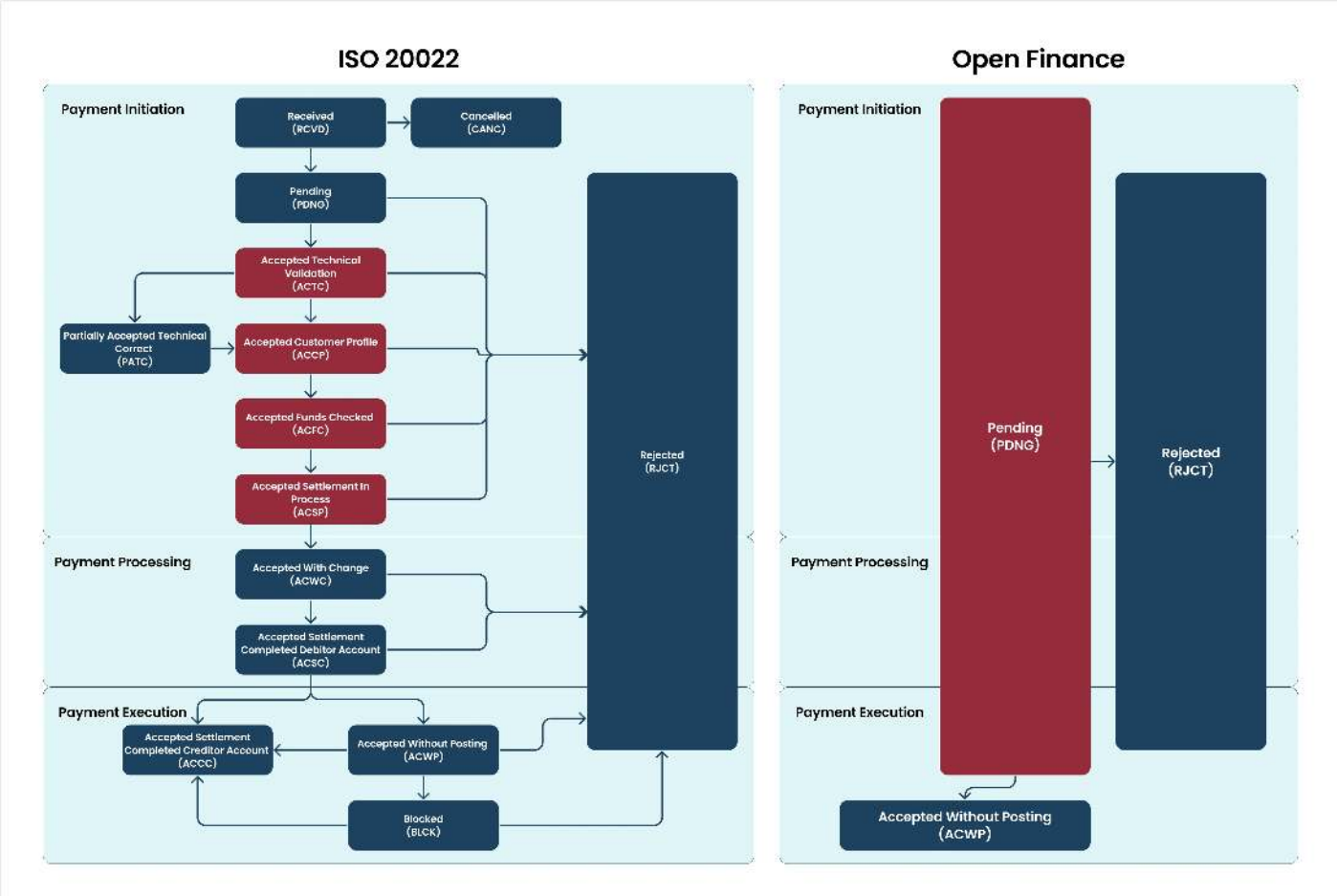


Figure 8: Payment Initiation completed

If the payment fails any validation or other check during the payment initiation process, the status of the Payment resource **MUST** be updated to Rejected. The LFI **MUST** provide all the appropriate error reason codes to inform the TPP of the reasons for the failure. For example, using ISO 20022 status reason codes for illustration only (Fawran Core and Tahweel do not define them):

Status Code	Status Reason Code	Status Reason Description
Rejected	AM04	Insufficient Funds
Rejected	AM14	Amount Exceeds Agreed Limit

A payment that cannot be executed via Fawran Core because the mandatory Debtor name or account details are missing is also rejected at this stage (COMM-207 to COMM-213).

A TPP can call the GET payment status endpoint at this stage to check whether the payment has remained Pending or has been Rejected. A TPP that has subscribed to event notifications **MUST** receive a notification from the API Hub of the change of status of the Payment resource to Rejected, using the registered Webhook URL.

> **Note**
>
> LFIs may decide to complete the payment initiation and processing validations and checks before responding to the payment Initiation request with an HTTP 201 status and creating the Payment resource. In this case, any rejection during technical validation, customer profile checking, funds checking or other related checks is notified to the TPP with an HTTP 4xx series response carrying the appropriate failure reason code(s). It is in the competitive space of LFIs how they implement this and at which stage they respond positively to the payment Initiation request, providing any further status updates and error reason codes via the Payment resource.

Payment Processing Status

The Payment resource may undergo further checks during payment processing. If any of these checks fail, the status of the Payment resource **MUST** be set to Rejected.

In some instances the LFI may process the payment with a change. For example, the payment rail may change from Fawran Core to Tahweel, because the Creditor LFI is not a Fawran Core participant (COMM-204), Fawran Core is unavailable (COMM-206) or the amount exceeds the Fawran Core maximum (COMM-261). As per the ISO 20022 model, the status of the Payment resource would become Accepted With Change (ACWC). As per the mapping to the Open Finance Payment Status model, the Payment resource **MUST** remain Pending.

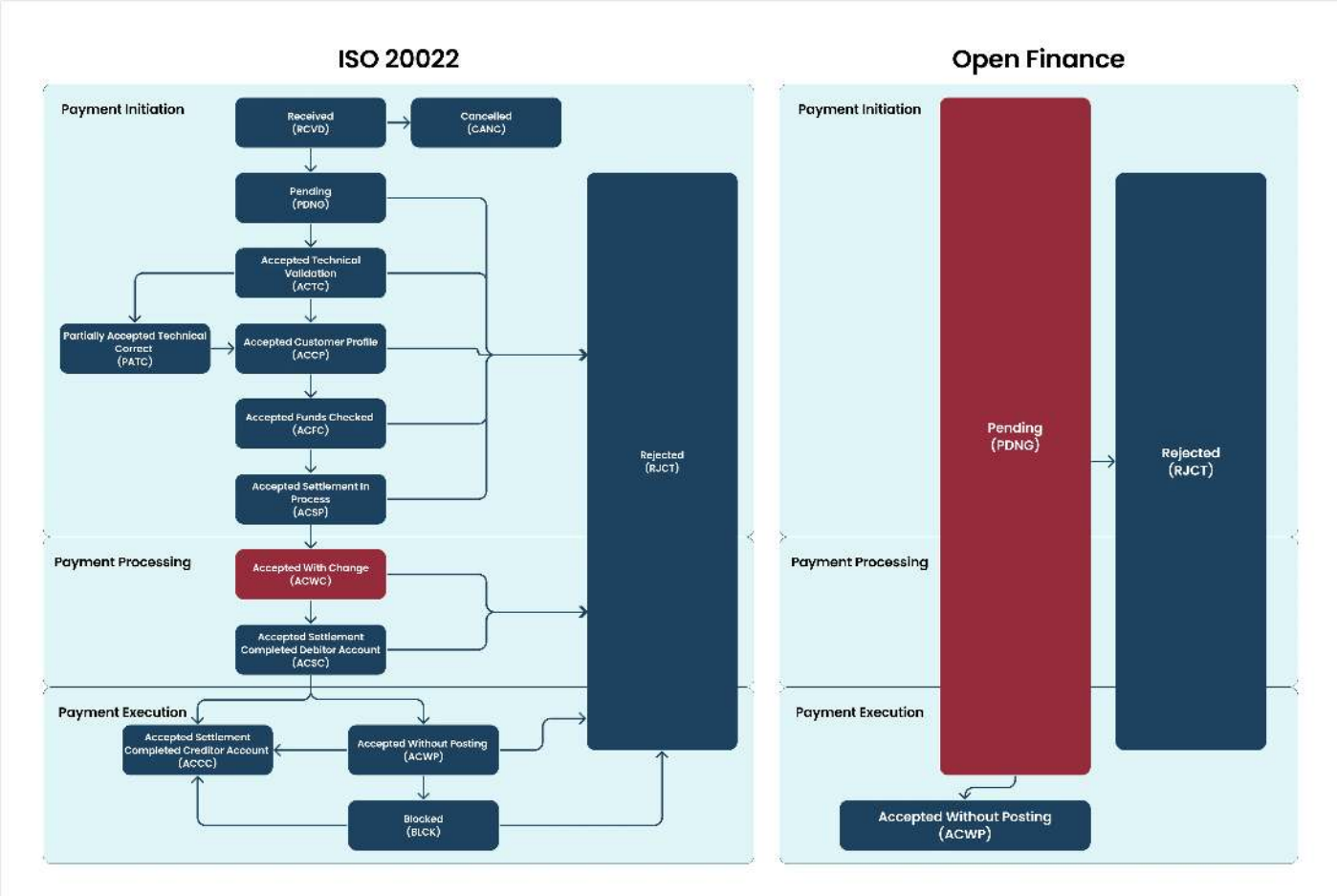


Figure 9: Payment Processing started

When payment processing has completed successfully, the Debtor LFI submits the payment to the payment rail. The Payment resource **MUST** remain Pending until the payment rail reports that the payment has been accepted or rejected, as described in Payment Execution Status.

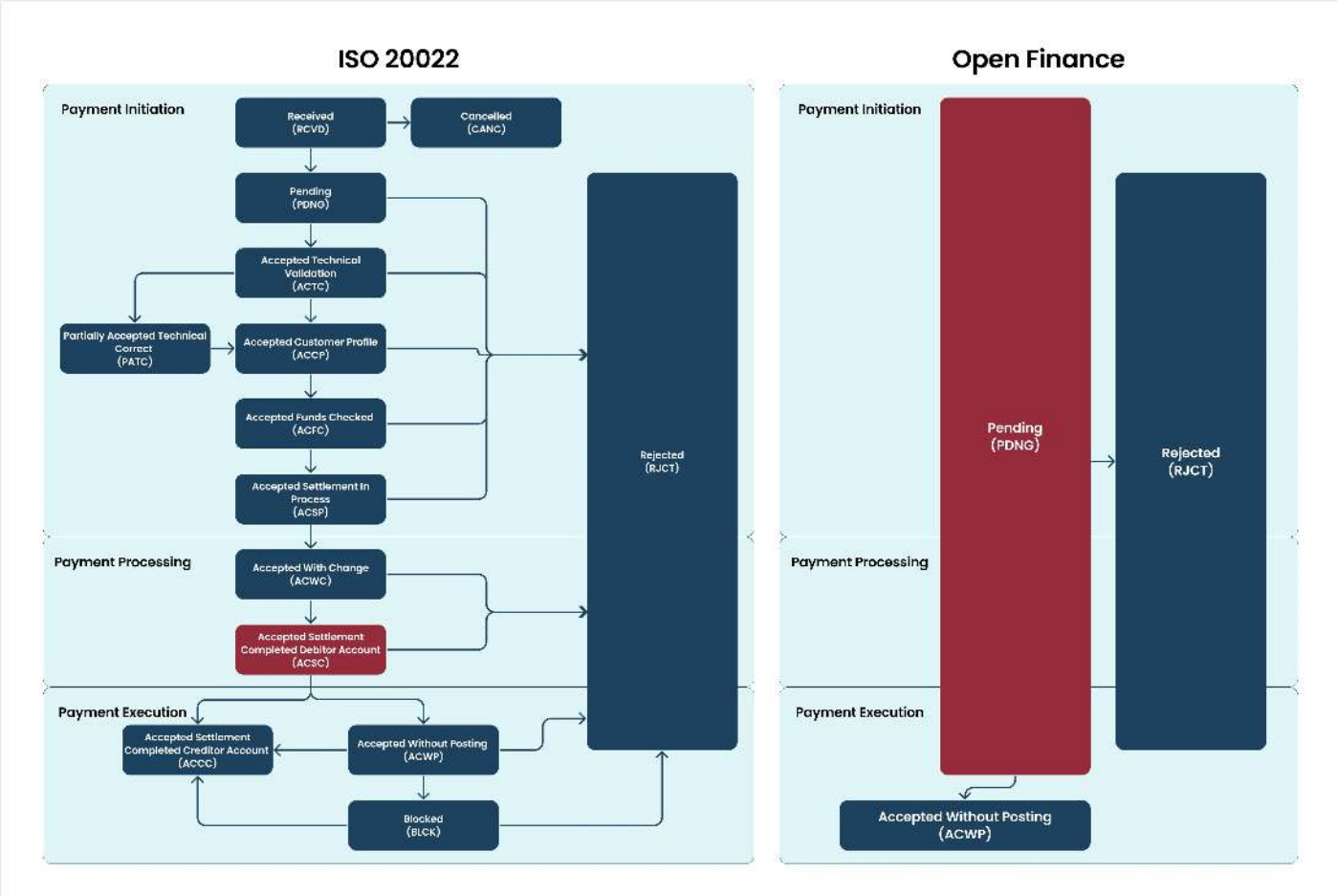


Figure 10: Payment Processing completed

If the payment fails at any point during payment processing, the status of the Payment resource **MUST** be updated to Rejected. The LFI **MUST** provide all the appropriate error reason codes to inform the TPP of the reasons for the failure.

Payment Execution Status

Once the payment has been submitted to the payment rail, it undergoes the payment rail's technical and business validations and the Creditor LFI's technical and business validations. For Fawran Core, the Creditor LFI can reject the payment for valid business reasons such as invalid customer or account status, exceeding limits or suspected fraud (Fawran Implementation Guide v5.4, sections 2.2.2 to 2.2.5). Further to these checks:

- When the Debtor LFI receives an acknowledgement from the payment rail that it has accepted the payment for processing, such as Tahweel's validation status report, the Payment resource **MUST** remain Pending (COMM-273).
- If any of the checks fail, the Payment resource status **MUST** be set to Rejected. For example:

Status Code	Status Reason Code	Status Reason Description
Rejected	U908	Reject sent due to Restrictions on account (Fawran Implementation Guide v5.4, Appendix F: Status Reason Codes)

- Neither Fawran Core nor Tahweel reports a Blocked (BLCK) status. A Creditor LFI that does not accept the payment rejects it, and the payment rail reports the rejection to the Debtor LFI.
- When the Debtor LFI receives the payment rail's final accepted status for the payment, the Payment resource status **MUST** be set to AcceptedWithoutPosting (COMM-160). Where the payment rail provides a status reason code with the acceptance, the Debtor LFI **MUST** provide it alongside the Payment resource status.

Payment Rail	Status Code	Status Reason Code	Status Reason Description
Fawran Core	AcceptedWithoutPosting	U000	Success/Transaction Accepted/Settlement Confirmation (Fawran Implementation Guide v5.4, Appendix F: Status Reason Codes)
Tahweel	AcceptedWithoutPosting	<empty>, unless the Creditor LFI provides one (QATCH Advanced Link Interface Specifications v2.7, section 2.3 Payment Status Report)	<empty>

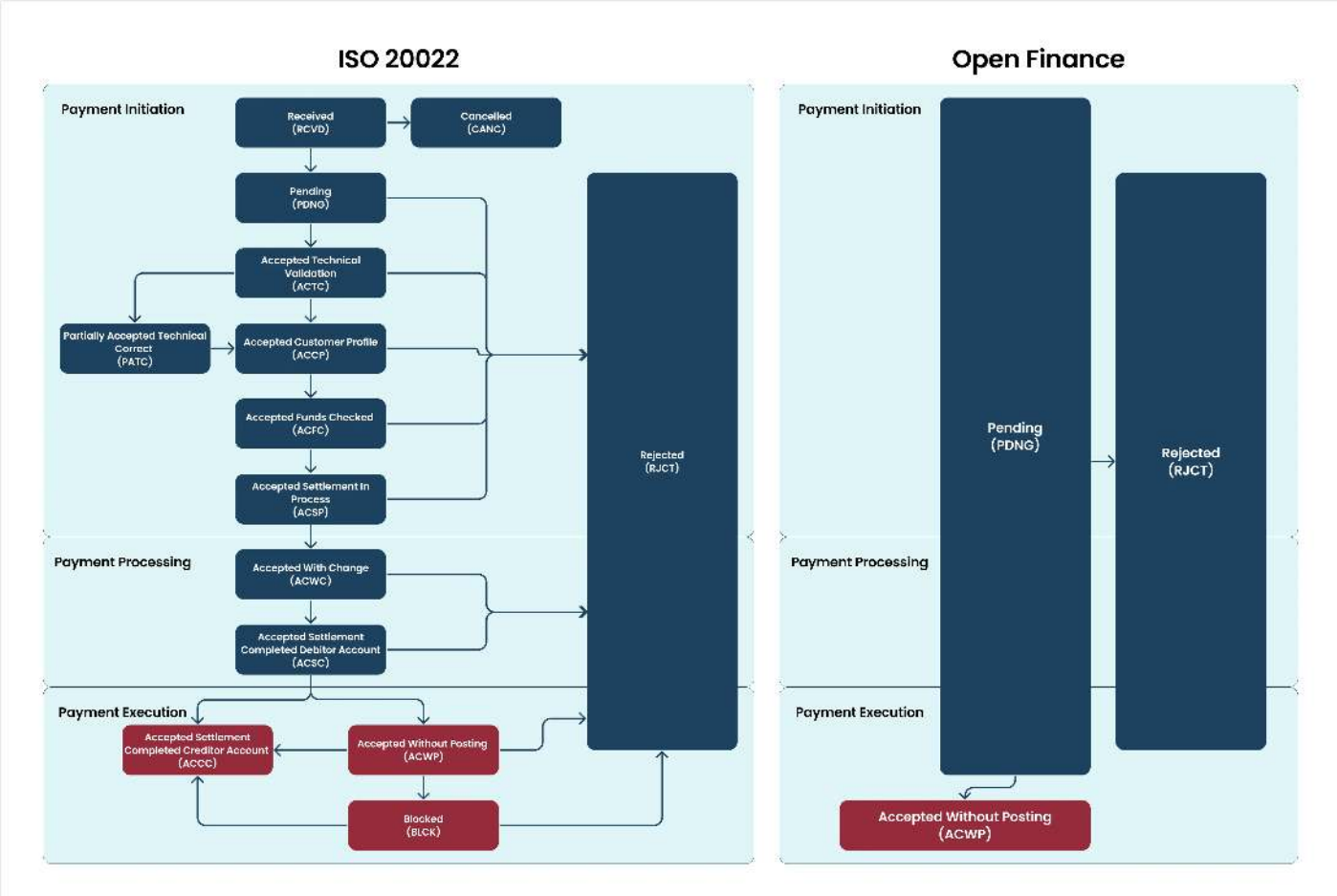


Figure 11: Payment Execution completed

A TPP can call the GET status endpoint at this stage to find out whether the payment is still Pending, has completed successfully (AcceptedWithoutPosting), or has failed (Rejected). A TPP that has subscribed to event notifications **MUST** receive a notification from the API Hub of each change of status of the Payment resource, using the registered Webhook URL.

As settlement breaks may occur after the payment rail has reported its final accepted status, TPPs supporting merchants under an acquirer model **SHOULD** verify the actual settlement of funds by checking the merchant's account, and **MUST NOT** rely solely on the AcceptedWithoutPosting status (COMM-158, COMM-159).

> **Note**
>
> For Fawran Core, a payment that has been completed can later be returned as a separate Payment Return, initiated by the Creditor LFI or the receiving customer (Fawran Implementation Guide v5.4, section 2.3 Payment Return). A Payment Return is a separate payment and does not change the status of the original payment. For Tahweel, a payment settled in a previous clearing session can be returned by the Creditor LFI with a Payment Return (pacs.004). The original payment is flagged as returned and its status does not change. The Debtor LFI responds to the return within the clearing session; if it does not, Tahweel accepts the return automatically when the session ends (QCB QATCH Return Specifications v1.2, sections 3, 3.1 and 3.2).

Payment Execution Status for Fawran Core

Fawran Core sends the Debtor LFI status messages for a payment submitted to it as a pacs.008 Credit Transfer. The messages, and the point at which each is sent, are described in Fawran Implementation Guide v5.4, sections 2.2.2 to 2.2.7 and Appendix E: Status Codes, and the reason codes in Appendix F: Status Reason Codes. The Debtor LFI maps them as follows (COMM-160, COMM-274 to COMM-276):

Fawran Core response to the Debtor LFI	Open Finance Payment Status
Status RCVD, reason code U004 Received (Appendix E and Appendix F)	Pending
Status ACTC, reason code U002 Success/Transaction Accepted Stored in SAF (Appendix F). The guide does not describe when this is sent	**<TBC with QCB>**
Rejection following technical or business validation at Fawran Core (sections 2.2.4 and 2.2.5)	Rejected
Rejection by the Creditor LFI (section 2.2.3)	Rejected
Rejection because Fawran Core received no response from the Creditor LFI within its time out period, including where the Creditor LFI's acceptance arrives late (section 2.2.6)	Rejected

Fawran Core response to the Debtor LFI	Open Finance Payment Status
Acceptance (status ACTC, reason code U000), sent once the Creditor LFI has accepted the payment (section 2.2.2)	AcceptedWithoutPosting
Settlement confirmation (status ACSC), sent immediately after the acceptance (section 2.2.2)	AcceptedWithoutPosting (no change)

Where the Debtor LFI does not receive Fawran Core's response, it requests the payment status from Fawran Core (section 2.2.7), and the Payment resource remains Pending until the response is received.

AcceptedWithoutPosting is a terminal state for a payment executed via Fawran Core, as the Debtor LFI receives no confirmation from Fawran Core after the Creditor LFI has applied the credit to the Creditor account. The Debtor LFI returns the Fawran Core status and reason codes to the OFP alongside the Payment resource status (COMM-153, COMM-156).

Payment Execution Status for Tahweel

Tahweel sends the Debtor LFI two kinds of status report for a payment directed to it: its own validation status report, and the Creditor LFI's Payment Status Report, which Tahweel passes on to the Debtor LFI. The validation status report is described in QATCH Advanced Link Interface Specifications v2.7, section 3.1 Status of Validation, and the Payment Status Report in section 2.3 Payment Status Report. The Debtor LFI tells them apart by the Instructing Agent of the message (section 3 Acknowledgement Messages). Tahweel also sends its settlement reports at each daily incremental settlement (QATCH Incremental Settlement v1.0, section 1.3), and its response to any Request for Cancellation (QATCH DD/DC Cancellation Process v1.1, section 2.1.3). The Debtor LFI maps them as follows (COMM-160, COMM-273, COMM-276):

Tahweel response to the Debtor LFI	Open Finance Payment Status
Validation status report: batch rejected	Rejected
Validation status report: batch accepted, with the settlement window (COMM-273)	Pending
Validation status report: batch partially accepted, and the payment is one of the rejected transactions	Rejected

Tahweel response to the Debtor LFI	Open Finance Payment Status
Validation status report: batch partially accepted, and the payment is not rejected (COMM-273)	Pending
Payment Status Report: payment rejected by the Creditor LFI	Rejected
Payment Status Report: payment accepted by the Creditor LFI	AcceptedWithoutPosting
Settlement reports at Tahweel's daily incremental settlement: payment settled (ACSC) (QATCH Incremental Settlement v1.0, sections 1.2 and 1.3; COMM-275)	AcceptedWithoutPosting (no change)
Response to a Request for Cancellation: cancellation accepted, and Tahweel changes the original payment to rejected (QATCH DD/DC Cancellation Process v1.1, sections 2 and 2.1.3; COMM-269)	Rejected
Response to a Request for Cancellation: cancellation request rejected at validation (QATCH DD/DC Cancellation Process v1.1, section 2.1.3)	No change

For a payment directed to Tahweel:

- The LFI provides the OFP with the settlement window information returned by Tahweel (COMM-268).
- A rejection can be received days after the payment was initiated. Where the payment relates to a Variable On-Demand Consent, the OFP decrements the Consent Control Parameter Values (COMM-150), and the Control Period totals only if the payment was made in the current Control Period (COMM-151).
- The specifications do not define what the Debtor LFI receives if the Creditor LFI does not respond **<TBC with QCB>**.
- AcceptedWithoutPosting is a terminal state. After the Creditor LFI's Payment Status Report, Tahweel settles the payment at its daily incremental settlement and reports it as settled in its settlement reports, which does not change the status (COMM-275). None of the QATCH documents confirms that the Creditor account has been credited.
- Tahweel allows the sending bank to cancel a payment until it is settled (QATCH DD/DC Cancellation Process v1.1, section 1). COMM-269 limits the LFI to requesting a cancellation

before the Creditor LFI accepts the payment, so that AcceptedWithoutPosting remains final.

- Whether a payment accepted by the Creditor LFI can still fail at settlement is **<TBC with QCB>**.

Summary

The TPP **MUST** be able to check all the statuses that the payment has gone through, from payment initiation to payment execution, using the `GET` payment details endpoint or the event notifications it has subscribed to. TPPs **MUST** also be able to see the historical reasons why the payment was in a specific state before it reached a terminal state. The only terminal states are:

Terminal State	Meaning
AcceptedWithoutPosting (ACWP)	The Creditor LFI has accepted the payment, as reported by the payment rail. The credit to the Creditor account is not confirmed.
Rejected (RJCT)	The Debtor LFI, the payment rail or the Creditor LFI has rejected the payment, or the payment has been cancelled.

General

ISO 20022 Payment Transaction Status Model

The ISO 20022 Payment Transaction Status codes (`ExternalPaymentTransactionStatus1Code`) are listed below, for reference against the mapping in Mapping to the ISO 20022 Payment Transaction Status Model.

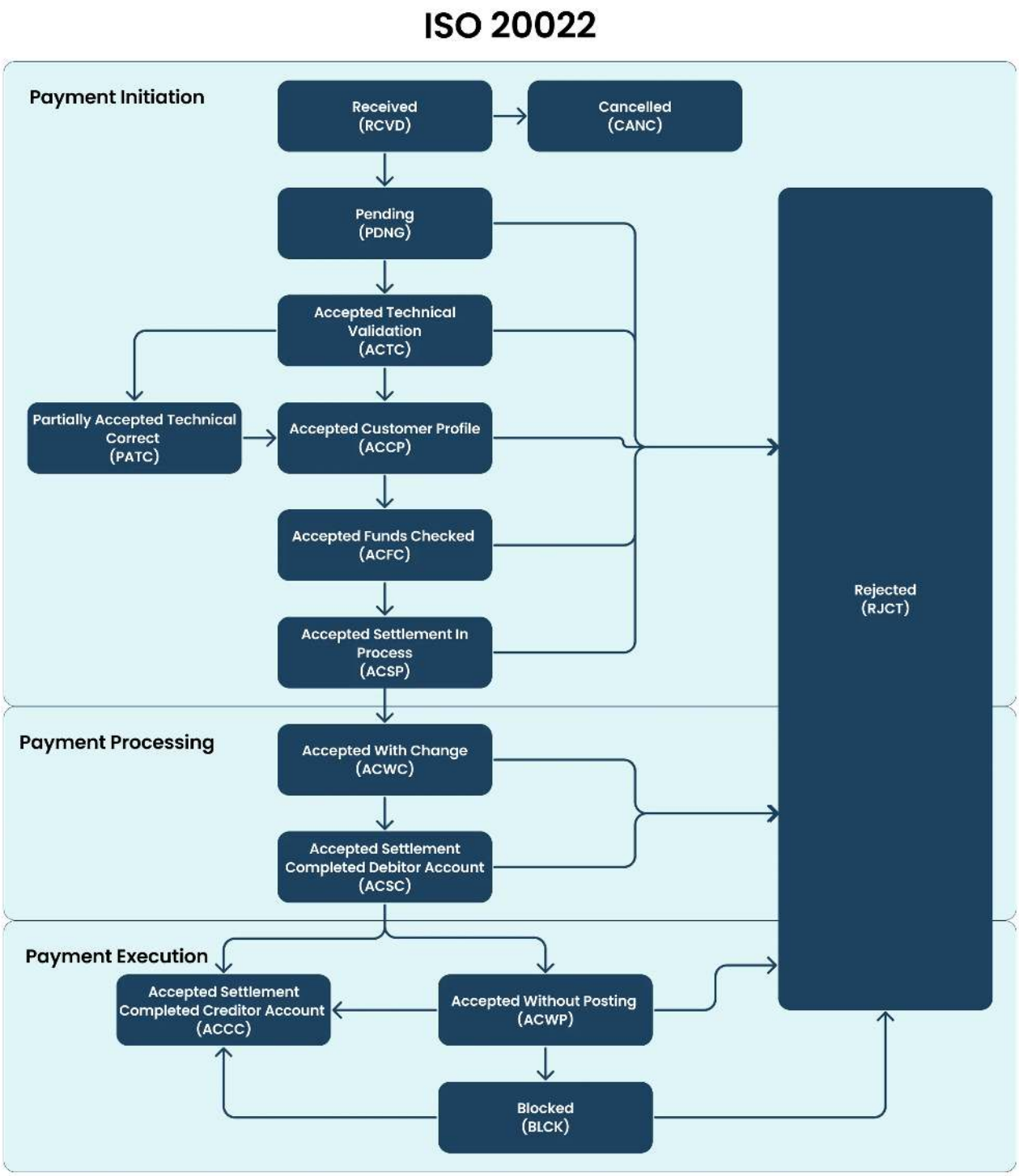


Figure 12: ISO 20022 Transaction Status Diagram

> **Note**
>
> Whether technical validation (ACTC) precedes the customer profile check (ACCP), or the reverse, is up to each LFI. Similarly, other checks such as funds checking (ACFC) may occur in a different order, and even during payment processing rather than payment initiation.

Code	Name	Definition
ACCC	AcceptedSettlementCompletedCreditorAccount	Settlement on the Creditor account has been completed.
ACCP	AcceptedCustomerProfile	The preceding technical validation was successful, and the customer profile check was also successful.
ACFC	AcceptedFundsChecked	The preceding technical validation and customer profile checks were successful, and an automatic funds check was positive.
ACSC	AcceptedSettlementCompletedDebitorAccount	Settlement on the Debtor account has been completed.
ACSP	AcceptedSettlementInProcess	All preceding checks, such as technical validation and customer profile, were successful, and the payment instruction has been accepted for execution.
ACTC	AcceptedTechnicalValidation	Authentication and syntactical and semantic validation are successful.
ACWC	AcceptedWithChange	The instruction is accepted, but a change will be made, such as to the date, or remittance information not sent.
ACWP	AcceptedWithoutPosting	The payment instruction included in the credit transfer is accepted without being posted to the Creditor customer’s account.

Code	Name	Definition
BLCK	Blocked	A payment previously reported as ACWP is blocked: the funds will neither be posted to the Creditor account nor returned to the Debtor.
CANC	Cancelled	The payment initiation has been cancelled after a request for cancellation was received.
PATC	PartiallyAcceptedTechnicalCorrect	The payment initiation needs multiple authentications, some but not all of which have been performed. Syntactical and semantic validations are successful.
PDNG	Pending	The payment instruction is pending. Further checks and status updates will be performed.
RCVD	Received	The payment instruction has been received.
RJCT	Rejected	The payment instruction has been rejected.

Confirmation to User

TPPs MUST:

COMM-162 Confirm to Users the outcome of their Consent Authorization with the LFI, in case of Long-lived Consents.

COMM-163 Inform Users if further authorizations are required for the Consent and provide additional information about the Authorizers supplied by the OFP as described in Common Rules and Guidelines.

COMM-164 Confirm to Users the outcome of the Service Initiation based on the payment status received by the OFP.

COMM-165 Confirm the payment outcome to Users using confirmation screens in line with the customer journey, for every Service Initiation that Users are present.

Payment Notifications

TPPs MUST:

COMM-166 Send a notification on the outcome of each Service Initiation to the beneficiary, if the beneficiary is onboarded by the TPP.

- COMM-167 17.1.1 Notify the Debtor about any upcoming payments one day before the payment date. The notification must include details of the payment amount, date, and beneficiary and should be delivered through the same channels used for payment-related notifications (e.g., SMS, email, app notifications).

LFIs MUST:

COMM-168 Send a notification to Users on the success or failure of a Service Initiation in the following scenarios:

- A Single Use Consent payment is initiated by Users and is either successfully received by the Creditor or there is an error in processing the payment. This may include Single Instant Payments or Future Dated Payments.
- A Long-lived Consent is set up by Users, including information of the TPP or beneficiary party on behalf of the TPP which acquired the User Consent
- A Service Initiation request as part of a Long-lived Consent submitted by the TPP is either successfully received by the Creditor or there is an error in processing the payment

COMM-270 For a payment directed to Tahweel, send the notification of its success or failure when Tahweel reports that the payment has been accepted or rejected by the Creditor LFI (see QATCH Advanced Link Interface Specifications v2.7, section 2.3 Payment Status Report), or rejected at validation (section 3.1 Status of Validation), not when the payment is submitted or accepted at validation.

COMM-169 Use SMS as a minimum channel to notify Users and optionally any other allowable channels (e.g. email, app notifications) currently used for account access or payment-related notifications to their customers depending on QCB rules and regulations, for all Open Finance related notifications to Users. This requirement is separate from the rules for User authentication: an SMS notification carries no authentication value, and nothing in this requirement permits the use of SMS one-time passcodes as an authentication factor.

COMM-170 Identify in the notifications the TPP initiating the payment, the beneficiary of the payment and additionally include the information provided in the on-behalf field.

Multi-User Authorization Flow Notifications

In scenarios of multi-user accounts such as shared accounts where multiple users have delegated authority to approve the payments.

LFIs MUST:

COMM-171 Inform all Users about the submission of a payment Consent for authorization.

COMM-172 Notify the other authorizers about any Single Use or Long-lived Consent that is pending their authorization, using all of the existing channels (e.g. SMS, email, app notifications) currently used for payment-related authorizations.

TPPs MUST:

COMM-173 Notify the Users of the updated status of the multi-user authorization received from the OFP, displaying the authorizers who have accepted, rejected or still have to authorize the payment.

Customer Notifications - Beneficiary and TPP Identification

LFIs MUST:

COMM-174 Create an alignment between the notification contents and the relevant consent object used to authorize an Open Finance event / transaction, this is so the customer can recognize the action they approved. In particular:

- COMM-175 All customer facing payment notifications (including push notifications, SMS, email, and in-app alerts) MUST clearly identify the beneficiary of the transaction. [COMM-177] The beneficiary's bank should NOT be the main feature of the contents of the any notification.

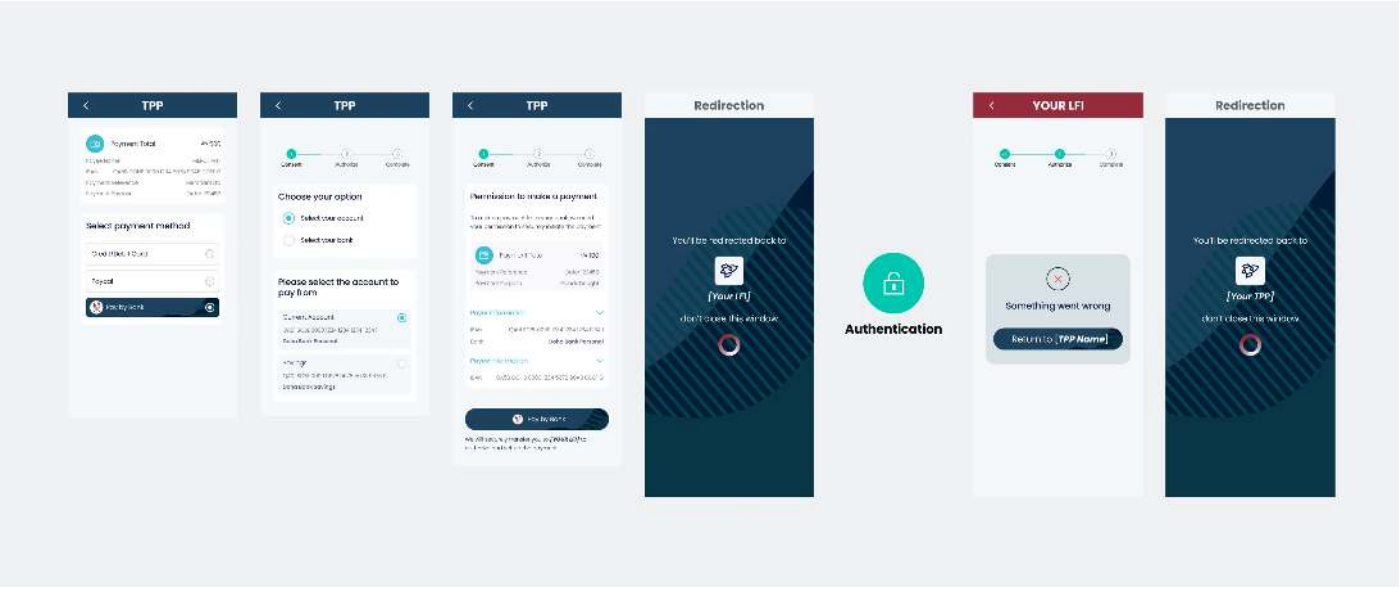
- COMM-176 Where the beneficiary cannot be explicitly identified (due to e.g. pooled accounts funding e-wallet top-ups), the notification MUST include the TPP's name so that the customer can reasonably associate the notification with the consented action.

Exception Handling

Exception Handling User Journey

LFIs MUST:

1. COMM-178 Determine the situations where the customer journey cannot be completed due to an exception and an exception error is required to be shown to Users, keeping consideration of the User Experience Principles that LFIs MUST maintain on the Open Finance journeys.
2. COMM-179 Prevent the User from continuing the journey they were intending to complete
3. COMM-180 Provide to Users a meaningful description of the cause of the exception from the intended journey
4. COMM-181 Allow Users the capability to redirect back to the TPP if the LFI has the capability to
5. COMM-182 Update the consent object with an appropriate error code



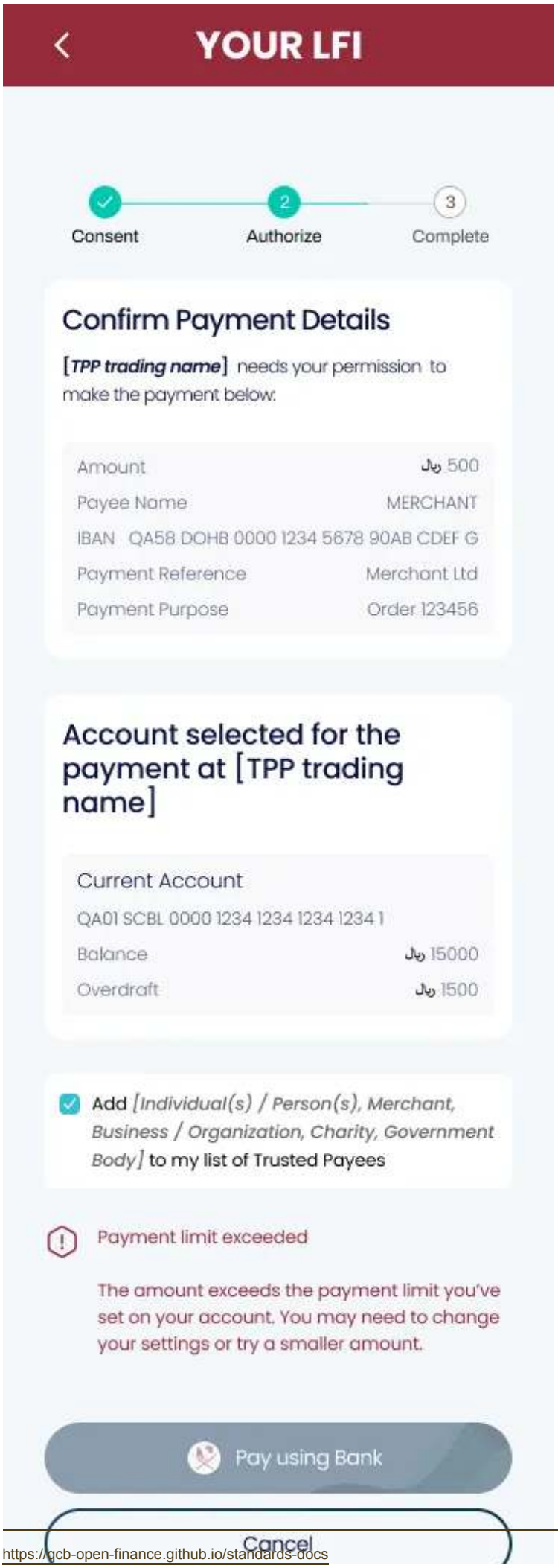
Summary authorization screen with greyed out button

LFIs MUST:

1. COMM-183 When it is identified that the consent authorization cannot proceed due to an issue with the selected account or because the payment request exceeds LFI or account policy, the **'Pay using QCB'** button MUST be greyed out and disabled to prevent the User from proceeding with the consent authorization.

2. COMM-184 The reason why the authorization cannot proceed MUST be clearly displayed to the User.

3. COMM-185 To end the consent authorization, the User MUST select **'Cancel'** to be re-directed back to the TPP.


YOUR LFI
Consent
2
Authorize
3
Complete
Confirm Payment Details
[TPP trading name] needs your permission to make the payment below:
Amount ﷼ 500
Payee Name MERCHANT
IBAN QA58 DOHB 0000 1234 5678 90AB CDEF G
Payment Reference Merchant Ltd
Payment Purpose Order 123456
Account selected for the payment at [TPP trading name]
Current Account
QA01 SCBL 0000 1234 1234 1234 1234 1
Balance ﷼ 15000
Overdraft ﷼ 1500
Add [Individual(s) / Person(s), Merchant, Business / Organization, Charity, Government Body] to my list of Trusted Payees
Payment limit exceeded
The amount exceeds the payment limit you've set on your account. You may need to change your settings or try a smaller amount.
Pay using Bank
Cancel

Known Scenarios

COMM-186 The LFI must provide a meaningful description of the exception. Suggested language is included below.

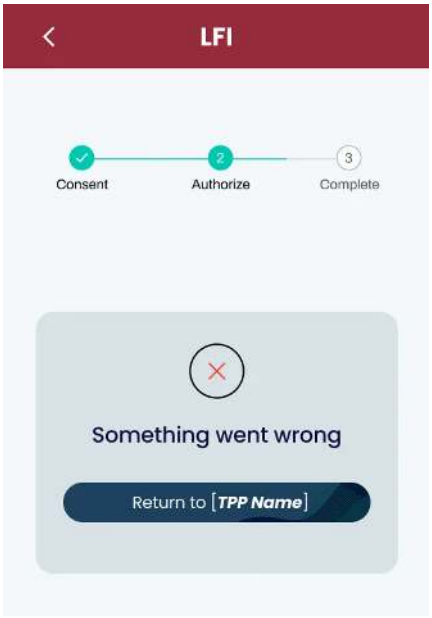
For more known scenarios and suggested exception language please refer to LFI Exception & Warning Journey Scenarios - Knowledge Base - Confluence

#	Scenario	CX Screen	Headline (UI)	Example Exception Language (Body Text)
1	No eligible accounts/products/policies	Out of scope exceptions	*We couldn't find any eligible data to share*	We couldn't locate any accounts, products, or policies under your profile that are currently eligible to be shared through Open Finance. Please check directly with **[LFI]** if you think this is an error. *We'll redirect you back to **[TPP Name]***
2	Account not eligible for payment	Out of scope exceptions	*Account not eligible for payments*	The selected account cannot be used to make payments through Open Finance. Please check

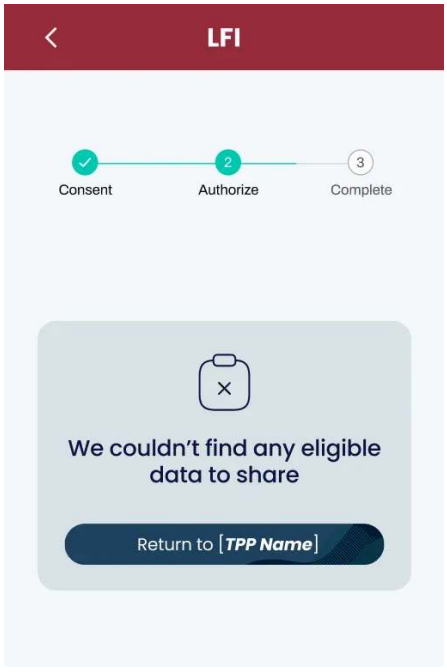
#	Scenario	CX Screen	Headline (UI)	Example Exception Language (Body Text)
				directly with **[LFI]** if you think this is an error. *We'll redirect you back to **[TPP Name]***
3	System error or technical failure	Technical errors	*Something went wrong*	We encountered an unexpected issue and couldn't complete your request. Please try again later. *We'll redirect you back to **[TPP Name]***
4	Authentication error	No redirect back to TPP	*We weren't able to verify your identity*	Please check directly with **[LFI]** if you think this is an error.
5	Minor accounts not in scope	Out of scope exceptions	*You are not allowed to access these services*	You do not meet the minimum age to access these services. Please check directly with **[LFI]** if you think this is an error. *We'll redirect you*

#	Scenario	CX Screen	Headline (UI)	Example Exception Language (Body Text)
				*back to **[TPP Name]***
6	LFI-side defined payment limit exceeded (user-defined)	Summary authorization screen with greyed out button	*Payment limit exceeded*	The amount exceeds the payment limit you've set on your account with **[LFI]**. You may need to change your settings with **[LFI]** or try a smaller amount. *We'll redirect you back to **[TPP Name]***

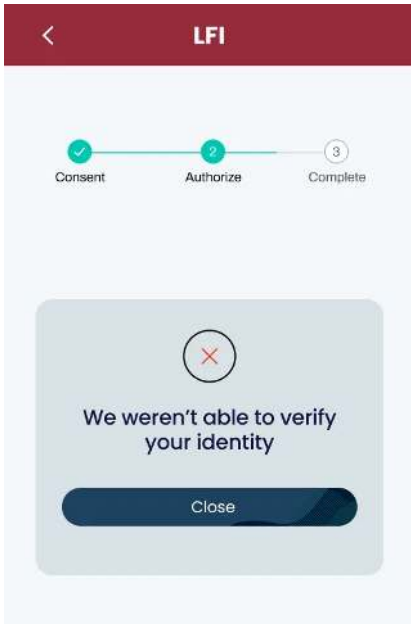
Technical errors



Out of scope exceptions



No redirect back to TPP

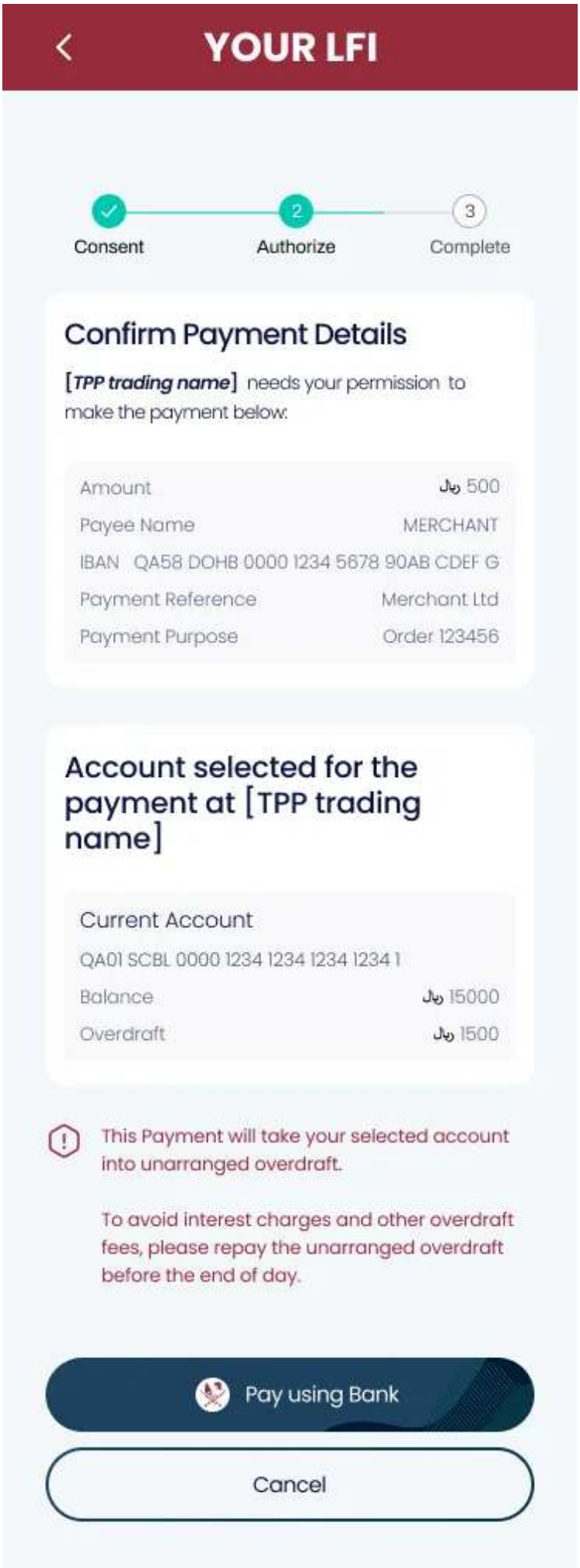


Supplementary Information

Summary authorization screen with warning on bottom

LFIs MUST:

1. COMM-187 Determine the situations where a warning or supplementary information is required to be shown to Users, keeping consideration of the User Experience Principles that LFIs MUST maintain on the Open Finance journeys.
2. COMM-188 Not prevent the User from completing the authorization
3. COMM-189 Prompt the User with a meaningful description of the warning or supplemental information including the cause and potential impact if they continue the intended journey on the bottom of the LFI authorization screen they were intending to complete
4. COMM-190 Allow Users the capability to revise any allowed inputs or redirect back to the TPP
5. COMM-191 Allow the User to complete authorization of the intended journey
6. COMM-192 Update the consent object with an appropriate error code if the customer elects to cancel the journey


YOUR LFI
Consent
2
Authorize
3
Complete
Confirm Payment Details
[TPP trading name] needs your permission to make the payment below:
Amount
﷼ 500
Payee Name
MERCHANT
IBAN QA58 DOHB 0000 1234 5678 90AB CDEF G
Payment Reference
Merchant Ltd
Payment Purpose
Order 123456
Account selected for the payment at [TPP trading name]
Current Account
QA01 SCBL 0000 1234 1234 1234 1234 1
Balance
﷼ 15000
Overdraft
﷼ 1500
This Payment will take your selected account into unarranged overdraft.
To avoid interest charges and other overdraft fees, please repay the unarranged overdraft before the end of day.
Pay using Bank
Cancel

Known Scenarios

COMM-193 The LFI must provide a meaningful description of the warning. Suggested language is included below.

For more known scenarios and suggested exception language please refer to LFI Exception & Warning Journey Scenarios - Knowledge Base - Confluence

#	Scenario	CX Screen	Headline (UI)	Example Rationale Language (Body Text)
1	Insufficient balance	Summary authorization with warning on bottom	Not enough funds	Your account doesn't have enough balance to complete this payment. Please check your balance and try again.
2	Account will enter overdraft	Summary authorization with warning on bottom	This payment will use your overdraft	Your account doesn't have enough balance to cover this payment, but you do have an overdraft facility. Completing this payment will place your account into overdraft. Please confirm if you'd like to proceed — or select a different account.
3	Duplicate payment attempt	Summary authorization with warning on bottom	This looks like a duplicate	The payment you're trying to make appears to be the same as one already submitted. Please check if it has been processed.

LFI Exception & Warning Journey Scenarios

Introduction

This Knowledge Base article is supplemental to the standards rules and guidelines regarding Exception and Warning handling — Common Rules and Guidelines | Exception Handling and Common Rules and Guidelines | Supplementary Information — and provides a complete list of all known scenarios in which the User's Open Finance journey with the Licensed Financial Institution (LFI) cannot proceed due to unmet criteria or unsupported conditions, the intended journey needs to be stopped, or a warning needs to be provided. This Knowledge Base article will be continuously updated as new scenarios are identified, thus providing LFIs with a full list of scenarios and suggested CX language.

COMM-194 Users must be clearly informed about why their journey was interrupted and provided with options to:

1. Return to the Third-Party Provider (TPP) if the journey cannot be completed.
2. Override warnings or revise inputs to continue the journey, where applicable.

Exception Handling Scenarios and Guidelines

Known Scenarios

These scenarios prevent the journey from continuing, requiring redirection to the TPP. The following guidance ensures clear communication to Users about the issue and next steps.

#	Scenario	CX screen	Headline (UI)	Example Exception Language (Body Text)
1	No eligible accounts/products/policies	Out of scope exceptions	*We couldn't find any eligible data to share*	We couldn't locate any accounts, products, or policies under your profile that are currently eligible to be shared through Open Finance.

#	Scenario	CX screen	Headline (UI)	Example Exception Language (Body Text)
				Please check directly with **[LFI]** if you think this is an error. *We'll redirect you back to **[TPP Name]***
2	Account not eligible for payment	Out of scope exceptions	*Account not eligible for payments*	The selected account cannot be used to make payments through Open Finance. Please check directly with **[LFI]** if you think this is an error. *We'll redirect you back to **[TPP Name]***
3	System error or technical failure	Technical errors	*Something went wrong*	We encountered an unexpected issue and couldn't complete your request. Please try again later. *We'll redirect you back to **[TPP Name]***

#	Scenario	CX screen	Headline (UI)	Example Exception Language (Body Text)
4	Authentication error	No redirect back to TPP	*We weren't able to verify your identity*	Please check directly with **[LFI]** if you think this is an error.
5	Minor accounts not in scope	Out of scope exceptions	*You are not allowed to access these services*	You do not meet the minimum age to access these services. Please check directly with **[LFI]** if you think this is an error. *We'll redirect you back to **[TPP Name]***
6	LFI-side defined payment limit exceeded (user-defined)	Summary authorization screen with greyed out button	*Payment limit exceeded*	The amount exceeds the payment limit you've set on your account with **[LFI]**. You may need to change your settings with **[LFI]** or try a smaller amount. *We'll redirect you back to **[TPP Name]***

#	Scenario	CX screen	Headline (UI)	Example Exception Language (Body Text)
7	Multi-factor authentication required	Additional authentication needed	*Additional authentication needed*	Your request requires additional authentication. Please complete the multi-factor authentication process. You will be redirected to **[TPP Name]** if not completed.
8	Account frozen or restricted	Account is restricted	*Account restricted*	Your account is currently frozen or restricted due to regulatory checks. Please wait for this to be resolved or select a different account to continue. You will be redirected to **[TPP Name]** if not completed.

Warning / Supplemental Information Scenarios

Known Scenarios

These scenarios possibly allow Users to revise inputs or confirm actions to continue their journey. The following guidance ensures Users understand the issue and their options.

#	Scenario	CX Screen	Headline (UI)	Example Rationale Language (Body Text)
1	Insufficient balance	Summary authorization with warning on bottom	*Not enough funds*	Your account doesn't have enough balance to complete this payment. Please check your balance and try again.
2	Account will enter overdraft	Summary authorization with warning on bottom	*This payment will use your overdraft*	Your account doesn't have enough balance to cover this payment, but you do have an overdraft facility. Completing this payment will place your account into overdraft. Please confirm if you'd like to proceed — or select a different account.
3	Duplicate payment attempt	Summary authorization with warning on bottom	*This looks like a duplicate*	The payment you're trying to make appears to be the same as one already submitted. Please check if it has been processed.
4	Payment limit exceeded (user-defined)	Payment limit exceeded	*Payment limit exceeded*	The requested amount exceeds your account's payment limit. Please adjust your settings or select a smaller amount to continue.
5	Account not eligible for payment	Account not eligible	*Account not eligible*	The selected account cannot be used for payments through Open Finance. Please choose a different account to proceed.
6	International transfer charges warning	International Transfer	*Correspondent bank charges may apply*	Additional fees may be applied by correspondent or beneficiary banks and financial institutions involved in

#	Scenario	CX Screen	Headline (UI)	Example Rationale Language (Body Text)
				processing your transfer. Please review the destination and confirm that you would like to proceed.

Supported Accounts & Payment Rails

LFIs MUST:

COMM-195 Provide Open Finance APIs for as many of the following account types, held by Retail and SME Users, as they allow their Users to access in their existing Digital channels:

- **Bank Data Sharing:** Current accounts, Savings accounts (including deposits), Credit cards, Loans, Mortgages, Wallets and Currency accounts
- **Bank Service Initiation:** Current accounts

COMM-196 Provide Open Finance APIs for the aforementioned account types in all currencies supported. The scope of currencies is as follows:

- COMM-197 For Bank Data Sharing, the account types in COMM-195 MUST be supported in ALL available currencies, including Currency accounts and any Credit card, Loan or Mortgage account held in a currency other than QAR.
- For Bank Service Initiation, the following scope rules apply:
  - COMM-198 For domestic payments (i.e. payment accounts within the Qatar Jurisdiction) **both** Debtor (initiating) payment account and Creditor (beneficiary) payment account MUST be in QAR.
  - Domestic payments (i.e. payment accounts within the Qatar Jurisdiction) involving FX payment conversion at the Debtor (initiating) payment account (for example from USD to QAR or GBP to QAR) before domestic payment initiation are NOT in the scope of the current version of the Open Finance Standard.

COMM-200 Allow Users to authorize consents only for eligible accounts.

- COMM-201 Filter out dormant and inactive accounts from the list of eligible accounts presented to Users during account selection, consent authorization, and payment initiation flows.
- COMM-202 Reject staged consents or payment initiation requests that reference an account with dormant or inactive status, and return a standardized error code and description to the TPP via the Open Finance Platform (OFP).

COMM-203 Provide Open Finance APIs with domestic Payment Initiation capabilities from the aforementioned account types using the Instant Payment Platform (Fawran Core) as the default payment rail, and the ACH batch-clearing payment system (Tahweel) as the fallback payment rail for payments that Fawran Core cannot process. Fawran Core operates 24 hours a day, 7 days a week.

- COMM-204 In the scenario that the Creditor account is with an LFI that has not been onboarded with the Fawran Core platform (see the participant list in Fawran Implementation Guide v5.4, Appendix C: Participant Banks), the LFIs **MUST** perform their BAU check and in cases of payments to Creditor accounts not supported by the Fawran Core platform, direct these payments to Tahweel.

- COMM-205 In the scenario that Fawran Core is unavailable, for example, it is going through maintenance and planned downtime, the LFIs **MUST NOT** reject the payment.

- COMM-206 In these cases, LFIs **MUST** direct these payments to Tahweel.

- COMM-261 In the scenario that the payment amount exceeds the maximum payment value accepted by Fawran Core (see Fawran Implementation Guide v5.4, section 1.4 Payment Validation Service, Maximum Amount Check), the LFIs **MUST NOT** reject the payment for that reason, and **MUST** direct it to Tahweel.

- COMM-207 Where an Open Finance payment should be executed via the Fawran network, the LFI MUST validate that all mandatory Fawran data elements required to successfully submit a payment (see Fawran Implementation Guide v5.4, section 4.9 pacs.008.001.08) are available prior to initiating the payment, including the Debtor name and account details

    - Where the LFI determines that the required information is not available at the point of payment initiation, the LFI MUST:

        - COMM-208 Update the payment status to “Rejected”

        - COMM-209 Return an error response to the API Hub indicating that the payment cannot be processed via the Fawran network due to missing mandatory Debtor name or account details

            - COMM-210 The API Hub MUST propagate the error response to the TPP without modification.

            - COMM-211 The TPP MUST present a clear, non-technical message to the Retail Debtor explaining that their name or account details held by the LFI must be completed before the payment can be processed.

    - Where the required information to successfully submit an Fawran payment is not available, the LFI:

        - COMM-212 MUST NOT initiate the payment via the Fawran network

        - COMM-213 MUST NOT silently reroute the same Open Finance payment instruction via an alternative payment rail. Missing Fawran information is not a reason to direct the payment to Tahweel under COMM-204, COMM-206 or COMM-261.
    - COMM-214 The LFI MUST ensure that the required Fawran information is obtained and recorded through its KYC or equivalent customer information management processes before a subsequent Open Finance payment can be successfully initiated via Fawran Core

- COMM-263 Where an Open Finance payment is directed to Tahweel, the LFI MUST populate all the data elements required by Tahweel, as specified in QATCH Advanced Link Interface Specifications v2.7, section 2.1 Direct Credit and section 3.6 General IBAN Specifications.

- COMM-264 Submit a payment with a future execution date to Fawran Core on its execution date (see Fawran Implementation Guide v5.4, section 1.4 Payment Validation Service, Date Sent Tolerance Check).

- COMM-272 Use the payment rail's unique transaction reference as the payment's Transaction ID, and provide it to the OFP: for Fawran Core, `CdtTrfTxInf/PmtId/EndToEndId`; for Tahweel, `CdtTrfTxInf/PmtId/TxId`; for intra-bank payments, the LFI's own unique transaction reference. The Transaction ID **MUST** be unique for each payment, so that the Debtor, the Creditor, the TPP and QCB can identify the payment by it.

- COMM-216 **NOT** impose any additional constraints to the Open Finance initiated payments on top of the LFI limits (minimum and maximum) stated in the Open Finance Standard. [COMM-217] More specifically, subject to meeting the minimum values stated in Limits and Constants LFIs MUST set the maximum transaction limits to the same value or higher as the transaction limits imposed on the following existing digital channels and use cases:

    - Applicable Channels & Use Cases
        - Debit and Credit Card Transactions: Transactions made via point-of-sale (POS) and online purchases, encompassing payments to merchants onboarded by Third-Party Providers (Pay by account).
        - Proprietary online banking & Mobile banking applications: Transactions executed via the LFIs' web portal or mobile app, including transfers, bill payments, and peer-to-peer (P2P) payments, enabling the following use cases

        - P2P/B2P Payments: Peer-to-peer payments between individuals and business-to-person payments.
        - Payments-to-self: Transfers made by customers to their accounts, whether within the same LFI or to accounts at other financial institutions.
        - Payments to Businesses non-onboarded by TPPs: Payments made to businesses not onboarded by TPPs.

- COMM-218 **NOT** apply a cooling-off period to Open Finance initiated payments, or otherwise hold or defer a payment between the User's authorization and its submission to the payment rail, whether or not the beneficiary is new to the User. This does not cover the settlement time of the payment rail itself: a payment directed to Tahweel under COMM-204, COMM-206 or COMM-261 settles at the next Tahweel settlement window. Nor impose any other timing constraints on Open Finance initiated payments, such as reduced transactional limits for new beneficiaries or constraints related to the number or value of payment transactions.

- COMM-269 Where the User asks to cancel an Open Finance payment directed to Tahweel, **MAY** request its cancellation using the Request for Cancellation (see QATCH Advanced Link Interface Specifications v2.7, section 2.4 Request for Cancellation, and QATCH DD/DC Cancellation Process v1.1, section 2.1.3), provided the Creditor LFI has not yet accepted the payment. Where the cancellation succeeds, the LFI **MUST** update the payment status to Rejected.

COMM-219 This applies to Open Finance initiated payments only. If the customer makes a payment through the LFI's own channels to a beneficiary saved during an Open Finance journey, the LFI may block that payment under its existing cooling-off policies for that account.

- COMM-220 Follow the principle that transaction limits and other constraints applied to the Open Finance services are not more restrictive than those applied across other commonly used banking digital channels. This establishes parity, ensuring that customer experience is consistent regardless of the digital channel used.

TPP Obligations towards Customer Verification

TPPs MUST:

COMM-251 Carry out full customer due diligence, in line with Qatar's anti-money laundering and counter-terrorist financing framework and the applicable QCB rules Refer the Regulations document for details, when Users are onboarded for any customer-facing product that leverages Open Finance and carries its own regulatory requirement for customer due diligence (for example, a wallet or a remittance service).

COMM-252 Carry out at least basic verification, such as verification of the User's email address and mobile number, when Users are onboarded for any other product that leverages Open Finance.

COMM-253 Have the capability, within a customer-facing product that supports multiple users within the same account, to create profiles with different sets of permissions. Certain profiles should be able to use specific OF features like authorizing a new Service/Data Request consent or using a Long-Lived consent setup by some other user with that account.

Authentication and Authorization | Overview

The QCB Standards will support two approaches of carrying out the authentication procedure of the User, namely redirection and decoupled.

Furthermore, the Authenticating Entity (AE), which will authenticate the User using a Multi-Factor Authentication (MFA), can either be the LFI, or the LFI could use a trusted partner to authenticate the User on their behalf.

The authentication journeys are categorized based on the device and the application where the consumption of the User-facing TPP service and authentication takes place.

Authentication and Authorization | Redirection

The User consumes the User-facing TPP service and authenticates for the OF request with the AE on separate applications on the same device. The authentication data is exchanged only between the User and the AE through the AE, and the User-facing TPP has no visibility of this.

Redirection uses the principle of deep-linking, when the User’s action within the User-facing TPP app/website invokes the AE app/website.

Authentication and Authorization | Decoupled

The User consumes the User-facing TPP service and authenticates with the AE on separate applications on separate devices. The authentication data is exchanged only between the User and the AE through the AE application, and the User-facing TPP has no visibility of this. A Decoupled experience on the two devices can be achieved by using Redirection, where the User uses a deep-link within the User-facing TPP app/website on one device to invoke their AE app/website on another device.

> **NOTE**
>
> Using the Redirection implementation, the User-facing TPPs can implement a redirection flow on a single device, or a Decoupled redirection flow using two devices, depending on the customer experience they want to support. A Decoupled Redirection does not require the AE to implement anything in addition to the Redirection flow they will be implementing.

Authentication by LFI | Overview

This section covers redirection and decoupled redirection, supported by the QCB Open Finance Standard (the Standard), to allow a User of a TPP to use the same authentication mechanisms as they do when accessing their LFI directly.

The general principles relating to the authentication models in this section that apply are:

1. COMP-001 **LFIs authenticate:** The User MUST go through Multi-Factor Authentication (MFA) at their LFI for a User-facing TPP request (i.e. Service Initiation or Data Sharing) to be actioned by the LFI.
2. COMP-002 **Availability of normal authentication methods:** The User MUST be able to use the elements they prefer to authenticate with their LFI if supported when interacting directly with their LFI.
3. COMP-003 **Not replicating online journeys:** The LFI's online channels may require doing an MFA multiple times, once for the User to login and then for any additional activity like adding new beneficiaries. However, for an Open Finance journey, the User MUST go through an MFA with the LFI only once before they authorize the consent. There should be no additional authentication required.
4. COMP-004 **No Obstacles:** LFIs MUST NOT create unnecessary delay or friction during authentication including unnecessary or superfluous steps, attributes, or unclear language, e.g. advertising of LFI products or services, language that could discourage the use of User-facing TPP services or additional features that may divert the User from the authentication process.

Authentication by LFI | Redirection

Redirection-based authentication has a range of possible experiences for a User based on whether the User has an LFI app or not and the device on which the User is consuming the User-facing TPP service.

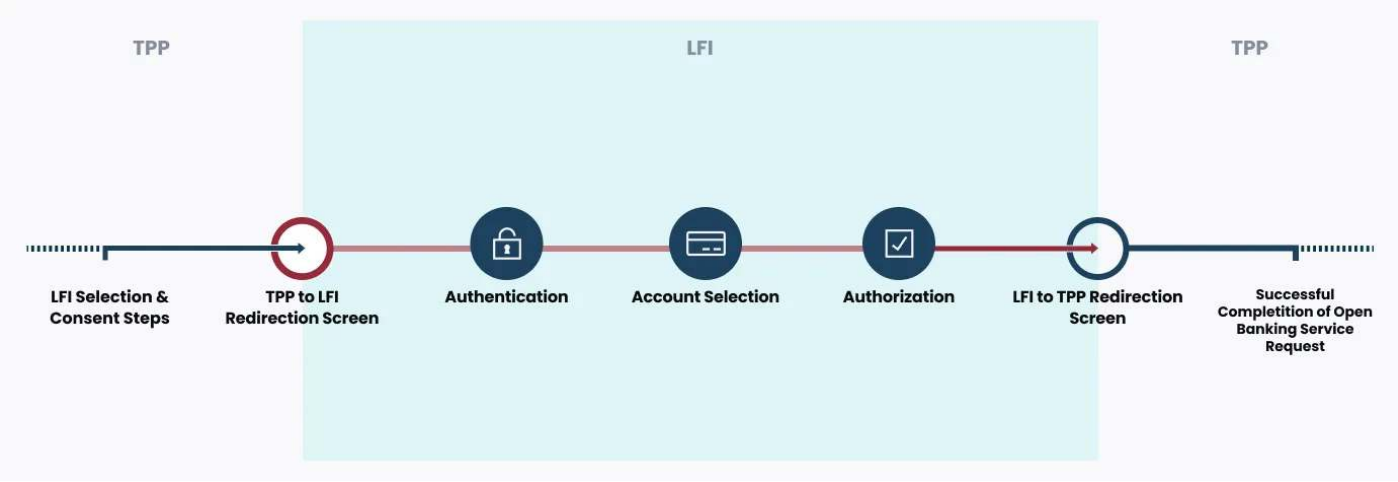
COMP-005 When using a User-facing TPP for Data Sharing Requests (DSR) and Service Initiation Requests (SIR), Users MUST be able to authenticate with the LFI using the same authentication methods they are accustomed to using when accessing the LFI directly.

We have used one example of a User-facing TPP and DSR journey to demonstrate how a redirection flow MUST work. (The same applies to SIR and service requests.)

App-to-App

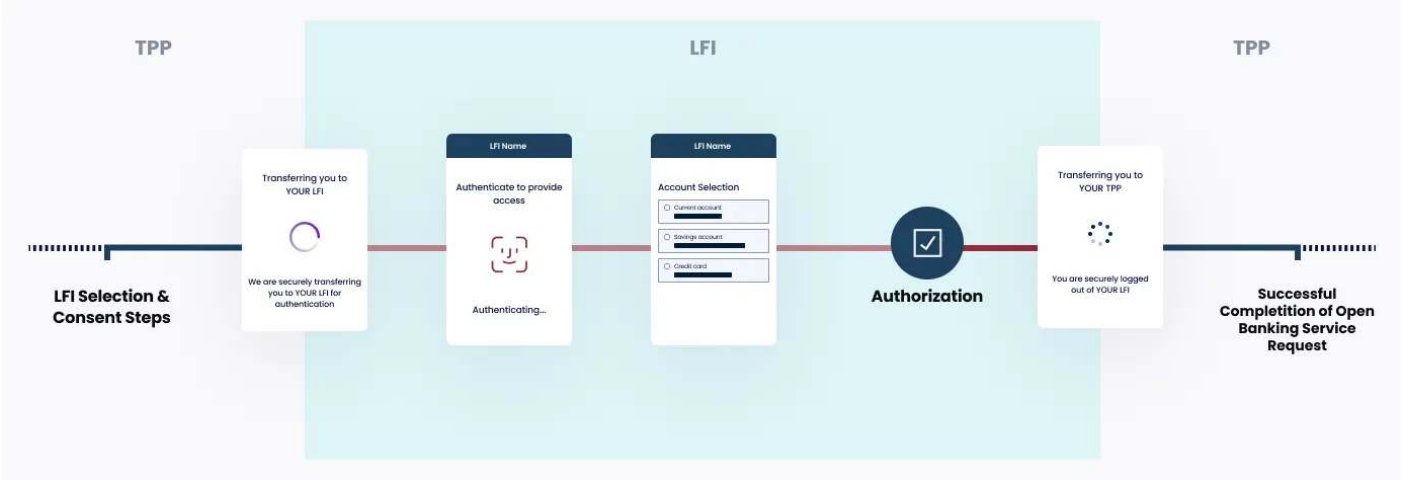
User authentication with the LFI using the LFI mobile app installed on the same device on which the User is consuming the User-facing TPP service. The User starts the journey from a User-facing TPP app.

User Journey



Customer Experience

A prototype of this journey showcasing a Personal Finance Use Case can be found here: Prototypes for QCB docs



This enables the User to authenticate with the LFI while using a User-facing TPP for Open Finance services (i.e. DSR, SIR, & Service Request) using the same LFI app-based authentication method which they use when accessing the LFI mobile channel directly.

User-facing TPP service COULD be web-based or app-based. [COMP-006] The redirection MUST directly invoke the LFI app to enable the User to authenticate and [COMP-007] MUST NOT require the User to provide any User identifier or other credentials to the User-facing TPP.

To demonstrate an app-based redirection part of the journey, we have used the User-facing TPP initial setup as one example.

Rules & Guidelines

1. COMP-008 User-facing TPPs MUST initially ask the User to identify the LFI so that the consent request can be constructed in line with the LFI's data group and/or service initiation capabilities.
2. COMP-009 User-facing TPPs SHOULD make the User aware on the inbound redirection screen (User-facing TPP to LFI) that they will be taken to their LFI for authentication for data sharing.
3. COMP-010 If the User has an LFI app installed on the same device, the redirection MUST invoke the LFI's app for authentication purposes only without introducing any additional screens. [COMP-011] The LFI's app-based authentication MUST have no more than the number of steps that the User would experience when directly accessing the LFI app (biometric, passcode, credentials) and offer the same authentication method(s) available to the User when authenticating in their LFI's direct channels.
4. COMP-012 After authentication, the User MUST be deep linked within the app to confirm the account(s) to which they would like the User-facing TPP to have access.
5. COMP-013 The LFI app is invoked only if the User has not chosen to block being redirected to the app. If the User has blocked redirection to the app, then the User will be redirected to

a browser-based authentication and authorization journey of the LFI.

6. COMP-014 LFIs SHOULD have an outbound redirection screen which indicates the status of the request and informs the User that they will be automatically taken back to the User-facing TPP.
7. COMP-015 LFIs SHOULD inform the User on the outbound redirection screen that their session with the LFI was closed.
8. COMP-016 User-facing TPPs SHOULD confirm the successful completion of the Open Finance Service Request (DSR, SIR).

Browser-to-App

Conversely, User-facing TPP may be browser-based only. However, if the User:

- is using a browser on a mobile device and
- has the LFI app installed on their device

then [COMP-017] the User-facing TPP website MUST redirect the User's browser to the LFI. [COMP-018] The LFI, on receiving the redirection request, MUST invoke the LFI app for authentication on the User's device. [COMP-019] Following authentication, the User MUST be redirected back to the User-facing TPP browser on the mobile device.

Browser-to-Browser

COMP-020 If a User is using a desktop browser to access the User-facing TPP, then under the redirection model the journey MUST be completed on the LFI browser channel.

Browser-to-browser redirection is also applicable when:

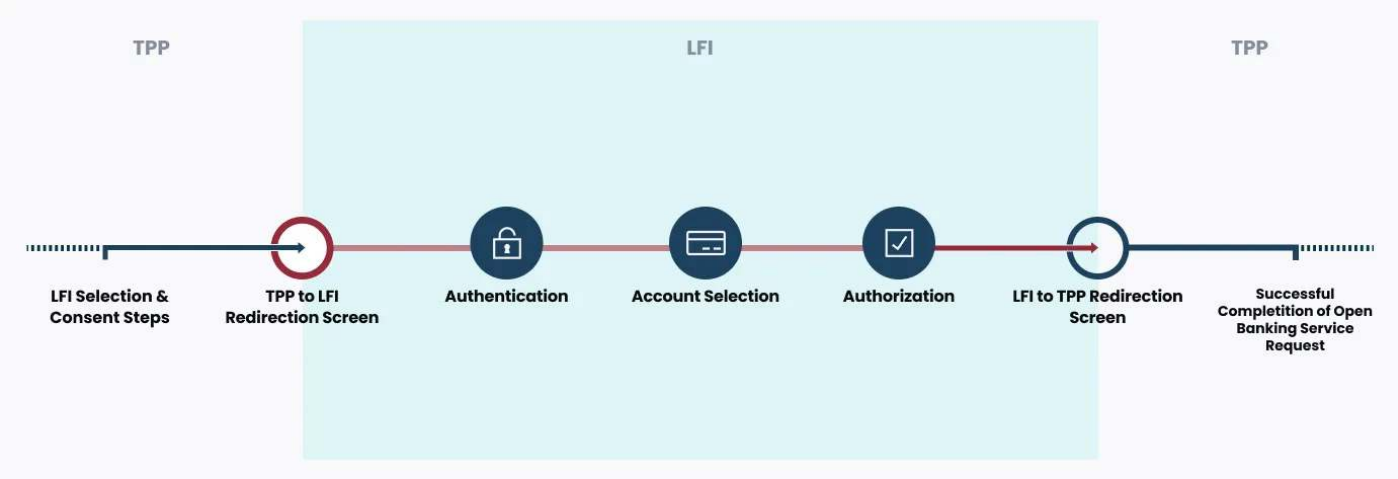
- the User-facing TPP is browser-based only
- the User is using a browser on a mobile device and
- the mobile device does not have the LFI mobile app installed, or the LFI does not provide an app at all.

It is imperative in these circumstances that the LFI browser channel has been optimized for mobile browsers and device types.

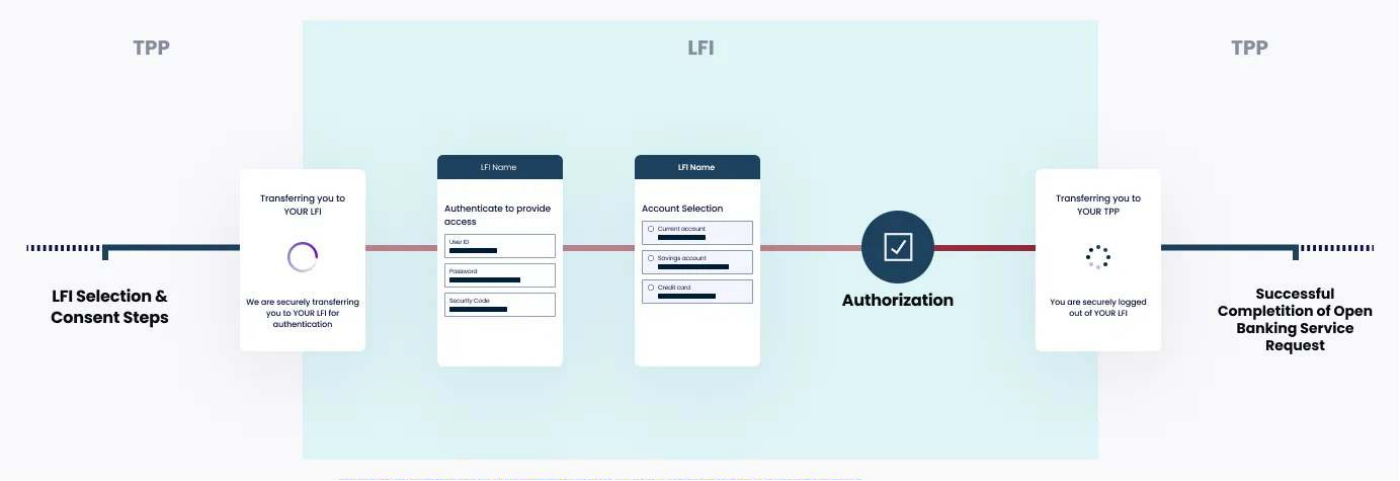
The following alternative experience MUST be implemented by LFIs to allow customers to use their mobile banking app to complete Authentication and Authorization:

1. COMP-021 The LFI MUST support a web-based landing page that opens on redirection with a Call to Action (CTA) to trigger an interaction using the User's mobile banking app.
2. The CTA provided on the page must be:
    - COMP-022 For non-mobile devices, a QR Code that can be scanned by the User. Direction must be displayed that indicates to the User that they must scan the QR Code with a device that has the LFI app installed.
    - COMP-023 For mobile devices without the LFI app installed, a CTA that enables the User to download the app from the relevant app store.
3. COMP-024 The QR Code displayed MUST be scannable directly by any mobile device camera and resolve into a deep link which will invoke the LFI mobile app on that device. The deep link will result in the User being prompted to complete Multi-Factor Authentication and be presented with a screen that allows them to complete consent authorization.
4. COMP-025 Where the CTA results in the User installing the LFI mobile app, the LFI must inform the User that they may have to reinitiate the request from the TPP, as the delay introduced in installing and setting up the LFI app is likely to expire the authorization window set by the TPP.
5. COMP-026 The LFI MUST provide the means for the User to abandon handoff to a mobile device and instead choose to complete Authentication and Authorization using the LFI web channel, where supported.

User Journey



Wireframes



Rules & Guidelines

1. COMP-027 User-facing TPPs MUST initially ask the User to identify the LFI so that the consent request can be constructed in line with the LFI's data group and/or service initiation capabilities.
2. COMP-028 User-facing TPPs SHOULD make the User aware on the inbound redirection screen (User-facing TPP to LFI) that they will be taken to their LFI for authentication for data sharing.
3. COMP-029 The redirection MUST take the User to the LFI web page (desktop/mobile) for authentication purposes only without introducing any additional screens. [COMP-030] The web-based authentication MUST have no more than the number of steps that the User would experience when directly accessing the web-based LFI channel (desktop/mobile).
4. COMP-031 After authentication, the User MUST be deep linked within the app to confirm the account(s) to which they would like the User-facing TPP to have access to.
5. COMP-032 LFIs SHOULD have an outbound redirection screen which indicates the status of the request and informs the User that they will be automatically taken back to the User-facing TPP.
6. COMP-033 LFIs SHOULD inform the User on the outbound redirection screen that their session with the LFI was closed.
7. COMP-034 User-facing TPPs SHOULD confirm the successful completion of the Open Finance Service Request (DSR, SIR).

App-to-Browser

It is possible that a User is using the User-facing TPP app on a mobile device which does not have their LFI mobile app installed, or their LFI does not provide an app at all.

COMP-035 In these instances, the User-facing TPP app MUST redirect the User to use the default mobile browser to present the User with their LFI's web channel to authenticate.

COMP-036 The browser for the User authentication MUST NOT be an embedded view within the User-facing TPP app.

Again, in these circumstances, it is imperative that the LFI browser channel is optimized for mobile browsers and device types.

A prototype of this journey showcasing a Personal Finance Use Case can be found here: Prototypes for QCB docs

Authentication by LFI | Decoupled Redirection

In “Decoupled” authentication, typically the User uses a separate secondary device to authenticate with the LFI. This model allows for several innovative solutions and has the added benefit of allowing a User to use their mobile phone to authenticate, taking advantage of biometrics for MFA, where they are engaging with a User-facing TPP through a separate device, such as a desktop/laptop or a point of sale (POS) device.

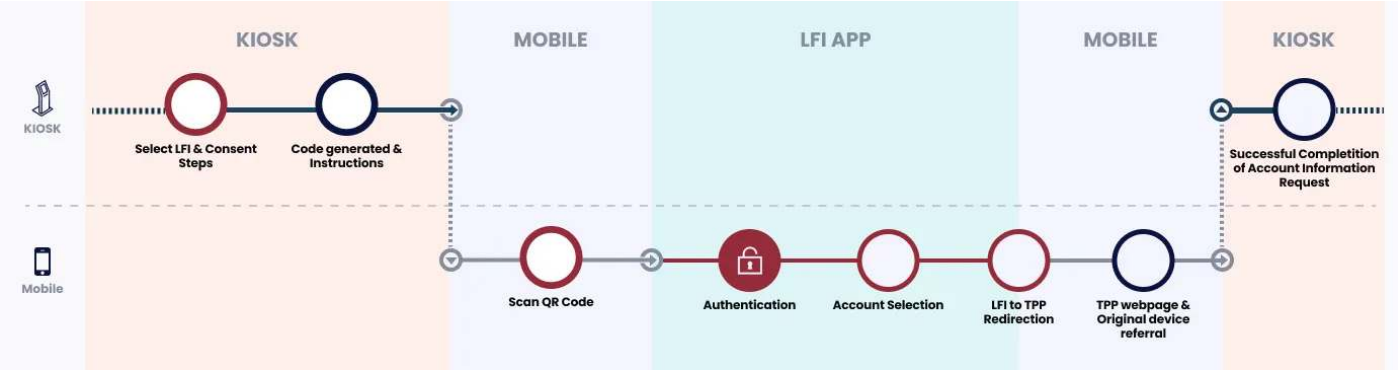
In a Decoupled Redirection flow, the User uses a deeplink within the User-facing TPP app/website on one device to invoke their LFI app/website on another device using the same redirection mechanism as in Authentication by LFI | Redirection.

The following User experiences are available using Decoupled Redirection:

Selection of LFI on the First Device

An authentication flow in which a dynamic link (e.g. via a QR Code or NFC) is generated by the User-facing TPP on one device (e.g. desktop/laptop or kiosk) and is scanned by the User on another device (e.g. the User’s mobile device). This dynamic link will invoke the LFI’s mobile banking app, which uses the information from the dynamic link to then redirect the User to process the request after authenticating the User. This journey does not introduce any changes in the redirection flow between the User-facing TPP and the LFI.

User Journey



Wireframes

To demonstrate a Redirection flow on different devices, we have used one variation of the DSR journey as an example where the LFI receives all the details of the request from the TPP.



We have illustrated an example where the dynamic identifier is a QR code. [COMP-037] The code SHOULD contain a DeepLink (as within the Redirection flow) supported by the LFI, which SHOULD invoke the LFI app when the User scans the QR Code.

Rules & Guidelines

1. COMP-038 User-facing TPPs MUST initially ask the User to identify the LFI so that the consent request can be constructed in line with the LFI's data group and/or service initiation capabilities.
2. COMP-039 User-facing TPPs MUST present Users with the authentication options supported by the LFI, which in turn can be supported by the User-facing TPP device/channel (e.g. a User-facing TPP kiosk that can only support authentication by LFI mobile app).
3. COMP-040 The User-facing TPP MUST display a code which is scannable or readable (QR Code/NFC) by another User device. [COMP-041] The User-facing TPP MUST present the information on how to use the code with their mobile device (e.g. scan QR code with the mobile phone camera).

> **NOTE:**
>
> COMP-042 The QR code MUST be a deep link (as within the Redirection flow) supported by the LFI, which MUST invoke the LFI app on scanning.

4. COMP-043 The User MUST be able to easily scan the code (e.g. scan the code from the Kiosk in this instance) without much friction (like manually entering any URLs).
5. COMP-044 After the User scans the code from the User-facing TPP with a device camera, the LFI app MUST be invoked to perform the MFA.
6. COMP-045 The LFI app-based authentication MUST have no more than the number of steps that the User would experience when directly accessing the LFI mobile app

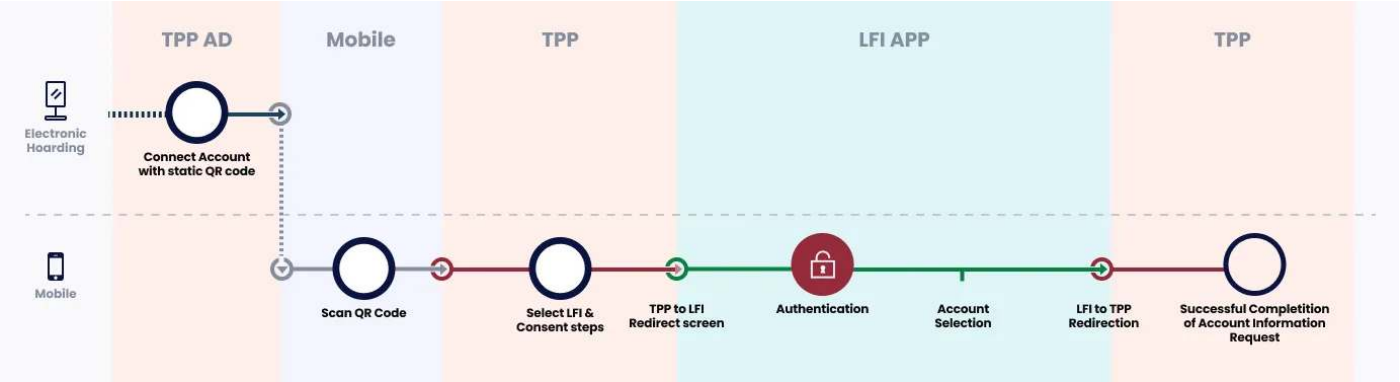
(biometric, passcode, credentials).

7. COMP-046 LFIs SHOULD have an outbound redirection screen which indicates the status of the request and informs the User that they will be automatically taken back to the User-facing TPP.
8. COMP-047 LFIs SHOULD inform the User on the outbound redirection screen that their session with the LFI was closed.
9. COMP-048 For this experience it is essential that the User-facing TPP MUST have a web page where the LFI can redirect control back to the User-facing TPP on the second device. The User-facing TPP must confirm on this mobile web page the successful completion of the request and inform the User to continue on the other device (Kiosk/desktop) where they had started their journey.

Selection of LFI on the Second Device

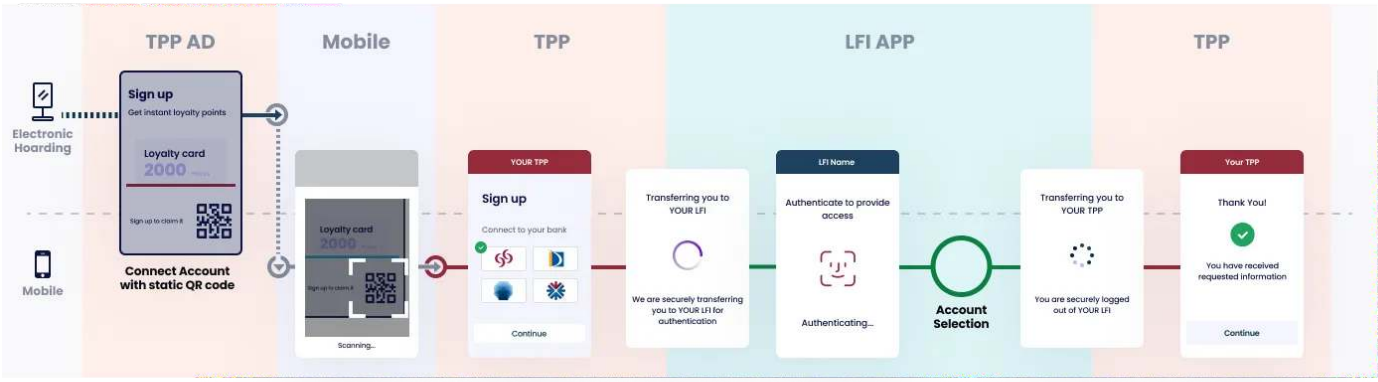
This journey is a variation of the User-facing TPP flow within the standard redirection journey, which could be triggered from a non-interactive interface of the User-facing TPP (e.g. an advertising board) which then progresses on the User's mobile device. The LFI selection and consent request happens on the User's mobile device, which then invokes the LFI app/webpage on the same device to complete the authorization using a redirection flow. This journey does not introduce any changes in the redirection flow between the User-facing TPP and the LFI.

User Journey



Wireframes

To demonstrate a Redirection flow on different devices, we have used one variation of the DSR journey as an example where the LFI receives all the details of the request from the TPP.



We have illustrated an example where the static code on an advertisement board is a QR code. [COMP-049] The code SHOULD contain a deep link supported by the User-facing TPP, which SHOULD invoke the User-facing TPP app/webpage on scanning.

Guidelines

1. COMP-050 The Call to Action (CTA) COULD be a static QR Code/NFC tag. [COMP-051] Scanning of the static code by the User SHOULD invoke a User-facing TPP page/app, deep linking them to the service represented by the code.
2. COMP-052 User-facing TPPs MUST initially ask the User to identify the LFI so that the consent request can be constructed in line with the LFI's data group and/or service initiation capabilities.
3. COMP-053 User-facing TPPs SHOULD make the User aware on the inbound redirection screen (User-facing TPP to LFI) that they will be taken to their LFI for authentication for data sharing.
4. COMP-054 If the User has an LFI app installed on the same device, the redirection MUST invoke the LFI's app for authentication purposes only without introducing any additional screens. [COMP-055] The LFI's app-based authentication MUST have no more than the number of steps that the User would experience when directly accessing the LFI app (biometric, passcode, credentials) and offer the same authentication method(s) available to the User when authenticating in their LFI's direct channels.
5. COMP-056 After authentication, the User MUST be deep linked within the app to confirm the account(s) to which they would like the User-facing TPP to have access.
6. COMP-057 LFIs SHOULD have an outbound redirection screen which indicates the status of the request and informs the User that they will be automatically taken back to the User-facing TPP.
7. COMP-058 LFIs SHOULD inform the User on the outbound redirection screen that their session with the LFI was closed.

8. COMP-059 User-facing TPPs SHOULD confirm the successful completion of the Open Finance Service Request (DSR & SIR).

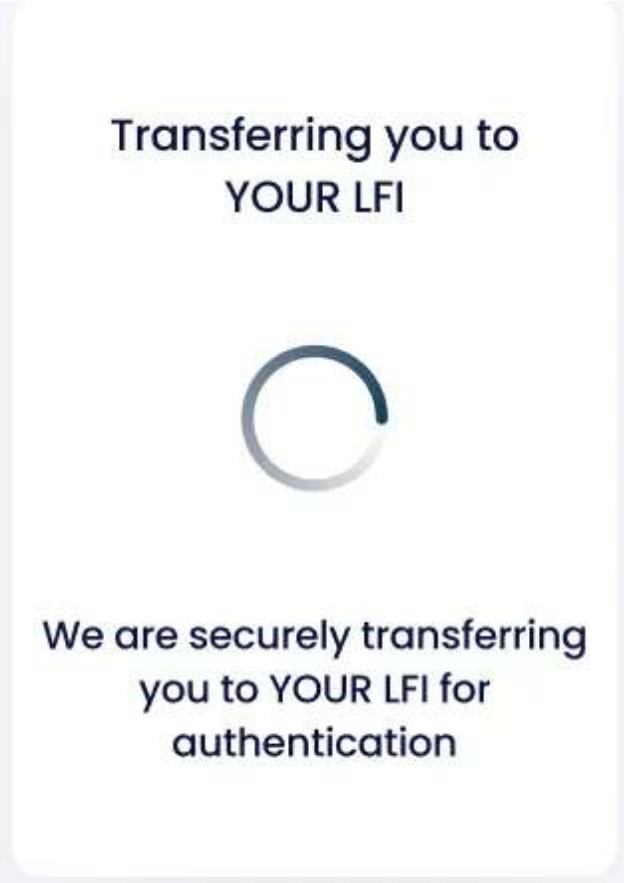
Effective Use of Redirection Screens

During the Open Finance consent authorisation journey, the User must be presented with two redirection screens to confirm that they are being transferred either to their LFI or back to the TPP.

COMP-060 Each redirection screen MUST display the name and logo of the LFI or TPP to which the User is being redirected, as illustrated in the examples below.

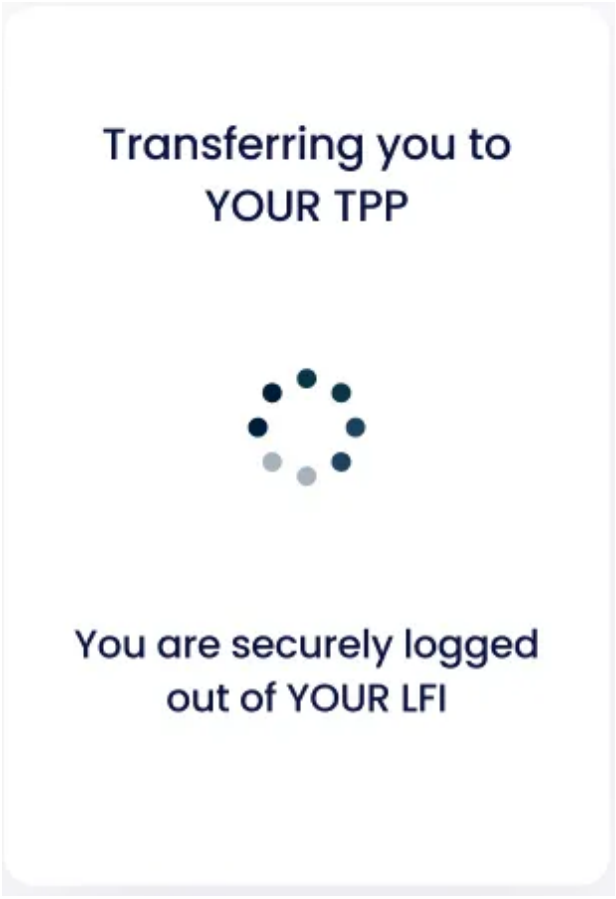
Inbound Redirection Screen

This screen is owned by the User-facing TPP and is shown when the User is redirected from the TPP domain to the LFI domain after providing consent to the TPP for the Open Finance service. [COMP-061] For the avoidance of doubt, LFIs MUST NOT present any additional inbound redirection screens.



Outbound Redirection Screen

This screen is owned by the LFI and is shown when the User is redirected from the LFI domain back to the User-facing TPP domain after the LFI has authenticated the User and completed the Open Finance consent authorization.



Why the Redirection Screens Are Required

Redirection screens play an important role in supporting a clear and trusted User experience. The following benefits apply:

- They help Users navigate their online journey and clearly explain what will happen next.
- They create a clear sense of separation between the User-facing TPP's domain and the LFI's domain.
- The inclusion of the LFI/TPP name and logo helps reassure the User that they are in control of the process and supports trust by clearly identifying which party they are being transferred to.

Redirection Screen Display Duration

COMP-062 The duration for which redirection screens are shown MUST be long enough for the User to clearly register the transition and understand that they are being redirected from the current app to their LFI or back to the TPP.

Error Scenarios During Redirection

As per OIDC, there are 2 main scenarios when an error occurs at the LFI side during the authorization process:

Malformed Authorization Request: The authorization request may be malformed when submitted by the User-facing TPP. For example, it may include an invalid redirection URL, invalid parameters or an invalid signature on the request object. OAuth 2.0 and OIDC define a whole list of potential errors. These are abnormal situations which indicate a significant technical issue at the User-facing TPP's end, or even an attacker trying to act as a User-facing TPP. [COMP-063] For safety (and as per the Standard) the LFI MUST NOT redirect the User back to the User-facing TPP in such situations. [COMP-064] The LFI MUST display an error message and stop the execution at this point. It is at the LFI's discretion to display to the User the message they find most appropriate for this error case; however, an error message MUST be displayed.

In this situation, User-facing TPPs do not receive a response from the LFIs about the malformed authorization request. Therefore, User-facing TPPs are not able to display any message to the User in this situation.

The User relies completely on the LFI to be notified about the occurrence of an error.

User interaction error: An error occurred during user interaction — for example, Users click on cancel or are unable to validate their credentials, or the Debtor account in the authorization request does not belong to the User. [COMP-065] In this case, the LFI MUST redirect the User back to the User-facing TPP and provide in the response the appropriate OIDC error message.

The LFI COULD potentially display an error message to the User before redirecting back to the User-facing TPP, but it is solely at their discretion. [COMP-066] The User-facing TPP SHOULD check the OIDC error code and display additional messages to the User with further information.

Error responses are documented as below:

- For OIDC: Final: OpenID Connect Core 1.0 incorporating errata set 2
- For OAuth 2: RFC 6749: The OAuth 2.0 Authorization Framework

Please note that both of the above scenarios are strictly checked by the FAPI conformance suite from OIDF.

Following the above requirements in the case of these error scenarios will prevent occurrences of bad customer experiences due to errors managed insufficiently.

Strong Customer Authentication | Introduction

Strong Customer Authentication (SCA) is a generic term used to describe multi-factor user authentication (MFA) supported by proofs-of-authentication. SCA came to prominence in Open Finance due to its inclusion as a Regulatory Technical Standard (RTS) under Payment Service Directive 2 (PSD2) in the European Union (EU). It sets the requirements for authenticating a Payment Services User — the account-holding customer who is making a payment — that must be adhered to in local law.

There are two key provisions that were heavily focused on in creating standards for Open Finance in the EU, namely:

- An authentication code indicating permission to make a payment must be based on at least two authentication factors: "the authentication shall be based on two or more elements which are categorized as knowledge, possession and inherence and shall result in the generation of an authentication code." (Article 4.1).
- The concept of "dynamic linking", which requires that the payee and amount are inputs to the creation of authentication codes that indicate proofs that payment has been authorized by the User (Article 5).

SCA is therefore very strongly allied to multi-factor authentication, and specifically outlines the acceptable factors, namely:

- **Inherence:** "Something you are" — a biometric credential indelibly linked to you.
- **Knowledge:** "Something you know" — a secret known to you and only you.
- **Possession:** "Something you have" — a device that can only be accessed and activated by you.

However, PSD2 is a directive and had to be interpreted in local law by different EU jurisdictions. SCA therefore has different implementations across the EU, and the dynamic linking requirement is manifested in a number of ways. The vast majority of implementations for Open Finance APIs involve the mobile banking app belonging to a bank, but there are two inherent limitations in this approach:

- **Proofs-of-authentication:** Invoking authentication through a biometric gesture on a given device results in an assertion at the mobile operating system level. This assertion is not, however, especially portable and provides proof only to the app invoking the request.

- **Alternative SCA providers:** Broadening the scope of available identity providers is more cumbersome, as the bank is the de facto identity provider in the context of an authentication flow.

Bringing together a best practice based on existing implementations across Open Finance markets is difficult to achieve. However, and despite the variance between EU countries, best practices can be achieved based on:

- A principles-based approach, which will help guide implementers to incorporate the correct features that provide both multi-factor authentication and proofs-of-authentication.
- Relevant controls that provide guidance on specific implementation approaches to ensure that SCA is performed in a sufficiently secure context.
- Alignment to salient regulatory frameworks for authentication established in Qatar.
- A view of emerging technologies and standards that can be leveraged in implementing SCA.

This page therefore provides guidance for ecosystem participants, to help make standardized choices in extending authentication options as the adoption of open finance is extended in the market, with the goal of creating an extensible ecosystem for SCA that complies with regulatory requirements in Qatar.

Strong Customer Authentication | Principles

The following are the principles on which the implementation of User authentication MUST be based.

#	Principle	Rationale
COMP-067	User authentication is implemented using MFA, where two factors are required to authenticate a given User	MFA is an existing best practice for User authentication that is prevalent in most Open Finance implementations.
COMP-068	MFA is based on the attributes of Inherence, Knowledge and Possession	Inherence, Knowledge and Possession are established factors and can be readily supported in the ecosystem.
COMP-069	A given User authentication operation is uniquely identified	An audit trail should be provided to link a User authentication operation with an action in the open finance ecosystem. Establishing this link will provide solid foundations for activities such as fraud prevention and dispute resolution.
COMP-070	A given User authentication operation is correlated with a given data sharing or service initiation consent	A given User authentication operation should be linked to a consent that has been agreed between a TPP and the User, either as a first-time consent or reauthorization of a long-term consent. Establishing this link provides accurate access controls established as a consequence of the User authentication operation and subsequent authorization undertaken by the User.
COMP-071	Attestation of User authentication is available as a cryptographically-verifiable proof-of-authentication	A given User authentication operation should be linked to a private key unique to the User. The presentation of a cryptographic signature generated using the private key can be verified using a corresponding public key that can be

#	Principle	Rationale
		accessed by a relying party who depends on the User authentication operation to provide access to services or data.
COMP-072	Open standards are leveraged wherever possible	Using open standards is a recognized means to create an interoperable and extensible ecosystem. Organizations should look to leverage the standards and implementations described in the Industry Standards section below.

Strong Customer Authentication | Controls

The following controls should be applied when implementing User authentication functionality for use in the open finance ecosystem.

These controls are intended to be normative, but not exclusive. Other controls, such as the OWASP Mobile Top 10, should be considered, together with existing organizational controls that govern mobile and internet banking.

#	Control	Rationale
COMP-073	Mobile apps that are used to authenticate Users are installed from an authorized and certified source	Provides an authorized source for an app designed to authenticate the User. Provides a means for an app to be installed from a source trusted by the User.
COMP-074	Mobile apps that are used to authenticate Users verify they are installed on a mobile operating system version for which they are approved	Ensures that the mobile app works as expected based on the functionality available on the mobile operating system. Lowers the risk of inadvertent leaking of User information based on unexpected results.
COMP-075	A given installation of an application that performs User authentication is correlated to the signature of the device on which it is installed	Ensures that when a User authentication operation is elicited by an application there is a clear binding between a given installation and a given device, providing assurance of the provenance of a given User authentication operation.
COMP-076	A given User is correlated to a private key, used for the purposes of User authentication, which in turn is correlated to a given device	Provides binding between a given device and the private key that represents the User.
COMP-077	Private keys created on a device for purposes of authentication are stored in the device security module	Reflects OWASP mobile app best practices. Accepted industry standard.

#	Control	Rationale
COMP-078	Private keys are represented by their corresponding public key or certificate, which can be published for use by relying parties	Allows a relying party to verify any cryptographically-signed assertions provided from a given device.
COMP-079	Use a biometric gesture (digit, face) as an authentication factor	Provides a strong proof of User identity when coupled with other controls.
COMP-080	A given authentication operation accepts an input parameter that uniquely links a given authentication operation to a given consent or consent signature	Provides a nonce that increases entropy in the cryptographic signing operation. Allows a signature to be indelibly linked to a given consent object.
COMP-081	The identifier used to link to a given consent or consent signature is validated before a User authentication operation is initiated	Ensure that User “intent” — where a first-time consent is being sought — is valid. Verify that an invalid consent reference has not been injected.
COMP-082	A given User authentication operation is asserted using a complex object that describes the conditions of the User authentication operation	Provides metadata about a given User authentication operation to relying parties, for the purposes of verifying the operation.
COMP-083	The assertion of a given User authentication operation is signed using the private key available in the device security module	Provides a cryptographically-based assertion of a given User authentication operation. Allows the assertion of a User authentication operation to be verified by relying parties, where supported by a suitable public key.
COMP-084	Signing of the assertion of a given User authentication operation is completed using	Prevents exposing the private key to untrusted components. Ensures the key does

#	Control	Rationale
	an appropriate function or method available on the device that provides signing access	not leave the secure storage area on the device.
COMP-085	Signing of the assertion of a given User authentication operation is consistent with the FAPI 2.0 Security Profile	Provides consistent security posture between components explicitly governed by FAPI 2.0 and the provisions of these controls.
COMP-086	Apps must verify the identity of external services on which they have a dependency for User authentication operations	Lowers the risk of allowing miscreants to misdirect the authentication flow, where a User must be redirected or handed off to an external service once MFA has taken place. Allows mobile app publishers to take advantage of approaches like certificate pinning to help ensure external services are correctly identified.
COMP-087	TPPs must safeguard against installation of their apps on jailbroken devices, and must prohibit initiating redirects for Authentication and Authorization when installed on a jailbroken device	Provides opportunities for activities by miscreants that compromise data sharing or service initiation.
COMP-088	TPPs must not allow installation or usage of their apps from within sanctioned countries, including but not limited to initiating redirects for Authentication and Authorization	Ensures the integrity of data sharing and service initiation operations by prohibiting activity in sanctioned countries.

Industry Standards

At the time of writing, the industry standards and technologies considered to be consistent with the principles and controls above are as follows:

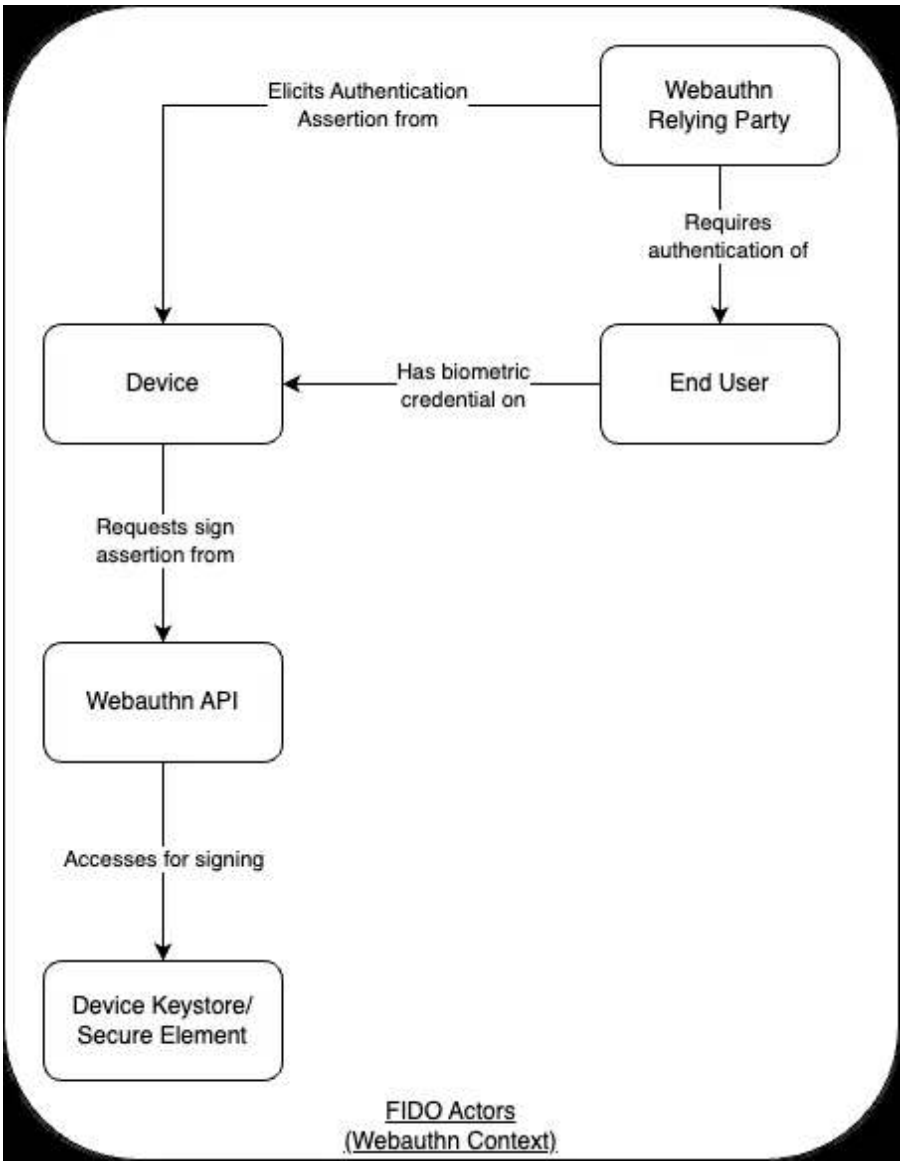
- FIDO2
- Passkeys
- OpenID for Verifiable Credentials
- Secure Payment Confirmation

Each is discussed in more detail in the sections below.

FIDO2

FIDO2 is a suite of protocols designed to offer strong proofs-of-authentication while eliminating the reliance on passwords. Unlike app-based biometric authentication, it provides proofs-of-authentication based on providing an Authentication Assertion, which is an object signed by a private key resident on a given device. The object signature can be verified by using a corresponding public key, which is held by the Webauthn Relying Party following an enrolment ceremony. Authentication Assertions can also complement OpenID Connect and FAPI 2.0, as the Assertion can be carried in an Authorization Code flow using a custom claim or a standard argument like `login_hint`.

The diagram below provides the key actors and their dependencies:



FIDO2 protocols are very widely supported on all devices that have come into the market recently, and the Web Authentication (Webauthn) API is supported in all major browsers. FIDO2 is often associated with and considered to be supportive of SCA, due to:

- **Private key provisioned on device:** FIDO2 protocols specify that a private key is provisioned on and never leaves an appropriate device (Possession).
- Interaction is generally through the device biometrics (Inherence).
- Fallback is allowed to a passcode or password (Knowledge).

FIDO2 also supports dynamic linking requirements. One of the inputs to invoking the Webauthn API is a challenge that can be set by the Webauthn Relying Party and can include relevant arguments such as payee details and amount, encoded as a string. The response to a request for User authentication can provide an authentication code. The FIDO Alliance has long asserted that FIDO2 standards can meet PSD2 requirements, but the standards have continued to evolve without, so far, widespread adoption in Open Finance. An extension of FIDO2, called Secure

Payment Confirmation (SPC), is however already included in 3D Secure, so there is some adoption in the banking and payments space.

The following are links to the relevant information on FIDO2:

- FIDO Alliance FIDO2 Homepage: FIDO User Authentication Specifications | FIDO Alliance
- Dynamic Linking white paper: Deploy FIDO Standards to Meet PSD2 SCA Requirements

Passkeys

Passkeys are a cross-browser, roaming implementation of FIDO2 credentials, which has been developed by the FIDO Alliance in cooperation with industry protagonists like Apple and Google. Implementing Passkeys is not the preserve of these organizations, however, and both hardware and software vendors are implementing support as a Webauthn Provider or Webauthn Relying Party.

As Passkeys is a FIDO2 implementation, it offers the same benefits as the FIDO2 suite of protocols, but adds the ability to synchronize keys between devices and retrieve keys from backup in the case of device loss or failure, a key critique of the out-of-the-box standard. SCA is provided in this context by FIDO2, as Passkey is a FIDO2 implementation.

The key point on Passkeys over a new FIDO2 implementation built from scratch is one of scale, due to the fact that existing providers are building out their support for the implementation and thus can readily support proofs-of-authentication in the market. There is also the point of backup, recovery, and synchronization, which improves user experience.

Building a trust framework using Passkeys is therefore, from a technical sense, more straightforward. The following are links to relevant information on Passkeys:

- FIDO Alliance Passkeys Homepage: FIDO Passkeys: Passwordless Authentication | FIDO Alliance

OpenID for Verifiable Credentials

OpenID for Verifiable Credentials is a new protocol that extends OpenID to allow presentations of credentials on behalf of users. It allows multiple, limited scope credentials to be minted that can be used for specific use cases. Examples abound, including dedicated credentials for appliances, vehicles, or specific merchants, or combining several credentials to authorize access to a given resource, such as having both club membership and a payment credential to access premium features at the gym. It is one of the standards proposed to enable the EU Digital Wallet.

Credentials can be stored in a wallet, and tied to specific use cases. Credential issuance is built on OAuth 2.0, and can therefore complement the adoption of FAPI 2.0. A bootstrap process for credential issuance is required, which can rely on existing authentication and authorization flows, but once issued the credentials can be presented in the form of a signed JSON Web Token (JWT) that can be verified. The important point here is that the JWT can contain presentation metadata, which can meet dynamic linking requirements.

Applying SCA in this context requires more development, however, as the mechanism for credential retrieval from a wallet has yet to be formally standardized. Standards and reference implementations for the EU Digital Wallet are currently being built.

The following are links to relevant information on OpenID for Verifiable Credentials:

- Implementers Draft for Credential Issuance: OpenID for Verifiable Credential Issuance - draft 13
- Implementers Draft for Verifiable Presentations: OpenID for Verifiable Presentations - draft 18
- Overview of issuance process (by Curity): Issue Verifiable Credentials using OpenID4VC
- Reference Wallet implementations for the EU: European Digital Identity

Secure Payment Confirmation

Secure Payment Confirmation (SPC) is an extension of Webauthn built for payments use cases. A Webauthn Relying Party can prompt for a Secure Payment Assertion, which incorporates a user journey that explicitly binds payment parameters with the resultant Assertion. As previously mentioned, SPC is already supported in 3D Secure.

As SPC is an extension of Webauthn, it naturally supports SCA and was built specifically to incorporate dynamic linking support. It has not, however, been widely supported in Safari and may also be refactored to complement Passkeys more readily. It also only supports payments use cases, so it will need to be complemented by native Webauthn assertions for data sharing requests.

The following are links to relevant information on SPC:

- SPC Homepage: Secure Payment Confirmation

Consent Setup | Overview

This page covers a critical step of the customer journey where Users grant consent to TPPs to allow them access their accounts for the provision of Data Sharing or Service Initiation.

COMP-089 Users MUST be clearly informed about the consent they are providing and the service they are receiving.

Consent Setup | Codification

Consent is the agreement between the User and the TPP on the bounds of the data access or service initiation that a TPP can undertake on behalf of the User.

Consent is described schematically, which allows the creation of a Consent object described using OpenAPI Specification v3.0.4. One-or-more Consents, depending on the use case, are submitted by TPPs to the API Hub through an RFC 9126: OAuth 2.0 Pushed Authorization Requests (PAR).

PAR serves as the mechanism for Consent Codification, as it allows TPPs to securely transmit one-or-more RFC 9396: OAuth 2.0 Rich Authorization Requests (RAR) in a standardized structured format. The use of RAR and PAR to codify Consent ensures that the information presented to Users is consistent, complete, and understandable, and therefore improves transparency and trust for participants in the Open Finance Framework.

Data Sharing Consent

When obtaining consent from Users for the provision of Data Sharing services, TPPs MUST:

- COMP-090 Clearly explain to the User the purpose for which data is being shared.
- COMP-091 Indicate how long the data will be shared for.

COMP-092 The table below shows the information TPPs MUST include in their consent setup journey with a given User.

#	Agreement Parameter	Description
1	Purpose Statement	Why we need you to share your data with us
2	Direct Benefit Statement	What you will get from us in return
3	Data Request Statement	What we need you to share with us
4	Duration Period Statement	How long we will need access to your data (unless you revoke access)
5	Agreement	Do you agree to share your data with us on the terms above?

Service Initiation Consent

When obtaining consent from Users for performing Service Initiations (e.g. payment initiations) TPPs MUST:

- COMP-093 Clearly explain to the User the purpose of initiating a given service.
- COMP-094 Indicate what type of service is being initiated.
- COMP-095 Clearly explain the consequences of initiating the service.
- COMP-096 Indicate how long the service will be provided for.

COMP-097 The table below shows the information TPPs MUST include in their consent setup journey with a given User.

#	Agreement Parameter	Description
1	Purpose Statement	Why we need to perform a Service Initiation for you
2	Direct Benefit Statement	What will you get from us in return
3	Service Initiation Request Statement	What Service Initiation specifically we are going to perform with your agreement
4	Duration Period Statement	How long we will need access to your account for Service Initiation (unless you revoke access)
5	Agreement	Do you agree to give us access to your account for performing Service Initiation on the terms above?

Purpose Statements

COMP-098 To meet the requirements in sections “Data Sharing Consent” and “Service Initiation Consent” above, TPPs MUST make sure their purpose statements are easy for an average User (person) to understand. A User should be able to read the statement and clearly understand how their data will actually be used.

When writing purpose statements, TPPs MUST:

- COMP-099 Use simple, everyday language to explain why the data is being accessed, and clearly link this to the benefit for the customer
- COMP-100 Set clear expectations about how the data will be used, without being too vague or overly detailed

- COMP-101 Explain the purpose in a way that matches the service being provided and the data needed to deliver it
- COMP-102 Avoid repeating information that is already shown elsewhere in the consent screen

COMP-103 The final wording MUST accurately reflect how the data will be used, so that Users can make an informed decision before providing their consent.

Consent Types

There are two different types of consent that Users will be requested to agree to:

A. Single-Use Consent

COMP-104 A one-off consent that a User provides to a TPP which MUST be used for a single Data Sharing or Service Initiation, which will take place immediately after User Authorization at the LFI.

This consent supports the following functionality:

- Single Instant Payments
- Account Information(One-off requests for a specific use case).

B. Long-Lived Consent

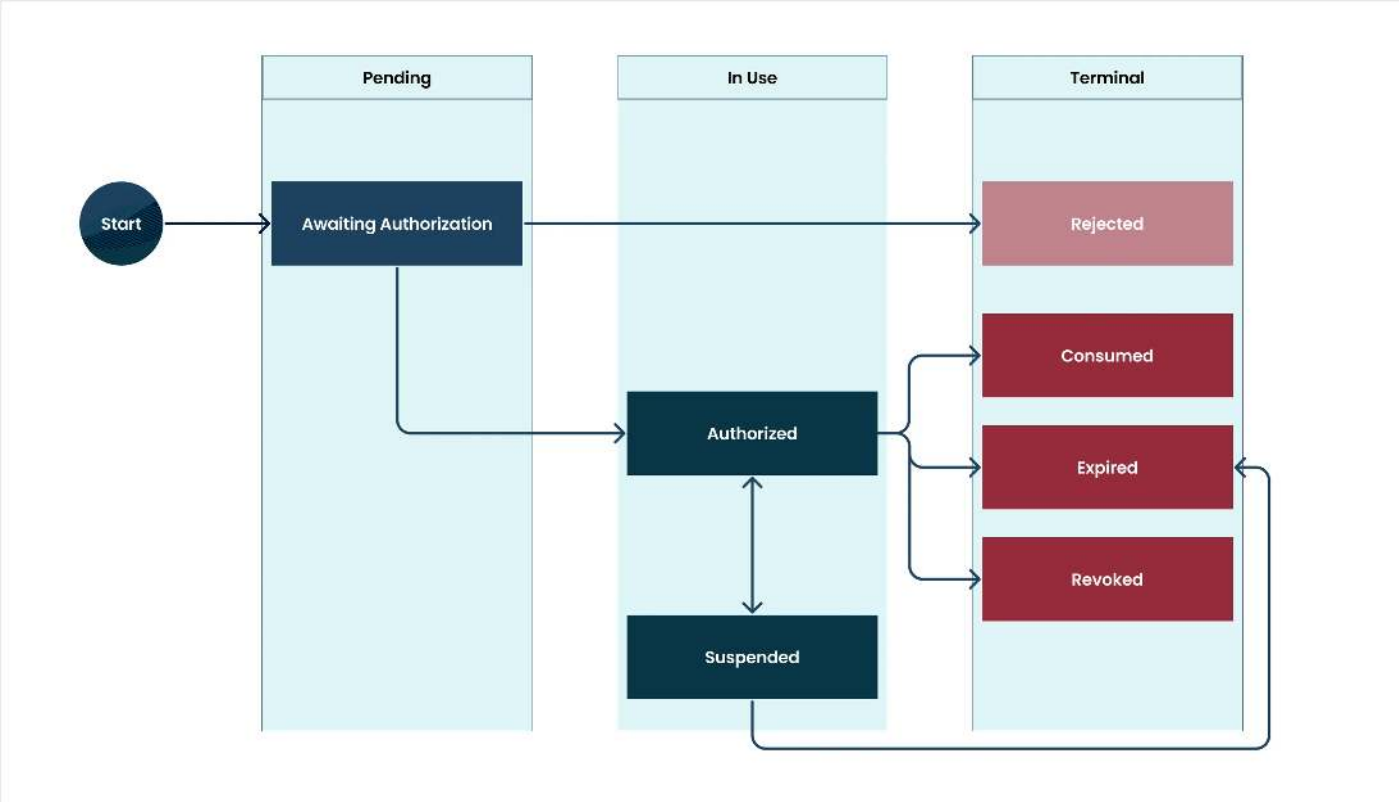
Consent that a User provides to a TPP for an ongoing Data Sharing access or for one-or-more Service Initiation operations in the future.

This consent supports the following functionality:

- Multi-Payments
- Account Information

Consent States

A Consent moves between these states in its lifecycle.



- **AwaitingAuthorization:** This is the initial state for all consents.
  - The TPP submits one-or-more RARs, containing one-or-more Consent objects, to the OFP.
  - The Consent is in a pending state waiting for the User to authenticate with the LFI and authorize the Consent.
  - > COMP-105 If multiple authorizers are required, the Consent MUST be authorized by all authorizers as per the authorization matrix of the LFI.
  - The Consent will remain in `AwaitingAuthorization` until all required authorizations are completed.
  - > COMP-106 Requests from TPPs for Data Sharing or Service Initiation MUST be rejected where the Consent is in the `AwaitingAuthorization` state.
- **Authorized:** In this state, the Consent has been fully authorized and it is now ready to use. The TPP can initiate requests to the OFP referencing this Consent.
  - The Consent will remain in this state until one of the following conditions is met:
    1. It has been fully used (`Consumed`).
    2. It has reached the end of its lifecycle and its validity has expired (`Expired`).
    3. It has been revoked by the User (`Revoked`).

4. It has been suspended by the LFI (`Suspended`)

- COMP-107 **Rejected:** A Consent MUST move from the `AwaitingAuthorization` to the `Rejected` state when either the User rejects the Consent, or the LFI rejects the processing of the Consent. `Rejected` is a terminal state.
- COMP-108 **Suspended:** A Consent MUST move from an `Authorized` state to a `Suspended` state when the LFI temporarily blocks access to the functionality the Consent covers in their systems—for example, if a User's Qatar ID (QID) has expired and a new Qatar ID (QID) must be provided to allow access to be restored.
  - COMP-109 Once the LFI has fully cleared the block, the Consent MUST move from `Suspended` back to `Authorized`. `Suspended` is not a terminal state.
- COMP-110 **Consumed:** A Consent MUST move from `Authorized` to `Consumed` when the action associated with the granted Consent has been fully used. `Consumed` is a terminal state.

> **NOTE**
>
> COMP-111 The `Consumed` state is only applicable to Service Initiation Consents and it is NOT to be used by Data Sharing Consents. A Data Sharing Consent, either of Single-Use or Long-Lived type, cannot be considered as consumed in any circumstances.

- COMP-112 **Expired:** A Consent MUST move from an In Use state (`Authorized` or `Suspended`) to `Expired` when the Consent has expired its time duration. `Expired` is a terminal state.
- COMP-113 **Revoked:** A Consent MUST move from an In Use state (`Authorized` or `Suspended`) to `Revoked` when the User has revoked the Consent. `Revoked` is a terminal state.

Consent Setup | Rules and Guidelines

TPPs MUST:

- COMP-114 Provide to Users, OFP and LFIs their trading/brand name clearly and the name of the User-facing entity, if the TPP is not the User facing entity.
  - It is the responsibility of TPPs to ensure that User-facing entities are identified and included in the Consent and the Consent Management Interface (CMI) information available to Users.
- COMP-115 Ensure that Users clearly understand the different elements of the Consent.
  - COMP-116 Where a Consent is part of a Consent Group, ensure that each Consent is presented to the User individually.
- COMP-117 Check the Consent state at the OFP and ensure that their systems accurately reflect the most-up-to-date state of the Consent item data.
  - COMP-118 Where a Consent is part of a Consent Group, check the state of each Consent in the group.
- COMP-119 Notify the OFP when the User has Revoked a Consent via the TPP.
  - COMP-120 Where a Consent is part of a Consent Group, notify the OFP where a specific Consent in the Consent Group has been Revoked.
- COMP-121 Inform customers if their data will be replicated outside Qatar.
  - COMP-122 The notification of data replication outside Qatar must include clear details about where the data will be processed and stored.
  - COMP-123 The TPP must obtain explicit agreement from the customer before proceeding with any data sharing or service initiation request.
- COMP-124 Where a Combined Consent is implemented, observe the constraints for CCGs when creating a PAR.
  - The OpenAPI description for Authorization Endpoints OpenAPI applies the CCG constraints that can be defined through the OpenAPI description language, namely:
    - Number of RARs.
    - Combinations of RARs.
    - Minimum and maximum number of RARs.

OFP MUST:

- COMP-125 Provide the system of record for the Consent object and the state of the Consent.

- COMP-126 Check the current date against the Consent Expiration Date. In case of expiration, OFP MUST set the Consent state to the Expired terminal state.
- COMP-127 Check the time elapsed from the Consent creation time against the Consent Authorization Time Window as defined in Common Rules and Guidelines: Authorization Time Window. In case of expiration, if the Consent is still in `AwaitingAuthorization` state, OFP MUST set the Consent state to the Rejected terminal state.
- COMP-128 Check the time elapsed from the Consent creation time against the Limits and Constants: Max Submitter Authorization Time Window. In case of expiration, if the Consent is still in `AwaitingAuthorization` state, OFP MUST set the Consent state to the Rejected terminal state, setting the appropriate error code for the TPP.
- COMP-129 In the case the Consent has been fully used, the OFP MUST set the Consent state to Consumed.
- COMP-130 Allow TPPs to check the status of the Consent and the Consent object.
- COMP-131 Allow LFIs to check the status of the Consent and the Consent object.
- COMP-132 Reject any Data Sharing or Service Initiation requests in relation to a Consent that is not in the Authorized state.
- COMP-133 NOT allow updates to a Consent state once the Consent has reached a terminal state.
- COMP-134 Where a CCG is submitted by a TPP, apply the constraints for CCGs.

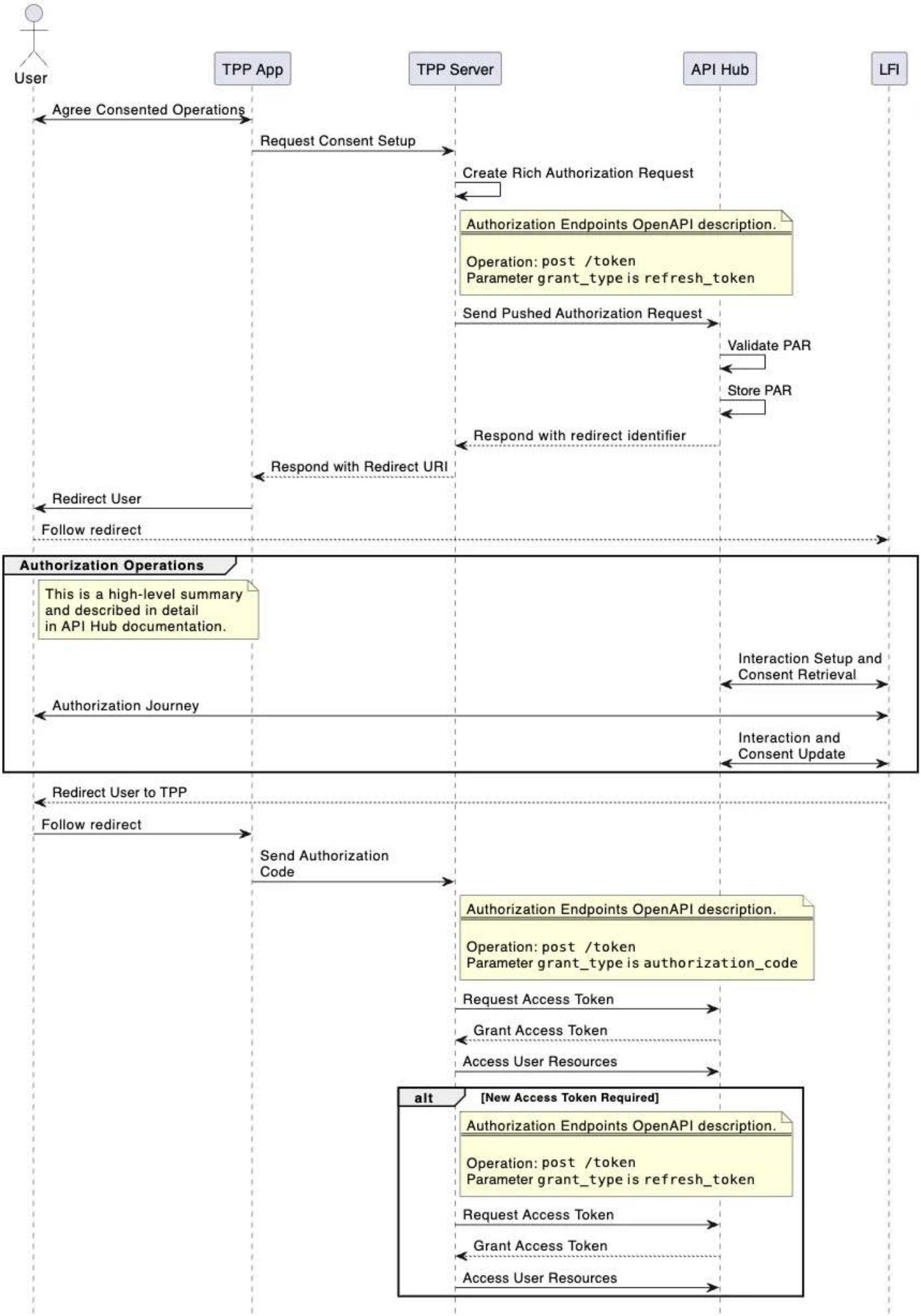
LFIs MUST:

- COMP-135 NOT allow Users to change any of the Consent parameters of a Consent in any state via the Consent Management Interface (CMI).
- COMP-136 Notify the OFP when the User has Revoked a Consent via the LFI.
- COMP-137 Notify the OFP when the Consent has been Suspended or moved out of Suspended.

Consent Setup | Implementation

The creation of Consent by TPPs and the authorization of consent by Users at LFIs is supported through the FAPI 2.0 Security Profile.

The diagram illustrates a high-level generalized view of Consent authorization flow below.



FAPI 2.0 implements the following OAuth 2.0 profiles that are highlighted in the diagram:

- PAR provides the means for TPPs to securely transmit one-or-more RARs, where Combined Consents are implemented, to the API Hub.
- RFC 6749: The OAuth 2.0 Authorization Framework allows TPPs to:
    - Initially obtain an Access Token to access a given resource or initiate a given service after Consent has been authorized, through the Authorization Code grant type.
    - Obtain a new Access Token on expiry of the initial Access Token grant, where applicable, through the Refresh Token grant type.
    - The details of the RAR(s) associated with a given Access Token grant can be retrieved from the Token endpoint, and includes the current state of the consent and the authorization status.
        - The Authorization Code and Refresh Token grant types both provide the RARs in the response.

The operations above are provided as an API description in OpenAPI Specification format, as per Authorization Endpoints OpenAPI, to which the API Hub conforms in the operations it implements.

Consent Management Interfaces | Overview

One of the primary objectives of the Standard is to provide simplification and consistency across all implementations. As such, we have defined a core set of principles for Consent Management Interface (CMI) dashboards that can and should be used, subject to the brand considerations of the User-facing TPPs and LFIs.

Dashboards are a common tool used in many industries to enable Users to manage their connections with service providers. From Facebook and Google to health providers and utility companies, various industries are leveraging dashboard functionality to provide Users with more control of their data and experience.

As the data & payment economies continue to expand, dashboards are set to become prominent features within digital landscapes, offering Users a range of capabilities from managing personal data and cookies to canceling subscriptions for services they no longer need.

Consent Management Interface (CMI) dashboards are introduced to the QCB Open Finance Standard to ensure Users can view and manage (i.e. cancel/revoke/update if desired) TPP and LFI Consent arrangements.

The Role of Consent Management Interfaces (CMIs)

- **User-facing TPPs' Consent Management Interface (CMI):** User-facing TPPs' facility to enable Users to view and revoke Consents given for Data Sharing or Service Initiation from their LFIs' account(s).
- **LFI Consent Management Interface (CMI):** LFI facility to enable Users to view TPPs that have been provided with Consent to access their account(s) for the provision of Data Sharing or Service Initiation and revoke a Consent, if desired.

Consent Management Interfaces (CMIs) are of utmost importance in reassuring Users that they have complete control of their Open Finance-enabled services. Well-designed and readily accessible Consent Management Interfaces (CMIs) are essential for building confidence around Open Finance.

An effective Consent Management Interface (CMI) provides Users with an overview of their Consent arrangements, clearly showing them what has been agreed to and providing them with the means to cancel or revoke such an agreement.

Principles Behind Effective Consent Management Interfaces (CMIs)

While the exact look and feel of Consent Management Interfaces (CMIs) is left to the discretion of TPPs and LFIs, there are design considerations that we strongly encourage all member organizations to consider:

Principle 1: Easy to Find and Locate

- To ensure Users can easily find the Open Finance Consents, avoid sub-menus and place it prominently in the main menu
- Designed and tested with the direct involvement of real Users
- Available on all relevant channels
- Named appropriately

Principle 2: Intuitive to Use and Understand

- Include clear and simple status messages and dates
- Ability to easily revoke live Consents
- Provide relevant additional detail as an optional view beneath the summary

Principle 3: Transparent as Possible

- Include a history for finished, rejected or canceled consents, with a commitment to retaining these in the history section for a minimum period of one year.
- Include all relevant parties on the Consent Management Interfaces (CMIs) extending to Beneficiaries and other User-facing entities where relevant

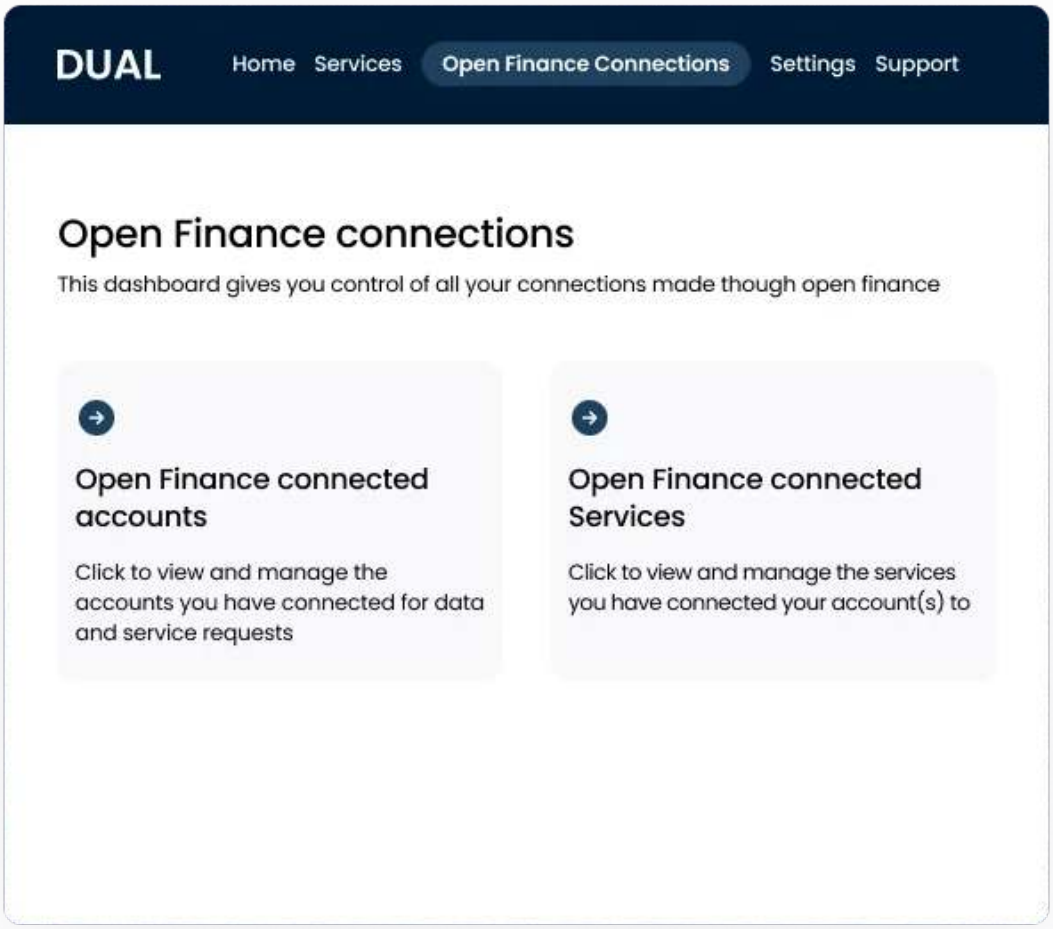
The rest of this section provides details on various aspects relating to Consent Management Interfaces (CMIs):

Naming of Consent Management Interfaces (CMIs)

The following names for Consent Management Interfaces (CMIs) are currently proposed:

Open Finance Connections	Where the entity is acting solely as a TPP, or a LFI, but not as both, the preferred term to describe the Consent Management Interface (CMI) is 'Open Finance Connections'. This is a term that avoids jargon and should improve familiarity with the concept of Open Finance and associated services. The term "Connections" is considered a useful catch-all term that is not off-putting to real Users and provides an accurate description of the link between TPPs and LFIs that can be managed through Consent Dashboards.
Open Finance Connected Accounts	Where the entity wants to clearly distinguish that it is presenting a User-facing TPP Consent Management Interface (CMI) (for example because they also act as a LFI), the preferred term is 'Open Finance Connected Accounts'. This is the term for User-facing TPPs to label their Consent Management Interfaces (CMIs) because they are providing a tool for Users to manage the payment accounts they have given the User-facing TPPs their consent for Data Sharing or Service Initiation.
Open Finance Connected Services	Where the entity wants to clearly distinguish that it is presenting a LFI Consent Management Interface (CMI) (for example because they also act as a TPP), the preferred term is 'Open Finance Connected Services'. This is the term for LFIs to label their Consent Management Interfaces (CMIs) because they are providing a tool for Users to manage the TPP services they have given their Consent for Data Sharing or Service Initiation.

COMP-138 Below we illustrate how we recommend an organization SHOULD name the respective Consent Management Interfaces (CMIs), when they operate as both a TPP and a LFI. Having navigated to "Open Finance Connections" in the top menu, the User would click "Open Finance Connected Accounts" to see their TPP Consent Management Interface (CMI), and "Open Finance Connected Services" to see their LFI Consent Management Interfaces (CMI):



We recognize that many providers in the market will fulfill multiple functions and this will have implications for their Consent Management Interfaces (CMIs). For example, there may be several providers who plan to operate as both LFIs and TPPs who will need to provide both Consent Management Interfaces (CMIs).

This figure shows guidelines for entities that are both a LFI and TPP. For the TPP Consent Management Interfaces and the LFI Consent Management Interfaces pages of this section, we have illustrated single-use Consent Management Interfaces (CMIs) only (i.e. Data Sharing on its own or Service Initiation on its own).

COMP-139 Providers who are acting in multiple capacities MUST ensure that their Consent Management Interfaces (CMIs) remain clear and transparent for Users, both at an individual level and when combined with other dashboards. [COMP-140] Providers MUST consider how Users will navigate between different dashboards and ensure that this is not confusing.

LFI Consent Management Interface | Overview

COMP-141 LFIs MUST provide Users with a facility to view and revoke ongoing Consents that they have given to any TPP for each account held at that LFI. Users may have consented for Data Sharing or Service Initiation from several accounts with many TPPs. This section describes:

- How to make the LFI Consent Management Interfaces (CMIs) an effective tool for Users
- How the revocation journey SHOULD be constructed

Consent Management Interfaces (CMIs) play an important role in clearly and transparently setting out what Consents Users have provided for TPPs. Therefore, Consent Management Interfaces (CMIs) play a role in multiple user journeys, including:

- Checking that the Consent is live (i.e. valid)
- Reminding Users to which TPPs they are providing Data Sharing or Service Initiation access from and for how long
- Informing Users how long TPP Consent will continue for
- Clarifying what types of Data Sharing access they have given TPPs
- Clarifying what type of Service Initiations TPPs can perform from their accounts
- Providing the ability to revoke TPP Consents

Being able to find the LFI Consent Management Interfaces (CMIs) easily and making it easy to understand plays a key role in building trust with Open Finance-enabled services.

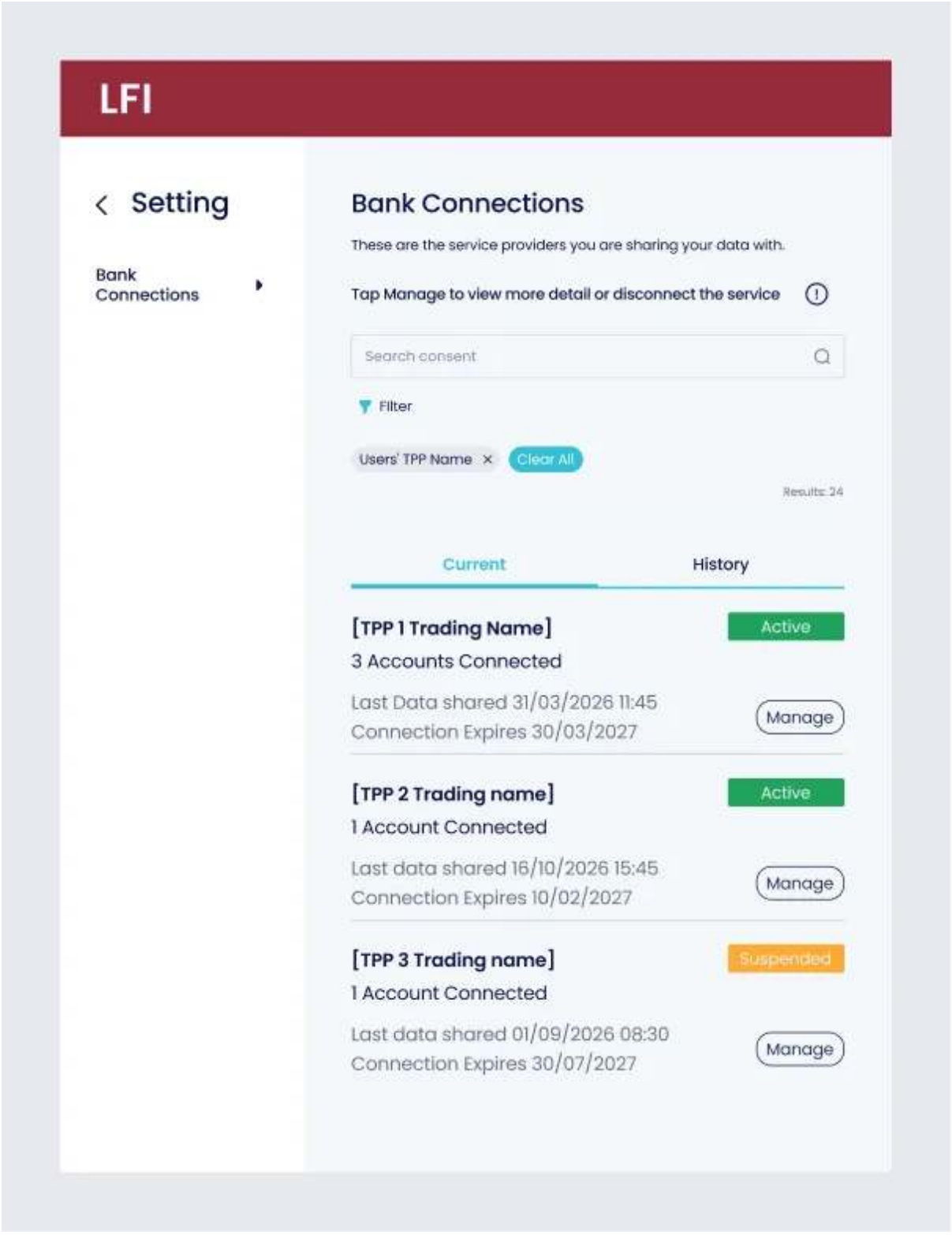
LFI Consent Management Interface (CMI) Examples

The following are example wireframes of LFI Consent Management Interfaces (CMIs), illustrating information that must be provided to Users for Data Sharing and Service Initiation Consents.

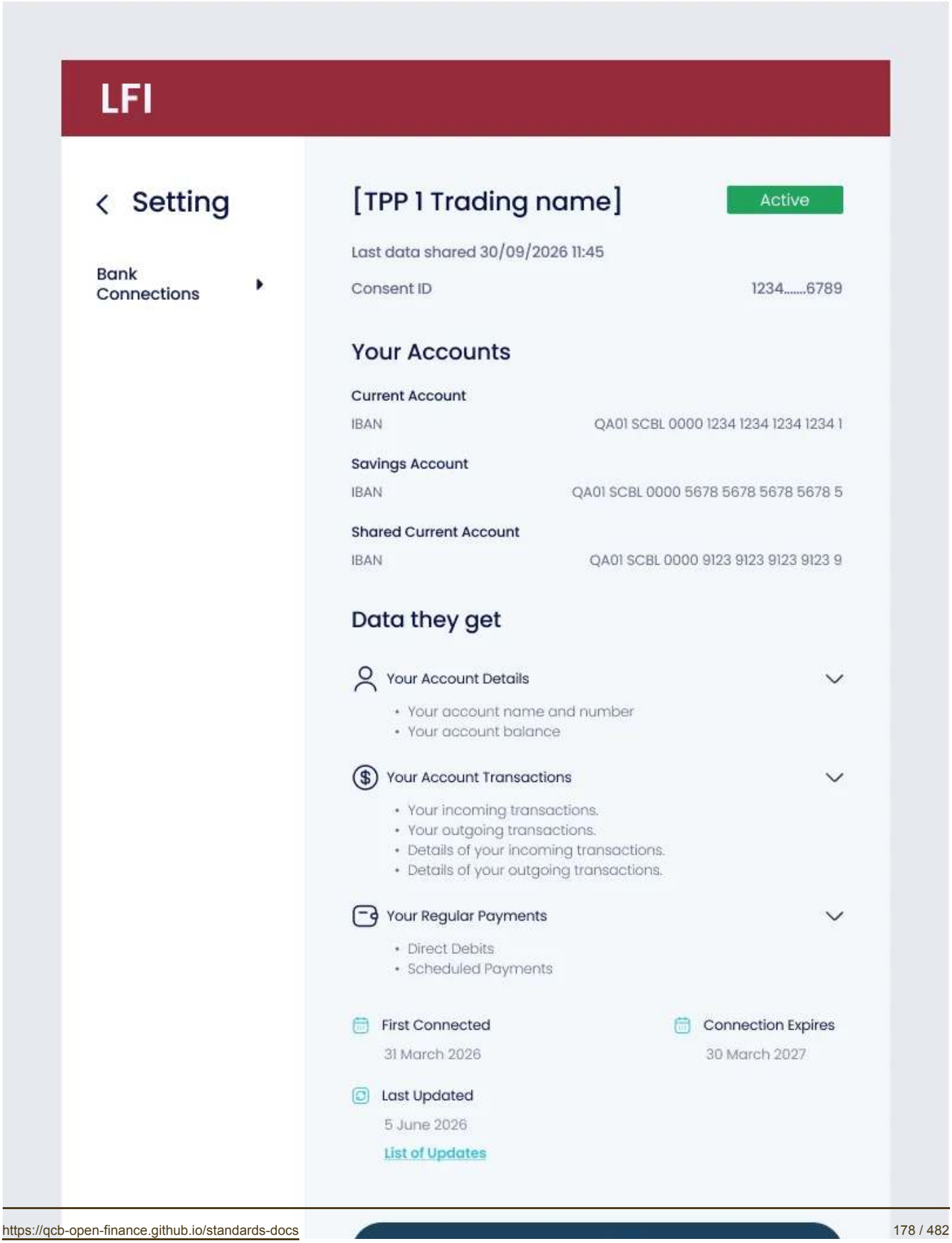
Data Sharing Consent Management Interface (CMI)

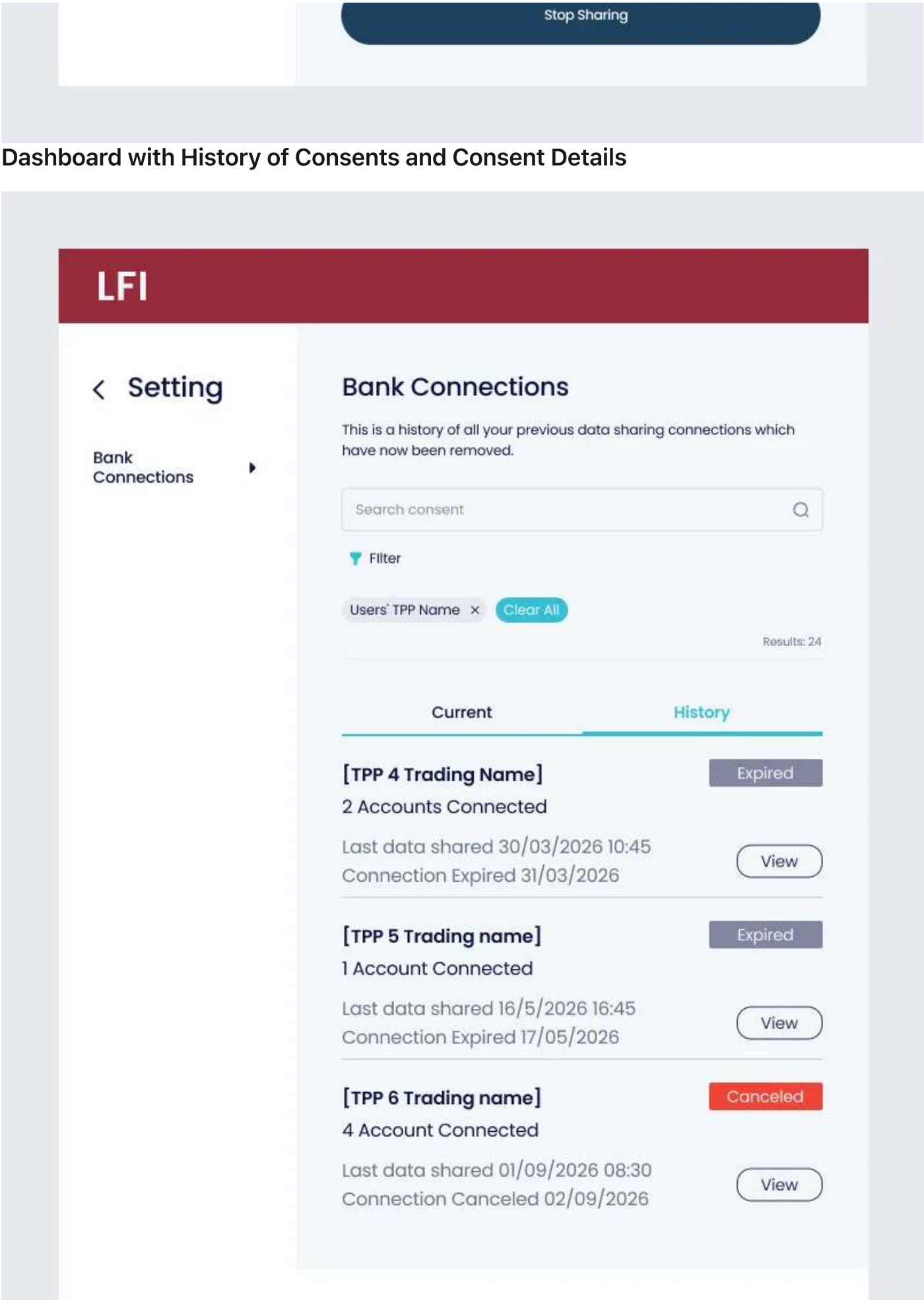
Desktop examples

Dashboard with Consents in use and Consent Details


LFI
Setting
Bank Connections
Bank Connections
These are the service providers you are sharing your data with.
Tap Manage to view more detail or disconnect the service
Search consent
Filter
Users' TPP Name
Clear All
Results: 24
Current
History
[TPP 1 Trading Name]
Active
3 Accounts Connected
Last Data shared 31/03/2026 11:45
Connection Expires 30/03/2027
Manage
[TPP 2 Trading name]
Active
1 Account Connected
Last data shared 16/10/2026 15:45
Connection Expires 10/02/2027
Manage
[TPP 3 Trading name]
Suspended
1 Account Connected
Last data shared 01/09/2026 08:30
Connection Expires 30/07/2027
Manage

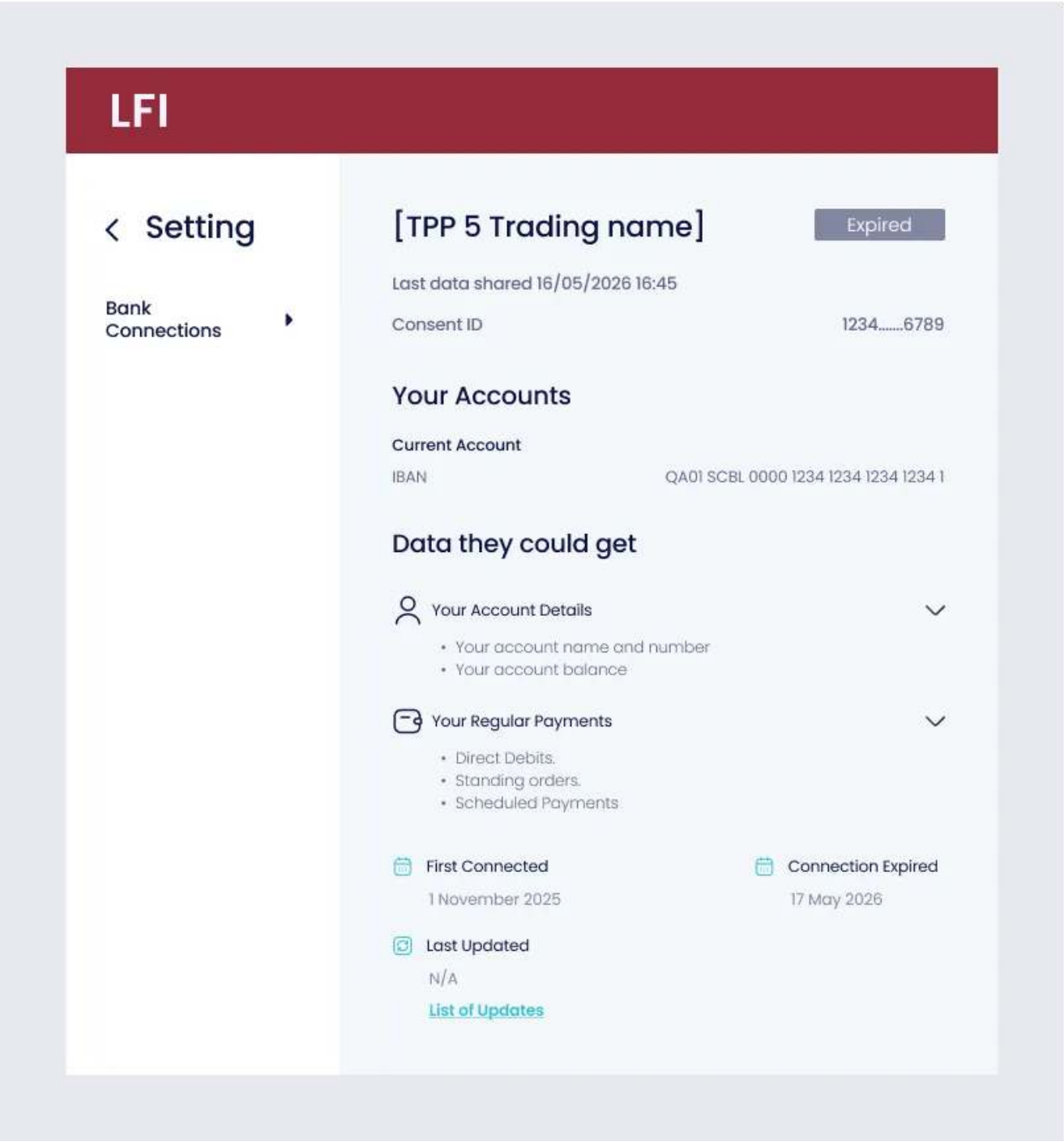
Data Sharing Connections Desktop Active Details





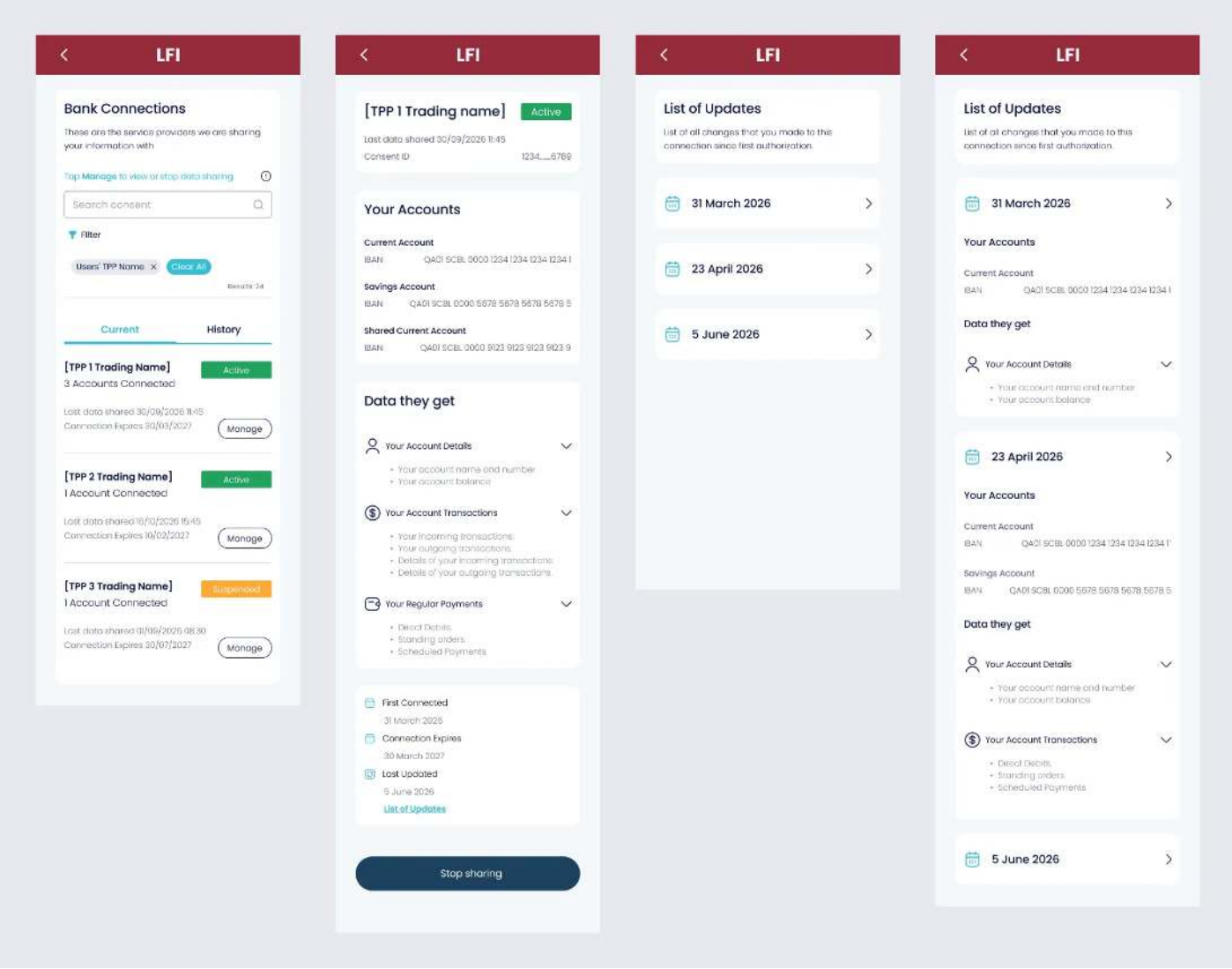
Dashboard with History of Consents and Consent Details

Data Sharing Connections Desktop Historical Details

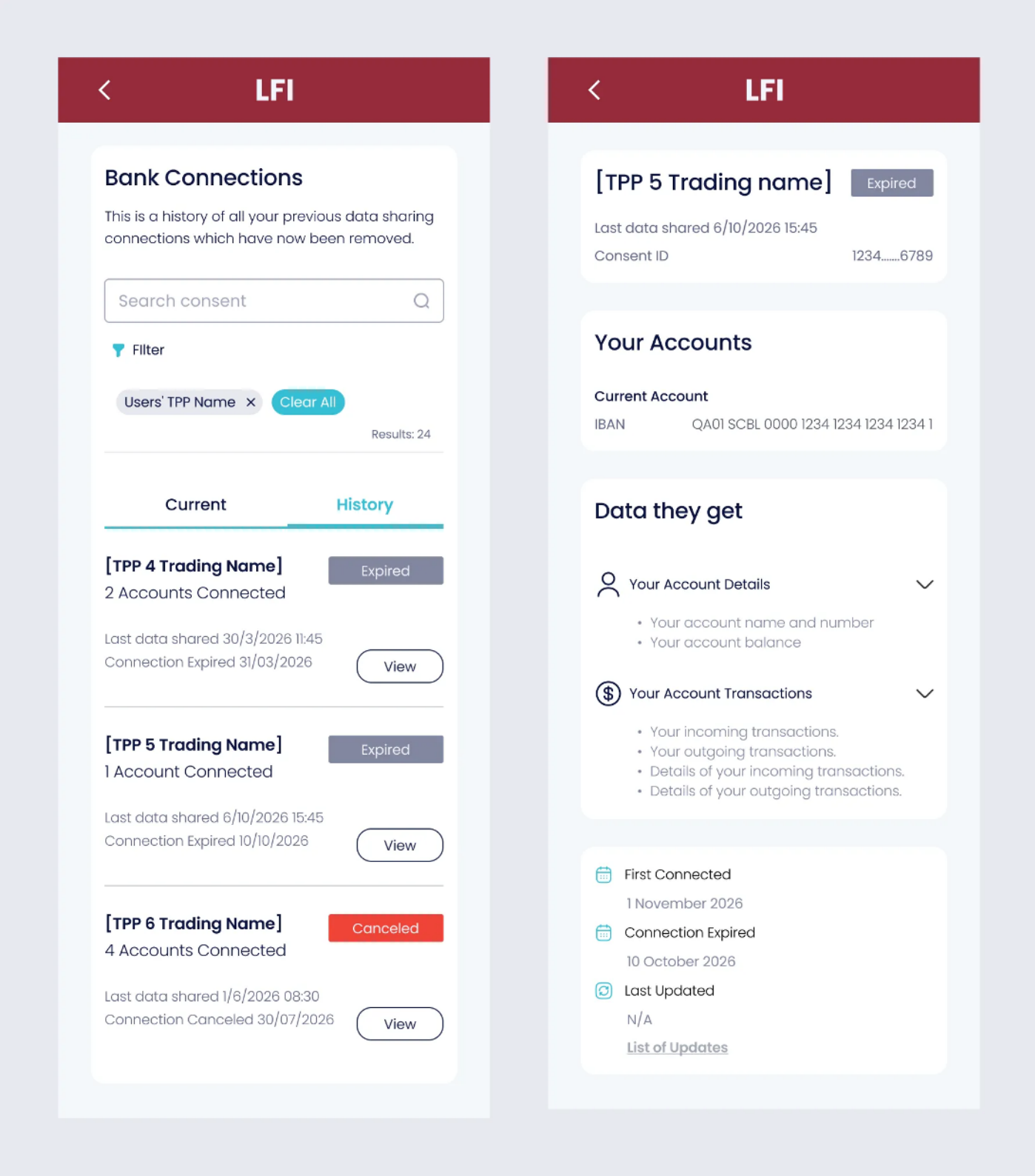


Mobile examples

Dashboard with Consents in use, Consent Details and List of Updates



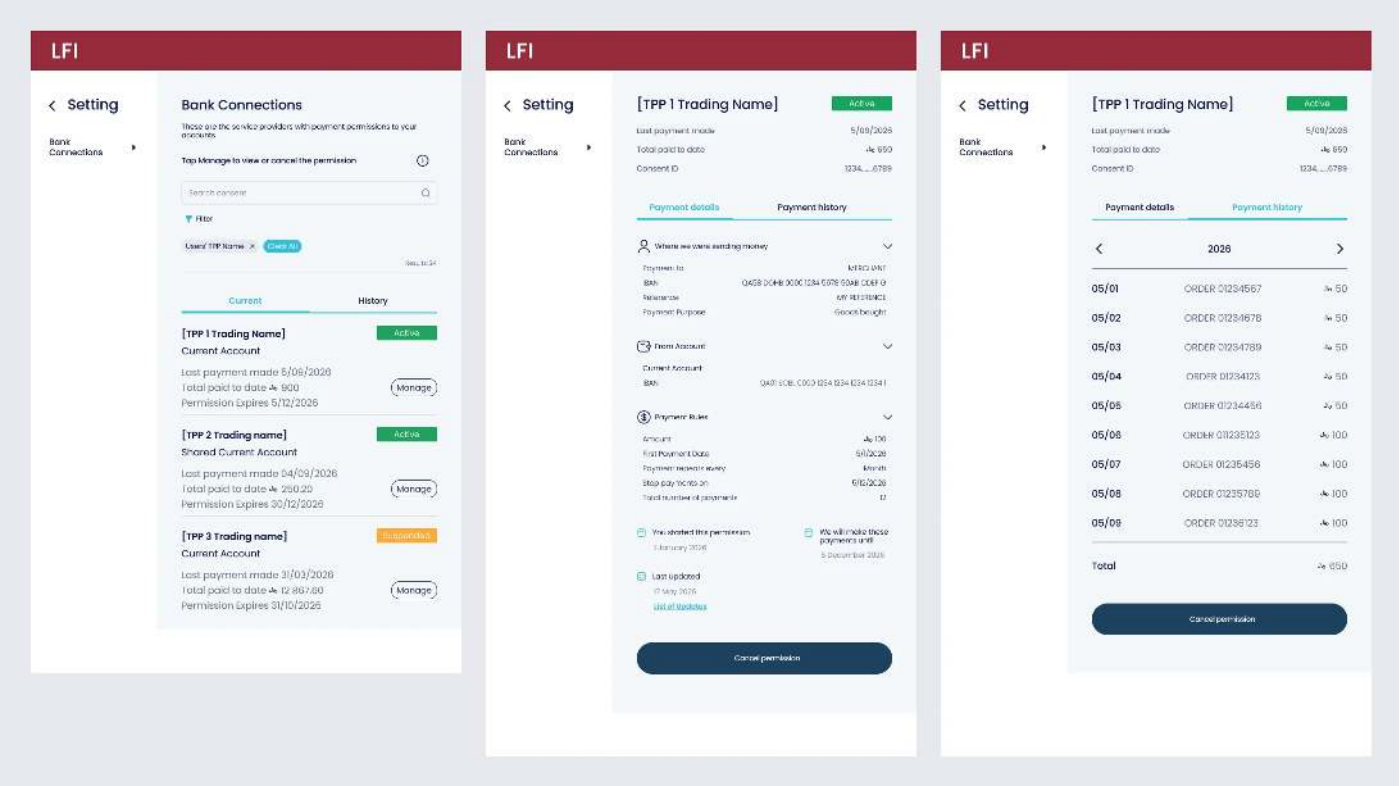
Dashboard with History of Consents and Consent Details



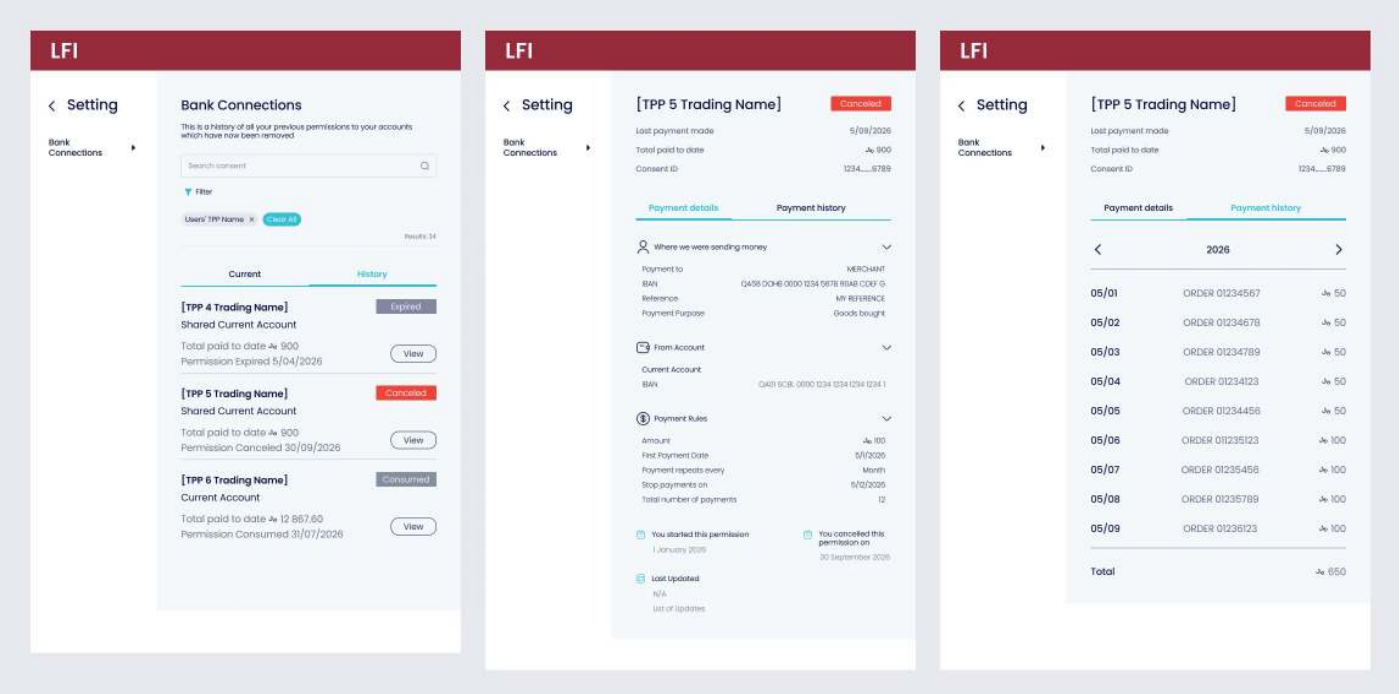
Service Initiation Consent Management Interface (CMI)

Desktop examples

Dashboard with Consents in use, Consent Details and Payment History

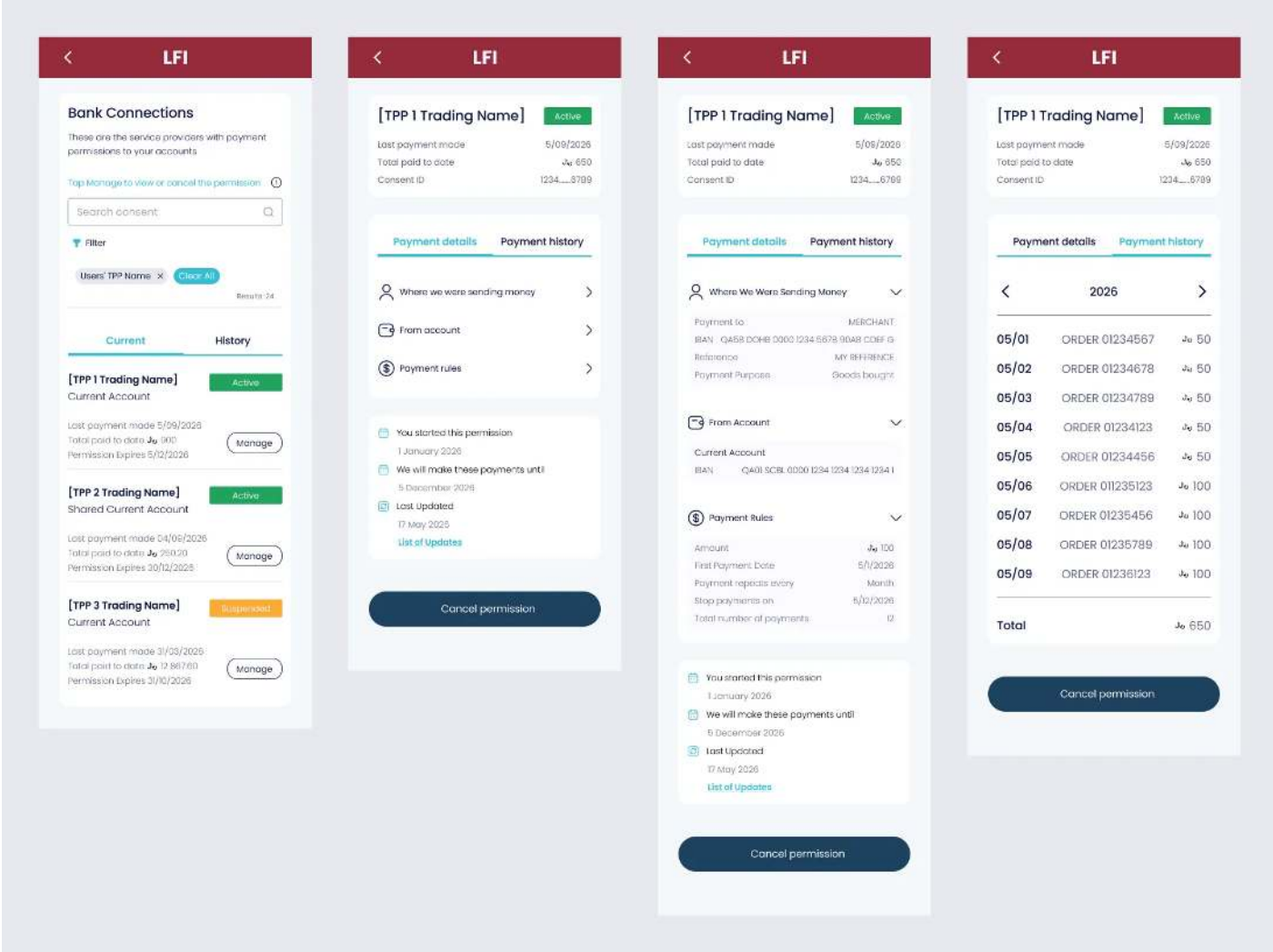


Dashboard with History of Consents, Consent Details and Payment History

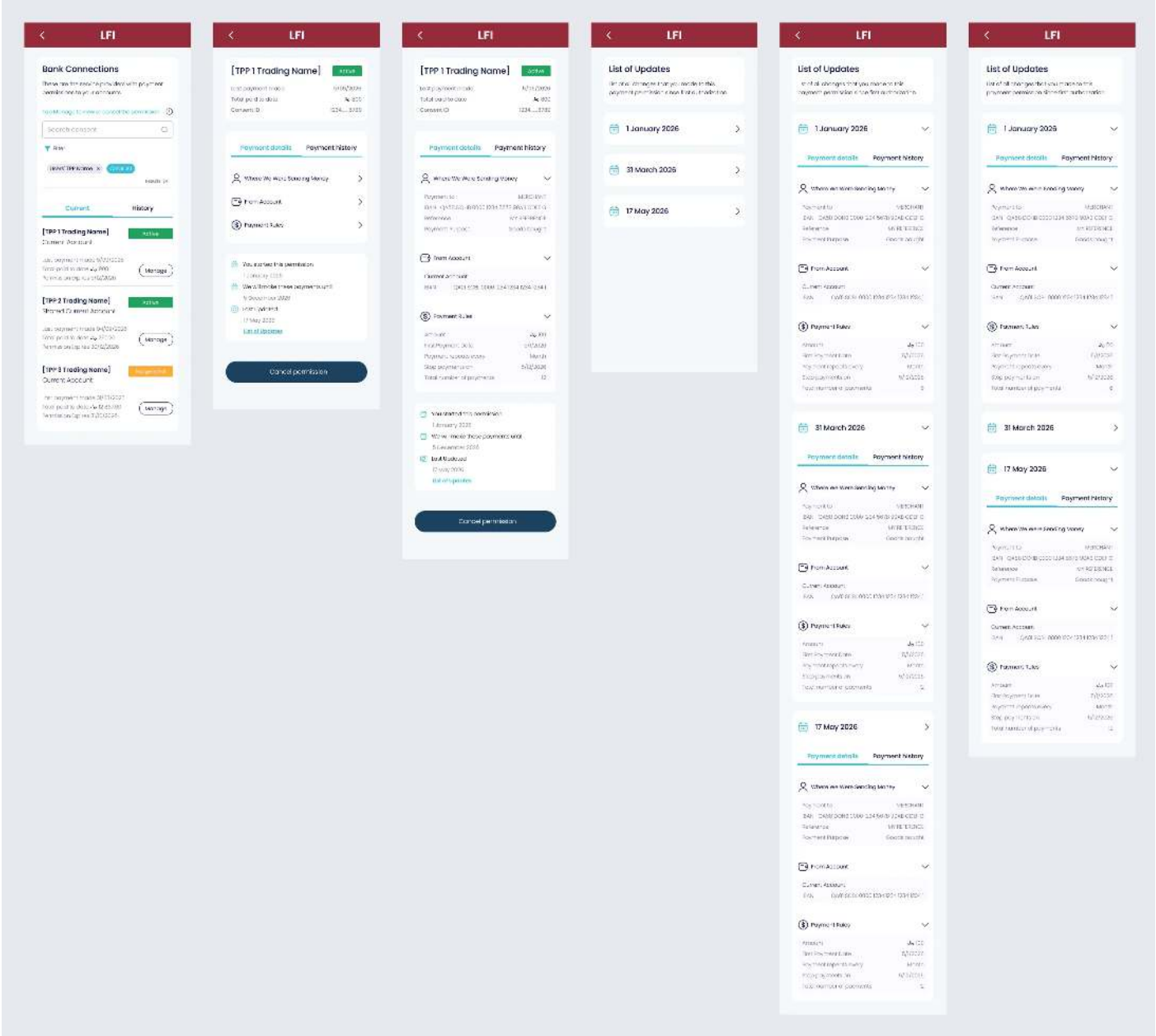


Mobile examples

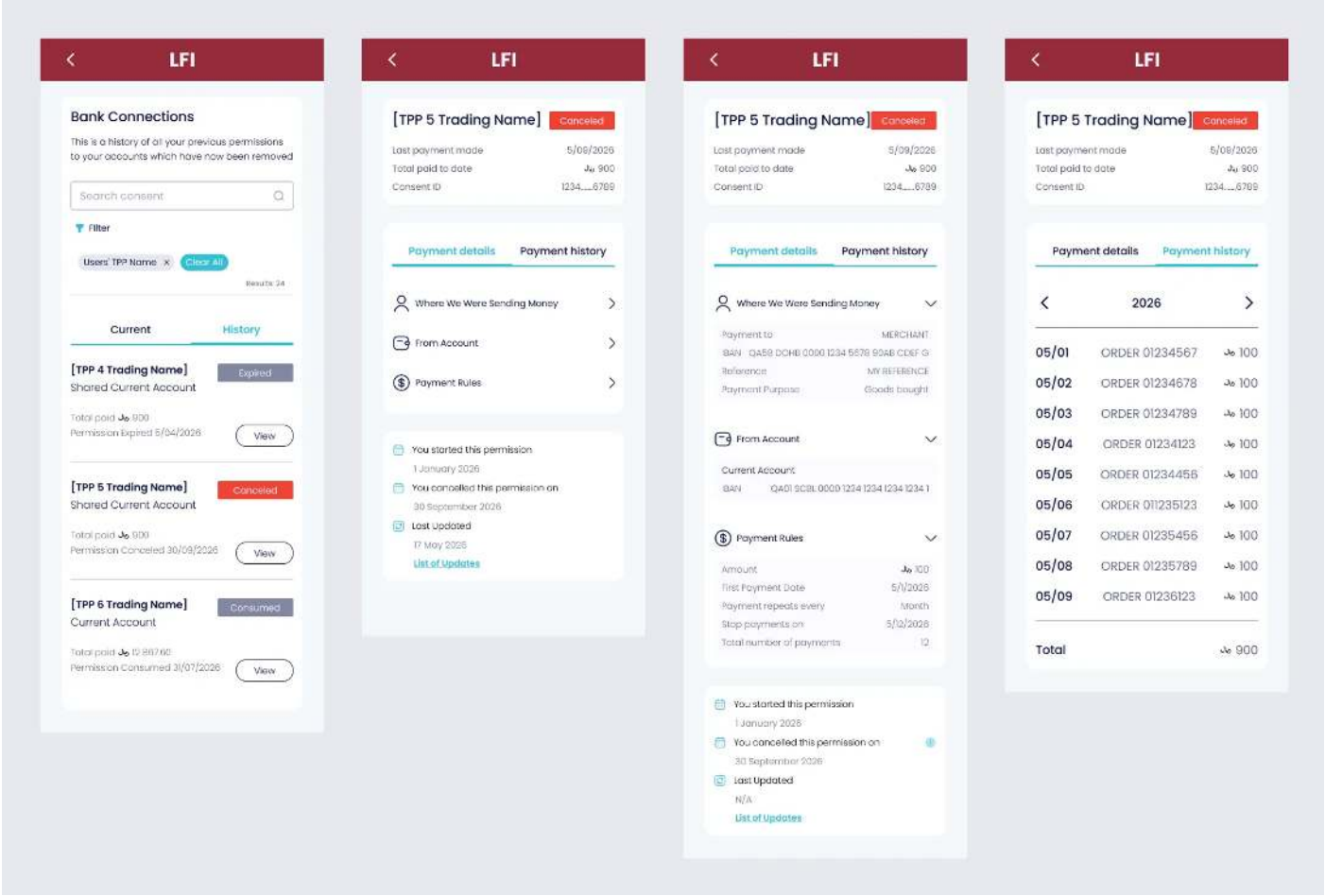
Dashboard with Consents in use, Consent Details and Payment History



Dashboard with Consents in use, Consent Details and List of Updates



Dashboard with History of Consents, Consent Details and Payment History



Rules & Guidelines

COMP-142 LFIs MUST keep the Data Sharing and Service Initiation Consents separate to ensure clarity and prevent any misunderstandings by Users.

COMP-143 LFIs MUST provide Consent Management Interfaces (CMIs) with an Overview page displaying high-level details for all Consents and a Detailed page for each Consent.

COMP-144 LFIs MUST display the TPPs' trading name/brand name to Users.

COMP-145 LFIs MAY also display the registered company name of TPPs, if it is different.

COMP-146 LFIs MUST use logos and the brand names of the LFIs as they are defined in the Trust Framework Directory.

COMP-147 LFIs MUST provide Users with comprehensive information to help them make informed decisions on the Consent Management Interface (CMI) Overview page. As a minimum, LFIs MUST display:

Consent Summary:

- Consent State ("Awaiting", "Authorized", "Rejected", "Canceled", "Consumed", "Suspended")

- Service Provider Name (i.e. the TPP trading name)
- Account type and the last 4 digits of account number
- The expiry date for when the Consent will end (Note: A countdown is an optional element. If provided, it SHOULD be used in addition to an expiry date and not instead of it)
- The date and time of the last occasion when Data Sharing or Service Initiation access occurred from the Users' connected account. This MAY be a specific date and time, or a range (e.g. within the last [x] days)
- "View Details" button to view the details of the Consent

COMP-148 LFIs MUST provide Users with comprehensive information to help them make informed decisions on the Data Sharing or Service Initiation details.

COMP-149 These details MUST include all the values which are defined within the specific Data Sharing or Service Initiation Consent, when it was requested. These details are covered within the Rules & Guidelines for each of the areas of functionality covered by the Standard.

COMP-150 LFIs MUST offer functionality (such as search, sort and filter) to enable Users to search for the relevant Consent. This will be of particular benefit as the number of Consents for different LFI accounts given by Users to TPPs increases.

COMP-151 LFIs MUST provide the following minimum set of filters in the Consent management Overview page:

- Service Provider Trading Name (and TPP name if different)
- Connected Account Number
- Consent Type
- Consent State
- Consent Date

COMP-152 LFIs SHOULD provide extra explanatory text to help Users understand complicated topics such as how to cancel their Consent (e.g. using information bubbles helps to keep information manageable).

COMP-153 The Consent ID that is presented at the User-Interface (UI) MUST be user-friendly.

COMP-154 LFIs MUST present the first and last four digits of the consent ID: 1234.....6789.

COMP-155 LFIs MUST offer a copy option that allows Users to copy the complete consent ID easily.

COMP-156 LFIs MUST provide the option to cancel a consent.

COMP-157 This option MUST be made available when:

- The consent is in an “Authorized” status following User authorization;
- The consent is in a “Suspended” status; or
- The LFI supports cancellation of a multi-user authorization consent prior to final authorization through its existing channels for consent submitters.

COMP-158 This option MUST NOT be made available for single-use consents associated with irrevocable service initiations, such as Single Instant Payments (SIPs).

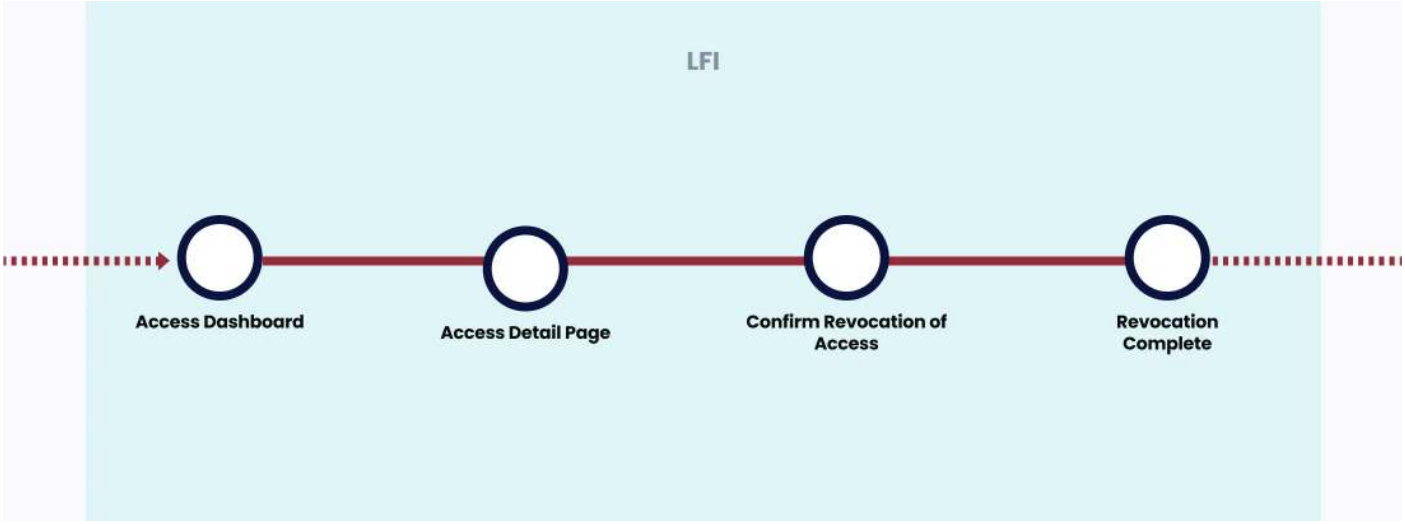
COMP-159 LFIs MUST provide a record of all past connections, including any previously granted Consents which had previously been:

- Canceled at the LFIs or the TPPs
- Expired
- Rejected
- Consumed

COMP-160 LFIs MUST ensure that the User who initiated the consent via the TPP can retrieve a record of all consents they have initiated that are in the “Awaiting Authorization” status.

LFI Consent Management Interface | Consent Revocation Journey

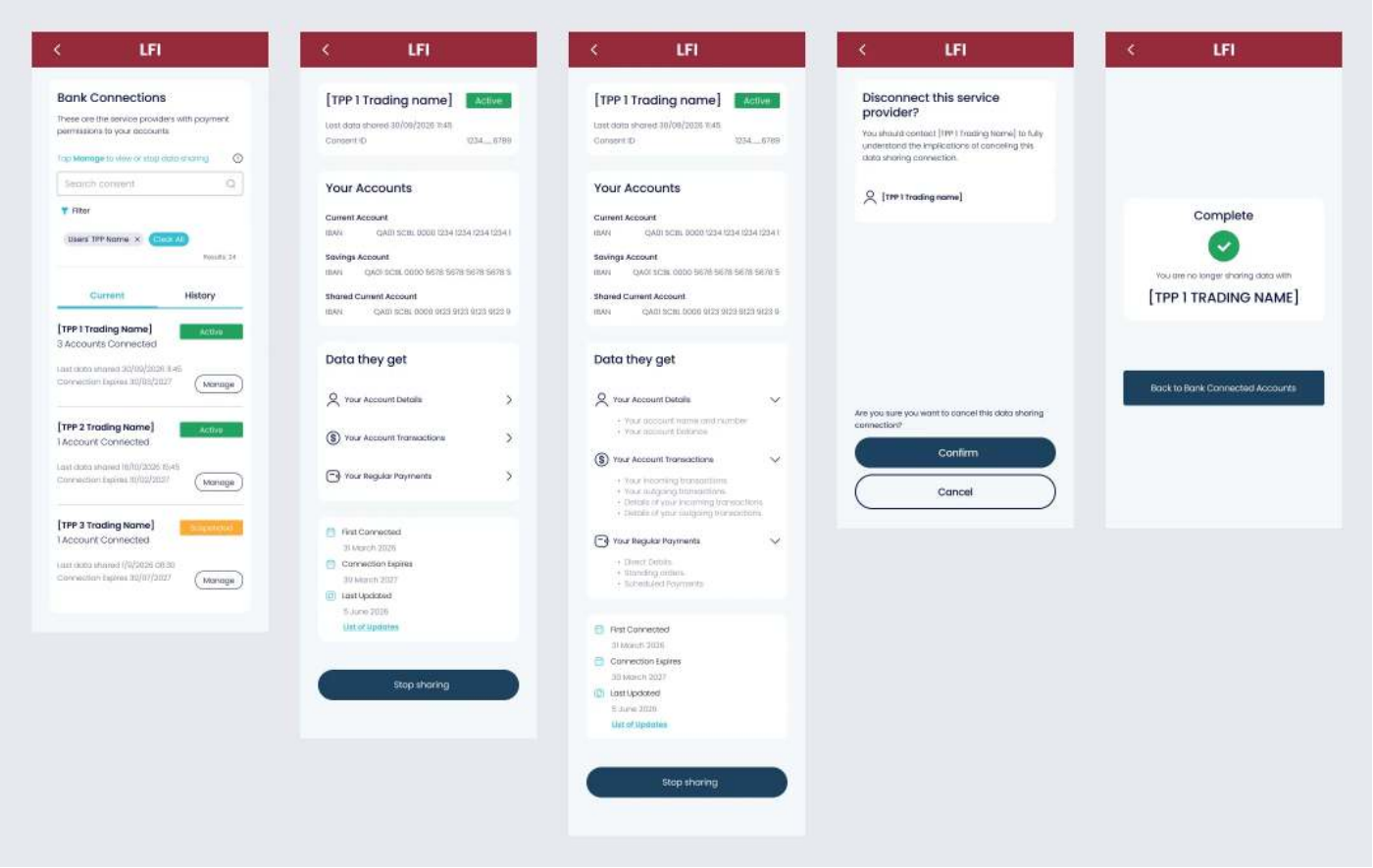
The ability to revoke TPP access to connected accounts is one of the primary roles of the LFI Consent Management Interface (CMI). The below journey illustrates the main steps involved:



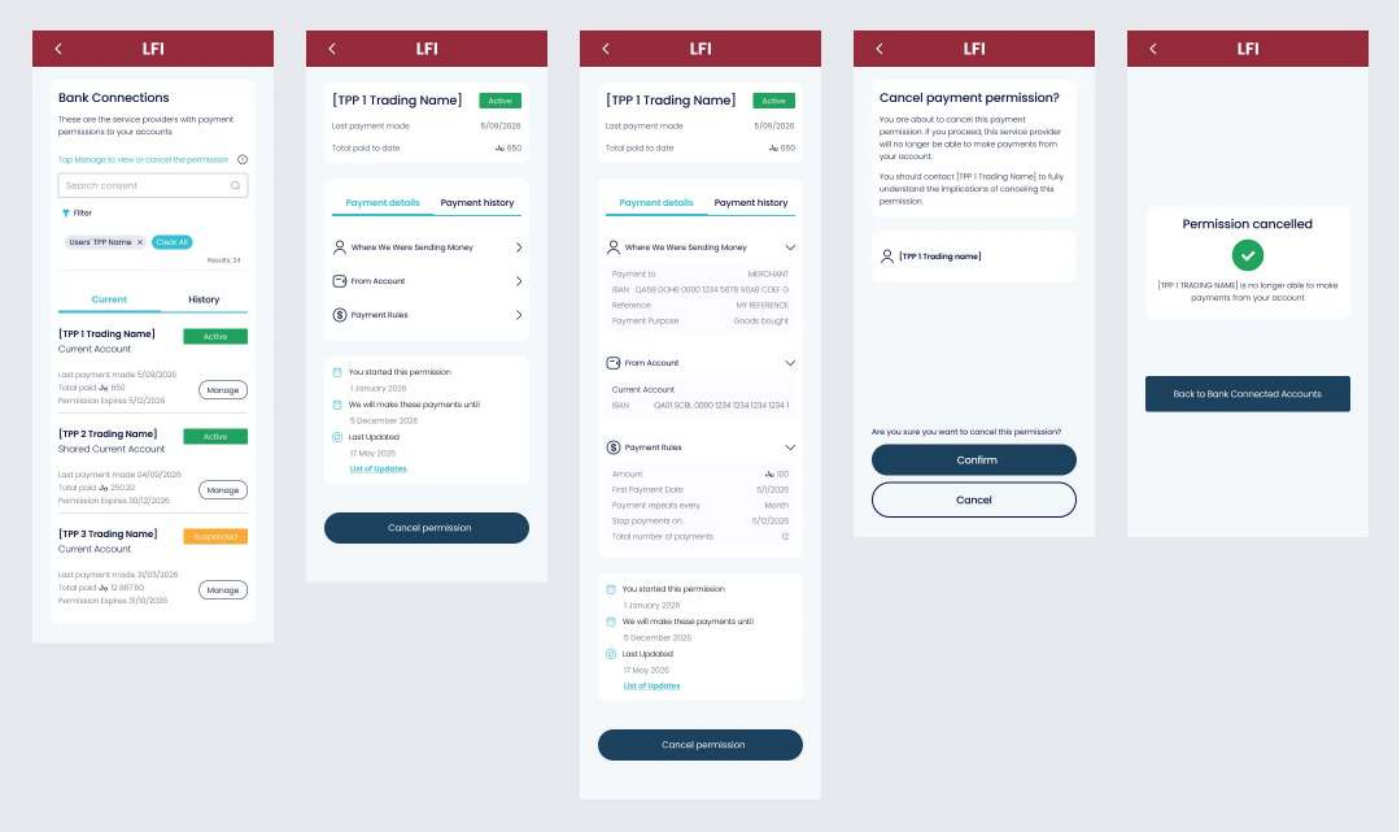
Wireframes

The below are example wireframes of the Consent Revocation journeys for Data Sharing and Service Initiation.

Data Sharing Consent Revocation Journey



Service Initiation Consent Revocation Journey



Rules and Guidelines

ID	Rules & Guidelines
COMP-161	In the **Consent Management Interface (CMI) Detailed** page, LFIs MUST show as a minimum: - When data was last shared or when Service Initiation was made by each TPP. This MAY be a specific date and time, or a range (e.g. within the last [x] days)). - The expiry date of the Consent.
COMP-163	In the **Consent Management Interface (CMI) Detailed** page, LFIs MUST allow Users to cancel the Consents they have provided to TPPs easily and without obstruction or excessive barriers.
COMP-164	In the **Consent Cancellation Notification** page (i.e. the last page), LFIs MUST advise Users that they SHOULD contact the associated TPP to inform them of the revocation of their Consent and to understand the consequences of their action.
COMP-165	In the **Consent Management Interface (CMI) Overview** page, LFIs MUST inform Users that the Consent has been withdrawn successfully and show “Canceled” state to the User.

LFI Consent Management Interface | Consent Management Interfaces (CMIs) when customer-facing service provider and TPP are different entities

COMP-166 If there are customer-facing service providers (e.g. Merchants) who are not TPPs but have commercial relationships with TPPs, the LFIs MUST display the customer-facing service provider name along with the TPP trading name on the Consent Management Interfaces (CMIs).

These entities are referred to as customer-facing service providers, as they are providing the end service to the end-Users, whereas TPPs in these cases are only undertaking the Data Sharing or Service Initiation activities. This could occur in merchant journeys for example, where a merchant contracts with a TPP to provide Variable Recurring Payments as a payment option on their platform.

Examples

TPP Trading Name (Client Name in the APIs)	Registered Legal Entity Name (Company Name/Organization Name)	Customer-facing entity name ('via' field in the APIs)	What to display
ABC Trades	ABC Company Ltd		ABC Trades
ABC Company Ltd	ABC Company Ltd		ABC Company Ltd
ABC Company Ltd	ABC Company Ltd	OBO Ltd	OBO Ltd via ABC Company Ltd
[TPP Trading Name]	[TPP Company name]	[Merchant Trading name]	[Merchant Trading name] via [TPP Trading Name]

TPP Consent Management Interface | Overview

COMP-167 TPPs MUST provide Users with Consent Management Interfaces (CMIs) to view and revoke ongoing Consents that they have given to the TPPs. This section describes:

- How to make these Consent Management Interfaces (CMIs) effective tools for Users
- How the revocation journey should be constructed

Consent Management Interfaces (CMIs) play an important role in clearly and transparently setting out what Consents a User has provided for a TPP. Therefore, Consent Management Interfaces (CMIs) play a role in multiple user journeys, including:

- Checking that the Consent is live (i.e. valid)
- Reminding a User where they are providing Data Sharing or Service Initiation access from and for how long
- Informing a User how long the access will continue for
- Clarifying what level of access the TPP has
- Revoking access
- Pausing access
- Reactivating Paused access

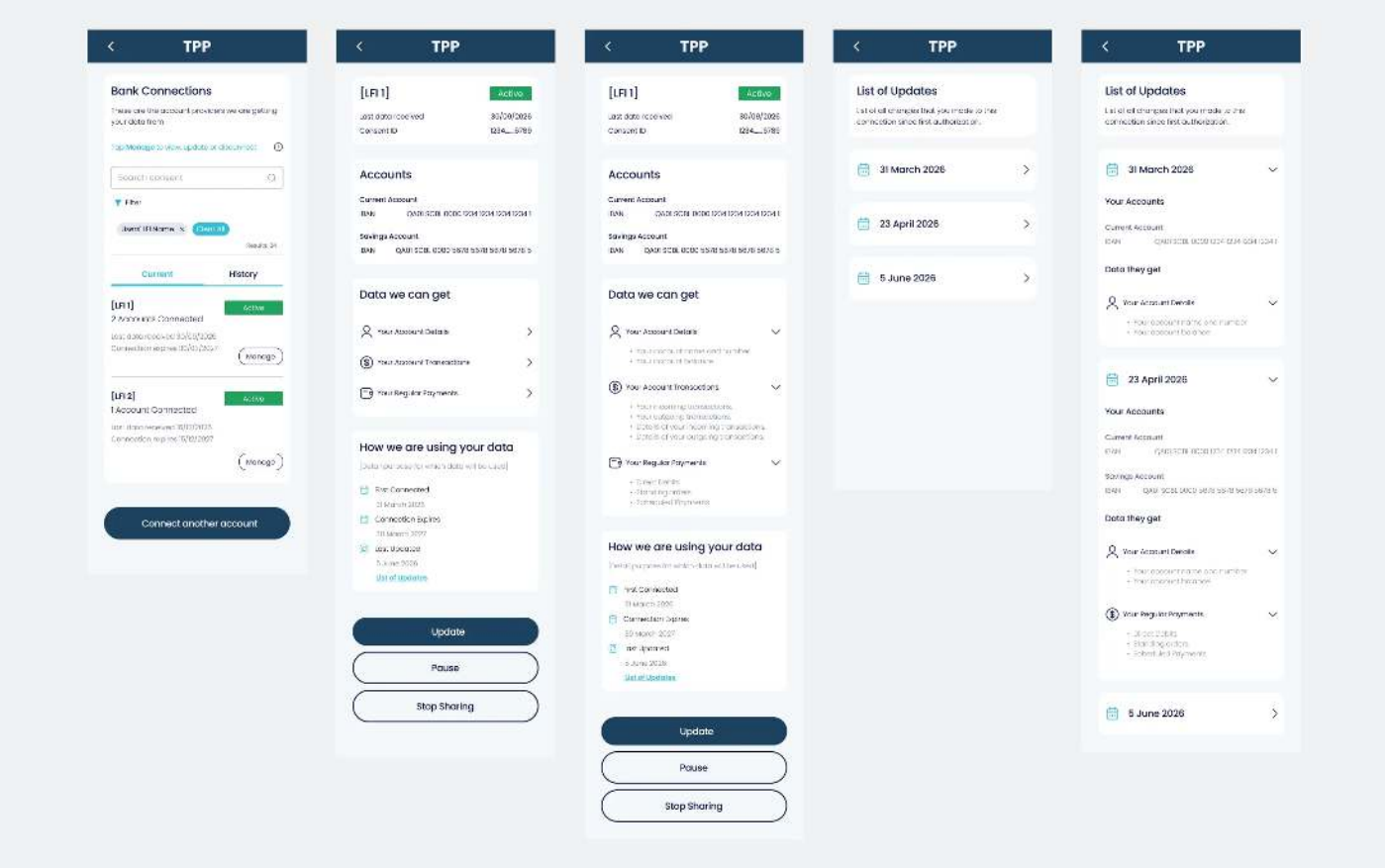
COMP-168 The TPP Consent Management Interface (CMI) SHOULD therefore be easy to find and easy to understand, which plays a key role in building trust with Open Finance-enabled services.

TPP Consent Management Interfaces (CMIs) examples

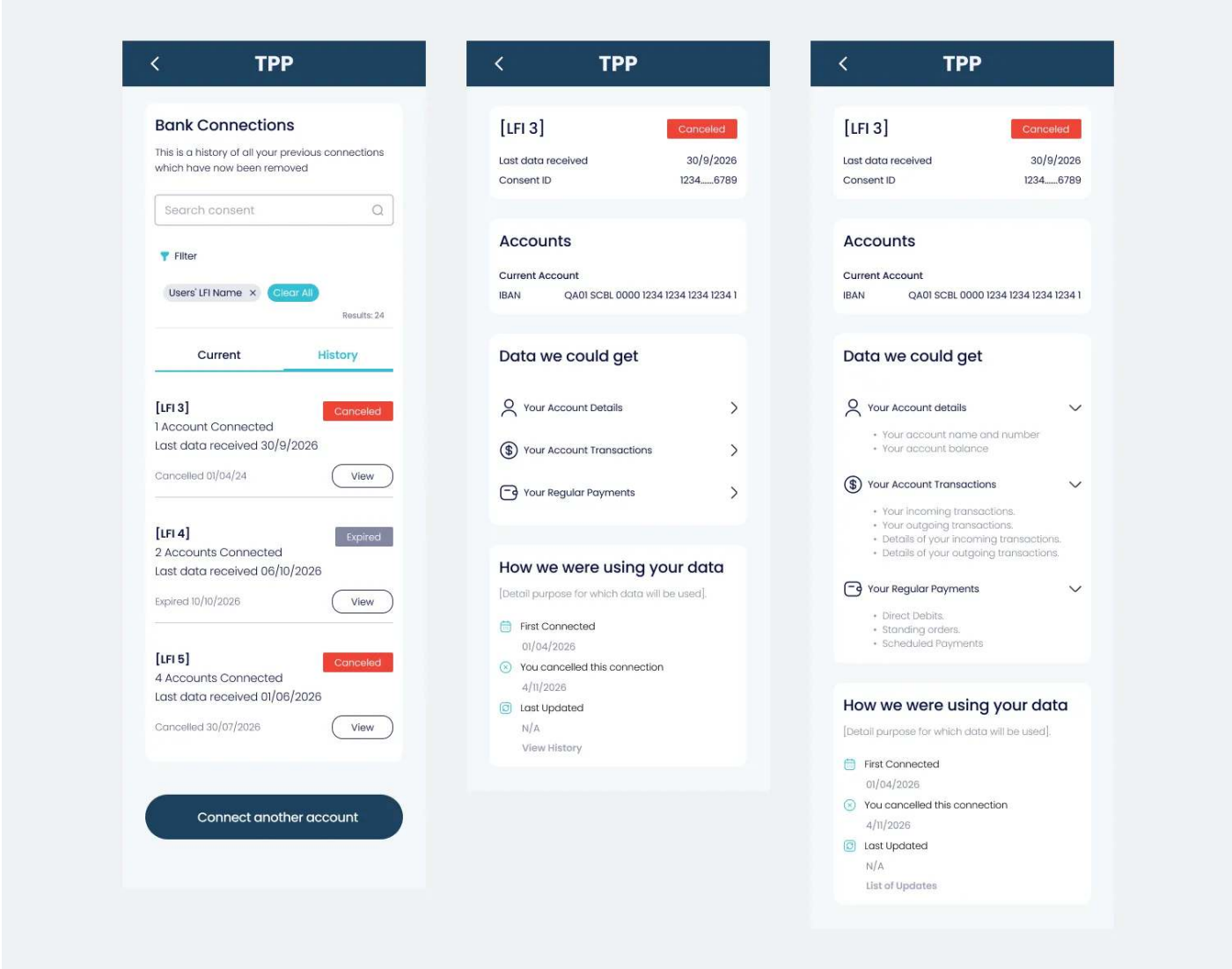
The following are example wireframes of TPP Consent Management Interfaces (CMIs), illustrating information that must be provided to Users for Data Sharing and Service Initiation Consents.

Data Sharing Consent Management Interface (CMI)

Dashboard with Active Consents, Consent Details and List of Updates

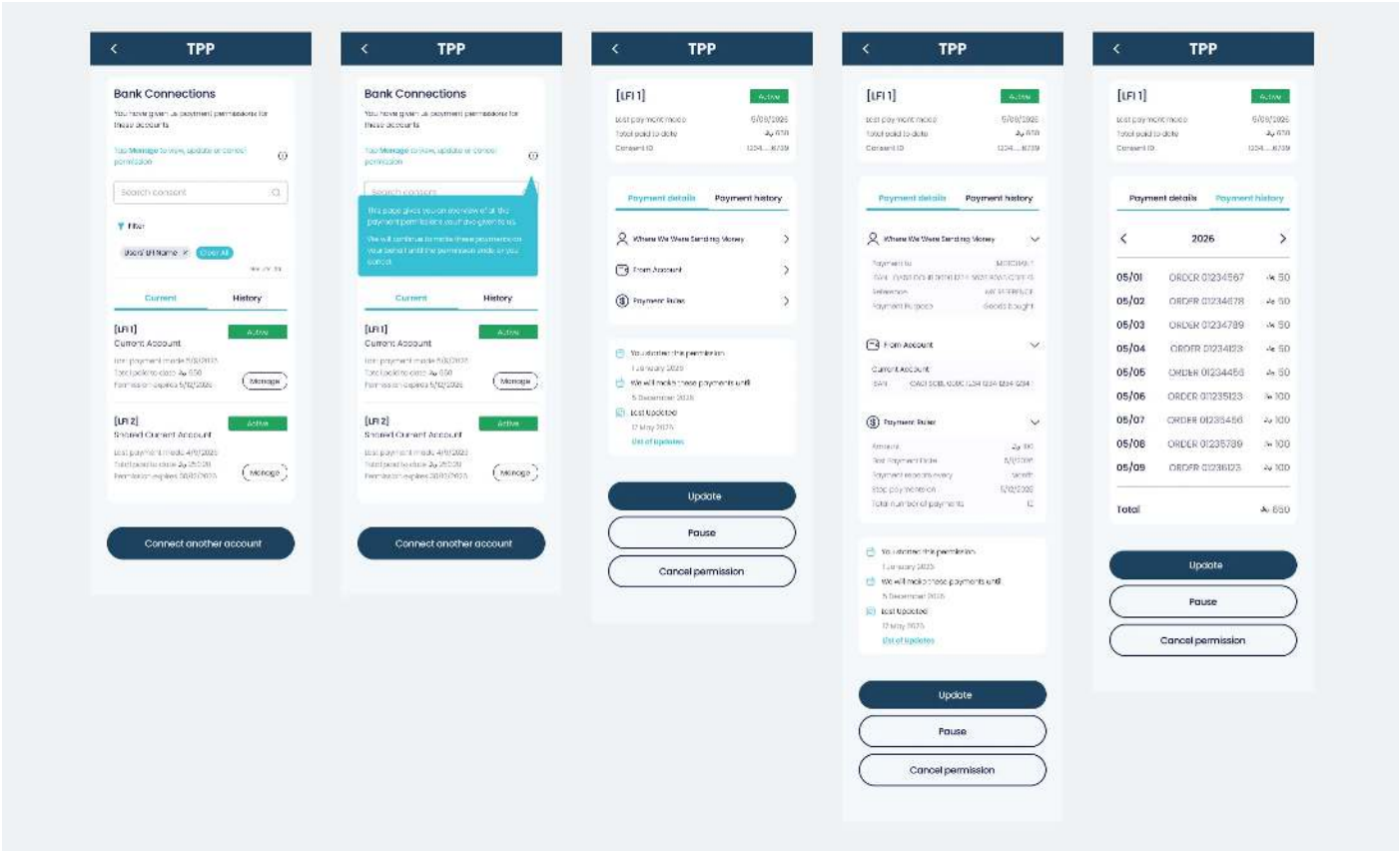


Dashboard with History of Consents and Consent Details

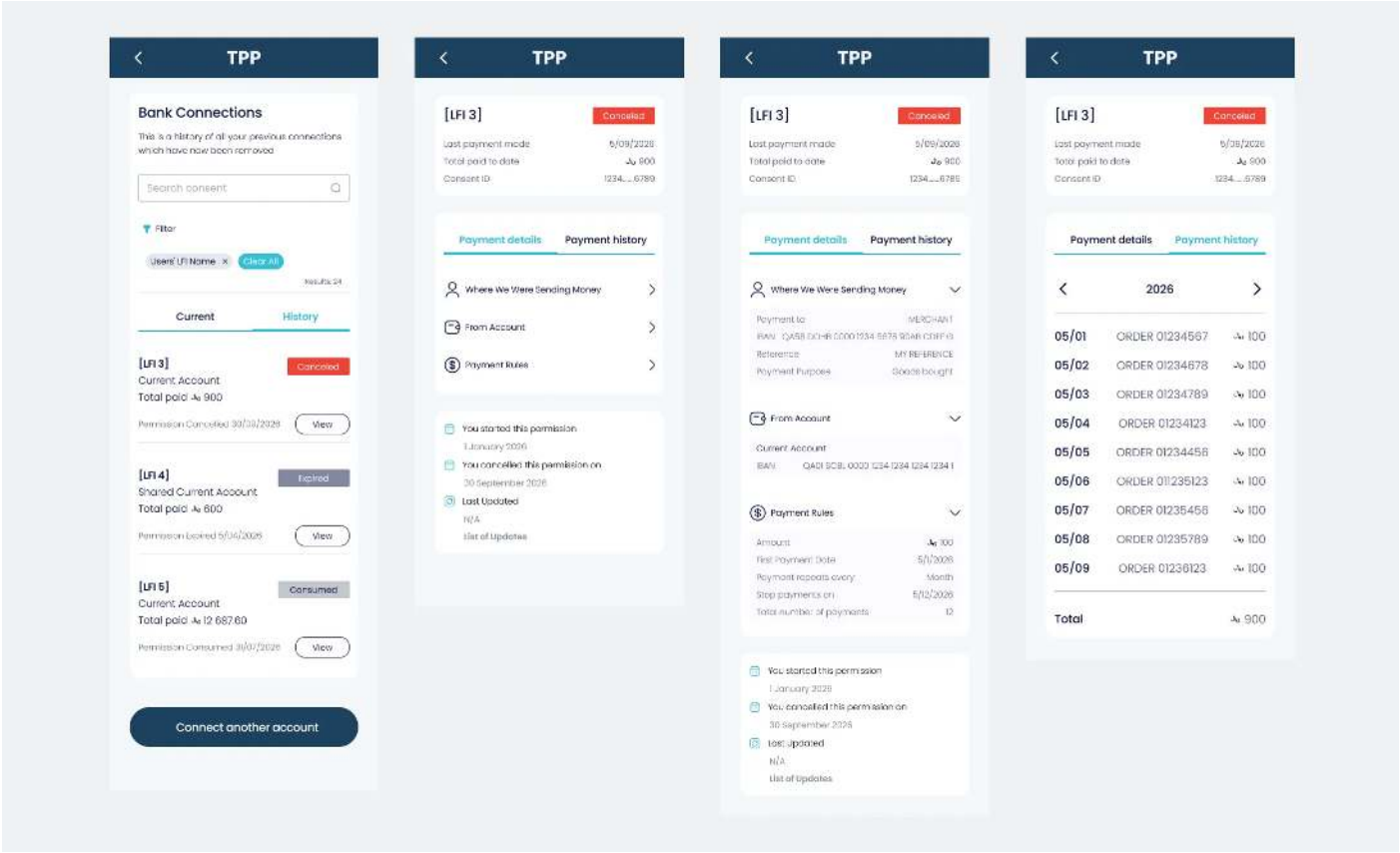


Service Initiation Consent Management Interfaces (CMI)

Dashboard with Active Consents, Consent Details and Payment History



Dashboard with History of Consents, Consent Details and Payment History



Rules & Guidelines

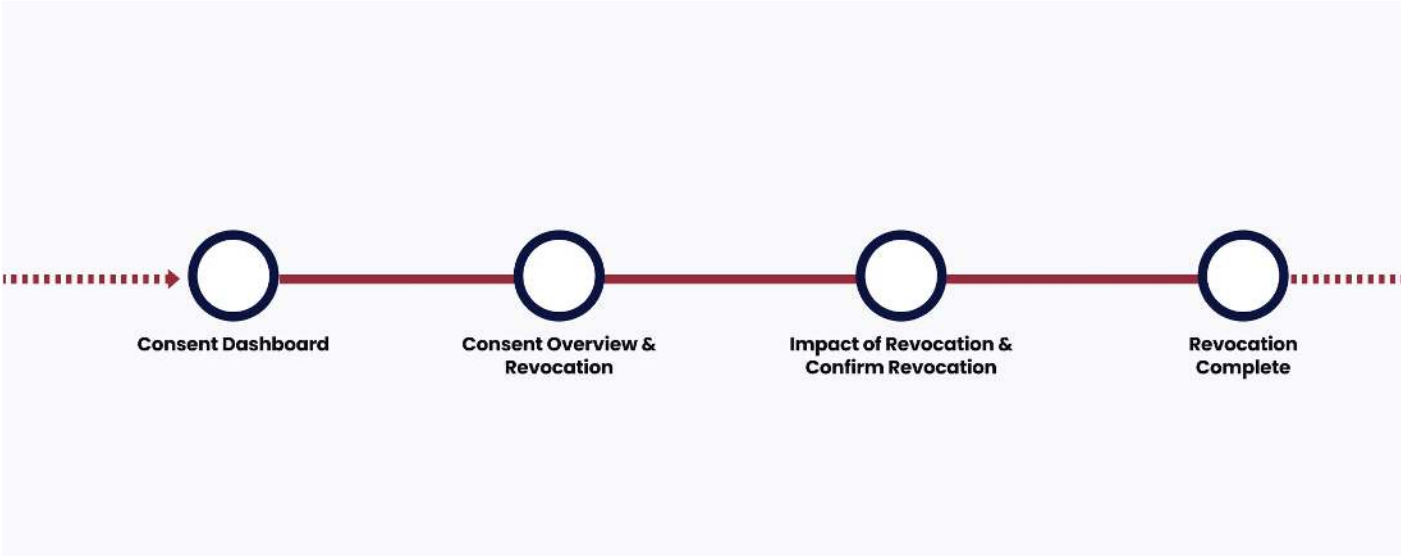
ID	Rules & Guidelines
COMP-169	The User-facing TPPs that offer both Data Sharing and Service Initiation MUST keep these Consent types separated to ensure clarity and prevent any misunderstandings by Users.
COMP-170	To aid clarity whilst providing detailed information when Users need it, Consent Management Interfaces (CMIs) MUST provide an Overview page that lists high-level information for all Consents, and a Detailed page for each Consent.
COMP-171 COMP-172 COMP-173	The User-facing TPPs MUST provide Users with sufficient information to enable them to make informed decisions on the Consent Management Interfaces (CMIs) Overview page. As a minimum, TPPs MUST show the following details: **Consent Summary:** - Consent State (“Awaiting”, “Authorized”, “Rejected”, “Canceled” or “Consumed”) - Account Provider Name (Trading Name/nickname if used) - Account type and the last 4 digits of account number (if provided) - The expiry date for when the Consent will end (*Note: A countdown is an optional element. If provided, it SHOULD be used in addition to an expiry date and not instead of it*) - The date and time of the last occasion when Data Sharing or Service Initiation occurred from the connected account - Warnings or alerts if the Consent is close to expiry. TPPs can define how close to the expiry date they SHOULD start to alert Users depending on their business models.
COMP-174 COMP-175	The User-facing TPPs MUST provide Users with sufficient information to enable them to make informed decisions on the **Data Sharing or Service Initiation** Consent given. These details MUST include all the values which are defined within the specific Data Sharing or Service Initiation Consent, when it was requested. These details are covered within the Rules & Guidelines for each of the areas of functionality covered by the Standard.

ID	Rules & Guidelines
COMP-176	The User-facing TPPs MUST offer functionality (such as search, sort and filter) to enable Users to search for the relevant Consent. This will be of particular benefit as the number of Consents for different LFI accounts given by Users to TPPs increases.
COMP-177	The User-facing TPPs MUST provide the following minimum set of filters in the Consent Management Interface (CMI) Overview page: - Users' LFI Name - Connected Account Number - Consent Type - Consent State - Consent Date
COMP-178	The User-facing TPPs SHOULD allow Users to edit their LFI account names by replacing the account name with a nickname such as "Household Account" or "Holiday Savings Pot."
COMP-179	The User-facing TPPs SHOULD provide extra explanatory text to help Users understand complicated topics like how to cancel their Consent. Using information bubbles helps to keep information manageable.
COMP-180	TPPs offering Service Initiation MUST provide the Service Initiation transaction details within the transaction log depending on the type of Service initiated. For details see the Rules & Guidelines for each of the areas of functionality covered by the Standard.
COMP-181 COMP-182	The Consent ID that is presented at the User-Interface (UI) MUST be user-friendly. TPPs MUST present the first and last four digits of the consent ID: 1234.....6789.
COMP-183	TPPs MUST offer a copy option that allows Users to copy the complete consent ID easily.
COMP-184 COMP-185	The User-facing TPPs MUST provide Users with a Transactions Log for every Service Initiation Consent. The Transactions Log SHOULD be easily accessible to Users by placing it on the Consent Management Interface's Detailed page.

ID	Rules & Guidelines
COMP-186	The User-facing TPPs MUST also provide a cancel (disconnect) button that allows Users to revoke their Consent. This is not applicable for irrevocable single-use Consents for Service Initiations such as Single Immediate Payments.
COMP-187	The User-facing TPPs MUST provide records of all past connections, including any previously granted Consents which had previously been: - Canceled at the LFIs or TPPs - Expired - Authorization Time Window Elapsed - Rejected - Consumed
COMP-188	TPPs MUST use logos and the brand names of the LFIs as they are defined in the Trust Framework Directory.

TPP Consent Management Interface | Consent Revocation Journey

One of the most important pieces of functionality of the Consent Management Interfaces (CMIs) is to cancel a Long-lived Consent. The below journey illustrates the main steps involved:

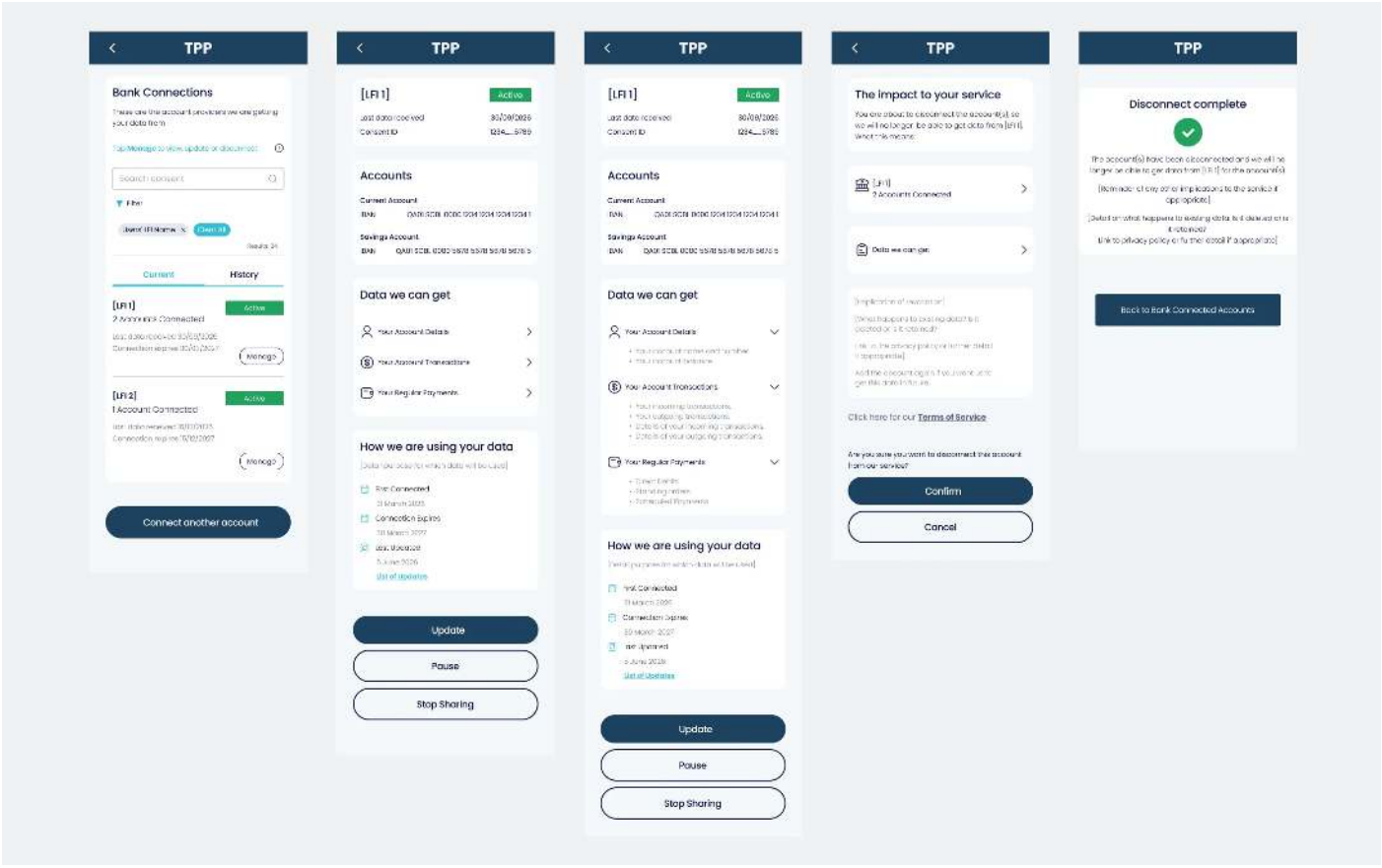


Additionally, it is essential that TPPs make clear to Users what will happen to the already-obtained data in their possession once the Consent is cancelled by the Users.

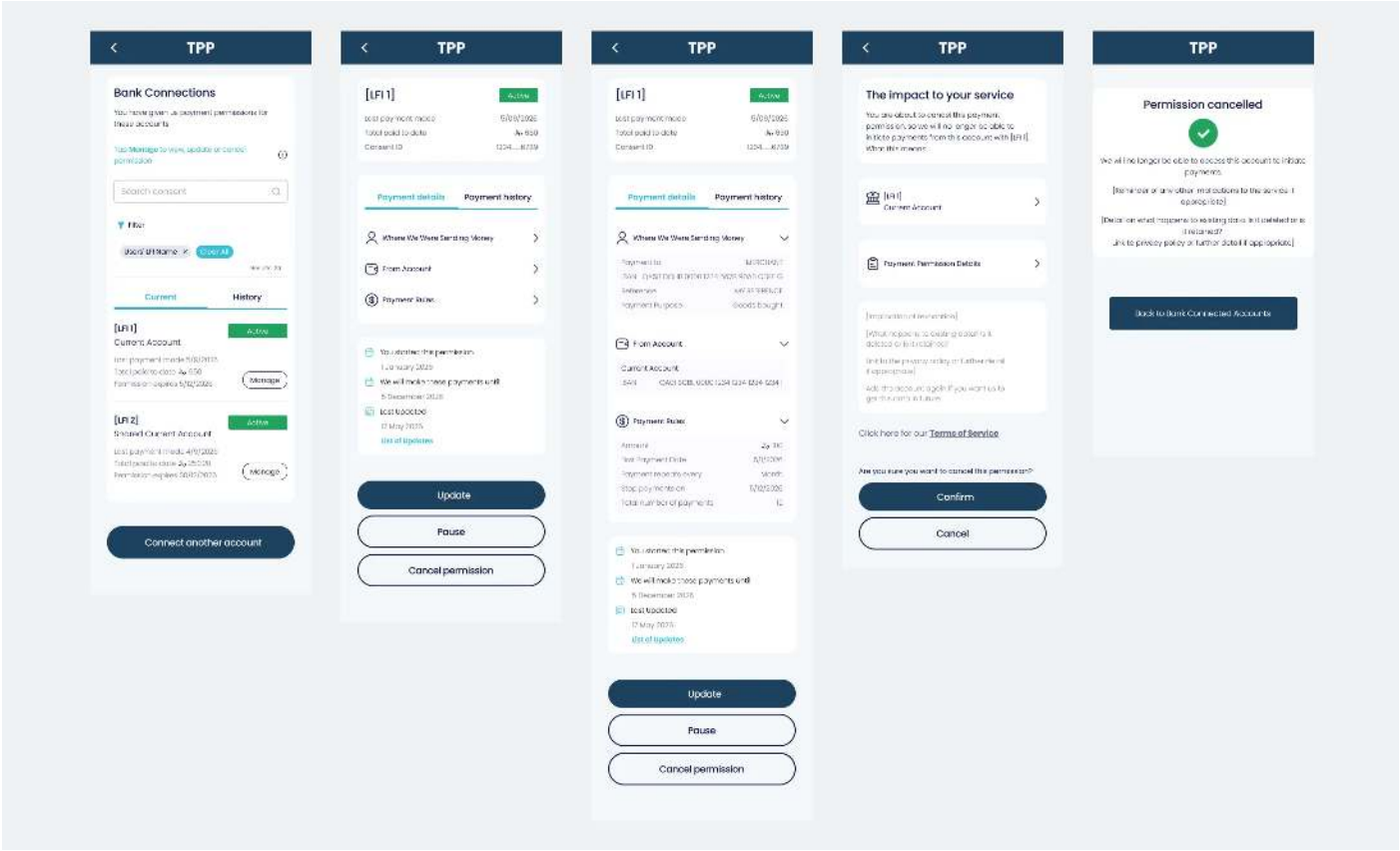
Wireframes

Below are example wireframes of the Consent Revocation journeys for Data Sharing and Service Initiation.

Data Sharing Consent Revocation Journey



Service Initiation Consent Revocation Journey



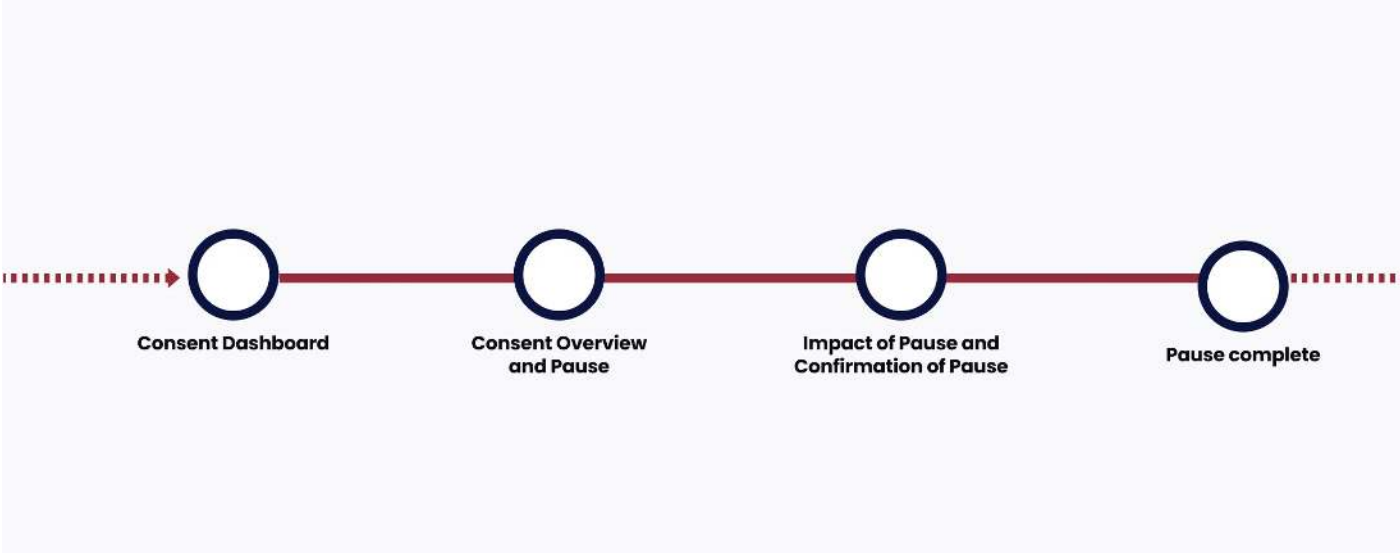
Rules and Guidelines

ID	Rules & Guidelines
COMP-189	The **Consent Management Interface (CMI) Detailed** page MUST allow Users to cancel the Consent they have provided easily and without obstruction or excessive barriers.
COMP-190 COMP-191	Once Users have clicked “Cancel” in the **Consent Management Interface (CMI) Detailed** page, TPPs MUST only provide one page before cancelling/revoking the Consent, which clearly sets out the implications of cancelling to ensure that Users want to proceed with the cancellation. The **Consent Cancellation Confirmation** page MUST be provided after this to confirm that the connection is no longer active.
COMP-192	The User-facing TPPs MUST make the exact consequences of cancelling the Consent clear to Users in the **Consent Cancellation Confirmation** page (i.e., they will no longer be able to provide the specific service to Users).
COMP-193 COMP-194 COMP-195 COMP-196	For Data Sharing Consent, the **Consent Cancellation Confirmation** page MUST confirm what happens to any existing data that TPPs have already retrieved. TPPs MUST ensure that they are fair and transparent about how existing data is processed and that no longer required data is deleted, where appropriate, in accordance to any relevant laws and regulations. For Data Sharing Consent, TPPs MUST present the data at a Data Cluster level and allow Users to expand the level of detail to show each Data Permission. For Service Initiation Consents, TPPs MUST present the conditions and controls related to the Consent and allow Users to expand the level of details to show each Service Initiation Consent details.
COMP-197 COMP-198	The User-facing TPPs MUST inform LFIs that Users have withdrawn their Consents, and the Consent state “Canceled” MUST be reflected to the User in the **Consent Management Interface (CMI) Overview** page.
COMP-199 COMP-200	In the **Consent Cancellation Notification** page (i.e. the last page) the User-facing TPPs MUST inform Users that the connection to their account has been cancelled and no further Data Sharing will be provided or no

ID	Rules & Guidelines
	further Service Initiations are to be performed by the LFIs as the LFIs have withdrawn the Consent given to the TPPs. After TPPs set the state of the Consent to ‘Canceled’, the LFIs are advised to inform Users via their own channels (for example via SMS or a notification on their mobile phone) that TPPs will no longer have Data Sharing or Service Initiation access to their account.

Consent Pause Journey

This capability allows the User to put a temporary pause on the Long-Lived Consent from the TPP and reactivate it again at a later period. The below journey illustrates the main steps involved:



Additionally, it is essential that TPPs make clear to Users any consequences of pausing the Long-Lived consent given their inability to offer functionality dependent on the Long-Lived consent.

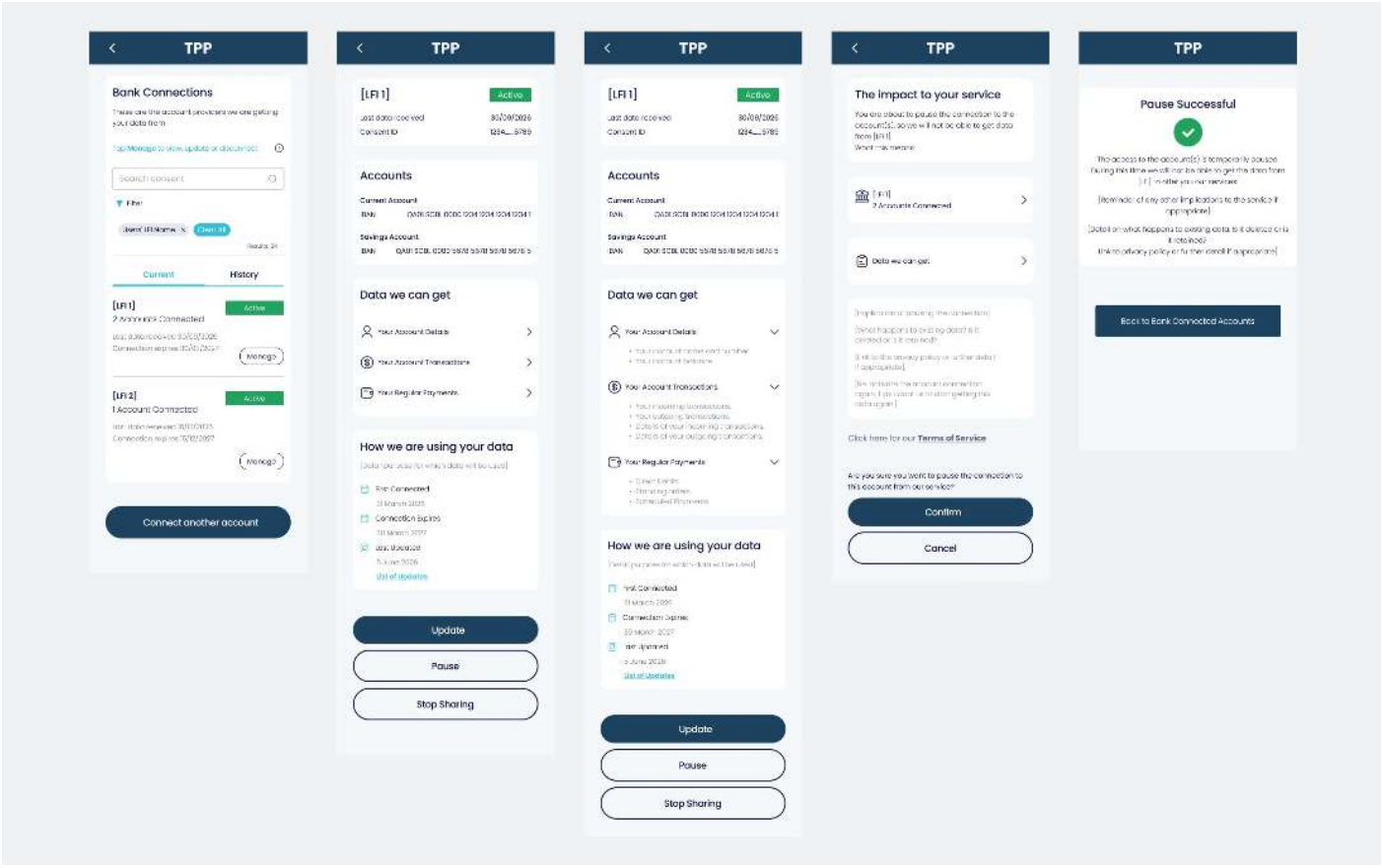
> ℹ️ **Note 1:** The pause functionality is a tool for Users to temporarily stop and re-enable access to their account, without going through the whole journey of re-authenticating with their LFIs. It is functionality equivalent to freezing one of their LFI accounts. Pausing the Consent does not alter the state of the Consent as viewed by the LFI. The Consent stays in the Authorized state and while the Consent appears as Paused in the TPP CMI, the Consent will appear as Active at the LFI CMI.

> ℹ️ **Note 2:** Pausing of the Consent must not be confused with the Consent Suspended state, which is a change of Consent state initiated by the LFI in cases of suspected fraudulent activities or other similar reasons. Please refer to Consent Setup: Consent States for more details.

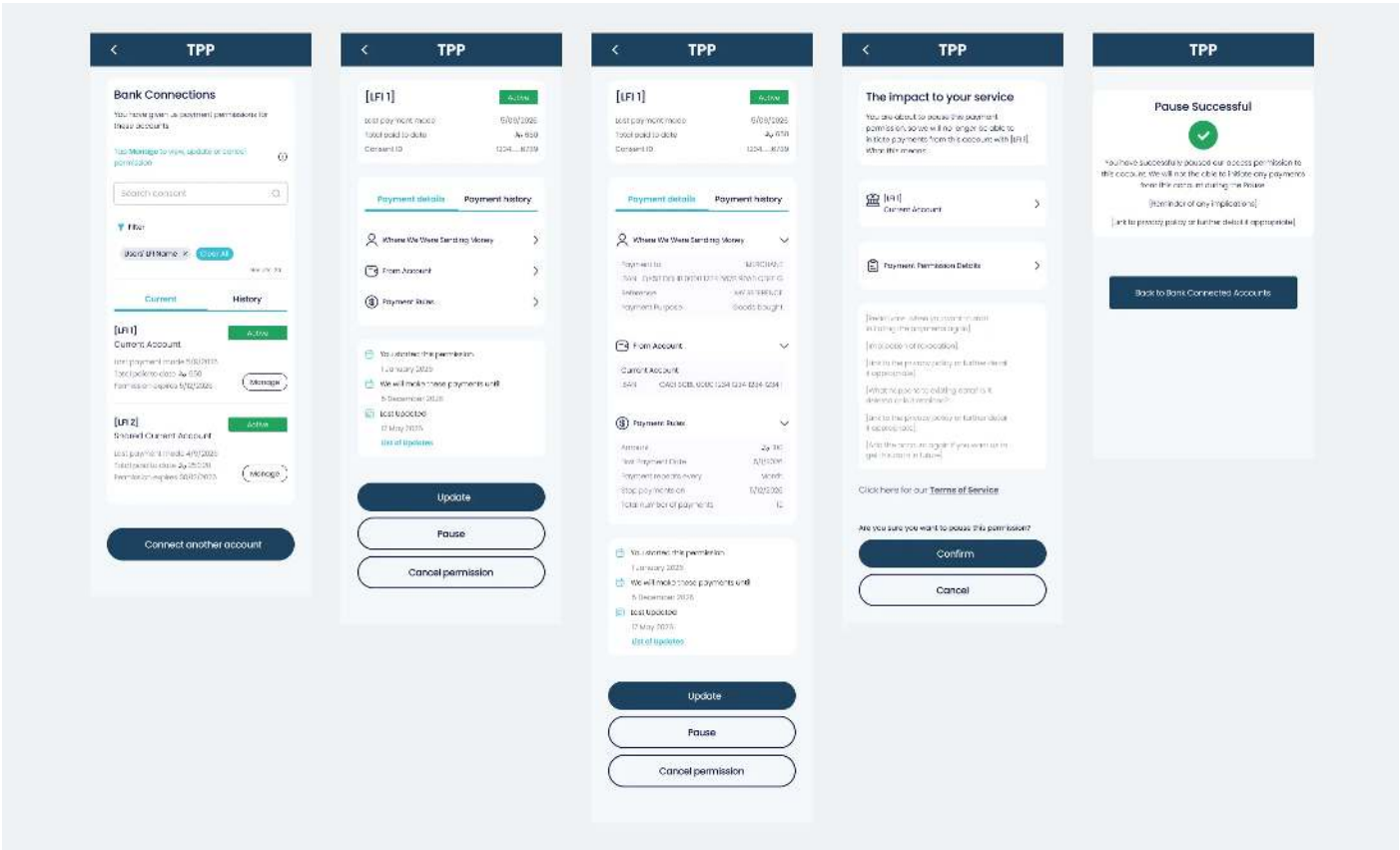
Wireframes

The below are example wireframes of the Consent Pause journeys for Data Sharing and Service Initiation.

Data Sharing Consent Pause Journey



Service Initiation Consent Pause Journey



Rules and Guidelines

ID	Rules & Guidelines
COMP-201	The **Consent Management Interface (CMI) Detailed** page MUST allow Users to Pause the Consent they have provided easily and without obstruction or excessive barriers.
COMP-202 COMP-203	Once Users have clicked “Pause” in the **Consent Management Interface (CMI) Detailed** page, TPPs MUST only provide one page before pausing the Consent, which clearly sets out the implications of pausing to ensure that Users want to proceed. The **Consent Pause Confirmation** page MUST be provided after this to confirm that the connection is temporarily inactive.
COMP-204	The User-facing TPPs MUST make the exact consequences of pausing the Consent clear to Users in the **Consent Pause Confirmation** page (i.e. they will not be able to provide the specific service to Users while the Pause is active).
COMP-205 COMP-206	For Data Sharing Consents, TPPs MUST present the data at a Data Cluster level and allow Users to expand the level of detail to show each Data Permission. For Service Initiation Consents, TPPs MUST present the conditions and controls related to the Consent and allow Users to expand the level of details to show each Service Initiation Consent details.
COMP-207	The User-facing TPPs MUST reflect the “Paused” status to the User in their **Consent Management Interface (CMI) Overview** page.
COMP-208	In the **Consent Pause Notification** page (i.e. the last page) the User-facing TPPs MUST inform Users that the connection to their account has been paused and no further Data Sharing will be provided or no further Service Initiations are to be performed by the LFIs till the pause is active.

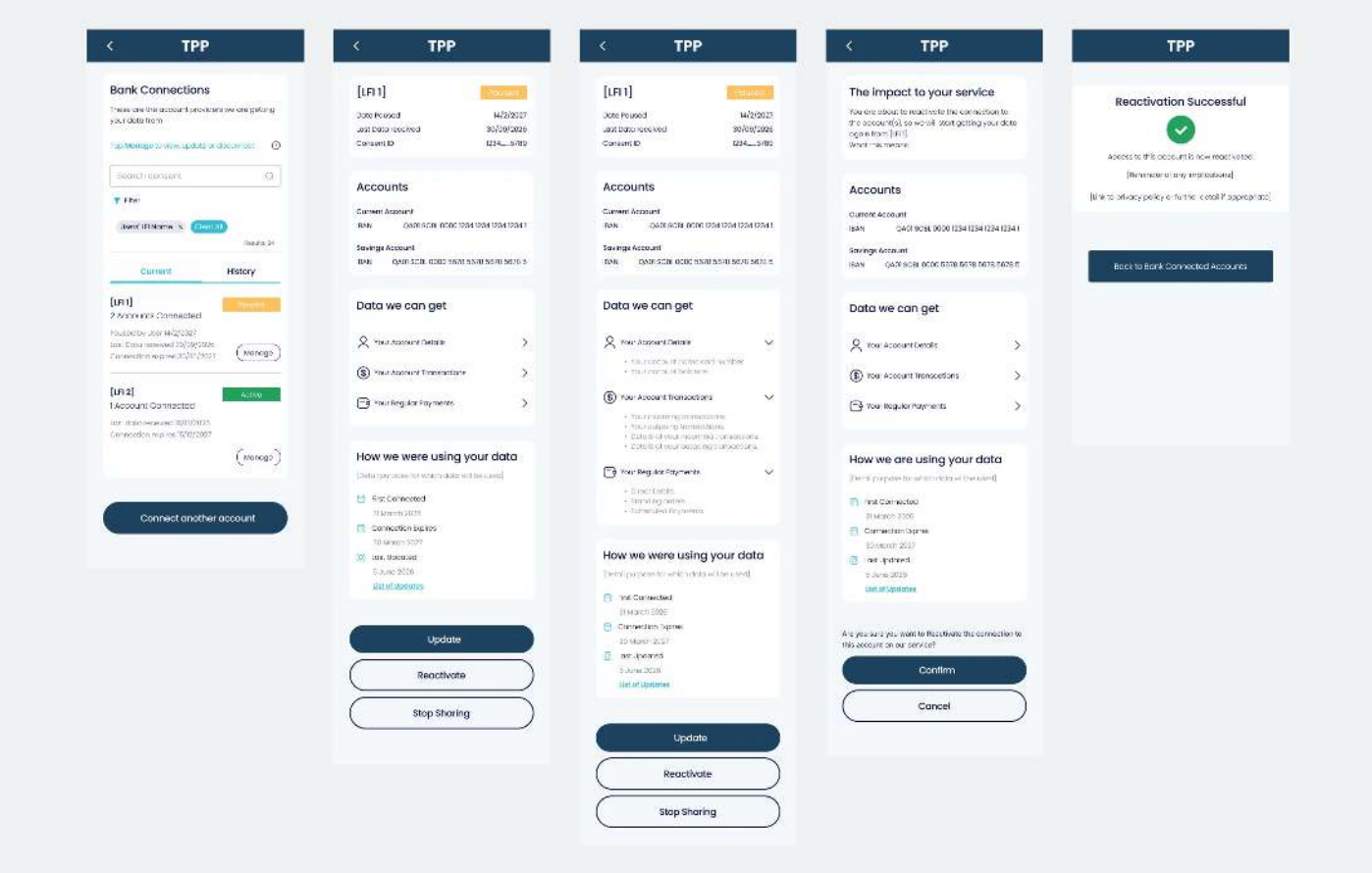
Reactivating a Paused Consent Journey

This capability allows the User to reactivate a previously paused Long-Lived Consent from the TPP's Consent Management Interface (CMI). The below journey illustrates the main steps involved:

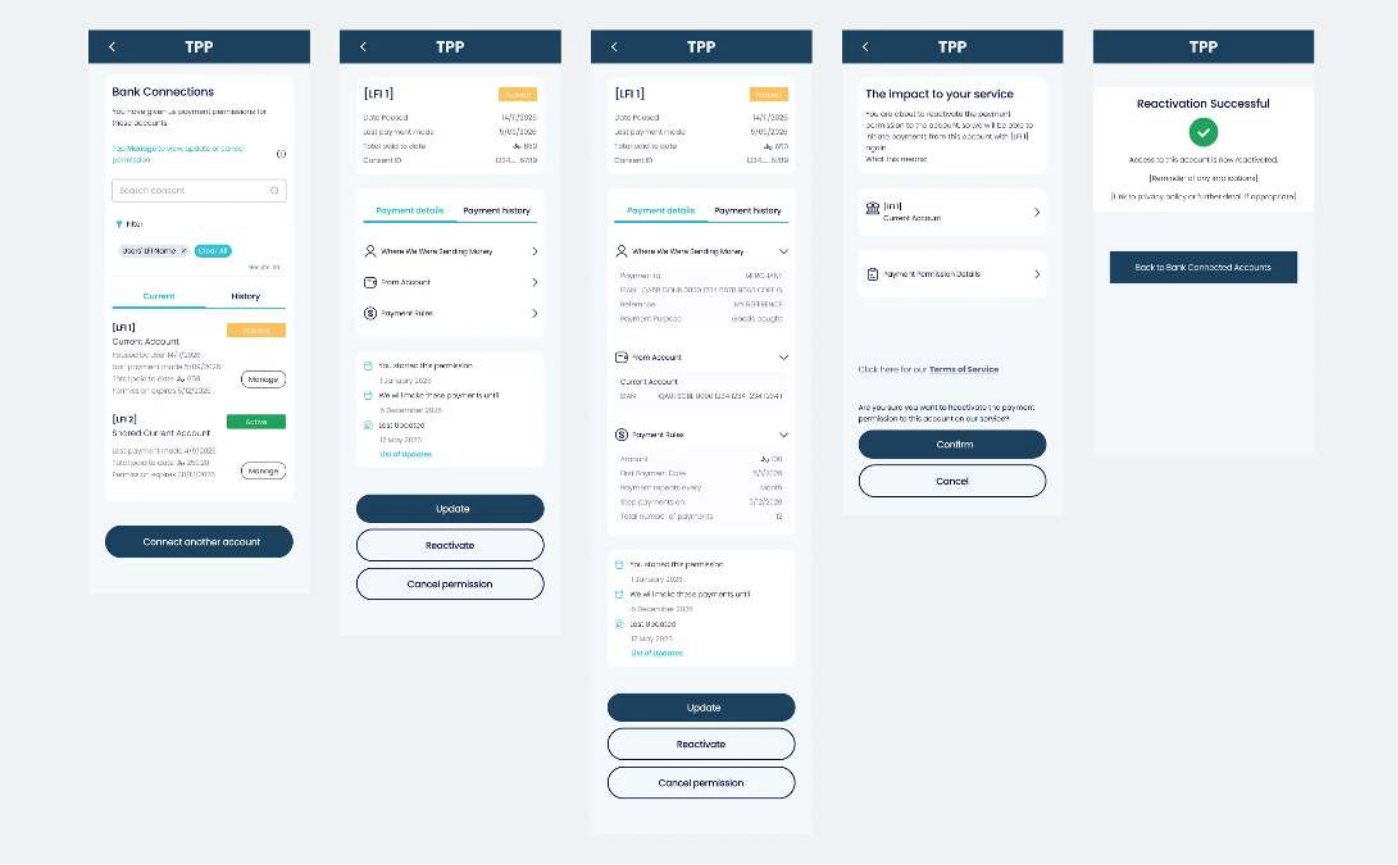
Wireframes

The below are example wireframes of the Consent Reactivation journeys for Data Sharing and Service Initiation.

Data Sharing Consent Reactivation Journey



Service Initiation Consent Reactivation Journey



Rules and Guidelines

ID	Rules & Guidelines
COMP-209	The **Consent Management Interface (CMI) Detailed** page MUST allow Users, easily and without obstruction or excessive barriers, to reactivate the Consent they had previously Paused.
COMP-210 COMP-211	For Data Sharing Consents, TPPs MUST present the data at a Data Cluster level and allow Users to expand the level of detail to show each Data Permission. For Service Initiation Consents, TPPs MUST present the conditions and controls related to the Consent and allow Users to expand the level of details to show each Service Initiation Consent details.
COMP-212	The activation of the consent by the User in the **Consent Management Interface (CMI) Detailed** page MUST not require the re-authentication of the User with the LFI or the Centralized Authentication and Authorization App.

ID	Rules & Guidelines
COMP-213	The Consent state “Active” MUST be reflected to the User on the **Consent Management Interface (CMI) Overview** page.

Consent Update Journey



The above User journey illustrates the steps Users have to follow when updating their Consent details from the TPP CMI Dashboard. The Update process largely depends on the TPPs' use cases and for which one of these the TPPs may allow Users to update their Consent details. However, in certain scenarios where update may be allowed, Users are able to initiate a Consent Update from the TPP CMI Dashboard. For example, Users may want and may be permitted by the TPPs to change their Variable Recurring Payments maximum payment amount, or their total cumulative amount (or value) per control period for OnDemand payments, or even the payment periodic schedule or the control period itself.

It is at the TPPs' discretion whether to allow the Consent updates and also whether to allow Users to propose (or set) new values for the Consent updated parameters or provide Users with a range of values. Additionally, the Consent update process can be used by TPPs to allow Users to renew a consent which is coming close to its validity period.

The QCB Open Finance Standard supports the Consent updates and enables TPPs to offer this capability to Users, if this is in accordance with their use case. The Standard's underlying API Descriptions support the Consent updates with the use of the Base Consent Id which is used to link all Consents which belong to the same group, as update versions of the original Consent.

> ℹ️ Please note that the Consent updates are only applicable to long-lived Data Sharing or Payment Consents.

Rules and Guidelines

ID	Rules & Guidelines
COMP-214	The **Consent Management Interface (CMI) Detailed** page SHOULD allow Users to initiate a Consent update easily and without obstruction or excessive barriers. ⚠️ It is in the TPPs discretion whether they will provide Users the capability to update their Consent details or not, based on the TPPs' supported use cases.
COMP-215	Once Users have clicked "Update" in the **Consent Management Interface (CMI) Detailed** page, TPPs SHOULD provide one page with all the Consent details, clearly highlighting which parameters of the Consent Users are allowed to change. ⚠️ It is at the TPPs' discretion which parameters to allow Users to update and also whether to allow Users to propose (or set) new values for these updated parameters or alternatively provide Users with a range of allowable or proposed values.
COMP-216	For Data Sharing Consents, TPPs SHOULD enable Users to amend one or more of the parameters of the long-lived Consent, as defined in Account Information: Consent Updates.
COMP-217	For Service Initiation Consents, TPPs SHOULD enable Users to amend one or more of the parameters of the long-lived Consent, as follows: - For Multi-Payment Consents, as defined in Multi-Payments: Consent Updates.
COMP-218	TPPs MUST enable Users to provide explicit consent for the forwarding of the new Consent update to the LFIs as per the parameters specified in the consent. TPPs will request Users to authenticate with their LFIs and authorize the Consent update.
COMP-219	TPPs MUST set the status of the original consent to 'Revoked' before forwarding the new Consent update to the LFIs.

The remaining steps for Authentication, Authorization, and Confirmation are as per the existing steps for the equivalent long-lived Consents for Data Sharing (Account Information: Rules & Guidelines and Multi-Payments (Multi-Payments: Rules & Guidelines) as defined in the QCB Open Finance Standard.

TPP Consent Management Interface | Consent Management Interfaces (CMIs) when customer-facing service provider and TPP are different entities

COMP-220 If there are customer-facing service providers (e.g. Merchants) who are not TPPs but have commercial relationships with TPPs, the TPPs MUST ensure that the Consent reflects this information correctly so that it can be displayed accurately to Users on the LFI Consent Management Interfaces (CMIs).

These entities are referred to as customer-facing service providers, as they are providing the end service to the end-Users, whereas TPPs in these cases are only undertaking the Data Sharing or Service Initiation activities. This could occur in merchant journeys for example, where a merchant contracts with a TPP to provide Variable Recurring Payments as a payment option on their platform.

Examples

TPP Trading Name (Client Name in the APIs)	Registered Legal Entity Name (Company Name/Organization Name)	Customer-facing entity name ('via' field in the APIs)	What to display on Dashboards
ABC Trades	ABC Company Ltd		ABC Trades
ABC Company Ltd	ABC Company Ltd		ABC Company Ltd
ABC Company Ltd	ABC Company Ltd	OBO Ltd	OBO Ltd via ABC Company Ltd
[TPP Trading Name]	[TPP Company name]	[Merchant Trading name]	[Merchant Trading name] via [TPP Trading Name]

User Experience Principles | Introduction

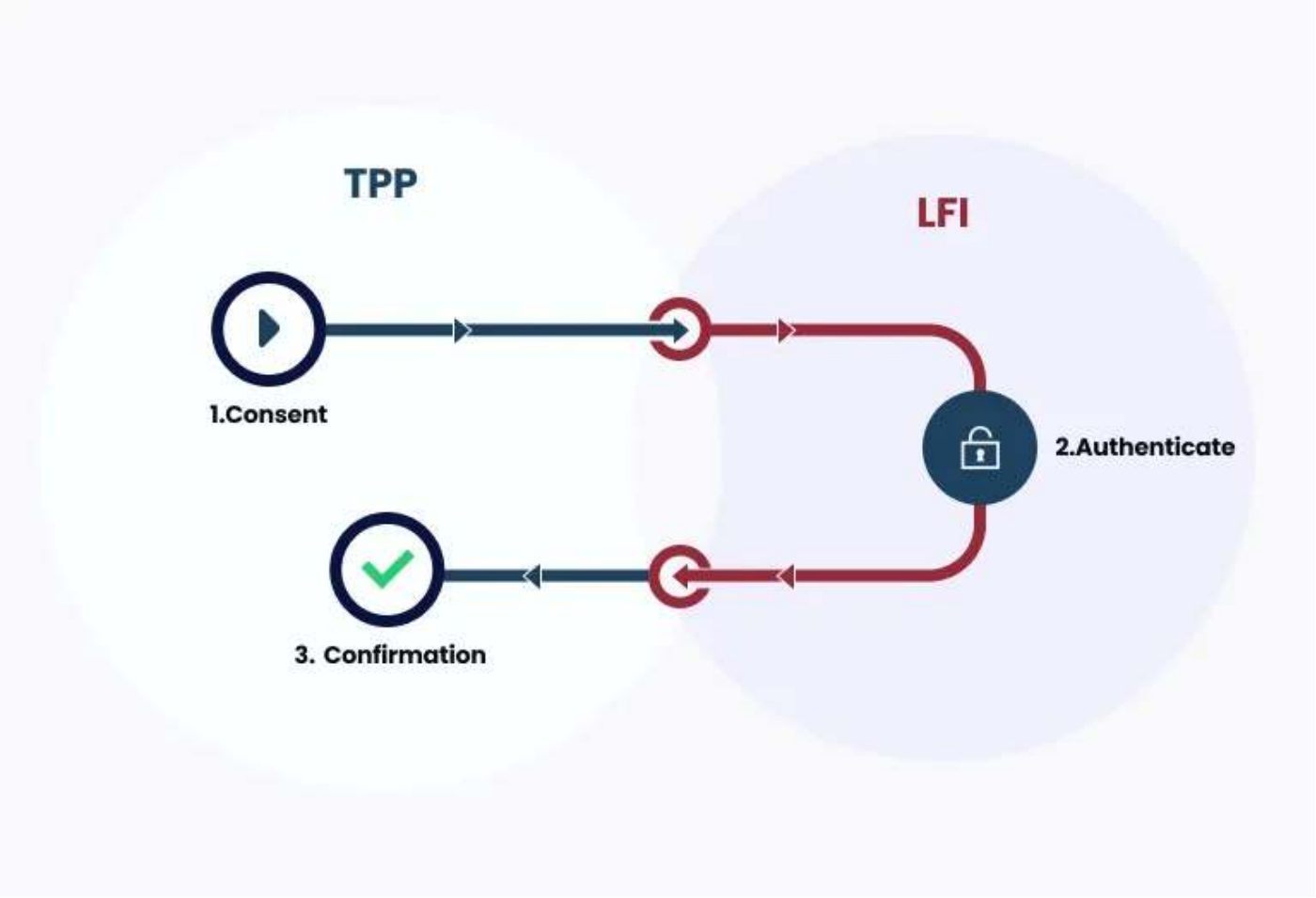
This section lays out the principles of informed decision-making, providing Users with well-designed experiences (using the principles of control, speed, transparency, security and trust). It also includes information that should be considered to support the protection of vulnerable Users.

COMP-221 LFIs and User-facing TPPs SHOULD design user journeys equivalent to or better than the journeys described in the Standard to deliver the best possible experience and outcomes.

Open Finance products and services MUST therefore enable:

- **Informed decision making**: [COMP-222] User journeys MUST be intuitive and information MUST be easily assimilated to ensure informed user decision making.
- **Simple and easy navigation**: [COMP-223] There MUST be no unnecessary steps, delays or friction in the user journey.
- **Parity of experience**: [COMP-224] The experience available to a User when authenticating a journey via a User-facing TPP SHOULD involve no more steps, delays or friction in the user journey than the equivalent experience they have when interacting directly with their LFI.
- **Familiarity and trust**: [COMP-225] The User MUST only need to use the login credentials provided by the LFI.

User Experience Principles | Customer Journey



At the core of all Open Finance customer journeys is the mechanism by which the User gives consent to a TPP to access their account held at their LFI for Data Service Requests or Service Initiation requests.

In general, simplified terms, a consent request is initiated in the TPP domain (step 1). The User is then directed to the domain of its LFI for authentication (step 2). Then, once authentication is complete, the LFI will be able to respond to the TPP's Data Service Requests or Service Initiation Requests and directs the User back to the TPP for confirmation and completion of the journey (step 3).

User Experience Principles

COMP-226 The Open Finance user experience MUST ensure informed decision-making while remaining understandable, intuitive and effective. [COMP-227] The user experience MUST be shaped and positioned into content and functionality that communicates and facilitates purpose, intent and relevance.

COMP-228 This is especially true in a transactional context where Users MUST know and understand at all times:

- Where they are in a specific process (and what they should expect from that process).
- Where they have come from.
- What options, actions or steps they have in front of them (if any).
- The (implicit) consequences of taking those actions or next steps.
- An unambiguous signal, feedback and/or response, once that action is taken.

It is essential to move beyond the pure mechanics of the transactional process and into a meaningful, supportive and trusted experience that directly addresses the User's needs, goals and concerns. This can be achieved in the way a transaction is structured but also in how it is expressed, designed for, and organized around a range of fluctuating human needs.

A series of "experience principles" are outlined here that can be, through careful design, baked into a process or transaction, and dialed up and down where certain interactions become more critical.

These experience principles are deeply user-centered; they are used to drive and focus design and User Experience (UX) decisions, i.e. what kind of widget, interaction, font, color, technology, UX and User Interface (UI) best serves the aspirations and requirements of the business but also meets the needs of the User in simple, effective ways.

It has been shown that in other countries there are certain recurring themes that Users deeply care about or are worried by. [COMP-229] To promote engagement and understanding and ensure adoption, these MUST be addressed, to varying degrees, within each of the Open Finance user journeys described in the Standard.

To support and achieve the goal of creating trust, these themes have been aggregated and synthesized into several driving experience principles for Open Finance. These principles underpin the range of core journeys and key User interactions described throughout the Standard.



An essential component of trust-building is the provision of Consent Dashboards by the User-facing TPPs and the LFI. Dashboards encapsulate the user experience principles below and provide a role in reassuring Users about the security of Open Finance-enabled services. In terms of the experience of using a Dashboard, the principles that apply to an authentication journey can be extended to support simple and easy navigation by enabling informed decision-making. Please refer to the Dashboards section to read more.

Control

- The introduction of any kind of new transaction, product or service – especially online – can create an opportunity for deeper engagement. However, it can also create barriers through poor implementation. From a User perspective, this is often about a perceived sense of control.
- If Users feel they understand what is going on in a process, and can make informed decisions and choices on their terms – including recourse to change their minds – it provides a sense of ownership and control over what is happening. In a transactional context, where money and data are potentially at stake, getting this right is essential.

- For Open Finance, control comes from providing the right tools and clarity of information at the right time (e.g. knowing that they can view and revoke consents given when they feel it is appropriate to do so).
- COMP-230 User-facing TPPs and LFIs MUST consider how they provide this sense of ownership and specific optionality throughout, enabling Users to feel this is a process they are both choosing and in charge of.

Speed

- COMP-231 The speed MUST be appropriate to Users and the journey they are undertaking. Convenient, speedy and intuitive design is a question of execution and interaction.
  - In the transactional context, anything that seems more time-consuming or onerous than Users are used to is going to represent a hurdle to adoption. We have to manage and optimize each interaction, as well as a hand-off between systems for speed, clarity and efficiency, but without sacrificing the principles of security and control.
  - In addition, we have to be mindful that the speed of transaction or interaction is not necessarily about the “fastest possible” experience. [COMP-232] As we have indicated, we MUST support informed decision-making through comprehension and clarity, allowing Users to, above all, move at a pace that suits them.
- COMP-233 User-facing TPPs and LFIs MUST ensure that Open Finance user journeys remain flexible enough to support different User contexts, expectations and situations and – critically – avoid any unnecessary friction in the completion of any journey.

Transparency

- Transparency of choice, action, and importantly the consequences of actions or sharing of account information, is crucial to promoting the benefits of Open Finance, creating engagement and supporting adoption.
- In new transactional scenarios where Users are being encouraged to share their financial information, this is critical. It is not only about communicating the benefits of a new service, but also being explicitly clear on what is required from Users, why it is required, and for what purposes. [COMP-234] Users MUST be able to make an informed decision and, in turn, understand the consequences of that decision.
- Sharing information is seen as unavoidable, and a trade-off for convenience and benefits. And while this is a great opportunity for User-facing TPPs and LFIs, the value exchange for the consumer needs to be explicitly clear.
- At the same time, we do not want to overburden Users or weigh down the business opportunity with excessive explanations. Transparency is about providing progressive levels

of information, in plain language, that inform and support User decisions.

Security

- In the context of Security, the key concerns for Users are fraud, which everyone understands, and the use of their account information and how it is protected.
- Explicit clarity and reassurance will be required about data definition, usage, security and, above all, protection.

- > COMP-235 As a new service, all security messaging SHOULD be clear and reassuring in tone, but not alarmist.

Trust

- Users are generally aware of the risks of sharing account information and, as expected, some types of Users, particularly older demographics, may initially express cautiousness and nervousness.
- It is therefore critical to establish and reinforce trustworthiness – trust in the service provider, trust in the transactional process and trust in the role and relationship with their LFIs.
- The principles of control, speed, transparency and security combine to create a trusted environment for Users.

- > COMP-236 User-facing TPPs and LFIs MUST consider, engender and promote values of trust through every part of their Open Finance user journeys, to foster understanding, acceptance and adoption of new innovative products and services.

Design to Maximize Transparency to Users

User-facing TPPs MUST:

- COMP-237 Clarify rights and responsibilities describing how the relationship works.
- COMP-238 Investigate and answer additional questions raised.
- COMP-239 Clarify the purpose of data collection and who is responsible for the use.
- COMP-240 Offer detailed explanations for why specific data groups are collected and used.
- COMP-241 Clarify what happens after the period ends.
- COMP-242 Clarify data use and removal after consent is revoked (e.g. what happens to past, present, and future data).
- COMP-243 Ensure all services are available at a minimum in English or Arabic.
- COMP-244 When providing alternative language support, ensure that there is an accurate translation of text.
- COMP-245 When providing alternative language support, ensure the User Experience adheres to best practices in terms of the adjustments for right-to-left or left-to-right text.

Design to Maximize User Trust

User-facing TPPs MUST:

- COMP-246 Convey strong data security. When required, use fingerprint and face recognition to help drive security and trust.
- COMP-247 When working with trusted bank brands, ensure to build in cognitive friction.
- COMP-248 Add support/contact details.
- COMP-249 Specify groups (or types) of 3rd parties, and how they support the service being offered.
- COMP-250 Clarify the intentions and benefits of historical data sharing.
- COMP-251 In all journeys, use the User's bank logos/icons rather than a generic one to support User familiarity.
- COMP-252 Use simpler, more direct, and less official-sounding language where possible.

Maximize the User's Sense of Control

User-facing TPPs MUST:

- COMP-253 Visually foreshadow stages of the process in advance.
- COMP-254 Give Users control of what data is shared and time frames around this.
- COMP-255 Describe the benefit of sharing with non-essential 3rd parties, but offer the option to de-select them.
- COMP-256 Describe how to revoke consent to access data (and make it easy).
- COMP-257 Consider navigation with less scrolling and more swiping.
- COMP-258 Allow Users to de-select specific data groups if a reduced service can be offered without them.

User in Control

Many Users are not yet familiar with Open Finance-enabled propositions. Users may not review information sufficiently and may make decisions that they might later wish to reconsider. It is extremely important to minimize unnecessary information and process, and then to package only the most important information in an easily understandable, intuitive manner so that Users can actually assimilate the information and therefore make better-informed decisions.

Protection for Vulnerable Users

Users deemed as vulnerable, or in vulnerable circumstances, may be significantly less able to effectively manage or represent their interests than the average User, and more likely to suffer detriment. This may take the form of unusual spending, taking on unnecessary financial commitments or inadvertently triggering an unwanted event. Any User can become vulnerable at any time in their life, for example through serious illness or personal problems such as divorce, bereavement or loss of income. In these scenarios, it is important to emphasize informed decision making, with appropriate steps and information in online experiences to help those with mental health problems make informed decisions, understand the potential consequence of their decisions, or even deter a particular course of action.

COMP-259 Entities have a particular responsibility to identify and protect vulnerable Users and SHOULD pay attention to possible indicators of vulnerability at a holistic level and have policies in place to deal with Users where those indicators suggest they may be at greater risk of harm. [COMP-260] For those Users identified as vulnerable, the policies applied SHOULD be implemented at the User level, not at the transaction level, nor specifically to Open Finance, just as is the case for vulnerable Users using other products provided by the LFI.

LFIs SHOULD take the following steps for vulnerable Users using products that make use of Open Finance:

- COMP-261 Provide support for vulnerable Users by incorporating information from the Open Finance channel. [COMP-262] LFIs SHOULD consider this issue holistically, treating Open Finance as they would any other User channel. [COMP-263] The LFI, having insight into user behavior, is well placed to provide the appropriate support, recognizing that no single Open Finance user journey SHOULD trigger vulnerability flags to the LFI.
- COMP-264 Provide useful and informative consent dashboards within the LFI's domain that give vulnerable Users the control they need over their financial affairs and access to their account information. [COMP-265] Vulnerable Users SHOULD be able to see full details of all the access granted to the User-facing TPP, the data shared, the expiry date, and have the ability to revoke their consent.
- COMP-266 It is suggested that a provision SHOULD be made in the LFI's consent dashboard enabling Users to switch on a summary information step as an opted-in choice.

Useful Elements in the User Journey

Many Users are prone to skim through the information presented to them when setting up online products because the information is not well presented. In their desire to achieve the promised benefit, insufficient notice is taken of the implications of their actions, or the terms and conditions. It is commonplace to discover, once they have completed the user journey, that they cannot spontaneously describe what they have just agreed to. It has been shown that a better understanding can be achieved by carefully designing the user journey. The solution is about the effective, intuitive presentation of information, and not about introducing steps to slow the User down or repeating information. The following methods are the most effective:

- Effective messages and navigation are appropriate to the redirection screens when the User is redirected from the User-facing TPP to the LFI, and then again when the User is redirected back from the LFI to the User-facing TPP. For a User that has granted consent to the User-facing TPP, the redirection screen creates a clear sense of separation as they enter the LFI's domain where they authenticate before clearly being passed back to the User-facing TPP. This provides a familiar and trusted experience to the User and signposts the User's journey from one domain to the other.
- Providing useful information presented in an intuitive and easily consumable way. The principle here is to ensure that the information that the User is presented with is kept to a minimum. If it is unavoidably necessary for the TPP to convey more complex information, it is more likely to be read and understood when presented as a series of smaller amounts of information across more than one screen. This is a much more effective method than the use of a single text-heavy screen.
- Providing supplementary information at specific points in the user journey is useful, helping the User to understand the process as well as ensuring comprehension of a product or offer and its implications. If executed well, it will enhance the user journey and does not lead to an increased propensity to drop-off.

Unhelpful Elements in the User Journey

Superfluous information, poor or confusing choice of words, repetition, large amounts of text, too many steps or avoidable delays in the user journey can lead to frustration, an even greater tendency to skim, and ultimately increase User drop-off. The following unhelpful elements MUST be avoided:

- COMP-267 A User authentication journey that takes too long and requires the use of separate devices such as one-time password generators, especially if applied multiple times in the user journey.
- COMP-268 Where there are fewer screens but a significant amount of text on the screen. This is particularly evident when this requires Users to scroll up and down the screen to progress the user journey.
- COMP-269 Providing superfluous information that does not add to the User's understanding or trust, especially when presented in a separate step or screen.
- COMP-270 Delays such as slow loading times, as well as web pages or apps that have not been effectively debugged, and unexpected crashing of web pages or apps.
- COMP-271 Inappropriate use of language, particularly language which may create a level of concern, uncertainty and doubt when going through the user journey.
- COMP-272 The use of language that is too long, complex or legalistic to be easily understood when going through the user journey.
- COMP-273 Asking for the same information twice, and asking for information for which there is no obvious purpose, e.g. replaying the consent to the User that was granted to the TPP, or asking for a PIN when it is not needed.
- COMP-274 Forcing the User to open a new browser window during the user journey, and having to toggle between screens to progress.
- COMP-275 Introducing the requirement for a User to input information that they do not readily have to hand, such as unique User reference numbers.
- COMP-276 Requesting input of information that could reasonably be expected to be pre-populated once the User has authenticated.
- COMP-277 Failing to differentiate between new Users and experienced regular Users who may want to shorten the user journey without exposing themselves to risk.

Other Rules for User Journeys

- COMP-278 The User-facing channel SHOULD be optimized for various devices, such as desktops, tablets, and smartphones, to ensure accessibility.
- COMP-279 The User-facing channel MUST ensure all services are available at a minimum in English or Arabic.
- COMP-280 TPPs MUST provide Users with a straightforward and user-friendly interface that offers clear instructions, helpful tips, and easy-to-understand labels throughout the whole process.
- COMP-281 TPPs MUST provide easy-to-access options for User support, such as chat, email, and phone, to ensure that Users can easily access help and receive prompt assistance.
- COMP-282 TPPs MUST provide their trading name to the Users and the LFIs. [COMP-283] If applicable, they must also disclose the names of other parties they offer their services to.
- COMP-284 TPPs MUST ensure that Users clearly understand the different elements of the Consent by providing a concise explanation and illustrative example for each element.
- COMP-285 LFIs and TPPs MUST implement their customer experience screens in line with what is provided in each Customer Experience section of the Standard for the relevant functionality. This includes colors, branding, spacing and component design.
    - The assets in the folder are provided to ensure alignment with the customer experience screens provided in the Standard: QCB Customer Experience Assets
- COMP-286 LFIs and TPPs MUST implement their customer experience screens in line with the QCB Brand Guidelines
- COMP-287 LFIs and TPPs MUST NOT make use of multi-press buttons whereby a User is required to press a button more than once for the same intended action.
- COMP-288 LFIs and TPPs MUST NOT create friction in the form of additional requests for confirmations.
- COMP-289 LFIs and TPPs MUST NOT introduce any additional friction, intermediate screens, or supplementary steps when a User is redirected from the TPP to the LFI for authentication and consent authorisation.
    - COMP-290 When the redirection occurs, the User MUST be presented directly with the LFI's authentication (log-in) interface in order to complete the authentication and commence the consent authorisation journey.
    - COMP-291 Any additional confirmation pages, informational screens, or workflow steps introduced prior to the LFI log-in page are NOT permitted, unless explicitly required by these Standards.

- COMP-292 The User MUST go through an MFA with the LFI only once before they authorize the consent. There should be no additional authentication required.
    - COMP-293 For Bank Service Initiation consent authorisations, once the consent has been authorised, the User MUST complete a payment authentication step in accordance with the LFI’s Strong Customer Authentication (SCA) requirements.

API Security | Overview

This section contains the security standards that apply to the QCB Open Finance APIs, and the material supporting them.

The standards define how an accredited institution (Data Receiver) authenticates itself to access protected resource endpoints at a Data Provider, and how it receives authorization from a Customer to access their data.

- Security Profile - FAPI: the QCB profile of the FAPI 2.0 Final Security Profile, covering authorization server and client requirements, algorithms, and the consent and authorization mechanism.

API Security | Security Profile - FAPI

1. Introduction

The QCB Open Finance Financial-grade API, a secure OAuth profile, is designed to offer detailed implementation guidelines for enhancing security and interoperability in QCB Open Finance APIs. This Profile, a profile of the FAPI 2.0 Final Security Profile, aims to streamline optionality within the framework. Additionally, it incorporates specific features to address the Consent and Authorization Requirements pertinent to QCB Open Finance use cases

2. Security Profile

2.1 Authorization Server Considerations (Open ID Provider)

The Authorization Server shall support the provisions specified in the 5.3.2 clause of the FAPI 2.0 Final Security Profile.

In addition, the Authorization Server:

1. **Shall** only issue sender-constrained access tokens using mTLS as described in RFC8705.
2. **Shall** ensure that the access token expiry is no longer than 10 minutes.
3. **Shall** authenticate the confidential client using `private_key_jwt`;
4. **Shall** cache clients `jwks_uris` for a maximum period of 20 minutes.
5. **Shall** support OAuth 2.0 Rich Authorization Requests (RAR) as a method of conveying information about the End User Authorization Request.
6. **Shall** only process authorization requests if the `type` specified in the `authorization_details` attribute matches any of the types listed in the `authorization_detail_types` metadata attribute of the Relying Party.
7. **Shall** use the `query` parameter as the mechanism for returning authorization response parameters as the response mode, which is the default for the `code` response type.
8. **Shall** advertise support for all signing, authentication mechanisms, and standards required to support this Specification on its OpenID Connect Discovery (`.well-known`) endpoint.

9. **Shall** support and require signed request objects according to the OAuth JWT-Secured Authorization Request (JAR) [RFC9101] at the PAR endpoint [RFC9126].

10. **Shall** require the request object to contain an `exp` claim that has a lifetime of no longer than 10 minutes after the `nbf` claim and an `nbf` claim that is not more than 10 minutes in the past.

11. **Shall** only rely on the parameters included in the signed request object passed via the `request_uri` parameter except for the `client_id`parameter.

12. **Shall** set the response header `x-fapi-interaction-id` to the value received from the corresponding client request header, or to a RFC4122 UUID if the request header was not provided, to track the interaction.

13. **Shall** issue `refresh_tokens` with an expiration period set to the same length as the expiration period of the longest consent with which the token can be used. For consents that do not have an expiry period, the authorization server **shall** issue `refresh_tokens` without an expiration period, or, if not technically achievable, with an expiration period greater than 100 years.

14. **Shall** enforce that all redirect URIs passed by the Relying Party are pre-registered in the client metadata as provided by the Trust Framework, in accordance with the Registration Framework, and shall reject any requests containing unregistered redirect URIs

15. **Shall** advertise mtls_endpoint_alias's including Token Endpoint, Pushed Authorization Request Endpoint (PAR) and all other endpoints where an authentication credential is presented

16. **Shall** honor and enforce all clients to support and utilize mtls endpoints as defined by client metadata use_mtls_endpoints_alias's and follow the orientations defined on RFC8705

17. **Shall** validate that the commonName (CN) of the client certificate used for mTLS matches the Software Statement ID obtained on the client registration.

2.2 Client Considerations (Relying Party)

The Authorization Server shall support the provisions specified in the 5.3.3 clause of the FAPI 2.0 Final Security Profile.

1. **Shall** support `private_key_jwt` as a token endpoint authentication mechanism (client authentication method).

2. **Shall** include the `request_uri` parameter in the authorization request as defined in the 6.2 section of OpenID Connect Core specification.

3. **Shall** send all parameters inside the authorization request's signed request object.

4. **Shall** support and require signed request objects according to the OAuth JWT-Secured Authorization Request (JAR) [RFC9101] at the PAR endpoint [RFC9126].

5. **Shall** send an `exp` claim in the request object that has a lifetime of no longer than 10 minutes;

6. **Shall** send an `nbf` claim in the request object.

7. **Shall** send the `x-fapi-interaction-id` request header, with its value being a unique RFC4122 UUID for each request, to help correlate log entries between the client and server, e.g: `x-fapi-interaction-id: c770aef3-6784-41f7-8e0e-ff5f97bddb3a`.

2.3 Algorithm Considerations

For JWS, both Client Applications and Authorization Servers **shall** use the PS256 algorithm.

For TLS, all of the Data Provider's Authorization Server endpoints and Resource Server endpoints shall support mTLS connections with the algorithms specified in the 5.2 section of the FAPI 2.0 Final Security Profile.

For JWE, Client Applications and Authorization Servers **shall** use RSA-OAEP with A256GCM algorithm.

2.4 Consent and Authorization Mechanism Considerations

The QCB Open Finance leverages Rich Text Authorization Requests (RAR) to enable the granularity level required for the Open Finance Consents and Use Cases.

The Consent and Authorization Mechanism and lifecycle details are specified in the "Authentication and Authorization Document".

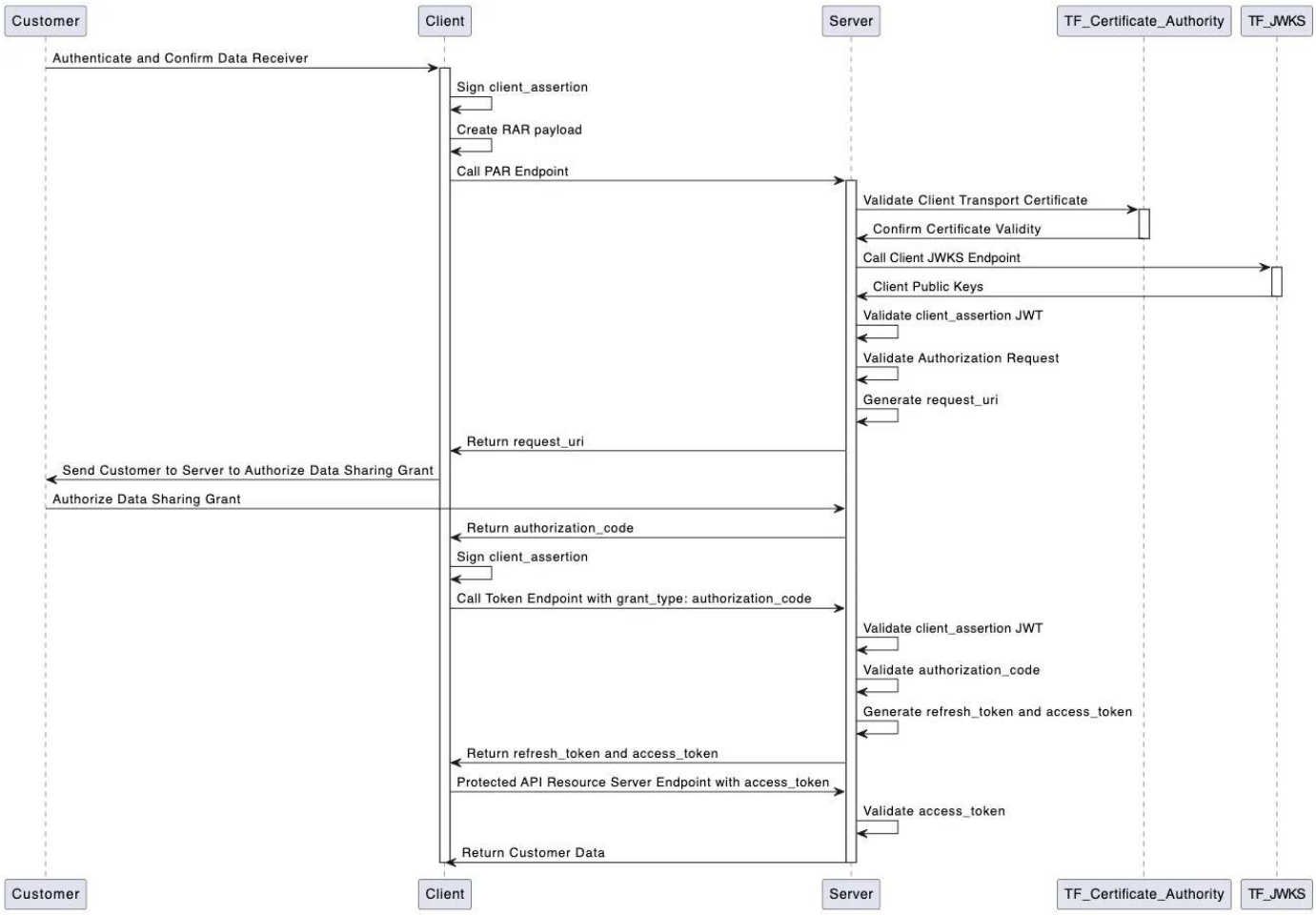
The schema for the `authorization_details` attribute of the RAR payload can be identified on:

- The **QCB Open Finance: Bank Service Initiation API** description, for the `urn:openfinanceqatar:service-initiation:*` types.
- The **QCB Open Finance: Bank Data Sharing API** description, for the `urn:openfinanceqatar:bank-data-sharing:*` type.

All of the documents defined above should be used in conjunction with this specification to ensure that all the parties can create and manage consents to resources in a standardized and secure way.

3. Consent Authorization Sequence Diagram

The diagram below provides a high-level visualization of the redirect journey, also known as authorization_code flow expected to be executed by Data Receivers (Relying Parties) with Data Transmitters (Open ID Providers) when obtaining an authorization grant from a customer that is willing to share data across institutions:



4. Design Considerations

4.1 Security

The QCB security profile has been based on the FAPI 2.0 Final Security Profile which is an API security profile based on the OAuth 2.0 Authorization Framework [RFC6749], that aims to reach the security goals laid out in the Attacker Model [Attacker Model]. The FAPI 2.0 Security Profile has been assessed by University of Stuttgart using formal analysis methods which is available for public review. By baselining the QCB Security Profile on FAPI 2.0 the ecosystem is leveraging the very latest in API Security Design.

4.2 Ecosystem Interoperability

A primary goal in reducing the available optionality is to increase and simplify interoperability across the Ecosystem. By narrowing the range of options in the Security Profile, it aims to decrease the number of allowed variations in the implementations of the FAPI 2.0 Security Profile.

While a high degree of optionality may offer Data Providers more flexibility in their Authorization Server implementation, it poses significant and material challenges for Data Receivers. Data Receivers (TPPs) must develop implementations capable of connecting to all Data Providers implementations, resulting in increased onboarding challenges and higher non-productive connectivity maintenance costs which otherwise can be spent on the delivery of value added propositions.

This move towards optionality reduction has been seen in modern Open Finance ecosystems including Open Finance Brazil and ConnectID in Australia where the FAPI 1.0 Advanced Profile was first used.

Initially, in 2021, the ecosystem's profile supported two token authentication methods – `tls_client_auth` and `private_key_jwt` – and made pushed authorization requests optional. This approach resulted in four possible variations of the same profile for Data Providers. Consequently, Data Receivers had to support all four profiles to achieve interoperability with all Ecosystem Data Providers. This design choice was originally made to facilitate the swift implementation by banks, allowing the ecosystem to go live quickly using some Banks legacy OpenID vendors.

However, from 2021 to 2023, the focus shifted from initial go-live to prioritizing interoperability and participant onboarding simplicity. This change in focus has resulted in the Ecosystem eliminating wherever possible any areas in the specification where optionality was given providers implementations. In March 2024, Open Finance Brazil mandated the introduction of

Pushed Authorization Requests (PAR) and adopted the use of `private_key_jwt` as the token authentication mechanism of choice. This change unified the ecosystem under a single profile, simplified and eradicated bank choice in favor of interoperability whilst simultaneously improving ecosystem security*.*

Given the above design considerations, the following recommendations are made to further profile the 'FAPI 2.0 Security Profile' to ensure a more interoperability domestic ecosystem:

Topic	What FAPI 2.0 Proposes	What QCB FAPI 2.0 Requires
Client Authentication Methods	tls_client_auth private_key_jwt	private_key_jwt
Method for Sender Constrained Access Tokens	MTLS DPoP	MTLS
Access Token Life Time	Not Specified	10 Minutes
JWS Signing Algorithms	PS256 ES256 EdDSA - Ed25519	PS256
Authorization Mechanism	Suggests RAR	Requires RAR
Message Tracing	No longer referenced.	Required - x-fapi-* headers for Logging and Tracking Purposes
Client Key Set Management	JWKS URI Recommended	Required - JWKS Uri
Authorization Non Repudiation	If Required - Signed Request Object (FAPI 2.0 Message Signing)	Required - Signed Request Object

Whilst not strictly a security consideration, to facilitate wider potential aspirations of the QCB ecosystem it is recommended that the ecosystem retain the use of Signed Request Objects as defined in OpenID Connect Core Section 6 and the OpenID FAPI Message Signing Profile. The use of signed request objects has two main benefits:

- It ensures that non-repudiation of Authorization Requests is easily achieved.
- Ensures broad interoperability for Relying Parties engaging in multiple Open Finance Initiatives around the world including the United Kingdom, Australia, The United States and Brazil.

4.3 Clarification around OFP and LFI Integrations

All documentation relating to the integrations and security layers between the Open Finance Platform (OFP) and the LFI will be provided as part of the OFP Documentation and are not part of the Ecosystem Security Profile, which should be used only for communications that happen directly between the Open ID Providers (Servers) and Open ID Relying Parties (Client)

Communications between the Open Finance Platform and the chosen LFIs will take place using certificates issued by the Trust Framework.

Account Information | Description

This bank service request enables TPPs to retrieve a User's data (with the User's consent) held with an LFI. The TPP agrees the boundaries of the data sharing consent with the User, then sends this data sharing consent to the LFI, where it is approved by the User. TPPs are then able to retrieve the status of the data sharing consent and receive data from the User's account.

This User journey requires an Account Access Consent.

The Bank Data Sharing service scope includes domestic accounts (i.e. accounts offered by LFIs located in Qatar) of the following types, in **all available currencies**: Current accounts, Savings accounts (including deposits), Credit cards, Loans, Mortgages, Wallets and Currency accounts. Please refer to Common Rules and Guidelines | Supported Accounts and Payment Rails for further details on the supported accounts.

Moreover, an LFI may request a data transfer from another LFI on behalf of a User with the User's consent. In the instance where an LFI is providing a service as a TPP, the LFI will be subject to the same rules and guidelines as a TPP would be, as specified in the Standard. This is illustrated in Account Information | LFI as a TPP Example.

Data Sharing Segments

The scope of the Account Information Sharing bank service covers the account segments shown below:

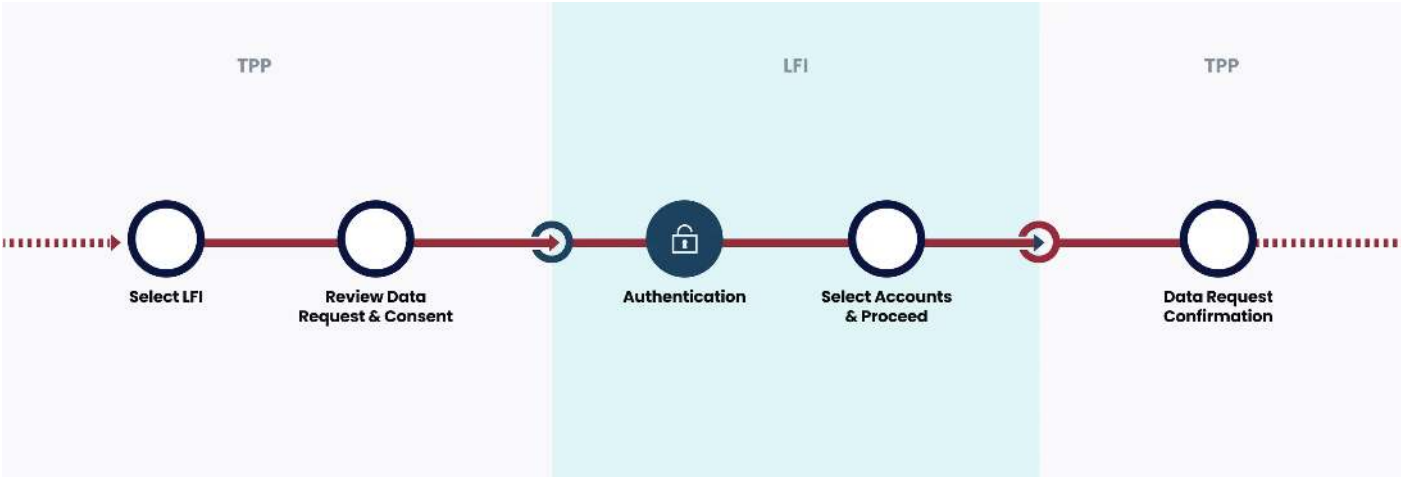
Consumers	SME
✓	✓

Account Information Sharing - Generic User Story

> **User Story**
>
> **As a** User (Consumer or SME),
>
> **I want to** provide my consent to a TPP to retrieve data about my account(s),
>
> **so that** I can access value added services offered by the TPP.

User Journey

Users can share data held with an LFI by providing their data sharing consent to a TPP.



Bank Data Sharing Consent Set-up & Authorization

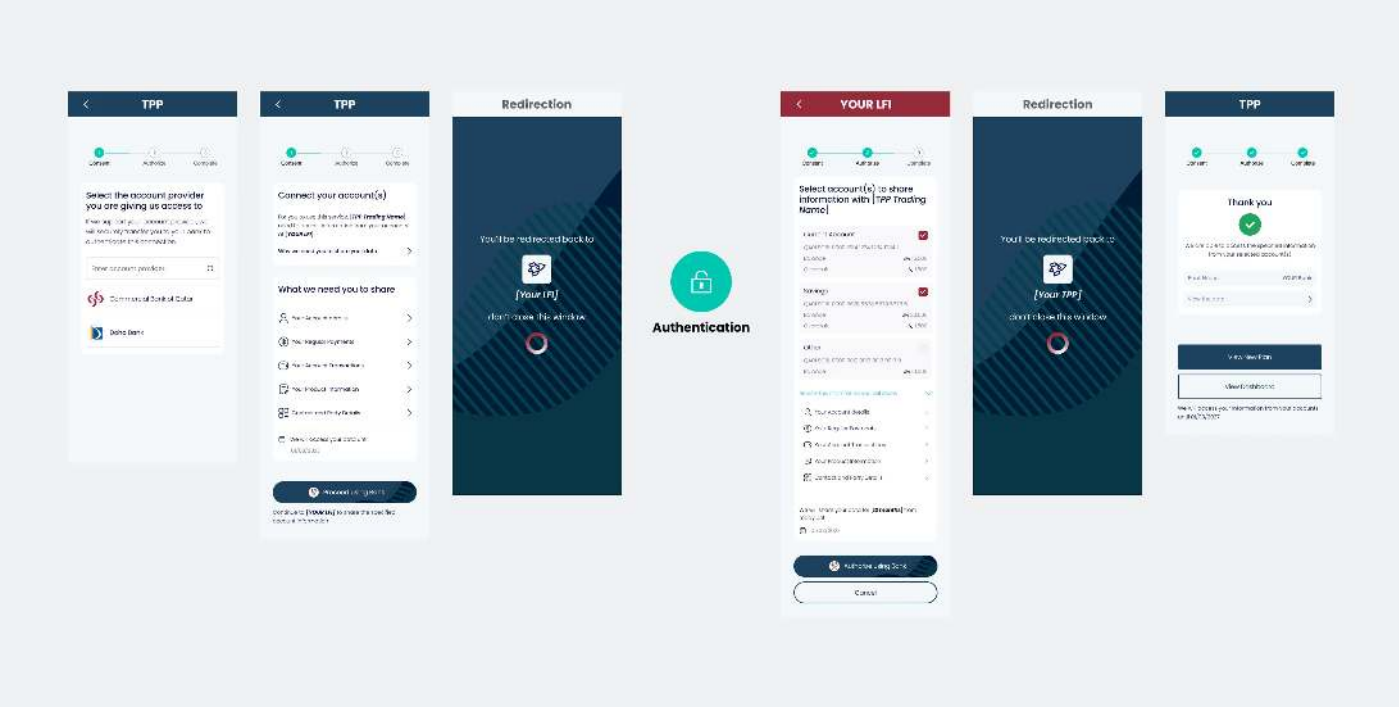
Customer Experience

A prototype of this journey, showcasing a Personal Finance Use Case, is illustrated in the example below:



Prototypes for QCB docs

Edited 5 hours ago



Bank Data Sharing Consent Rules & Guidelines

Initiate User Set-up (Conditional)

Depending on the use case, the User may have to be onboarded with the TPP by agreeing to any relevant terms and conditions (e.g. regarding sharing and storage of personal data) and privacy notice, and setting up an account with the TPP, if required.

TPPs MUST:

ACCI-001 Provide the User with Terms & Conditions and Privacy Notice documents, outlining the applicable rights and responsibilities in the context of relevant regulation and legal principles. This may also need to include any onward sharing of personal data, recipients or categories of recipients who receive that data, and the lawful basis for processing the User's personal data, as per Consent Setup | Consent Codification.

ACCI-002 Obtain the User's agreement to the above before setting them up, and be able to request the User's consent as per the next step in the process.

ACCI-003 Provide an option for the User to cancel the flow, if preferred.

Data Sharing Consent

Basic Consent Parameters

TPPs MUST:

ACCI-004 Request only the data required to perform their service (or use case).

ACCI-005 Use the data language standards to describe the Data Clusters and Data Permissions in user-facing interactions, so that the User clearly understands the data that will be requested from their LFI to provide the service requested.

- ACCI-006 Display to the User the data clusters that will be collected and the purpose the data will be used for.
  - ACCI-007 In the scenario that the use case of the TPP allows the User to select the data they wish the TPP to access, the TPP may provide the User with a list of data clusters for selection. However, the TPP MUST enable this selection to take place earlier in the user journey, prior to the Consent screen.

ACCI-008 Confirm the duration of the data access requested, or, depending on the use case, allow the User to select how long the TPP can access their data.

- ACCI-009 When a “single-use” (i.e. short-lived) account access consent is required, confirm to the User that their accounts will only be accessed for a maximum period of time, as defined in Limits and Constants | Single-use Consent Duration for Data.
  - ACCI-010 Confirm to the User what will happen to their data once the service has been provided and data access has been completed.
- ACCI-011 When a “long-lived” account access consent is required for the service to be provided:
  - ACCI-012 Confirm to the User that when the consent reaches the end of the sharing duration period, the consent will expire and may need to be renewed.
  - ACCI-013 When the consent expires, confirm to the User what will happen to their data and the service being provided, if not renewed.
- ACCI-014 Ensure that there is always an expiry date which defines a maximum validity period, as defined in Limits and Constants | Max Consent Validity Period. Consents with no expiry date will be rejected by the OFP.

ACCI-015 Define the start and end dates for the period needed for historical transaction data, if required for specific use cases and the TPP has requested any of the transaction-related permissions, as defined in Account Information | Data Cluster Definitions.

- ACCI-016 The end date of the period of historical data that can be requested by TPPs, and which MUST be returned by LFIs for a data sharing request, is as per the period defined in Limits and Constants | End Date of historical data for Data Sharing Request.

ACCI-017 Confirm to the User that they can withdraw their consent at any time before the consent expires.

ACCI-018 Provide to the User, the OFP and the LFI, their trading/brand name clearly and the name of any other parties they are supporting (if applicable).

ACCI-019 Allow the User to identify and select the LFIs for the Consent.

- ACCI-020 Provide a way for the User to search for their LFI.

Additional Consent Parameters

TPPs MUST:

ACCI-021 Set or clear the Single Authorization Indicator as appropriate (as per Common Rules and Guidelines | Is Single Authorization flag.

ACCI-022 Set the Authorization Expiration DateTime (as per Common Rules and Guidelines | Authorization Expiration DateTime if there are specific timing requirements that must be met for the Consent authorization. This is also relevant to cases where multiple authorizers are required to authorize the consent (please refer to Common Rules and Guidelines | Multi User Authorization Flow.

ACCI-023 Obtain the User's explicit consent to access information from the payment account products held at the selected LFI.

Consent Staging

ACCI-024 As per Common Rules and Guidelines | Consent Staging.

Hand-off to LFI

TPP MUST:

ACCI-025 Notify the User that they will be transferred to the selected LFI to undertake their authentication and consent authorization, as per Common Rules and Guidelines | Hand-off to LFI.

Authentication

LFI Authentication Only

LFI MUST:

ACCI-026 Enable Users to perform authentication with their LFI, as per the following sections:

- Authentication by LFI | Redirection
- Authentication by LFI | Decoupled Redirection

ACCI-027 Redirect Users back to the TPP, with information that the Consent has not been authorized, if User authentication has failed or the User opts to cancel the authentication/authorization process.

Disclosure Consent

LFIs MUST:

ACCI-028 Enable Users to authenticate using Multi-Factor Authentication (MFA) in order to review and authorize the data sharing single-use or long-lived Consent.

ACCI-029 Retrieve from the OFP the data sharing Consent details staged by the TPP, using the unique Consent Identifier.

ACCI-030 Display details of the data that will be shared and for how long.

ACCI-031 Use the data language standards to describe data clusters and permissions in user-facing interactions, so that the same information is displayed to the User, as defined in Account Information | Permissions and Data Clusters.

Select Accounts

LFI MUST:

ACCI-032 Display a list of eligible accounts for data sharing to the User, as per Common Rules and Guidelines | Supported Accounts.

- > ACCI-033 LFIs can display the eligible accounts using recognized nicknames, icons, account numbers, and account type.

ACCI-034 Only display the applicable accounts for User selection when the TPP has specified the Account Type and Account Sub-Type in the consent request object.

ACCI-035 Enable the account selection process as per Common Rules and Guidelines | Payment Account Selection at LFI.

ACCI-036 Allow the User to select which of their accounts to share data from, if the consent includes account-specific data permissions and there are multiple accounts available.

ACCI-037 **NOT** request Users to select any account from their eligible account list when the Data Request Consent does not include any account-specific data permissions and instead includes only User data permissions (e.g. in the case of the Parties endpoint). TPPs MUST NOT display any list of eligible accounts in this case — please refer to the variation described in Account Information | Data Sharing for Contact and Party data cluster.

- ACCI-038 Display to Users the full details of their personal data that will be shared with the TPP for the requested period of the Consent.

ACCI-039 Allow Users to proceed to the Data Sharing Consent authorization without selecting any accounts, in the case of Consents solely including non-account-specific data permissions.

ACCI-040 Display to the User any accounts that are unavailable to select and share with the TPP, and communicate why these accounts cannot be selected.

ACCI-041 Check that the authorization status of the selected payment account is in accordance with the TPP's Accepted Authorization Type, as per Common Rules and Guidelines | Check Accepted Authorization Type.

Confirmation / Authorization

LFIs MUST:

ACCI-042 Present to Users all the details in relation to the data sharing Consent.

ACCI-043 NOT allow Users to change any of the Consent parameters (e.g. permissions) staged by the TPP.

ACCI-044 Request Users to authorize the data sharing Consent.

ACCI-045 Enable Users to cancel the data sharing Consent request from within the authorization journey.

ACCI-046 Redirect Users back to the TPP, with information that the Consent has not been authorized, if Users opt to cancel the Consent authorization process before final authorization.

ACCI-047 Check that the Authorization Time window is valid, as per Common Rules and Guidelines | Check Authorization Time Window.

ACCI-048 Change the state of the data sharing Consent from “Awaiting Authorization” to “Authorized” once all Authorizers (one or more) have authorized the data sharing Consent.

ACCI-049 Update the data sharing Consent details stored in the OFP with all the information included in the data sharing Consent authorized by the User.

OFP MUST:

ACCI-050 Confirm back to the LFI that the data sharing Consent details have been updated successfully.

Hand-off back to the TPP

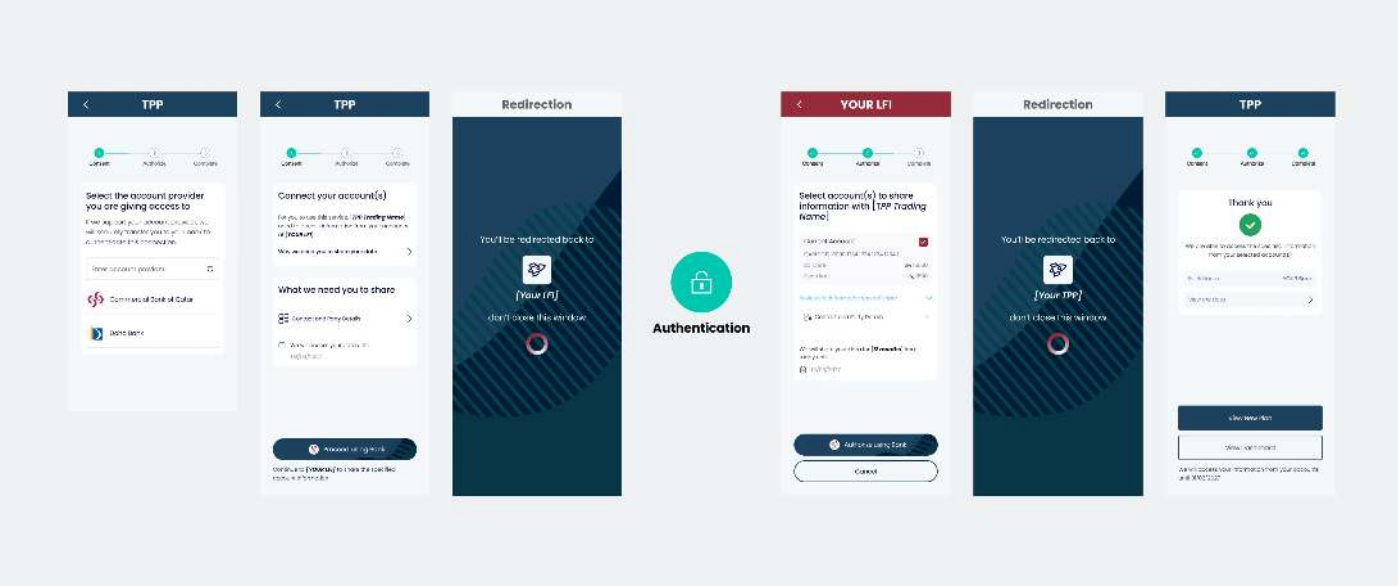
ACCI-051 As per Common Rules and Guidelines | Hand-off back to the TPP.

Confirmation to User

ACCI-052 As per Common Rules and Guidelines | Confirmation to User.

Journey Variations

Data Sharing for Contact and Party data cluster



Data Sharing Requests
Rules & Guidelines

Data Sharing Request

TPPs MUST:

ACCI-053 Only request specific data within the scope of their service (or use case).

ACCI-054 Only submit to the OFP data sharing requests for the data clusters and permissions consented to by the User, as per the data sharing Consent authorized by the User.

Processing of Data Sharing Requests

OFP MUST:

ACCI-055 Allow TPPs to submit data sharing requests in relation to a data sharing Consent authorized by Users, without any additional MFA or authorization by the Users.

ACCI-056 Check that the received data sharing request relates to a valid data sharing Consent authorized by the User. The Consent MUST be in the “Authorized” state. The OFP MUST reject any data sharing requests related to a data sharing Consent in a different state (e.g. expired) and respond back to the TPP with the appropriate error message/code.

- ACCI-057 Provide ongoing consented information (data and accounts) only if the User has consented to a long-lived data sharing Consent.
- ACCI-058 Provide short-lived data sharing access to consented information only if the User has consented to a single-use data sharing Consent.

ACCI-059 **Reject** the data sharing request and provide the necessary error message to the TPP, if any checks on the data sharing request fail against the authorized data sharing Consent.

- ACCI-060 Only allow data sharing requests from TPPs for the data clusters and permissions consented to by the User, as per the data sharing Consent authorized by the User.

ACCI-061 Send the data sharing requests to the LFI for the data clusters and permissions consented to by the User, as per the data sharing Consent authorized by the User.

LFIs MUST:

ACCI-062 Allow the OFP to submit the data sharing requests without any additional MFA or authorization from the User.

ACCI-063 Reject the data sharing request received from the OFP, in case there are valid reasons for the data sharing Consent to be suspended, as per Consent Setup | Consent States, or due to any other BAU check failure.

ACCI-064 Share data requested by the OFP in relation to the authorized data sharing Consent.

- ACCI-065 Ensure that the data provided to the OFP allows the TPP to reconcile the account transactions received from the LFI.

OFP MUST:

ACCI-066 Send an appropriate error response to the TPP, in case the data sharing request is rejected due to violating any of the LFI's BAU checks.

ACCI-067 Provide the TPP with all the available data for the data clusters and permissions requested in relation to the data sharing Consent authorized by the User.

Beneficiaries Data Sharing

When a User has granted either the `ReadBeneficiariesBasic` or `ReadBeneficiariesDetail` data permission, the TPP is permitted to access the User's trusted beneficiaries (payees) information for a specific `AccountId`.

ACCI-068 To clearly distinguish the source of beneficiary creation, LFIs MUST maintain two separate lists:

- Beneficiaries created through the authorization of an Open Finance payment.
- Beneficiaries created through the LFI's own banking channels.

TPPs can access the complete list of beneficiaries through the Get Beneficiaries API (`GET /accounts/{AccountId}/beneficiaries`).

ACCI-069 The API response MUST include beneficiaries created through both Open Finance and LFI channels. The field `AddedViaOF`, when set to `true`, indicates that the beneficiary was created as part of an Open Finance payment.

Account Information | Consent Updates

Consent Update

TPPs SHOULD:

ACCI-070 Enable Users to use the TPP Consent Management Interface (CMI) to amend one or more of the following parameters of a data sharing long-lived consent:

- Data clusters that are associated with the Consent - for example, enabling an additional data cluster to get other services from a TPP.
- The duration for which the User wants the consent to be valid, subject to the maximum validity period as defined in Limits and Constants | Max Consent Validity Period.

TPPs MUST:

ACCI-071 Require Users to authenticate with their LFI and authorize the Consent update, if they were allowed to make changes.

LFIs MUST:

ACCI-072 Allow Users to amend one or more of the accounts related to a long-lived data sharing consent, following a consent update initiated by the TPP CMI and prior to allowing Users to authorize the updated consent.

Permissions and Data Clusters

In the Bank Data Sharing API design, data elements are logically grouped together into “permissions”. Grouping permissions together and adding another layer of logical grouping and description helps Users’ understanding of the data they are being asked to consent to share.

All fields in the standards are required. Fields can only be optional if not stored at all.

If mandatory fields are not stored, they need to start being captured, stored, and included in the OF APIs.

The fields returned by each Bank Data Sharing endpoint, and whether each is technically required, are defined in the QCB Account Information API. A field that is not technically required in the API is still required by the Standard where the LFI stores it.

Some data fields are not technically required but are conditionally mandatory in certain circumstances. These are defined in the business rules.

As a principle, the Standards require that all available data be supplied when the corresponding data permission has been granted by the User through an authorized data sharing consent.

Permissions

As stated above, data elements are logically grouped together into “permissions”. It is at this level that TPPs request data access. If they request access to a specific permission, they will have access to all the data elements within that permission. This provides a pragmatic approach, allowing TPPs to be selective while at the same time creating a consent process that is at an acceptable level of granularity for Users. Details of the data elements within each permission will be included in the API technical specifications.

Data Clusters

Grouping permissions together and adding another layer of description aids Users’ understanding of the data they are being asked to consent to share. This approach also allows consistency of language across TPPs and LFIs, providing additional comfort to Users that they are sharing the data they intended to. Consistent language used across all member organizations drives User familiarity and adoption. These groups of permissions are known as Data Clusters.

Data Clusters are not reflected in the API specifications; they are purely a presentational layer on top of permissions to aid User understanding.

Data Cluster Structure & Language

Data Cluster Definitions

ACCI-073 The following table describes how permissions MUST be grouped into Data Clusters, and the language that MUST be used to describe the data at each of these clusters.

ACCI-074 Both TPPs and LFIs MUST describe the data being shared at a Data Cluster level, and allow Users to "drill down" to see the detail at the Permission level, using the permission language set out in the table below.

Where both Basic and Detail permissions are available for a set of data elements, the Detail permission contains all data elements of the Basic permission plus the additional elements described in the table. For more details about the information in the Data Clusters and permissions, please refer to the Bank Data Sharing API Specifications.

Data Cluster language	Permissions	Permissions Language	Examples of Information available
Your Account Details	Accounts Basic (`ReadAccountsBasic`)	*Any other name by which you refer to this account*	Currency of the account, nickname of account assigned by the account owner (e.g. "Fahd's Household account"). Each account will have a unique and immutable AccountId.
	Accounts Detail (`ReadAccountsDetail`)	*Your account name and number*	Account name, account identifier (may include account number, IBAN, mobile number, email address or other unique identifier) (plus all data provided in Accounts Basic).

Data Cluster language	Permissions	Permissions Language	Examples of Information available
	Balances (`ReadBalances`)	*Your account balance*	Amount, currency, credit/debit, type of balance, date/time, credit line.
Your Account Transactions	Transactions Basic (`ReadTransactionsBasic`)	*Your transactions*	Transaction information on payments, both credits in and debits out of the User's account (reference, amount, status, booking date information, value date information, transaction code). Does not include information about the Debtor/Creditor.
	Transactions Detail (`ReadTransactionsDetail`)	*Details of your transactions*	Transaction information on payments, both credits in and debits out of the User's account (reference, amount, status, booking date information, value date information, transaction code, merchant category code). Includes information about the Debtor/Creditor.

ACCI-075 With respect to the Data Clusters and Permissions language, LFIs SHOULD consider whether the language displayed to Users is appropriate when the information being accessed relates to more than one party. For example, "Your data" may need to be adapted to just "data" to indicate to Users that the account information being displayed may not be solely specific to them

- for example, in the case of joint accounts, when the account information of both parties is requested.

Data Cluster Requirements

The QCB Account Information API defines the response payload of each Bank Data Sharing endpoint. The permissions granted for each Data Cluster, as set out in the table above, determine which of those fields the LFI returns.

Relevance of Data Clusters by Product Type

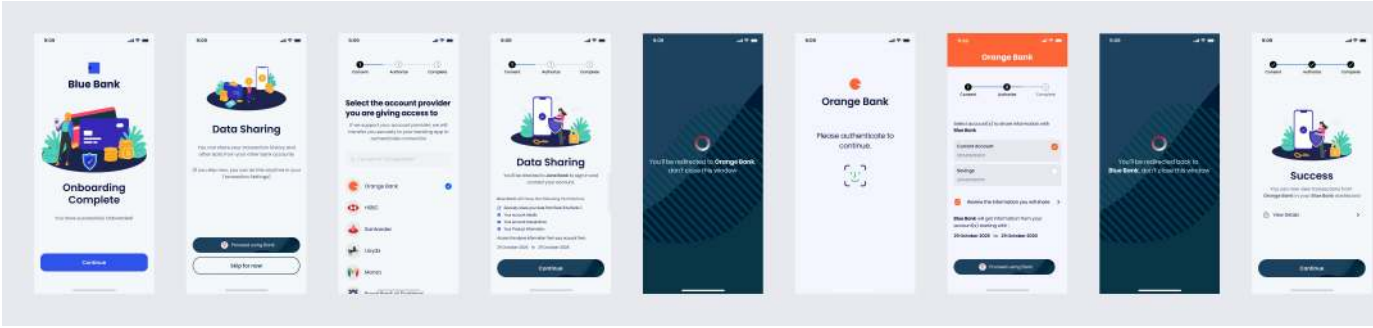
ACCI-076 TPPs MUST ensure they have business rules that manage the relationship between Data Clusters and product types, and omit access to Data Clusters that are irrelevant to a specific product type, as well as to their service offering.

ACCI-077 If a TPP requests a Data Cluster of data that is irrelevant to the product type associated with the payment account — for example, the Scheduled Transfers Data Cluster is requested for a Savings Account product type — the LFI MAY provide that Data Cluster as empty.

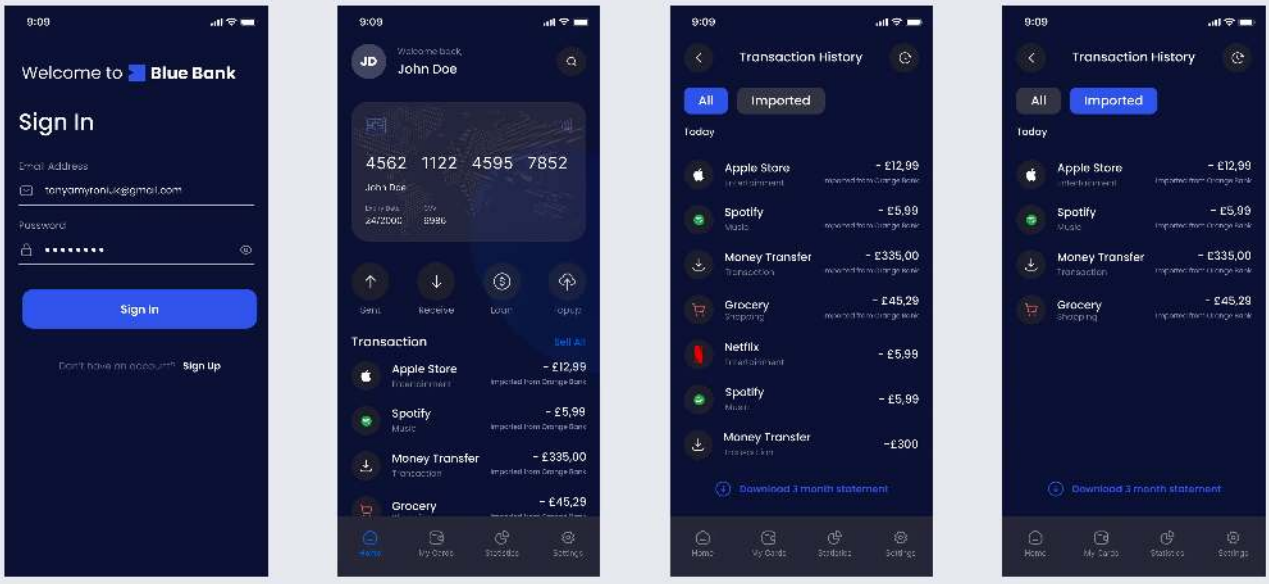
LFI as a TPP Example

The first flow illustrates a User providing consent for an LFI to access data from the User's other LFI. The second flow is an example of how an LFI could consolidate transaction history from both LFIs into a single statement. As an extension of this example, the consolidated statement could be used in a credit assessment.

User provides consent to share data from one LFI to another



User views consolidated transaction history



Single Instant Payment | Description

This bank service request enables TPPs to initiate domestic Single Instant Payments (SIPs), with the User's consent, from the User's online payment account held at the LFI to a business or a personal beneficiary. SIPs are executed immediately. TPPs are then further able to retrieve the status of the submitted payment order.

This user journey requires a Single Use Consent of type **Single Instant Payment Consent**.

Debtor and Creditor Segments

The scope of the SIP bank service initiation, in relation to the segments of Debtors and Creditors, is shown below:

Debtor		**Creditor**	
Consumer	**SME**	**Consumer**	**SME**
✓	✓	✓	✓

Example User Story

> **User Story**
>
> **As a** User (Consumer or SME),
>
> **I want to** provide my consent to a TPP to use my payment account for initiating a Single Instant Payment of a fixed amount to a domestic beneficiary account owned by a business or an individual,
>
> **so that** I can pay the relevant beneficiary immediately.

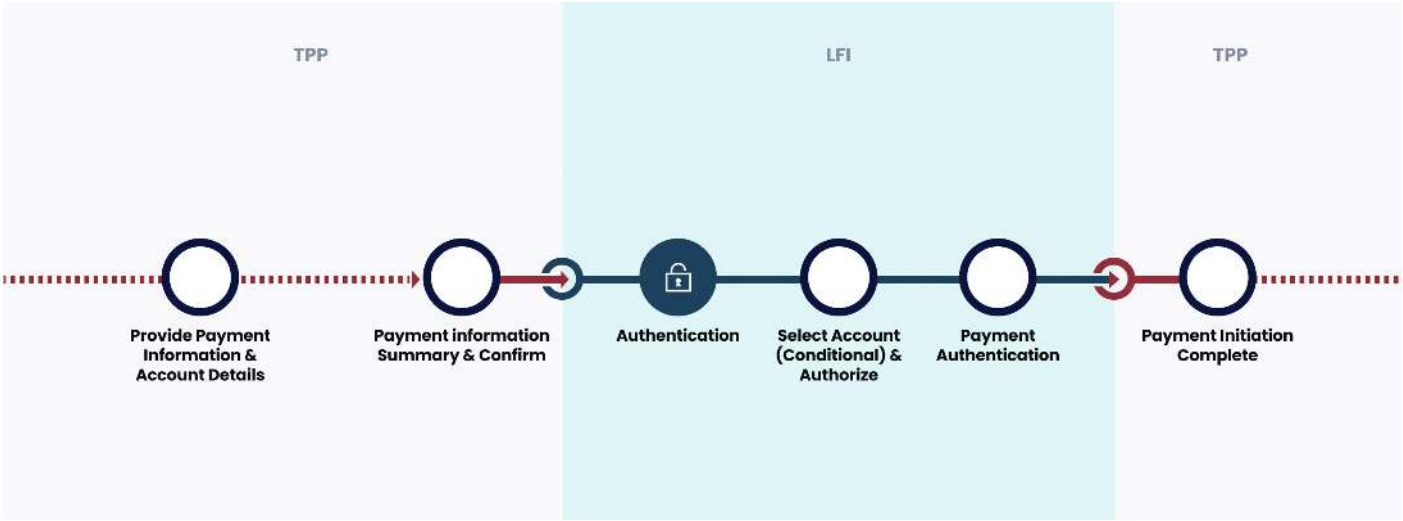
User Story

As a User (Consumer or SME),

I want to provide my consent to a TPP to use my payment account for initiating a Single Instant Payment of a fixed amount to a domestic beneficiary account owned by a business or an individual and allow the TPP to retrieve details about my payment account,

so that I can pay the relevant beneficiary immediately, and the TPP can provide me payment-linked services based on data about my payment account

User Journey



Users can initiate, by providing their consent to TPPs, a payment order to their LFIs to make a one-off single payment of a specific amount to a specific Creditor.

Single Instant Payment | Customer Experience

A prototype of this journey, showcasing a Single Instant Payment example, can be seen here: Prototypes for QCB docs

Standard Journey

SIPD-001 The display of the account balance by the LFI as part of the account selection process in the payment initiation journey SHOULD NOT require an additional application of MFA. The application of MFA is a matter for individual LFIs.

This applies where the payment order submitted by TPPs to LFIs is incomplete, such as where the User's account selection has not yet occurred. In these scenarios, the QCB Open Finance Standard considers that Multi-Factor Authentication (MFA) only needs to be obtained once, as part of the initial interaction between the LFI and the User. The fact that the User then has to carry out account selection or provide other information does not invalidate the MFA just performed by the LFI.

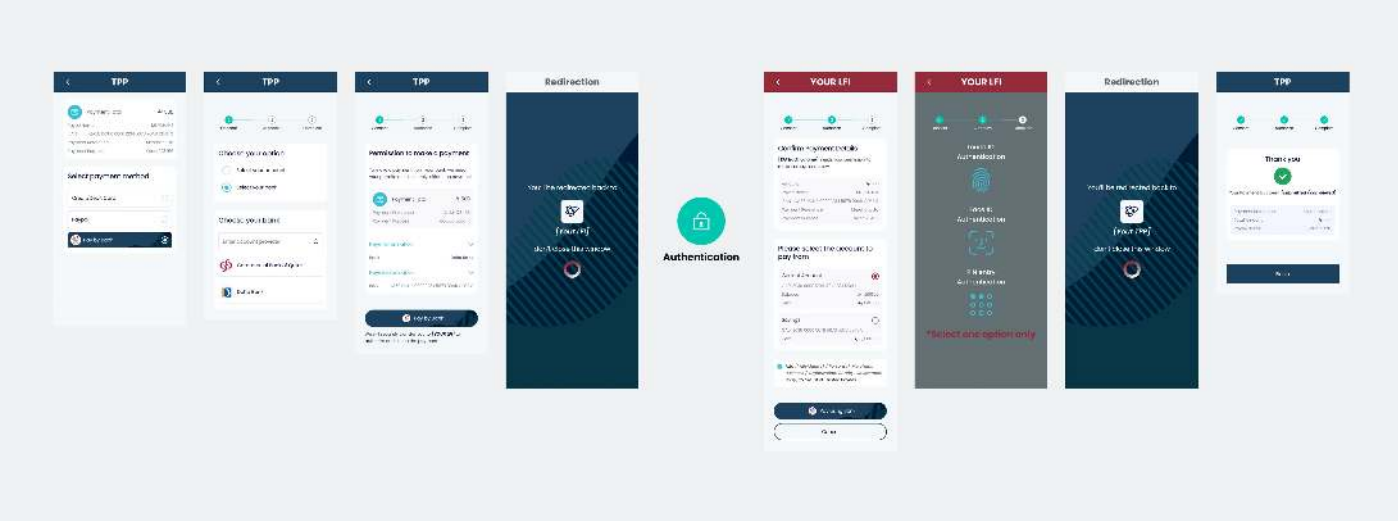
SIPD-002 **LFIs MUST** determine the situations where supplementary information is required, taking into account that the principle of maintaining parity between Open Finance journeys and the LFI's online channel journeys MUST be applied.

SIPD-003 If supplementary information is not provided directly to the User within the LFI's online channels, then it MUST NOT be provided during an Open Finance Service Initiation request journey.

SIPD-004 **LFIs MUST** also ensure that this information does not constitute an obstacle or an additional check on the Consent provided by the User to the TPP.

This applies where the payment order submitted by TPPs to LFIs has all the required information for the payment, but an additional step in the LFI's journey may be required to display supplementary information to the User, as defined in Common Rules and Guidelines | Supplementary Information..

User selects LFI at the TPP and account at the LFI



Rules & Guidelines

Single Instant Payment Consent

Basic Consent Parameters

TPPs MUST:

SIPD-005 Enable Users to provide and review the parameters related to the SIP to which they need to consent. These parameters include:

- User Payment Account or User LFI (as per Common Rules and Guidelines | User Payment Account Selection.
- Payment Amount & Currency (as per Common Rules and Guidelines | Payment Amount & Currency.
- Creditor Identification details (as per Common Rules and Guidelines | Creditor Identification.
- Debtor Reference (as per Common Rules and Guidelines | Debtor Reference (Optional).
- Creditor Reference (as per Common Rules and Guidelines | Creditor Reference.
- Purpose of Payment (as per Common Rules and Guidelines | Purpose of Payment.

Additional Consent Parameters

TPPs MUST:

SIPD-006 Set or clear the “Is Single Authorization” flag as appropriate (as per Common Rules and Guidelines | Is Single Authorization flag.

SIPD-007 Set the Authorization Expiration DateTime (as per Common Rules and Guidelines | Authorization Expiration DateTime if there are specific timing requirements that must be met for the consent authorization. This is also relevant to cases where multiple authorizers are required

to authorize the payment consent (Refer to Common Rules and Guidelines | Multi User Authorization Flow.

SIPD-008 Set the Risk Information Block (as per Common Rules and Guidelines | Risk Information Block.

TPPs MUST:

SIPD-009 Enable Users to provide explicit consent for the initiation of a SIP payment order from their online payment account held at their LFI, as per the payment details specified in the payment Consent.

Consent Staging

SIPD-010 As per Common Rules and Guidelines | Consent Staging.

Hand-off to LFI

SIPD-011 As per Common Rules and Guidelines | Hand-off to LFI.

Example wording to use: "We will securely transfer you to your LFI to authenticate and make the payment."

Authentication

LFI Authentication Only

LFIs MUST:

SIPD-012 Enable Users to perform authentication with their LFIs, as per the following sections:

- Authentication by LFI | Redirection
- Authentication by LFI | Decoupled Redirection

SIPD-013 Redirect Users back to the TPP, with information that the Consent has not been authorized, if User authentication has failed or the User opted to cancel the authentication/authorization process.

Authorization

LFIs MUST:

SIPD-014 Enable Users to authenticate using Multi-Factor Authentication (MFA) in order to review and authorize the Single Instant Payment (SIP) Consent.

SIPD-015 Retrieve from the OFP the Single Instant Payment (SIP) Consent details staged by the TPP and present the relevant details to the User.

- SIPD-016 **Reject** the Single Instant Payment (SIP) Consent if the Purpose of Payment violates any existing BAU rules for blocking/rejecting payments, as currently applied for other digital channels (Refer to Common Rules and Guidelines | Purpose of Payment). The OFP MUST be notified of this rejection with an appropriate error message.

SIPD-017 Allow Users to select a payment account for the initiation of the Single Instant Payment (SIP), if this was not provided in the retrieved staged payment Consent details, as per Common Rules and Guidelines | Payment Account Selection at LFI.

- SIPD-018 **NOT** allow Users to select a payment account, from their list of available payment accounts, that has insufficient funds for the Single Instant Payment (SIP) initiation. This only applies where the User does not select a payment account when providing Consent to the TPP.
- SIPD-019 **Reject** the Single Instant Payment (SIP) Consent if the payment account was part of the payment Consent provided by the TPP and that payment account has insufficient funds. The OFP MUST be notified of this rejection with an appropriate error message.

SIPD-020 Verify that the authorization status of the chosen payment account aligns with the "Is Single Authorization" flag specified by the TPP, as per Common Rules and Guidelines | Is Single Authorization flag.

SIPD-021 Display to Users the TPP Trading Name of the TPP that initiated the Single Instant Payment (SIP) Consent.

- SIPD-022 If there are customer-facing service providers (e.g. Merchants) who are not TPPs but have a commercial relationship with a TPP, the **LFI MUST** display the customer-facing service provider name along with the TPP trading name.

SIPD-023 Present to Users the following minimum required information for authorizing the Single Instant Payment (SIP) Consent:

- User Payment Account
- Payment Amount & Currency
- Creditor Identification details, including:

    - Creditor Name
    - Creditor Account
    - Creditor Account Holding LFI (Optional)
- Debtor Note (Conditional)
- Creditor Reference
- Purpose of Payment

SIPD-024 Request Users to authorize the Single Instant Payment (SIP) Consent, so that a single instant payment can be initiated.

SIPD-025 Provide Users the ability to **cancel** the payment journey if the User decides to terminate the request. The **LFI MUST** hand the User back off to the TPP, providing the necessary error message to the OFP, and reject the Single Instant Payment (SIP) Consent.

SIPD-026 Change the state of the Single Instant Payment (SIP) Consent from Awaiting Authorization to Authorized once all Authorizers (one or more) have authorized the payment Consent.

SIPD-027 Update the Single Instant Payment (SIP) Consent details stored in the OFP.

OFP MUST:

SIPD-028 Check that the Authorization Time window is valid, as per Common Rules and Guidelines | Check Authorization Time Window.

SIPD-029 Confirm back to the LFI that the Single Instant Payment (SIP) Consent details have been updated successfully.

Multi-Authorization Journey Only

SIPD-030 As per Common Rules and Guidelines | Multi User Authorization Flow.

Hand-off back to the TPP

SIPD-031 As per Common Rules and Guidelines | Hand-off back to the TPP.

Payment Initiation

TPPs MUST:

SIPD-032 Submit to the OFP the payment initiation request with the same parameters as the Payment Consent authorized by the User.

- SIPD-033 Submit the SIP payment initiation request to the OFP immediately after receiving the SIP Payment Consent authorization confirmation. The SIP payment initiation request MUST be received by the OFP within the Max SIP Payment Initiation Time Interval, as defined in Limits and Constants | Limits.

OFP MUST:

SIPD-034 Allow the TPP to submit the individual payment initiation request under the Payment Consent authorized by the User, without any additional MFA or authorization from the User.

SIPD-035 Check that the received payment initiation request relates to a valid Payment Consent authorized by the User. The Consent MUST be in the Authorized state. The OFP MUST reject a payment initiation message related to a Payment Consent in a different state and respond to the TPP with the appropriate error message/code.

SIPD-036 Check the payment initiation request parameters against the authorized Payment Consent. All parameters MUST match exactly.

- SIPD-037 **Reject** the payment initiation and provide the necessary error message to the TPP if any check of the payment initiation request parameters fails against the parameters of the authorized Payment Consent.

SIPD-038 Send the SIP payment initiation request to the LFI for initiating an instant payment, using the payment parameters included in the payment initiation request, including:

- Authorized Payment Consent Identifier
- Payment Amount & Currency
- Debtor Reference (if provided)
- Creditor Reference
- Purpose of Payment

LFIs MUST:

SIPD-039 Trigger the payment initiation process for the payment Consent immediately after receiving the payment initiation request from the OFP.

- SIPD-040 Retrieve the Creditor Identification details from the encrypted PII information block included in the original Payment Consent message.

- SIPD-041 Apply all existing BAU checks and validation processes to the Creditor Identification details. In case of failure, the **LFI MUST** reject the payment initiation request

> and notify the OFP of this rejection with an appropriate error message.

SIPD-042 Additionally apply all existing BAU payment account controls and limits, such as single transaction value limit, total transaction value limit, AML checking (if applicable) and others, as if the payment request had been initiated through the LFI's existing channels, except that no cooling-off period or other hold between authorization and submission to the payment rail may be applied (see COMM-218). The **LFI MUST** send an appropriate error response to the OFP if the payment is rejected for violating any of these limits.

SIPD-043 **Reject** the payment initiation if the payment account selected for the payment has insufficient funds. The OFP MUST be notified of this rejection with an appropriate error message.

SIPD-044 Subject to successful BAU checking, validation and payment processing, proceed with execution of the payment without delay, either by submitting it to the underlying payment rails or by executing it internally as an intra-bank payment.

SIPD-045 Provide the OFP with all available information relating to the initiated payment instruction, including the payment's unique identifier, the payment Transaction ID (see COMM-272).

SIPD-046 Ensure that the Creditor Reference provided in the SIP payment initiation is made available in the Creditor account information in the case of intra-bank payments within the same LFI.

OFP MUST:

SIPD-047 Send an appropriate error response to the TPP if the payment is rejected for violating any of the LFI's BAU payment account checks or limits.

SIPD-048 Send the TPP an appropriate error message if the payment initiation was rejected by the LFI due to insufficient funds in the selected payment account.

SIPD-049 Provide the TPP with all available information relating to the initiated Single Instant Payment (SIP) instruction, including the payment's unique identifier, the payment Transaction ID (see COMM-272).

TPPs MUST:

SIPD-050 Display confirmation to the User that the payment has either been submitted or completed.

- SIPD-051 **Submitted** must be displayed when the payment initiation request has been submitted to the LFI and the payment status is Pending.
  - SIPD-071 Where the payment has been directed to Tahweel, the TPP **MUST** also inform the User that the payment will be settled in a later Tahweel settlement window, not immediately, using the settlement window information provided under COMM-268 (**<TBC with QCB>**).
- SIPD-052 **Completed** must be displayed when the payment initiation request has been submitted to the LFI and confirmation has been received that the payment status has changed to AcceptedWithoutPosting, the final accepted state in the Payment Status Model.

Payment Status Update

SIPD-053 As per Common Rules and Guidelines | Payment Status Update.

Confirmation to User

SIPD-054 As per Common Rules and Guidelines | Confirmation to User.

Payment Notifications

SIPD-055 As per Common Rules and Guidelines | Payment Notifications.

Payment-linked Account Data Sharing

Prior to Payment Initiation, TPPs can optionally retrieve the User's payment account details to provide Payment Services (e.g. populating a user's payment options within a merchant site for next payment or making payments back to the user), subject to have received Account Basic or Account Details permission during the Single Instant Payment Consent setup.

Request Account Details

TPPs MUST:

SIPD-056 Be able to request account information (e.g. account number) using the authorized Single Instant Payment Consent before initiating a payment.

SIPD-057 Use this capability only in relation to Payment Services such as populating a user's payment options within a merchant site for next payment or TPPs making payments back to the user.

SIPD-058 Have existing Privacy Policies or Terms & Conditions which cover User data retention and include a process which a User can use to request data deletion.

SIPD-059 Only pass User's account details to a third party which the TPP has a contractual relationship which includes data retention and liability clauses, and only pass the required information for the use which is explicitly consented by the user (e.g. populate user's payment options within a merchant site).

SIPD-060 **NOT** use this capability as an alternative for Account Information Service, for example, to satisfy use cases like Personal Finance Manager (PFM), where the account information including balance is being refreshed.

SIPD-061 Display a Checkbox UI/UX element with Supplementary Information at consent setup which requests consent and highlights to the user what data will be shared and for what purpose(s), including separate descriptions of all uses of shared data.

SIPD-062 Make sure the use case Supplementary Information language is easy for an average user (person) to understand. A user should be able to read the statement and clearly understand how their data will actually be used.

SIPD-063 In the following Use Cases, allow the User to "opt in" of consenting to share their account details by displaying a unselected Checkbox and the provided Supplementary Information:

- SIPD-064 Next Payment Use Case - The User's Account Details are to be shared with TPP and their third party Merchant for ease of making next purchase.

- *Next Payment Supplementary Information - "You allow us to share my account details with [Merchant Name] for future purchases. My details will be stored securely and cannot be used for other purposes."*

Account Details Request Processing

LFIs MUST:

SIPD-065 Provide the OFP with the information related to the User's account, which was used for authorizing the Single Instant Payment Consent. This information **MUST** be the same information as defined under the `ReadAccountBasic` or `ReadAccountDetail` permission in Account Information | Data Cluster Structure & Language.

SIPD-066 Display a Supplementary Information at consent authorization which highlights to the user the permission(s) being authorized.

OFP MUST:

SIPD-067 Provide TPPs with all the available information in relation to the Account Information request.

Account Details Storage and Retention

TPPs MUST:

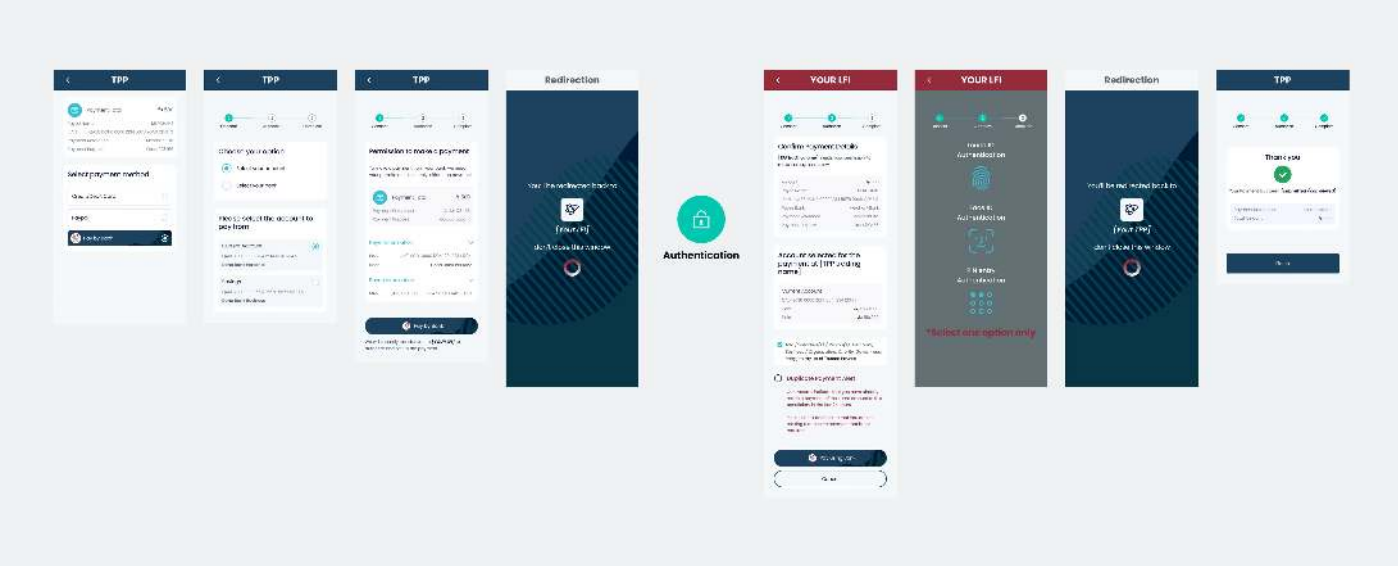
SIPD-068 Retain the User's account details until the expiry of the consent unless required due to regulatory, legal or services which the user has directly consented to with the TPP or third party (e.g. merchant).

SIPD-069 Allow a User to withdraw consent to retain and use their account details through direct communication to the TPP or TPP's contracted client (e.g. Merchant).

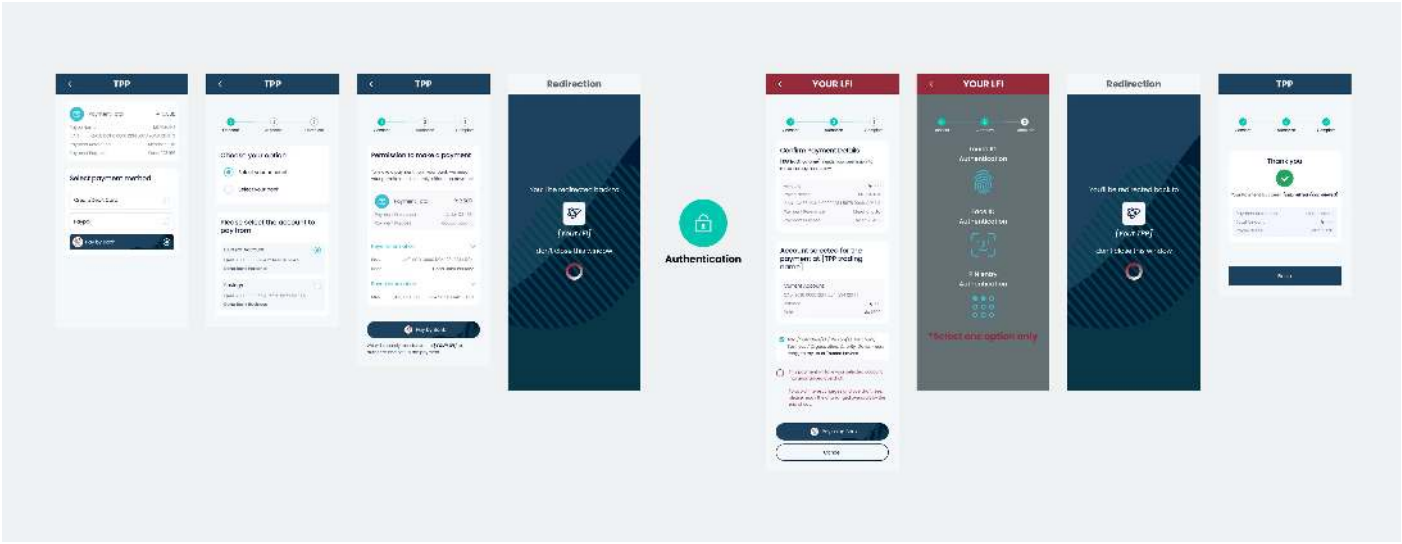
SIPD-070 Ensure their contracted client (e.g. Merchant) has properly deleted the account details.

Journey Variations

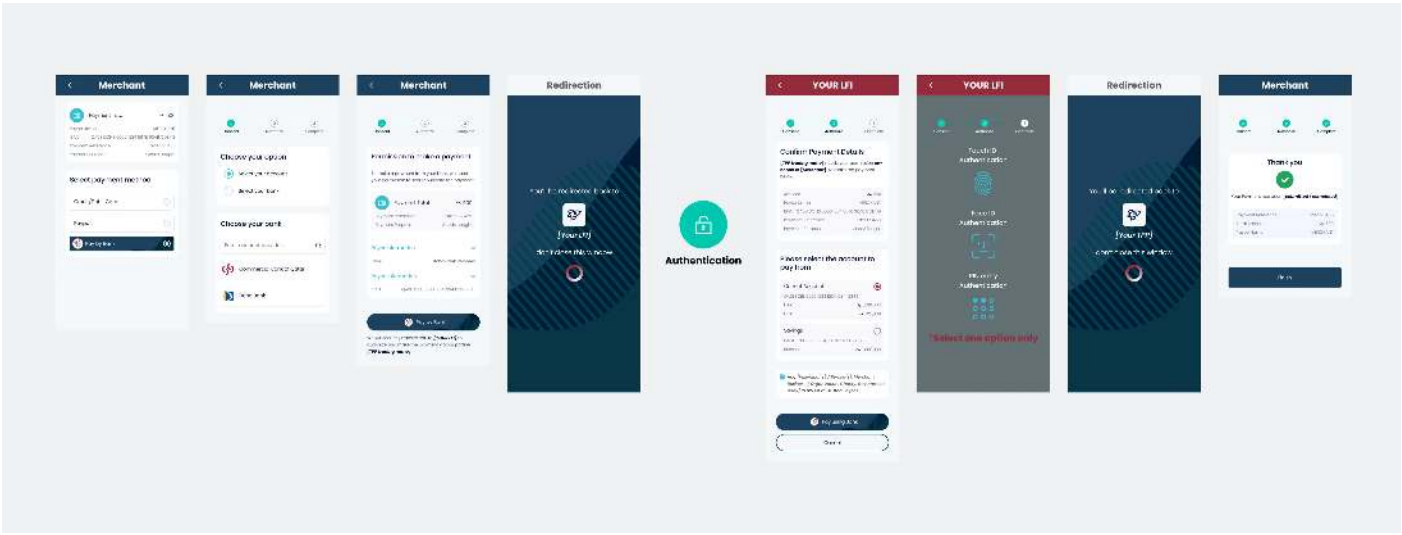
User selects account at the TPP & LFI provides Supplementary Information (Duplicate Payment)



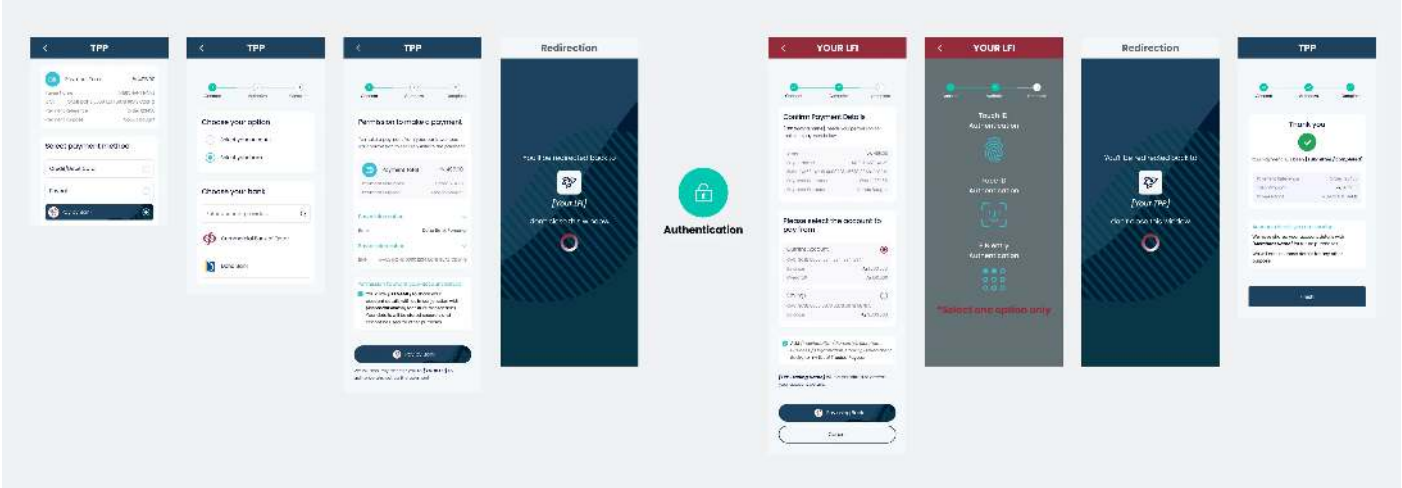
User selects account at the TPP & LFI provides Supplementary Information



User selects LFI at the TPP & merchant is the user-facing entity using a TPP for Open Finance payments



User makes payment and TPP requests to share account details to Merchant for future purchases



What are Multi-Payments

Multi-Payments refer to a series of payments initiated by a TPP. Multi-payments are designed to support a range of long-lived payment use cases that TPPs may wish to implement.

MPGN-001 The Variable-on-Demand Multi-Payment consent **MUST** be authorised by the User through Multi-Factor Authentication (MFA) at their LFI during the initial setup stage. However, each individual payment subsequently instructed by the TPP under one of these Multi-Payment long-lived consents does not require further MFA by the LFI, as these are executed within the limits and controls established in the authorised consent.

Multi-Payment Types Set-up

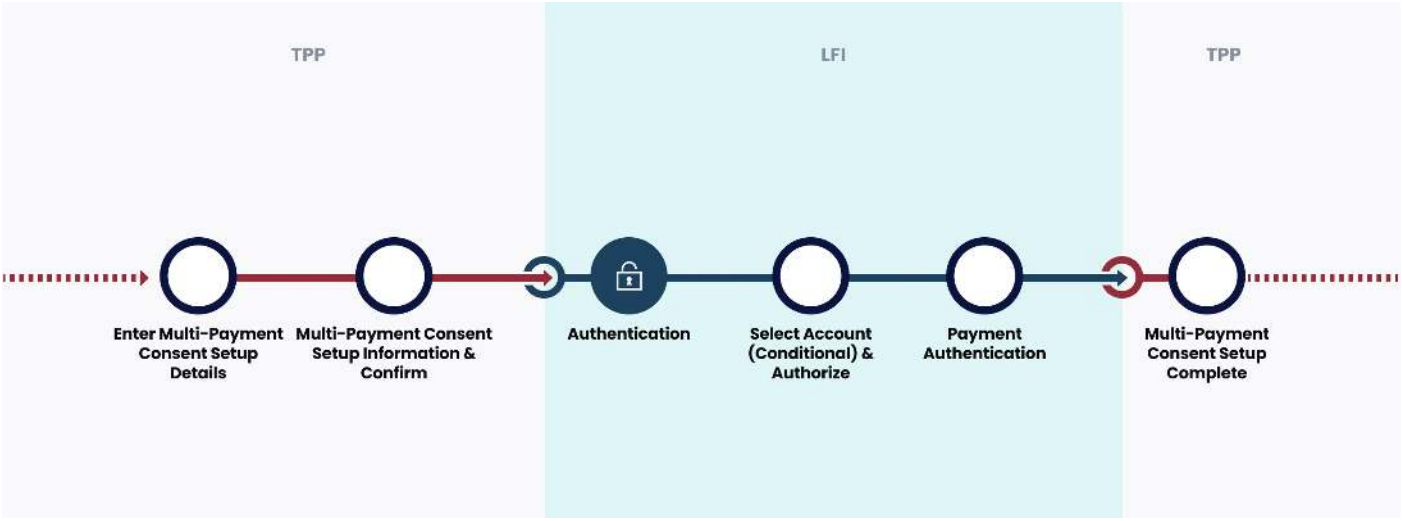
Multi-Payments Type	Payment Amount	Payment Schedule	Typical Usage Examples
Variable On-Demand Payments (`VariableOnDemand`)	Variable up to a defined Amount	Anytime	Account sweeping; cab-booking app

Debtor and Creditor Segments

The scope of the Multi-Payments bank service initiation related to the segments of Debtors and Creditors is shown below:

Debtor		Creditor	
Consumer	**SME**	**Consumer**	**SME**
✓	✓	✓	✓

User Journey


TPP
LFI
TPP
Enter Multi-Payment Consent Setup Details
Multi-Payment Consent Setup Information & Confirm
Authentication
Select Account (Conditional) & Authorize
Payment Authentication
Multi-Payment Consent Setup Complete

TPP Consent Staging Rules & Guidelines - Basic Consent Parameters

Multi-Payments Consent

MPGN-002 TPPs **MUST** enable Users to provide and review the parameters related to the initiation of a series of Multi-Payments for which consent is required. These parameters include, but are not limited to:

Debtor Account - Payment Account Selection

- The selection of the User's account (Debtor Account) for the payment initiation when the account details are already held by the TPP.
  - MPGN-003 In this scenario, the User's account details **MUST** be populated in the `DebtorAccount` object within the consent's Personal Identifiable Information (PII).
- The option to select the account for the payment initiation during the LFI's consent authorization flow when the required account is not held or known by the TPP.
  - MPGN-004 In this scenario, the `DebtorAccount` object **MUST NOT** be included in the consent's PII, to indicate to the LFI that account selection is required during consent authorization.
- See Common Rules and Guidelines | User Payment Account Selection for further details.

Creditor Details

- MPGN-005 Where the Creditor details are not known or held by the TPP, the TPP **MUST** allow the User to provide the required details to enable payment initiation.
- MPGN-006 The identification details of each Creditor **MUST** be populated in accordance with Common Rules and Guidelines | Creditor Identification.

MPGN-007 **Single known Creditor:** Where all payments under the consent will be made to the same single Creditor over the lifetime of the consent, the TPP **MUST** display the Creditor details to the User (see Common Rules and Guidelines | Creditor Identification.

MPGN-008 **Multiple defined Creditors:** Where payments may be made to multiple defined Creditors, the TPP **MUST** display the details of each Creditor to the User (see Multi-Payments | Creditor Identification for Variable-defined Beneficiaries.

MPGN-009 **Variable, undefined Creditors:** Where payments may be made to variable, undefined Creditors, the TPP **MUST** present clear messaging to the User stating that:

- the Payee will not be specified as part of the consent and will instead be defined during payment initiation;
- the Payee will be confirmed once specified and before the payment is executed; and
- MPGN-010 the User **MUST** explicitly agree that one or more factors (conditions) defined in the consent will be used by the TPP as their authorisation for each payment initiation.

PAR authorization_details

`authorization_details.consent` Attribute	**Required**	**Consent Set-up Rules**
`PersonalIdentifiableInformation`	Yes	See `PersonalIdentifiableInformatio` below.
`ControlParameters.ConsentSchedule`	No	Although this object is optional, it **MUST** be provided in order to define the required Multi-Payment parameters.
`ConsentId`	Yes	This **MUST** be a unique identifier for the Multi-Payment consent being set up.
`BaseConsentId`	No	A base consent identifier that links to previous Multi-Payment consent resources that have expired or been consumed for the same service. Wher the consent being staged is the first Multi-Payment consent for the service being provided, the OFP **MUST** set th `BaseConsentId` to match the `ConsentId` if this field has not been provided.

`authorization_details.consent` Attribute	Required	Consent Set-up Rules
`IsSingleAuthorization`	No	Specifies to the LFI that the consent authorization **MUST** be completed by single authorizer at the LFI. When this flag is provided and set to `true`, the LFI **MUST** ensure that: only accounts that can be authorised by a single authorizer (account holder) are displayed when the Debtor Account details have **not** been provided in the `PersonalIdentifiableInformatio` object; and when the Debtor Account details **have** been provided in the `PersonalIdentifiableInformatio` object, the payment consent **MUST** b authorised solely by the authenticatec end user. See [Common Rules and Guidelines
`AuthorizationExpirationDateTime`	No	The Authorization Expiry Date Time **MUST** be provided where there are specific timing requirements that appl to the consent authorization. This fielc is also relevant in scenarios where multiple authorizers are required to complete the payment consent authorization. See [Common Rules an Guidelines
`ExpirationDateTime`	Yes	The date and time at which the consent will expire or be consumed. The time element within the Expiratior Date Time must be set to the end of day (23:59:59).

`authorization_details.consent` Attribute	Required	Consent Set-up Rules
`Permissions`	No	If the ability to check the account balance prior to submitting a payment initiation is required, the `ReadBalances` data access permissio **MUST** be provided. In addition, one of `ReadAccountsBasic` or `ReadAccountsDetail` **MUST** also be provided.
`DebtorReference`	No	The reference provided by the Debtor that will be assigned to each payment initiation. See [Common Rules and Guidelines
`CreditorReference`	No	The reference required by the Credito to support the reconciliation of a giver payment instruction. See [Common Rules and Guidelines
`PaymentPurposeCode`	Yes	Must be provided to confirm the purpose of the multi payments. See [Common Rules and Guidelines

PersonalIdentifiableInformation

`PersonalIdentifiableInformation` Attribute	Required	Consent PII Set-up Rules
`Initiation`	No	**MUST** be provided when the Debtor Account has been selected as part of the TPP multi-payment set-up

`PersonalIdentifiableInformation` Attribute	Required	Consent PII Set-up Rules
		process, and/or when the payment is to be made to single or multiple defined beneficiaries.
`Initiation.DebtorAccount`	No	**MUST** be provided when the Debtor Account has been selected as part of the TPP multi-payment set-up process.
`Initiation.Creditor`	No	**MUST** be provided when the payment(s) will be made to single or multiple defined beneficiaries.
`Initiation.Creditor.CreditorAgent`	No	If applicable, the details of the Creditor Agent(s) should be provided when the payment(s) will be made to single or multiple defined beneficiaries.

`PersonalIdentifiableInformation` Attribute	Required	Consent PII Set-up Rules
`Initiation.Creditor.Creditor`	No	The details of the Creditor should be provided when the payment(s) will be made to single or multiple defined beneficiaries.
`Initiation.Creditor.CreditorAccount`	Conditional	**MUST** be provided when the payment(s) will be made to single or multiple defined beneficiaries.
`Initiation.Creditor.ConfirmationOfPayeeResponse`	No	The result of the confirmation of payee check performed for new Beneficiaries **MUST** be included.
`Risk`	No	Set the Risk Information Block as per [Common Rules and Guidelines

TPP Consent Staging Rules & Guidelines - Consent Staging

Consent Staging

TPPs **MUST**:

- MPGN-011 Before submitting the consent request to commence the authorization process, enable Users to provide explicit consent for the initiation of a series of future Multi-Payments in accordance with the payment parameters presented to them.
- MPGN-012 When the User has provided their explicit consent, submit the PAR request as per Common Rules and Guidelines | Consent Staging.

Hand-off to LFI

TPPs **MUST**:

- MPGN-013 When a PAR 200 OK response is received with the `request_uri`, inform the User that they will be transferred to their selected LFI to authorize the multi-payment consent.
- MPGN-014 Display a re-direction "holding" screen to the User confirming *"We will securely transfer you to [YOUR LFI] to authorize the payment setup"*. See Common Rules and Guidelines | Hand off to LFI.

When the User has been successfully re-directed from the TPP to the LFI, the **LFI MUST**:

Authentication

LFI Authentication Only

MPGN-015 As per the following sections:

- Authentication by LFI | Redirection
- Authentication by LFI | Decoupled Redirection

Authorization

LFIs MUST:

1. MPGN-016 Enable Users to authenticate using Multi-Factor Authentication (MFA) in order to review and authorize the long-lived payment Consent.
2. MPGN-017 Retrieve from the OFP the payment Consent details staged by the TPP using the unique Consent Identifier.

- MPGN-018 **NOT** reject the Multi-payment Consent, even if the Purpose of Payment violates any existing BAU rules for blocking/rejecting payments, as they use today for other digital channels. This is because the payment initiation messages sent by TPPs may use a different Purpose of Payment code which may not violate these rules.

3. MPGN-019 Allow Users to select a payment account for the initiation of the multi-payments, if this was not provided in the retrieved staged Payment Consent details, as per Common Rules and Guidelines | Payment Account Selection at LFI.

- MPGN-020 Allow Users to select a payment account for the initiation of the multi-payments even if it has insufficient funds at the time of the payment Consent authorization. This allows Users to fund the payment accounts appropriately before the dates of the payment initiation. [MPGN-021] However, the LFIs **MUST** inform the User if the selected payment account has insufficient funds.

4. MPGN-022 **NOT** earmark (i.e. block) any funds related to the payment Consent in the User's payment account at the point of Consent authorization.
5. MPGN-023 Check the authorization status of the selected payment account is in accordance with the TPPs' Is Single Authorization flag as per Common Rules and Guidelines | Is Single Authorization flag.
6. MPGN-024 Display to Users the TPP Trading Name of the TPP that initiated the long-lived payment Consent.

- MPGN-025 If there are customer-facing service providers (e.g. Merchants) who are not TPPs but have commercial relationships with TPPs, the LFIs **MUST** display the customer-facing service provider name along with the TPP trading name.

7. MPGN-026 Present to Users the following minimum required information for authorizing the long-lived payments Consent:

   - User Payment Account
   - Creditor Identification details — a single Creditor, or a predefined list of Creditors
   - Debtor Reference (Optional)

- Creditor Reference
- Purpose of Payment
- Consent Control Parameters — the maximum individual payment amount, and any optional cumulative value/number caps (lifetime and per-period)
- Consent Control Period & Start Date — the period the per-period caps reset on
- Consent Expiration Date & Time

8. MPGN-027 Request Users to authorize the Multi-payments Consent, so that the TPP can initiate payments from the payment account.

- MPGN-028 Request Users to additionally authorize the Balance Check Permission, in the case that TPPs have requested permission to check the balance of the Users' payment accounts.
- MPGN-029 When the multi payments will be made to a single Creditor, provide an option to the User to add the Creditor and the Creditor account details to the User's list of Trusted Payees (Beneficiaries) for Open Finance payments. See Common Rules and Guidelines | Adding a Trusted Payee (Beneficiary).

9. MPGN-030 Provide Users the ability to **cancel** the payment journey, if Users decide to terminate the request. [MPGN-031] The LFI **MUST** hand off the Users back to the TPP, providing the necessary error message to the OFP and reject the Multi-payments Consent.
10. MPGN-032 Change the state of the payment Consent from Awaiting Authorization to Authorized when all Authorizers (one or more) have authorized the payment Consent.
11. MPGN-033 Update the payment Consent details stored in the OFP with the relevant information.

OFP MUST:

12. MPGN-034 Check the Authorization Time Window is valid as per Common Rules and Guidelines | Check Authorization Time Window.
13. MPGN-035 Confirm back to the LFIs that the payment Consent details have been updated successfully.
14. MPGN-036 Start tracking the Consent Control Parameters for the Control Period at the mandatory Control Period Start Date. The Control Period starts from 00:00:00 of the day and ends at 23:59:59 of the Control Period end day, calculated based on the Control Period type.

Multi-Authorization Journey Only

15. MPGN-037 As per Common Rules and Guidelines | Multi User Authorization Flow.

Hand-off back to the TPP

MPGN-038 As per Common Rules and Guidelines | Hand off back to the TPP.

Confirmation to User

MPGN-039 As per Common Rules and Guidelines | Confirmation to User.

Multi-Payments Common Rules & Guidelines - Period & Payment Schedule

Period Type & Start Date

TPPs & OFP **MUST** enable their systems to support the following definitions:

- MPGN-040 **Start Date:** The date that the first repeating Period starts. It is defined at the start of the date at midnight (00:00:00).
- MPGN-041 **Period Type:** This **MUST** be one of the following:
  - **Day:** A continuous period of time, consisting of 24 consecutive hours, starting from midnight (00:00:00) and finishing at 23:59:59 of the same day. The Period will be repeating every day from the Start Date. For example, if the Start Date is the 10th of the month, then the Period will be repeating on the 11th, 12th, 13th, and so on.
  - **Week:** A continuous period of time, consisting of seven consecutive days, starting from midnight (00:00:00) of the Start Date and finishing at 23:59:59 of the 7th day elapsed from the Start Date. The Period will be repeating on the same day every week from the Start Date. For example, if the Start Date is Thursday 10th October 2024, then the Period will be repeating on Thursday the 17th October 2024, Thursday the 24th October 2024, Thursday the 31st October 2024, Thursday the 7th November 2024, and so on.
  - **Month:** A continuous period of time starting from midnight (00:00:00) of the Start Date and finishing at 23:59:59 of the day before the same date of the next month. The Period will be repeating on the same date every month from the Start Date. For example, if the Start Date is the 10th October 2024, then the Period will be repeating on the 10th November 2024, the 10th December 2024, the 10th January 2025, and so on. The exceptions to this rule are the following:
    - If the Start Date is a date not available every month, then the Period will be adjusted to the end date of the month which does not have the Start Date available. For example:
      - If the Start Date is the 31st, then the Period will be initiated on the 31st for January, March, May, July, August, October and December; on the 30th for April, June, September and November; and on the 28th or 29th (depending on leap year) for February.
      - If the Start Date is the 30th, then the Period will be initiated on the 30th for all months apart from February, for which it will be the 28th or 29th (depending on leap year).

        - If the Start Date is the 29th, then the Period will be initiated on the 29th for all months in a leap year, or for all months except February in a non-leap year, for which it will be the 28th.
    - **Year:** A continuous period of time, consisting of 12 months, starting from midnight (00:00:00) of the Start Date and finishing at 23:59:59 of the day before the same date of the next year. The Period will be repeating on the same date every year from the Start Date. For example, if the Start Date is the 10th October 2024, then the Period will be repeating on the 10th October 2025, the 10th October 2026, and so on. The exception to this rule is the following:
        - If the Start Date is a date not available every year (29th February on a leap year), then the Period will be initiated on the 28th of February on non-leap years and the 29th on leap years.

Balance Check Permission

Long-lived Multi-Payments Consents may also include the User's permission to perform a check of the User's payment account balance before initiating a payment under that consent. This permission is not a substitute for Bank Data Sharing.

This capability allows TPPs to:

- check the balance in advance of payments to be initiated as part of the Multi-Payments Consent and ask Users to take remedial action if the funds are insufficient; and
- display the balance to Users at the point of initiating a payment.

Balance Check

Prior to Payment Initiation, TPPs can optionally check the balance of the account for sufficient funds for the payment and display the balance to Users, subject to having received Balance Check permission during the Multi-Payments Consent setup.

Balance Check

TPPs **MUST**:

1. MPGN-042 Be able to request balance information using the authorized Multi-Payment Consent before initiating a payment.
2. MPGN-043 Use this capability only in relation to the Payment Initiation steps that follow.
3. MPGN-044 **NOT** use this capability as an alternative for the Account Information Service — for example, to satisfy use cases like Personal Finance Manager (PFM), where the account balance is being refreshed periodically.

Balance Check Request Processing

LFIs MUST:

1. MPGN-045 Provide the OFP with the information related to the balance of the User's account, which was used for authorizing the Multi-Payment Consent. This information **MUST** be the same information as defined under the Balances permission in *Account Information | Data Cluster Structure & Language*.

OFP MUST:

2. MPGN-046 Provide TPPs with all the available information in relation to the Balance Check request.
3. MPGN-047 Send an appropriate error response to TPPs in case the Balance Check request was not successful.

Payment Status Update & Payment Notifications

Payment Status Update

MPGN-048 As per Common Rules and Guidelines | Payment Status Update.

Payment Notifications

MPGN-049 As per Common Rules and Guidelines | Payment Notifications.

Multi-Payment (Generic) | Consent Updates
Consent Update

TPPs **SHOULD**:

1. MPGN-050 Enable Users to use the Consent Dashboard to amend the following parameters of a long-lived Payment consent:

- User Payment (Debtor) Account
- Debtor Reference (Optional)
- Creditor Reference
- Purpose of Payment
- Consent Control Parameters (`MaximumIndividualAmount`, cumulative value/number caps, lifetime and per-period)
- Consent Control Period & Start Date
- Consent Expiration Date & Time
- Predefined Beneficiary List — only when the consent uses the Defined List of Creditors variant

TPPs **MUST**:

2. MPGN-051 When staging a new Multi-Payment consent to replace an existing payment consent, provide the original `BaseConsentId` where the same payment service is being provided and the existing and new consents are required to be linked to ensure continuity of service.
    1. MPGN-052 When constructing the new consent, provide a new, unique `ConsentId`.
3. MPGN-053 Require the User to authenticate with their LFI and authorise the consent update where changes to the consent are permitted.

> **NOTE:**
> The parameters of the long-lived Multi-Payments Consent that Users are able to update depend on the use case and the business requirements of each TPP. There may be scenarios where TPPs are fine to allow Users to update one or more of the Consent parameters, and other scenarios where Users are not allowed to make any changes to the Consent parameters. In any case, Users are always able to revoke their Consent.

Creditor Requirements

Variable-on-Demand Multi-Payments can be configured to support different Creditor arrangements depending on the customer's intended use case and consent scope. The `Initiation.Creditor` array in the `PersonalIdentifiableInformation` object can be used to define whether payments will be made to a single Creditor, or a defined list of multiple Creditors.

Single Creditor: When all payments during the lifetime of the consent will be made to a single Creditor, TPPs **MUST**:

- MPOD-001 Provide the full Creditor details in the `PersonalIdentifiableInformation` object in the PAR.
- MPOD-002 Select Multi-Payment type as "Variable-on-Demand".
- MPOD-003 Populate the agreed payment details and limits within the `PeriodicSchedule` object.

Defined List of Creditors: When payments may be made to a defined list of Creditors during the lifetime of the consent, TPPs **MUST**:

- MPOD-004 Provide the full details for each Creditor (between two and ten) in the `PersonalIdentifiableInformation` object in the PAR.
- MPOD-005 When `VariableOnDemand` is populated as the `PeriodicSchedule.Type`, the TPP **MUST** include the control parameters and payment limits agreed with the customer in the PAR.

Variable On-Demand Multi-Payments to Multi-Creditors

A Variable On-Demand Multi-Payment Consent can be used to support payments made to Multi-Creditors.

Multi-Creditors

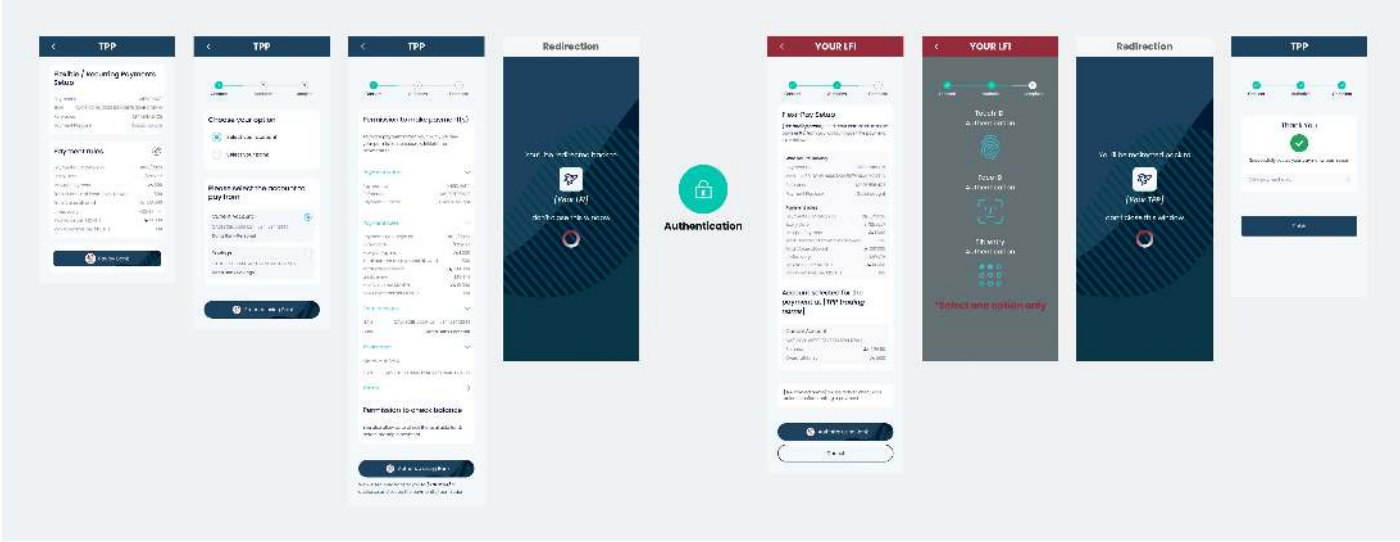
MPOD-006 Payments may be made to a defined list of Creditors (beneficiaries) that has been agreed in advance between the Customer and the User.

- The list of Creditors, including their Creditor Account details, **MUST** be provided in the PAR within the `PersonalIdentifiableInformation` object.
- The payment control parameters and limits **MUST** be defined within the consent to be authorized.
- The User authorizes these Creditors and payment control parameters when approving the consent with the LFI.
- Payments initiated to these authorized Creditors do not require additional User authorization at the time of execution.

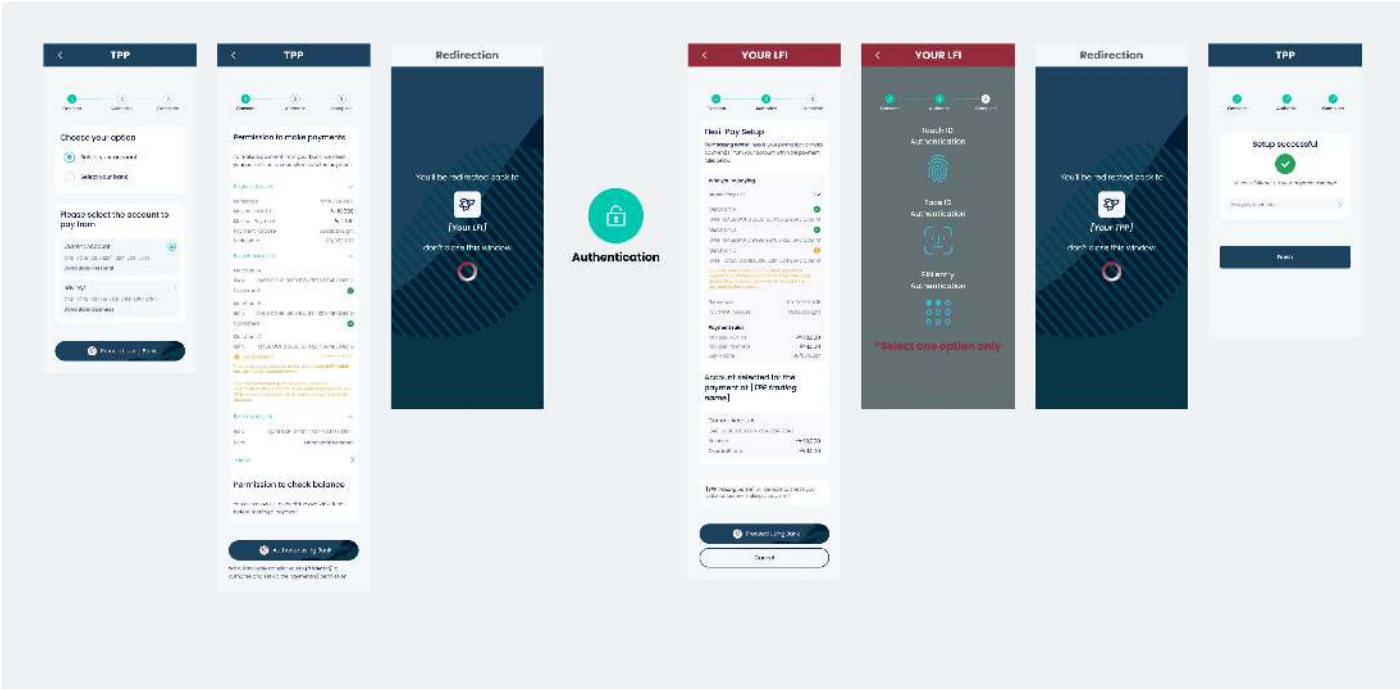
Example Multi-Creditors User Story

> **User Story**
>
> **As a** User (Consumer or Business),
>
> **I want to** use a TPP to set up a sequence of future on-demand payments of variable amounts to a predefined list of domestic beneficiary accounts owned by one or more businesses or individuals,
>
> **so that** I can arrange to transfer funds or pay the relevant beneficiaries automatically without needing to be present or take any action with the TPP each time a payment is initiated.

Consent Set-up - Variable On-Demand Payments



Consent Set-up - Variable On-Demand Payments to a Defined List of Creditors



TPP Consent Staging Rules & Guidelines - Variable On-Demand Consent Set-up

TPPs **MUST**:

Consent Cumulative Control Parameters

`MultiPayment` Attribute	**Required**	**Consent Cumulative Control Parameter Rules**
`MaximumCumulativeValueOfPayments`	No	This optional control parameter defines the maximum cumulative value of payments that may be initiated over the lifetime of the consent. This control limit is independent of the maximum cumulative number of payments, where defined, and any maximum cumulative value of payments per period. As a result, it does not need to match the calculated total derived from the number of periods multiplied by the maximum cumulative value per period.
`MaximumCumulativeNumberOfPayments`	No	This optional control parameter defines the maximum cumulative number of payments that may be initiated over the lifetime of the consent. This control limit is independent of the maximum cumulative value of payments, where defined, and any maximum cumulative number of payments per period. As a result, it does not need to match the calculated total derived from the number of periods multiplied

`MultiPayment` Attribute	Required	Consent Cumulative Control Parameter Rules
		by the maximum cumulative number per period.

Consent Periodic Control Parameters

`PeriodicSchedule` Attribute	Required	Consent Peri Parameter Ru
`PeriodType`	Yes	A mandatory defines the le recurring peri the maximum value of paym period and/or cumulative nu payments per applies. The s period option Week, Month
`PeriodStartDate`	Yes	A mandatory defines when and monthly r period starts. `PeriodType` i this value def from which pa be initiated.
`Controls.MaximumIndividualAmount`	At least one of the control parameters **MUST** be provided.	When provide parameter de maximum ind payment amo over the lifeti consent. The

`PeriodicSchedule` Attribute	Required	Consent Peri Parameter Ru
	Two or all three control parameters MAY be provided.	amount speci payment initia **MUST** be equ than this valu [Common Rul Guidelines
`Controls.MaximumCumulativeValueOfPaymentsPerPeriod`	At least one of the control parameters **MUST** be provided. Two or all three control parameters MAY be provided.	When provide parameter de maximum cur of payments t initiated withi period. This c independent maximum cur number of pa period, where **MUST** be gre `MaximumIndi` when that par provided.
`Controls.MaximumCumulativeNumberOfPaymentsPerPeriod`	At least one of the control parameters **MUST** be provided. Two or all three control parameters MAY be provided.	When provide parameter de maximum cur number of pa may be initiat defined perio limit is indepe maximum cur of payments where define

Consent Expiration DateTime

`ExpirationDateTime` Attribute	Required	Consent Expiration Date-Time Rules
`ExpirationDateTime`	Yes	TPPs **MUST** either: allow Users to manually enter or select the date at which the Variable On-Demand consent will end, or pre-populate the value for Users based on the specific business case and the receiving beneficiary party.

Multi-Payments Common Rules & Guidelines

Creditor Identification for Variable-defined Beneficiaries

TPPs **MUST** either allow Users to specify the below set of parameters, or pre-populate them for Users based on the specific use case or the requirements of their receiving beneficiary customers:

- MPOD-007 **Predefined Beneficiary List:** this is a list of pre-defined beneficiaries (i.e. Creditor identification details). The identification details of each Creditor **MUST** be populated as per Common Rules and Guidelines | Creditor Identification and can be in any of the supported Creditor identification formats. For example:
    - Predefined Beneficiary List as follows:
        - Creditor #1: IBAN, Name

Creditor Identification for Variable Beneficiaries

MPOD-008 TPPs **MUST** provide a message to Users clearly stating that:

- The Payee will not be specified as part of the Consent and will instead be defined during the payment initiation.
- The Payee will be confirmed when specified, before the actual payment takes place.
- The User **MUST** agree that one or more “factors” (i.e. conditions) stated in the Consent will be used by the TPP as their explicit authorization for each payment initiation.

TPP and User Events to Agree to Initiate Payments

MPOD-009 TPPs **MUST** require Users to agree there will be one (or more) event(s) that the TPP will use to signal the User's agreement for a payment initiation to be processed under the Multi-Payments consent.

Examples of these events could include the following:

- A User stating a single-purpose secret word or number string to a merchant, where the secret word or number string frequently changes and is known only to the User and the merchant via a secure channel.
- A User tapping a wristband (featuring RFID technology) on a merchant's scanner.
- A user ordering a coffee at a merchant's kiosk, where the User has registered with the merchant's app via a smart device and the merchant can detect that the User's device is present at the kiosk.
- A User, and the licence plate of their vehicle, both registered with a merchant's mobile app, having their vehicle refuelled at the same merchant's fuel pump, where their vehicle's licence plate is recognised by the merchant's cameras as being present while a refuelling request was made.

MPOD-010 TPPs **MUST** require one (or more) event(s) to demonstrate the User's agreement for a payment initiation to be processed. The event(s) will be agreed between TPPs and Users as the qualifying conditions for each payment initiation to be considered agreed for processing.

MPOD-011 The nature of these agreement events **MUST** be presented to Users while providing initial, explicit consent for the long-lived Multi-Payment consent to exist with their designated LFI.

MPOD-012 The event(s) applicable to any payment initiated by TPPs under a Multi-Payment **MUST** be recorded and stored by the TPPs in case a dispute is raised about the payment (initiation).

Payment Initiation — Submission Rules for Variable On-Demand Payments (VariableOnDemand)

MPOD-013 Once the Variable On-Demand consent has been successfully authorised, payments **MAY** be initiated at any time in accordance with the terms of the consent (individual and cumulative control parameters, and, if applicable, the agreed Creditor scope).

Payment Initiation

All Variable On-Demand consents — TPPs MUST:

1. MPOD-014 **NOT** submit any payment initiation request with an amount greater than the maximum individual payment amount specified in the authorised consent.
2. MPOD-015 Track the cumulative value of all payment initiations and **MUST NOT** submit any request that would result in the maximum cumulative payment value being exceeded, where such a limit is defined.
3. MPOD-016 Track the cumulative number of payment initiations and **MUST NOT** submit any request that would result in the maximum cumulative number of payments being exceeded, where such a limit is defined.
4. MPOD-017 Track the cumulative payment value of all initiations within each defined Control Period and **MUST NOT** submit any request that would exceed the maximum cumulative value per period, where defined.
5. MPOD-018 Track the cumulative number of payment initiations within each defined Control Period and **MUST NOT** submit any request that would exceed the maximum cumulative number of payments per period, where defined.

If this consent uses a Defined List of Creditors — TPPs MUST additionally:

6. MPOD-019 Include, in each payment initiation request, one of the Creditor Identification details from the Pre-defined Beneficiary List.

Processing of Payment Initiation Requests for Variable On-Demand Payments (VariableOnDemand)

Processing of Payment Initiation Requests

OFP MUST:

1. MPOD-020 Allow TPPs to submit individual payment initiation requests under a long-lived Payment Consent authorised by the User, without requiring any additional Multi-Factor Authentication (MFA) or further authorisation from the User.
2. MPOD-021 Verify that each received payment initiation request relates to a valid long-lived Payment Consent authorized by the User.
    1. MPOD-022 The Payment Consent **MUST** be in the **Authorized** state.
    2. MPOD-023 Reject any payment initiation request that relates to a Payment Consent in any other state (for example, *consumed* or *expired*) and **MUST** return an appropriate error code and error message to the TPP.
3. Validate the parameters of each payment initiation request against the authorised long-lived Payment Consent. As part of this validation, the OFP **MUST** verify the following:
    1. MPOD-024 **Payment execution date** — the payment execution date in the payment initiation request **MUST** fall within the validity period of the long-lived Payment Consent (that is, on or before the Consent Expiration DateTime).
    2. MPOD-025 **Payment amount** — the payment amount in the payment initiation request **MUST** be less than or equal to the maximum individual payment amount for the Variable On-Demand (`VariableOnDemand`) consent.

Maximum Cumulative Control Parameters

> **NOTE:**
>
> As the cumulative value and number of payments, for both the life of the consent and the current period, are retained and managed by the OFP, and these limits are validated when a payment request is received, there is no requirement for the LFI to retain these totals or perform this validation.

OFP MUST:

4. Validate each payment initiation request against any maximum cumulative control parameters defined in the authorised consent. As part of this validation, the OFP **MUST** verify that:
    1. MPOD-026 the cumulative total value of all payment initiations, including the submitted request, is ≤ `MaximumCumulativeValueOfPayments`, where provided;
    2. MPOD-027 the cumulative total number of payment initiations, including the submitted request, is ≤ `MaximumCumulativeNumberOfPayments`, where provided;
    3. MPOD-028 the cumulative total value of all payment initiations within the current Consent Control Period, including the submitted request, is ≤ `MaximumCumulativeValueOfPaymentsPerPeriod`, where provided;
    4. MPOD-029 the cumulative total number of payment initiations within the current Consent Control Period, including the submitted request, is ≤ `MaximumCumulativeNumberOfPaymentsPerPeriod`, where provided.
5. MPOD-030 Increment the cumulative Total Number and Total Value of payments under the Consent after the payment resource has been created and is in 'Pending' status and has not been rejected. Initial value **MUST** be zero for each authorized Consent.
6. MPOD-031 Increment the cumulative Total Number and Total Value of payment initiations per Control Period after the payment resource has been created and is in 'Pending' status and has not been rejected.
    1. MPOD-032 These per-Control-Period totals are reset to zero when a new Consent Control Period starts at 00:00:00 of the first day of the Control Period.
7. MPOD-033 Set the long-lived Payment Consent state to a terminal state (**Consumed**), if the cumulative total value and/or number of payment requests becomes equal to the `MaximumCumulativeValueOfPayments`/`MaximumCumulativeNumberOfPayments` parameter of the Consent, where provided.

OFP MUST:

8. MPOD-034 Allow the description of the Creditor Reference in the submitted request to differ from the one defined in the consent.
9. MPOD-035 **Reject** the payment initiation and provide the necessary error message to the TPP if any other checks fail against the consent's parameters.
    1. MPOD-036 In this case, the OFP **MUST** decrement the Consent Control Parameter Values and, where applicable, the Control Period totals, as defined in COMM-150 and COMM-151.
10. MPOD-037 Send a payment initiation request to the LFI including: User Payment Account (or account identifier), Payment Amount & Currency, PII object, Debtor Reference (if

provided); Creditor Reference; Purpose of Payment.

LFIs MUST:

11. MPOD-038 Trigger the payment initiation process **immediately** after receiving the request from the OFP.

    If this consent uses a single known Creditor ("Fixed Beneficiaries"):

    1. MPOD-039 Retrieve the Creditor Identification details from the encrypted PII block included in the original Payment Consent message.
    2. MPOD-040 Apply all existing BAU check and validation processes to those details. On failure, reject and notify the OFP with an appropriate error message.

    If this consent uses a Defined List of Creditors ("Variable-defined Beneficiaries"): 3. [MPOD-041] Check and validate the Creditor Identification details in the request against the Creditor details in the authorized consent. If they do not exactly match any entry in the Predefined Beneficiary List, reject and notify the OFP with an appropriate error message.

12. MPOD-042 Allow the OFP to submit the request without additional MFA or authorization from the User.

13. MPOD-043 Additionally apply all existing BAU payment account controls and limits (for example, single and total transaction value limits, and AML checking), as if initiated via the LFI's existing channels. Send an appropriate error response to the OFP on rejection.

    1. MPOD-044 **Reject** the payment initiation if the Purpose of Payment violates any existing BAU blocking/rejecting rules. Notify the OFP with an appropriate error message.

14. MPOD-045 **Reject** the payment initiation if the selected account has insufficient funds. Notify the OFP with an appropriate error message.

15. MPOD-046 Subject to successful BAU checking, proceed with execution via the underlying payment rails or as an intra-bank payment.

16. MPOD-047 Provide the OFP with all available information on the initiated payment instruction, including the Payment Transaction ID (see COMM-272). The format of the Payment Transaction ID can be found in the **QCB Open Finance Standard** specifications.

OFP MUST:

17. MPOD-048 Send an appropriate error response to the TPP if the payment is rejected due to the LFI's BAU checks or limits.
    1. MPOD-049 Decrement the Consent Control Parameter Values and, where applicable, the Control Period totals, as defined in COMM-150 and COMM-151.
18. MPOD-050 Send to the TPP the appropriate error message if the payment initiation was rejected due to insufficient funds.
19. MPOD-051 Provide the TPP with all available information on the initiated payment instruction, including the Payment Transaction ID (see COMM-272).

Multi-Payment Example Use Cases

Variable On-Demand Payments (`VariableOnDemand`) Example Use Cases

Sweeping Rounding-Up Use Case

A User signs up for an account sweeping service with a TPP that automatically transfers funds from the User's current account held with one LFI to their savings account held with another LFI.

The TPP's sweeping service is based on a "rounding-up" principle, which will automatically round up the User's current account transactions to the nearest 10 QAR or selected currency.

The value between the current account transaction amount and the rounded-up amount is automatically transferred to the User's savings account by the TPP using the Multi Payment `VariableOnDemand` payment consent.

The User has a savings goal of QAR 5,000 and wishes to save up to QAR 100.00 during a week.

Point of Sale (PoS) Purchases

A user opts to utilize an Open Finance payment option provided by their favourite coffee shop. This payment solution enables the coffee shop to directly request payments from the user's bank account whenever a purchase is made.

To process the payments, the coffee shop partners with a third-party payment initiation service who set up a Multi Payment `VariableOnDemand` payment consent with the User. This multi-payment consent is used as the payment amounts and initiation dates are unknown and depend on when the User makes a purchase at the coffee shop.

As part of the set-up, as the value of each purchase is unknown, the User sets a cap on the maximum individual payment that can be initiated along with a total daily spending limit to prevent unauthorized or accidental overcharges.

Controlled Financial Support (Funds Drawdown)

A user enrols in a TPP service that facilitates controlled financial support for a family member. Through this service, the User can set a predefined weekly or monthly allowance limit to cap the funds available to the family member within the selected period.

In this scenario, the User sets a monthly allowance limit of QAR 2,500, with access to the first funds becoming available on 4th October 2025.

As this arrangement allows the family member to draw down the entire QAR 2,500 limit in a single transaction or make multiple smaller individual requests up to the established limit each month a Multi Payment `VariableOnDemand` payment consent is set-up with the User.

Once the family member has drawn down the full QAR 2,500, either in a single transaction or over multiple smaller transactions, they must wait until the next monthly period begins before making further requests. In this scenario, a new monthly period starts on the 4th of each month, ensuring that the allowance resets in alignment with the predefined schedule.

Constants and Limits | Limits

ID	Name	Description
LIMT-001	Max Consent Validity Period	This is the maximum validity period allowed for a Long-lived Consent. This period is defined to be 1 year.
LIMT-002	Max Authorization Time Window	The maximum allowable time from when the TPP submits (i.e. staged) the Consent to the LFI until it is authorized by all required authorizers (one or more) is defined as the consent expiry duration.
LIMT-003	Max Submitter Authorization Time Window	The maximum allowable time from the Consent being submitted (i.e. staged) by the TPP to the LFI until the Consent is authorized by the User who is initiating the Consent via the TPP. In case of multiple authorizers, this will be the first authorizer in the authorization matrix. This period is defined to be 2 hours.
LIMT-004	Max Variable On-Demand Payment Amount	The maximum allowable amount of each Variable On-Demand payment. This will be agreed between Users and TPPs.
LIMT-005	Max Variable On-Demand Consent Cumulative Value	The maximum Cumulative Value of all payment initiations for the entire Consent validity period. This will be agreed between Users and TPPs.
LIMT-006	Max Variable On-Demand Consent Cumulative Payments	The Maximum Cumulative Number of all payment initiations for the entire Consent validity period. This will be agreed between Users and the TPPs.
LIMT-007	Max Variable On-Demand Cumulative Value per Control Period	The Maximum Cumulative Value of all payment initiations per Consent Control Period. This will be agreed between Users and the TPPs.
LIMT-008	Max Variable On-Demand	The Maximum Cumulative Number of all payment initiations per Consent Control Period. This will be agreed between

ID	Name	Description
	Cumulative Payments per Control Period	Users and the TPPs.
LIMT-009	Maximum Number of Creditors in a Multi-Creditor Payment Initiation Consent	For Multi-Creditor Payment Initiation Consents, the number of allowed Creditors, which is between two and ten Creditors.
LIMT-010	Max Inter-bank Payment Amount	The Maximum payment amount for inter-bank payments allowed to be initiated by a TPP using the QCB Open Finance Standard. This is equal to the maximum allowable payment value supported by Tahweel, which is currently QAR 50,000,000. Payments above the maximum payment value accepted by the Fawran Core platform (see Fawran Implementation Guide v5.4, section 1.4 Payment Validation Service, Maximum Amount Check) are directed to Tahweel, as described in Supported Accounts & Payment Rails.
LIMT-011	End Date of historical data for Data Sharing Request	The end date of the period of historical data that can be requested by TPPs and which MUST be sent by LFIs for Data Sharing Requests. LFIs MUST provide the historical data for the period that they are stored in their systems of record and are available to Users in the LFIs' existing online or mobile channels. As a minimum, all LFIs MUST be able to provide at least 5 years of historical data, if it exists.
LIMT-012	Max SIP Payment Initiation Time Interval	This is the period of time within which TPPs MUST submit the Single Instant Payment Initiation Request to the OFP. The value defined for this period is currently 10 min. The OFP will reject the Payment Initiation Request if it is submitted outside this time window.

ID	Name	Description
LIMT-013	Payment Status Update (OFP SLA 2)	This is the time required by the LFI to update the Payment resource status, using a `PATCH /payments/{PaymentId}` endpoint, with the latest status update, after receiving a response from the generic payment system, if any. The value defined for this period is currently 500ms. Note: this time period requires execution both from the LFIs and the API Hub. Therefore, the internal SLAs are defined to be: - 250ms for the LFI, from the point in time the LFI receives a response about the payment from the generic payment system, if any, until the LFI sends the `PATCH /payments/{PaymentId}` endpoint call with the updated status. - 250ms for the `PATCH` endpoint call to reach the Consent Manager and be processed by the API Hub to update the payment record.
LIMT-014	Event Notification SLA (OFP SLA 3)	This is the time taken for the event notification `POST {WebhookUrl}` endpoint call to be initiated and sent to the TPP by the API Hub, following a change of status of the Consent resource or the Payment resource. The value defined for this period is currently 500ms.
LIMT-026	Tahweel Settlement Windows and Cut-off Times	The periods of a Tahweel settlement session, where Day 1 is the day the session starts: - Start: Day 1, 12:00 to 12:30 - Exchange, to send and receive payments: Day 1, 12:30 to Day 2, 12:30 - Exchange, response period for payments sent and received: Day 1, 12:30 to Day 4, 12:30 - Cut-off: Day 4, 12:30 to 12:40 - Settlement preparations: Day 4, 12:40 to 12:50 - Settlement: Day 4, 12:50 to 13:00 - Final cut-off: Day 4, 13:00 to 13:10 - Closure: Day 4, 13:10 to 13:15

ID	Name	Description
		A payment directed to Tahweel MUST be submitted during the exchange period to send and receive payments, to be settled in that session. Tahweel returns the settlement window in which the payment will be settled in its validation status report (see QATCH Advanced Link Interface Specifications v2.7, section 3.1 Status of Validation).
LIMT-027	Settlement Reporting Times	The times by which settlement is reported and posted for each payment rail: - Fawran Core: settlement reports are available by 11:30, and settlement is posted to RTGS by 13:45. - Tahweel: Creditor LFIs respond to payments within the exchange response period of the settlement session (see LIMT-026); settlement reports are available by 15:00, and settlement is posted to RTGS between 12:00 and 13:30.

Constants and Limits | Constants

ID	Name	Description
LIMT-019	Single-use Consent Duration for Data	In the context of data sharing, a single-use consent refers to a short-lived consent valid for a period of time from the time the User provides explicit Consent to the TPP. The value defined for this period is currently 24 hours.
LIMT-023	Max Payment Reference Length	The maximum length of the Debtor Reference and the Creditor Reference. This is equal to the lowest maximum remittance information length supported by the agreed domestic payment rails, so that a reference accepted at payment initiation can be carried by whichever rail the payment is routed through. The value defined for this length is currently 140 characters, the maximum length of the ISO 20022 unstructured remittance information field (see Fawran Implementation Guide v5.4, section 4.9 pacs.008.001.08 and Appendix A: XSD(s); and QATCH Advanced Link Interface Specifications v2.7, section 2.1 Direct Credit).
LIMT-024	Payment Reference Character Set	The characters allowed in the Debtor Reference and the Creditor Reference. The payment rails place no specific requirement on these references, so the allowed characters are those accepted by the QCB Open Finance API: A–Z, a–z, 0–9, space and the characters / ? : ( ) . , ′ + -.
LIMT-025	Payment Purpose Codes	The category purpose codes, and their descriptions, allowed for domestic Open Finance payments. These are the domestic category purpose codes defined in Fawran Implementation Guide v5.4, Appendix G: External Category Purpose-Code (section 11.1 Individuals and section 11.2 Corporates), in the format defined there. For payments directed to Tahweel, the LFI converts the code as described in Purpose of Payment.

LFI Channel Limits

ID	Name	Description
LIMT-020	Me-to-me Payments (Single or Variable On-Demand) Limit	The per-transaction limit for Me-to-Me payments MUST be set to the higher of 5,000 QAR or the equivalent LFI Channel/User limit. *For clarity: this means that the maximum allowed amount for an Open Finance Me-to-Me transaction must be set to the greater of:* *1. The maximum transaction amount currently allowed for the customer via a comparable digital channel (e.g., a transfer through the mobile banking app), or* *2. 5,000 QAR* *In short: Me-to-Me limits should not fall below 5,000 QAR, but they can go higher if the equivalent internal channel allows it. A payment above the maximum payment value accepted by Fawran Core is not rejected for that reason, but directed to Tahweel (COMM-261), up to the Max Inter-bank Payment Amount (LIMT-010).*
LIMT-021	Third-Party Payments (Single or Variable On-Demand) Limit	The per-transaction limit for Third-Party payments MUST be set to the higher of 200 QAR or the equivalent LFI Channel/User limit.
LIMT-022	Open Beneficiary Variable On-Demand Payments Limit	The per-transaction limit for Open Beneficiary Variable On-Demand payments MUST be set to the higher of 800 QAR or the equivalent LFI Channel/User limit.

Overview

The Implementation Guides show how to put the Qatar Open Finance Standards into practice. Where the business and technical rules set out what participants must do and the API descriptions define the contract, these guides walk through the sequence of calls end to end — establishing consent, obtaining an access token, and exercising the relevant endpoints.

They are written for the developers and integrators building on behalf of Financial Institutions (FIs) and Third Party Providers (TPPs). Follow the guide for the capability you are implementing.

Retrieving Accounts Information Data →

Establish consent and call the Account Information endpoints to read accounts, balances and transactions.

Making a Single Instant Payment →

Create a payment consent, authorize it, and submit the payment order.

Making a Variable On-Demand Payment →

Set up a long-lived consent for a series of future payments of variable amounts, and initiate against it.

Retrieving Accounts Information Data | Overview

Bank Data Sharing is a bank service request. It lets a TPP retrieve a User's account data, held at an LFI, with the User's consent, under an Account Access Consent that may be Single-Use (short-lived) or Long-Lived. See Account Information | Description for the full scope, and Consent Set-up & Authorization for the ACCI rules this guide implements. Those sit on top of the Common Rules and Guidelines that apply across every Open Finance journey.

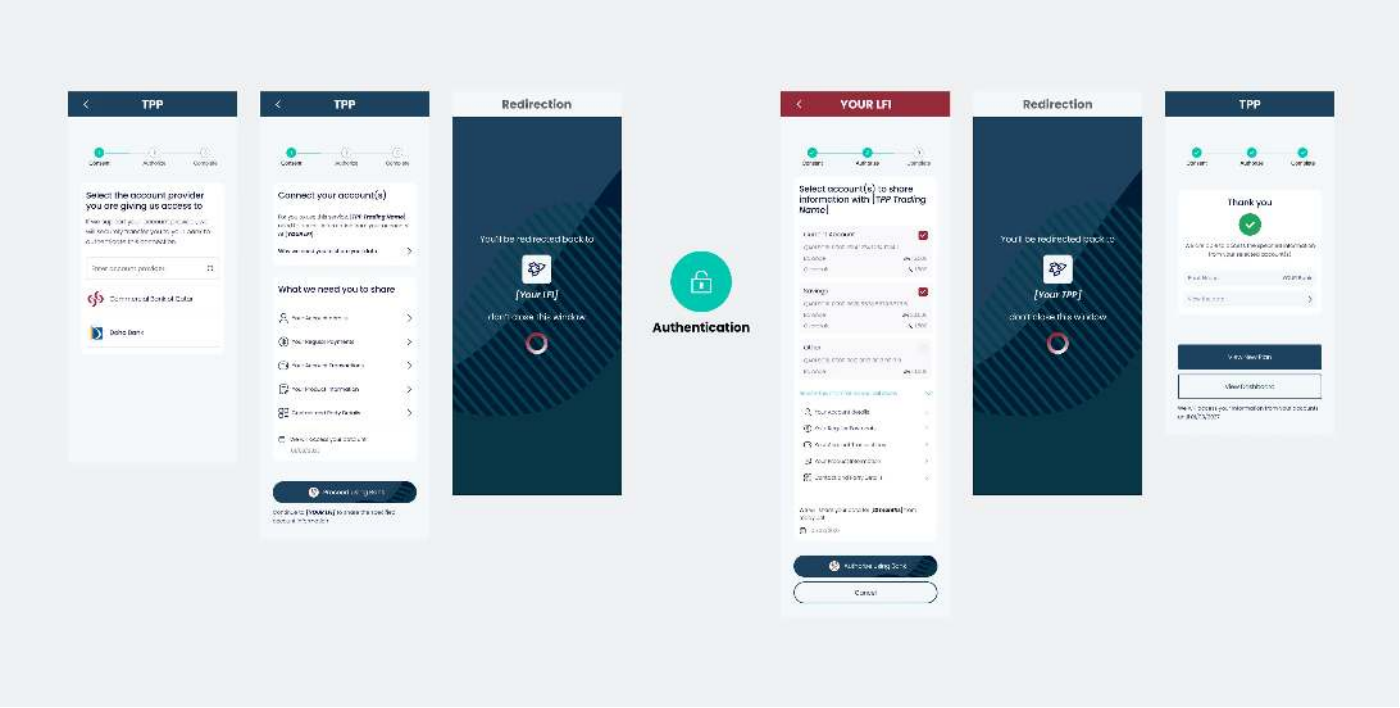


Three roles, three guides

Three parties build something to make Bank Data Sharing happen. The TPP collects consent and requests data. The LFI authenticates and authorizes the User, then shares the data. The OFP brokers both hand-offs and routes every data request in between. Each role gets its own end-to-end guide, covering every rule relevant to that role:

- **TPP Implementation Guide**: onboard the User, collect and stage the consent, hand the User off to the LFI, submit data sharing requests once authorized, and manage the consent over its lifetime (revocation, updates).
- **LFI Implementation Guide**: receive the hand-off, authenticate the User, disclose the consent, handle account selection, present and capture authorization, share the requested data, and support consent lifecycle events.
- **OFP Implementation Guide**: validate the staged consent, broker both hand-offs, maintain the consent state machine, and route data sharing requests.

All three converge on the same underlying flow:



1. The TPP collects consent parameters (data clusters, duration, LFI selection) and stages the Consent with the OFP. The OFP validates the Consent and records it as Awaiting Authorization.
2. The TPP hands the User off to the LFI, through the OFP. The OFP brokers the hand-off and provides the Consent details to the LFI.
3. The LFI authenticates the User, discloses what will be shared, resolves account selection, and presents the Consent for authorization. The OFP updates the Consent state as the LFI reports the outcome.
4. The LFI hands the User back to the TPP, through the OFP. The OFP brokers the hand-off back, returning the authorization code to the TPP.
5. The TPP submits data sharing requests. The OFP validates them against the authorized Consent and routes them to the LFI, which returns the data.
6. The TPP confirms the outcome to the User. For Long-Lived Consents, the User can revoke, pause, or update it at any time through the TPP's Consent Management Interface. The OFP records revocations and updates in the Consent state for both the TPP and the LFI; a pause is held by the TPP only and does not change the Consent state.

Each role's guide states plainly, step by step, which rule it implements, whether that's a ACCI rule or a common one.

> **Note**
>
> This covers the standard journey. For non-account-specific consents (Contact and Party data only, for example) or an LFI acting as a TPP, see Handle Journey Variations.

Retrieving Accounts Information Data | TPP Implementation Guide

This guide is for TPPs implementing Bank Data Sharing: retrieving a User's account data held at an LFI, with the User's consent, under an Account Access Consent that may be Single-Use or Long-Lived. See Account Information | Description for the full scope (Consumer and SME data-sharing segments).

Every step below cites the exact rule it implements, drawing on Account Information | Consent Set-up & Authorization | ACCI rules, Account Information | Data Sharing Requests | ACCI rules, the Common Rules and Guidelines, Common Components, and Constants and Limits.

1. Onboard the User (where required)

Implements: Initiate User Set-up (Conditional).

Depending on the use case, onboard the User before requesting data-sharing consent:

1. Provide Terms & Conditions and a Privacy Notice covering the applicable rights and responsibilities. Cover any onward sharing of personal data, who receives it, and the lawful basis for processing it. See Consent Setup | Consent Codification.
2. Obtain the User's agreement before setting them up, then request Consent as the next step.
3. Give the User an option to cancel the flow at this stage.

2. Collect and review the data sharing consent parameters

Implements: Data Sharing Consent.

Ground every consent screen in the codification rules that apply to all Data Sharing consent. Clearly explain the purpose data is being shared for and how long it's shared for, using five plain-language agreement parameters: Purpose Statement, Direct Benefit Statement, Data Request Statement, Duration Period Statement, and the Agreement itself. Purpose statements specifically must use simple language an average User can understand, tied to the actual benefit, without vague or excessive detail. See Consent Setup | Consent Codification | Data Sharing Consent.

On top of that baseline, build:

1. **Scope the request.** Only request the data required for the service or use case.
2. **Describe it in Data Cluster language.** Use the standard Data Cluster names and descriptions, not raw permission names, so the User understands what's being requested, and let them drill into the underlying permissions. See Permissions and Data Clusters for the full Data Cluster table (Your Account Details, Your Regular Payments, Your Account Transactions, Your Account Statements, Your Product Information, Contact and Party Details).
    - If the use case lets the User choose which data to share, offer that selection before the Consent screen, not on it.
3. **Confirm the duration.** Let the User select it where the use case allows:
    - **Single-use (short-lived).** State the maximum access period, per Limits and Constants | Single-use Consent Duration for Data (24 hours), and what happens to the data once the service is delivered.
    - **Long-lived.** Confirm the consent expires, and may need renewal, at the end of the sharing period, and what happens to the data and service if it isn't renewed.
    - Either way, always set an expiry date, bounded by the Max Consent Validity Period (1 year). The OFP rejects a consent with no expiry date at all.
4. **Set the historical data window**, if any requested permission is transaction-related. See Data Cluster Definitions for which permissions are transaction-related, and the End Date of historical data for Data Sharing Request for how far back an LFI must be able to return data (a minimum of 5 years).
5. **Confirm withdrawal rights.** Tell the User they can withdraw consent at any time before it expires.
6. **Show your trading or brand name** to the User, the OFP, and the LFI, clearly, plus the name of any other party you're supporting, if applicable. See Step 9 for the different-entity case.
7. **Let the User pick their LFI**, with a way to search for it.

Then set the additional consent controls, which you don't display to the User:

8. **Single Authorization Indicator.** Set or clear it as appropriate. See Is Single Authorization flag.
9. **Authorization Expiration DateTime.** Set it where there are specific timing requirements (this matters for multi-authorizer consents too). See Multi User Authorization Flow and Authorization Expiration Date Time.
10. **Capture explicit consent** to access the products held at the selected LFI.

3. Stage the consent

Implements: Consent Staging, which applies Consent Staging.

Submit the codified consent, as one or more RARs via a PAR request, per Consent Setup | Codification, to the OFP before you redirect the User to the LFI. See the Making a Single Instant Payment guide's equivalent step for the general staging mechanics, which apply here unchanged.

4. Hand off the User to the LFI

Implements: Hand-off to LFI.

Notify the User they're being transferred to the selected LFI to authenticate and authorize, per Hand-off to LFI. The same redirection-screen mechanics described in the SIP TPP guide | Step 4 apply here too.

5. Wait for the LFI to disclose and authorize

The LFI now authenticates the User, discloses exactly what data will be shared and for how long, handles account selection (including the no-account-list variation, see Step 11), and captures authorization or cancellation. See the LFI Implementation Guide for the full detail. There's nothing for you to build at this step beyond handling the hand-off back.

6. Receive the hand-off back from the LFI

Implements: Hand-off back to the TPP, which applies Hand-off back to the TPP.

Resume control once the OFP redirects the User back with the required authorization code.

7. Submit data sharing requests

Implements: Data Sharing Request.

1. Only request data within the scope of your service or use case.
2. Only submit requests to the OFP for the data clusters and permissions the User actually consented to.

The OFP validates the request against the authorized Consent (state, clusters, permissions) before forwarding it to the LFI. See the OFP Implementation Guide for what happens next. Handle a rejection from the OFP, whether from an invalid or expired Consent state or a mismatch against the authorized clusters and permissions, with an appropriate error rather than a generic failure.

8. Confirm to the User

Implements: Confirmation to User, which applies Confirmation to User.

Confirm to the User the outcome of their Consent Authorization with the LFI, for Long-lived Consents, including whether further authorizations are required, and confirm the outcome again once data is retrieved.

9. Manage the consent over its lifetime

Long-Lived data-sharing consents don't end at authorization. Build a Consent Management Interface (CMI) so Users can view, revoke, pause, and update them, per Consent Setup | Rules and Guidelines | TPPs MUST and TPP Consent Management Interfaces | Overview:

1. **Show your trading or brand name and the User-facing entity's name**, if you're not the User-facing entity yourself, in both the Consent and the CMI.
2. **Present each Consent individually**, including each Consent inside a Consent Group.
3. **Keep your systems in sync** with the Consent's actual state at the OFP.
4. **Notify the OFP the moment a User revokes** a Consent through your CMI.
5. **Respect Combined Consent Group (CCG) constraints** when building a PAR: number of RARs, allowed combinations, and minimum and maximum RAR counts, as defined in the CCG-aware OpenAPI description.

Revocation. Build the CMI's cancellation journey per Consent Revocation Journey. Let Users cancel without obstruction from the CMI Detailed page. Interpose exactly one implications-of-cancelling page before the final Consent Cancellation Confirmation. State the concrete consequences (no further data sharing) and what happens to data already retrieved, presented at Data Cluster level and drillable to Permission level. Tell the LFI the Consent was withdrawn, and reflect the "Canceled" state back to the User on the CMI Overview page.

Updates. Where your use case supports it, let Users amend a Long-Lived Consent's data clusters or validity duration, up to the Max Consent Validity Period, from the CMI, per Consent Updates and Consent Update Journey. Highlight which parameters are changeable, require the

User to authenticate with their LFI and authorize the update, and set the original Consent's status to `Revoked` before forwarding the new one. The LFI, in turn, must let the User amend the related accounts before authorizing the update.

Naming. Label the CMI "Open Finance Connections," or "Open Finance Connected Accounts" if you also operate an LFI CMI under the same brand, per Naming of Consent Management Interfaces. Design it to be easy to find, intuitive, and transparent, with a visible history of finished, rejected, and cancelled consents, retained at least a year, per Principles Behind Effective CMIs.

10. Access Beneficiaries data (conditional)

Implements: Data Sharing Requests | Beneficiaries Data Sharing.

Where the Consent grants `ReadBeneficiariesBasic` or `ReadBeneficiariesDetail`, retrieve the User's trusted beneficiaries for a given account via `GET /accounts/{AccountId}/beneficiaries`. The response includes beneficiaries from both Open Finance and the LFI's own channels, distinguished by `AddedViaOF: true`.

11. Account for journey variations

Where the Consent includes only User-level data permissions, the Parties endpoint, for example, and no account-specific permissions, don't display any list of eligible accounts to the User. See Handle Journey Variations for the full detail.

Retrieving Accounts Information Data | LFI Implementation Guide

This guide is for LFIs supporting Bank Data Sharing: disclosing a User's account data to a TPP under an authorized Account Access Consent. See Account Information | Description for the full scope.

Every step below cites the exact rule it implements, drawing on Account Information | Consent Set-up & Authorization | ACCI rules, Account Information | Data Sharing Requests | ACCI rules, the Common Rules and Guidelines, and Common Components.

1. Support eligible accounts

Implements: Supported Accounts & Payment Rails.

Bank Data Sharing covers Retail and SME current accounts, savings accounts (including deposits), credit cards, loans, mortgages, wallets and currency accounts. Unlike Single Instant Payment, which is limited to current accounts in QAR, it must be supported in every currency you offer on these accounts. Everything else in that rule (dormant and inactive account filtering, and no additional constraints beyond the Standard's limits) applies exactly as it does for payments.

2. Receive the hand-off from the TPP

Implements: Hand-off to LFI and Hand-off to LFI.

Present no additional inbound redirection screens, request the staged Consent from the OFP by its unique identifier, and initiate User authentication immediately. See Step 3.

3. Authenticate the User

Implements: Authentication.

Authenticate with MFA, using the same redirection models described in the SIP LFI guide | Step 3. Redirection, Decoupled Redirection, redirection-screen requirements, error handling, and the underlying SCA principles all apply unchanged to Data Sharing. If authentication fails or the User cancels, redirect back to the TPP with information that the Consent was not authorized.

4. Disclose the consent details

Implements: Disclosure Consent.

1. The MFA from Step 3 already covers reviewing and authorizing the Consent. You don't re-authenticate for it.
2. Retrieve the staged data sharing Consent from the OFP using its unique identifier.
3. Display what data will be shared and for how long.
4. Describe the data clusters and permissions using the same Data Cluster language the TPP used. See Permissions and Data Clusters for the definitive table. Consistency here is what lets the User trust that what they consented to at the TPP is what they're authorizing here.

5. Select accounts

Implements: Select Accounts and Payment Account Selection at LFI.

1. Display the User's eligible accounts. Nicknames, icons, account number, and account type all work fine for this list. See Supported Accounts & Payment Rails.
2. Only show accounts for selection at all if the TPP specified an Account Type and Sub-Type in the consent request.
3. Let the User pick which account or accounts to share from, where the Consent has account-specific permissions and more than one account is available.
4. **Skip account selection entirely** where the Consent has only non-account-specific User data permissions, the Parties endpoint, for example. Don't show any account list in that case. See Step 11. Instead, show the User the full personal data that will be shared for the Consent's period.
5. Where the Consent is solely non-account-specific, let the User proceed straight to authorization without selecting an account at all.
6. Show any ineligible accounts too, with a reason they can't be selected.
7. Check the selected account's authorization status against the TPP's Accepted Authorization Type. See Check Is Single Authorization flag.

6. Present the consent for authorization

Implements: Confirmation / Authorization.

1. Present the full data sharing Consent details to the User.

2. Never allow the User to change any parameter the TPP already staged, permissions, for example.
3. Request the User's authorization.
4. Let the User cancel from within the authorization journey. Redirect back to the TPP with "not authorized" if they do, before final authorization.
5. Check that the Authorization Time Window is still valid.
6. Move the Consent from `AwaitingAuthorization` to `Authorized` once every required Authorizer has acted, and update the Consent details at the OFP, which confirms the update back to you.

7. Hand off back to the TPP

Implements: Hand-off back to the TPP and Hand-off back to the TPP.

Confirm session completion to the OFP, tell the User they're being returned to the TPP, and redirect them immediately once the OFP provides the authorization code. This uses the same mechanics as the SIP LFI guide | Step 8.

8. Share the requested data

Implements: Processing of Data Sharing Requests (LFI obligations).

1. Accept data sharing requests from the OFP without requiring any further MFA or User authorization.
2. Reject the request if the Consent has a valid reason to be suspended (see Consent States) or fails any other BAU check.
3. Share the data the OFP requested, for the authorized Consent.
4. Ensure the data you provide lets the TPP reconcile the account transactions it receives.

9. Maintain Beneficiaries lists

Implements: Data Sharing Requests | Beneficiaries Data Sharing.

Keep two clearly separate beneficiary lists: one for beneficiaries created through an Open Finance payment, one for beneficiaries created through your own banking channels. Return both

in the `GET /accounts/{AccountId}/beneficiaries` response, and flag Open-Finance-sourced entries with `AddedViaOF: true`.

10. Support consent lifecycle management

Long-Lived data sharing consents need ongoing LFI-side support, per Consent Setup | Rules and Guidelines | LFIs MUST and the LFI Consent Management Interfaces section:

1. Never allow a User to change a Consent's parameters through your own Consent Management Interface (CMI), in any Consent state.
2. Notify the OFP the moment a User revokes a Consent through your CMI.
3. Notify the OFP whenever a Consent moves into, or out of, `Suspended`. For example, a User's Qatar ID (QID) might expire, blocking access until it's renewed. Move it back to `Authorized` once fully cleared. See Consent States for the complete state machine.
4. Support Consent updates initiated from the TPP's CMI, per Consent Update Journey: let the User amend the accounts related to the Consent before they authorize the update.
5. Where you also operate your own CMI, label it "Open Finance Connected Services," or "Open Finance Connections" if acting solely as an LFI, and follow the same easy-to-find, intuitive, transparent design principles as a TPP CMI. See Naming of Consent Management Interfaces.
6. `Consumed` is not a valid terminal state for a Data Sharing Consent, Single-Use or Long-Lived. It applies only to Service Initiation.

11. Account for journey variations

Where the Consent contains only non-account-specific data permissions, skip account selection entirely (see Step 5). See Handle Journey Variations for the full detail, including the Contact/Party data cluster case illustrated there.

Retrieving Accounts Information Data | OFP Implementation Guide

This guide is for anyone implementing the OFP (Open Finance Platform) role in Bank Data Sharing: validating and staging the consent, brokering both hand-offs between TPP and LFI, and routing data sharing requests through to the LFI and back. See Account Information | Description for the full scope.

Every step below cites the exact rule it implements, drawing on Account Information | Consent Set-up & Authorization | ACCI rules, Account Information | Data Sharing Requests | ACCI rules, the Common Rules and Guidelines, and Common Components.

1. Validate and confirm the staged consent

Implements: Consent Staging and Consent Staging.

Validate the codified consent parameters, format and business-rule compliance, that the TPP submits. Ask the LFI to run its own additional validation, and reject with an appropriate error if either check fails. These are the same mechanics as the SIP OFP guide | Step 1.

2. Broker the hand-off to the LFI

Implements: Hand-off to LFI and Hand-off to LFI.

Provide the Consent details to the LFI, keyed on the unique Consent identifier, once it requests them after the TPP triggers its authorization endpoint.

3. Maintain the consent state machine

Implements: Consent Setup | Rules and Guidelines | OFP MUST and Consent States.

You are the system of record for every data sharing Consent and its state:

1. Check the Consent Expiration Date against the current date. Expire it to `Expired` if it's passed.
2. Check elapsed time against the Authorization Time Window and the Max Submitter Authorization Time Window. Reject an unauthorized Consent still `AwaitingAuthorization`

to `Rejected`, with an error code for the TPP, once either elapses.

3. Let both TPPs and LFIs check a Consent's status at any time.
4. Reject any Data Sharing request against a Consent that isn't `Authorized`.
5. Never allow a Consent update once it's reached a terminal state (`Rejected`, `Expired`, `Revoked`, but not `Consumed`, which doesn't apply to Data Sharing Consents at all, Single-Use or Long-Lived).
6. Where a Combined Consent Group (CCG) is submitted, apply its declared constraints (RAR count, allowed combinations, minimum and maximum RARs).
7. Reflect a `Suspended` state, and its resolution back to `Authorized`, whenever the LFI reports one. An expired Qatar ID (QID) blocking access until renewed is one example.

4. Process data sharing requests

Implements: Processing of Data Sharing Requests (OFP obligations).

1. Accept a TPP's data sharing request without demanding further MFA or User authorization.
2. Check the request maps to a Consent that's genuinely `Authorized`. Reject, with an error and code, any request against a Consent in a different state, `Expired`, for example.
3. Differentiate access by Consent type:
    - **Long-Lived.** Provide ongoing consented data and accounts.
    - **Single-Use.** Provide only the short-lived access window the Consent covers.
4. Reject the request, with an appropriate error to the TPP, if it doesn't match the data clusters and permissions the User actually authorized.
5. Forward the request to the LFI, scoped to exactly those clusters and permissions.
6. Relay whatever comes back:
    - an error to the TPP, if the LFI rejected it for a BAU check failure, a suspended Consent, for example
    - otherwise, all available data for the requested clusters and permissions

5. Broker the hand-off back to the TPP

Implements: Hand-off back to the TPP and Hand-off back to the TPP.

Once the LFI confirms the interaction session is complete, initiate the User's redirection back to the TPP with everything it needs to resume, including the authorization code. Confirm back to

the LFI once the Consent record has been updated with the User's authorization (ACCI-050).

6. Support consent lifecycle management

Act on every lifecycle event either side reports through their Consent Management Interface, per Consent Setup | Rules and Guidelines:

- A TPP-side revocation notification moves the Consent toward `Revoked`.
- An LFI-side revocation, suspension, or un-suspension notification updates the Consent state accordingly. For a Consent inside a Consent Group, apply the change to that specific Consent, not the whole group.
- A TPP-side Consent update arrives as a new Consent submission with the original set to `Revoked` first. Process it exactly like any other newly staged Consent (see Step 1).

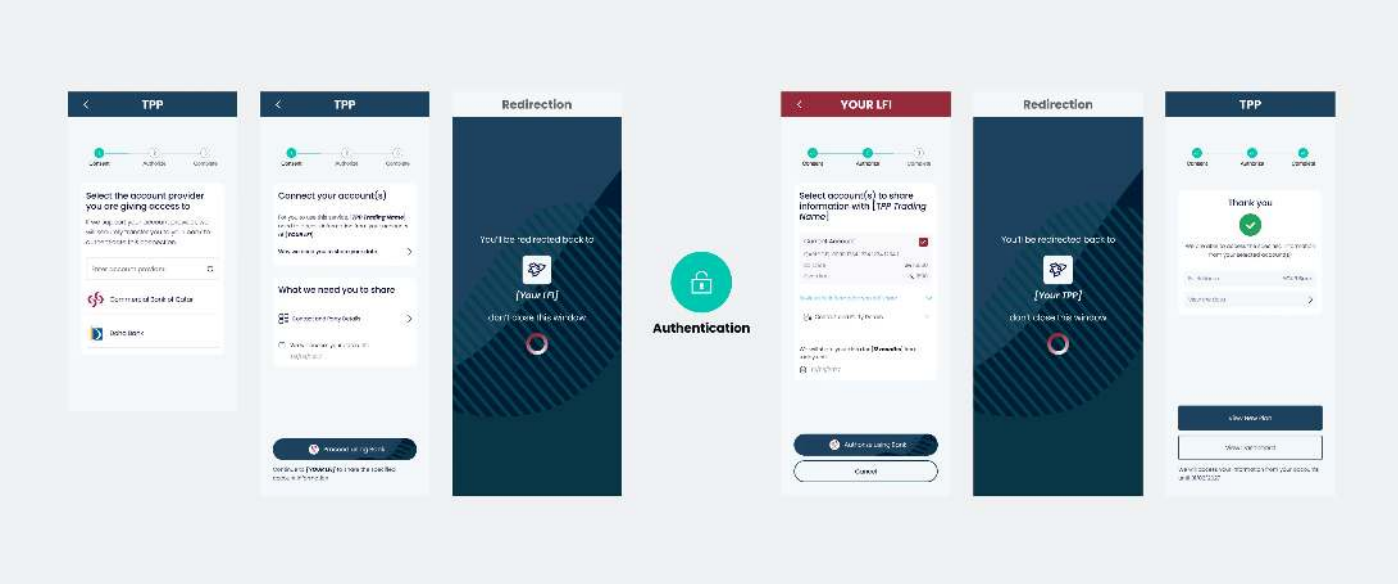
7. Account for journey variations

You have no direct UI role in the account-selection variation described in Handle Journey Variations, but you still route and validate non-account-specific data sharing requests (against the Parties endpoint, for example) exactly like any other request. The absence of an `AccountId` on the request doesn't change how you validate it against the authorized clusters and permissions.

Retrieving Accounts Information Data | Handle Journey Variations

The TPP, LFI, and OFP guides describe the standard journey, where the User selects one or more accounts to share data from. This page covers the documented variations on that journey.

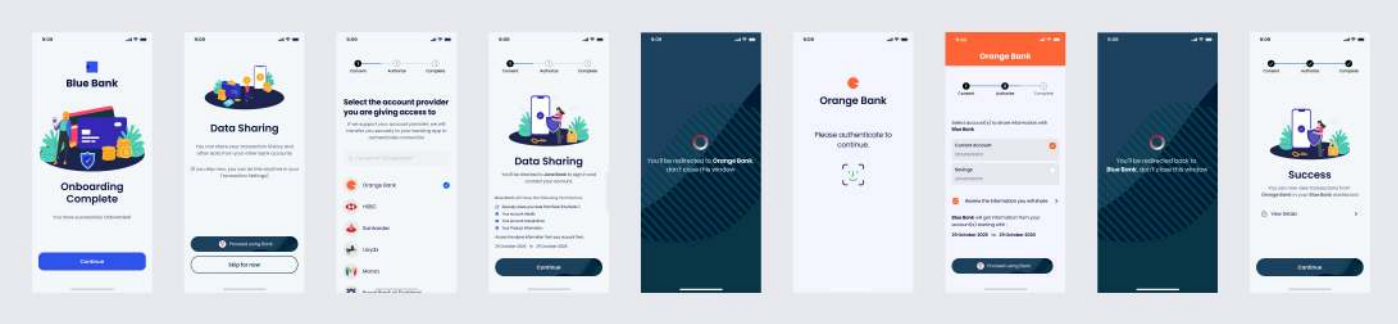
User consents to non-account-specific data only (e.g. Contact and Party details)



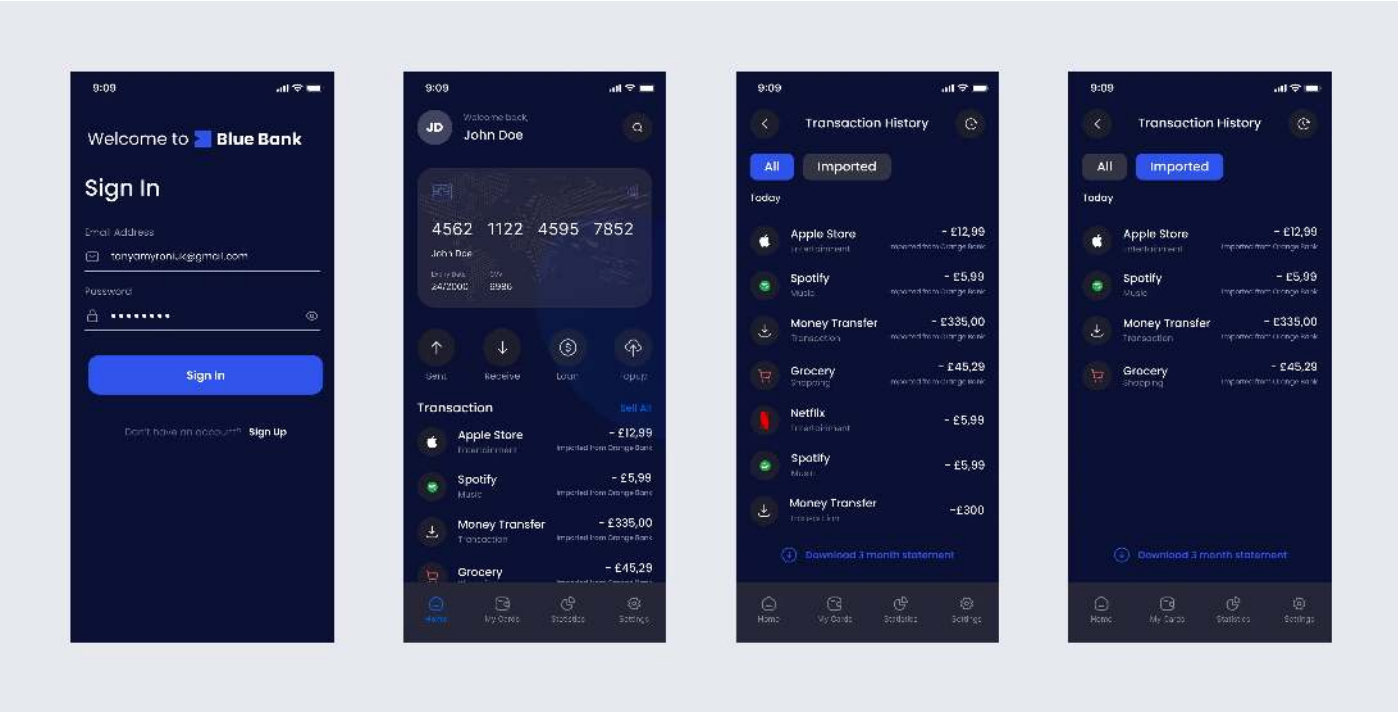
Where a Data Request Consent includes only User-level data permissions, `ReadPartyUser`, `ReadPartyUserIdentity`, or `ReadParty` (the "Contact and Party Details" data cluster, see Permissions and Data Clusters), and no account-specific permissions, there's no account to select.

- **LFI:** don't display any list of eligible accounts during Step 5 (Select accounts). Let the User proceed straight to authorization, showing them the full personal data that will be shared for the Consent's period.
- **TPP:** no change to Step 2 (Collect and review the consent parameters) beyond scoping the request to the non-account-specific permissions actually needed.
- **OFP:** validates and routes the request exactly as any other. See OFP guide | Step 7.

LFI acting as a TPP



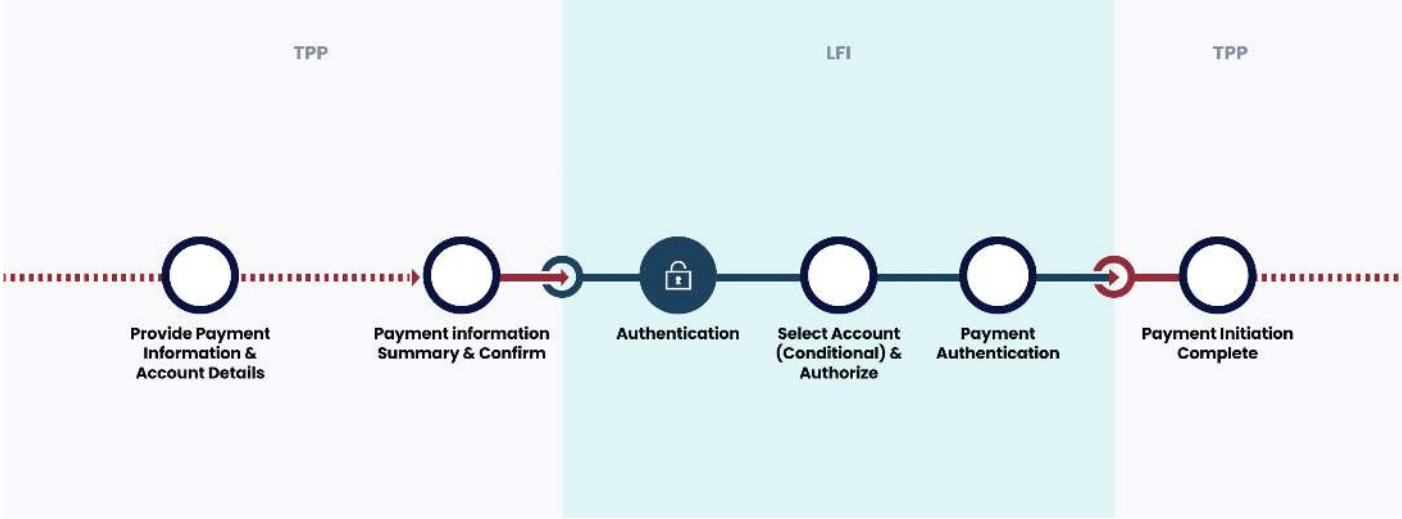
An LFI may itself request a data transfer from another LFI, on a User's behalf and with the User's consent, becoming the customer-facing entity while acting under exactly the same rules a TPP would. All three guides on this page apply to it unchanged in that role. One use of this pattern is consolidating transaction history from both LFIs into a single statement:



See Account Information | LFI as a TPP Example for the source illustration.

Making a Single Instant Payment | Overview

A Single Instant Payment (SIP) is a bank service request. It lets a TPP initiate, with the User's consent, a one-off domestic payment of a fixed amount from the User's payment account at an LFI to a business or personal beneficiary. The payment executes immediately. The TPP then tracks its status. See Single Instant Payment | Description for the full scope, and Rules & Guidelines for the SIP-specific rules (SIPD-001 through SIPD-070) this guide implements. Those sit on top of the Common Rules and Guidelines that apply across every Open Finance journey.



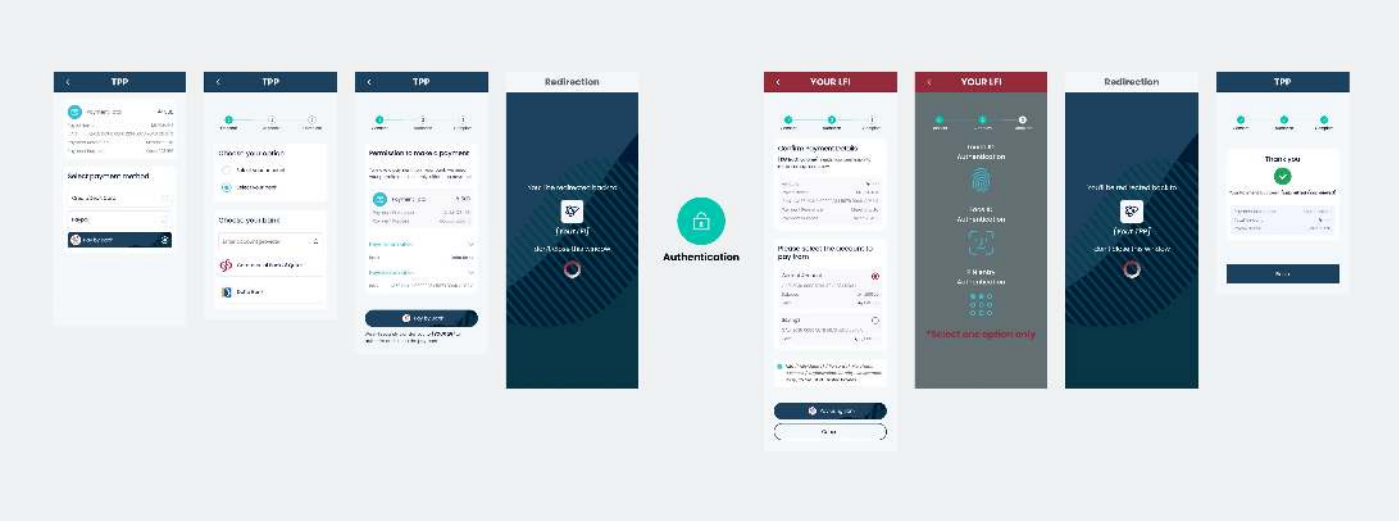
See a prototype of this journey here: Prototypes for QCB docs.

Three roles, three guides

Three parties build something to make a SIP happen. The TPP collects consent and submits the payment. The LFI authenticates and authorizes the User, then executes the payment. The OFP brokers both hand-offs and validates everything in between. Each role gets its own end-to-end guide, covering every rule relevant to that role:

- **TPP Implementation Guide**: collect and stage the consent, hand the User off to the LFI, submit the payment initiation once authorized, and track status, confirmation, and notifications.
- **LFI Implementation Guide**: receive the hand-off, authenticate the User, handle account selection, present and capture authorization, execute the payment, and handle exceptions and multi-user authorization.
- **OFP Implementation Guide**: validate the staged consent, broker both hand-offs, enforce idempotency, route the payment initiation, and propagate status.

All three converge on the same underlying flow:



1. The TPP collects consent parameters and stages the Consent with the OFP.
2. The TPP hands the User off to the LFI.
3. The LFI authenticates the User, resolves the payment account, and presents the Consent for authorization.
4. The LFI hands the User back to the TPP.
5. The TPP submits the payment initiation. The OFP validates it and routes it to the LFI for execution.
6. All three parties track status through to a confirmed outcome, and notify accordingly.

Not every one of the ten SIP-specific rules maps to a distinct step above. Some steps (consent staging, both hand-offs, status update, confirmation, notifications) run entirely on the Common Rules and Guidelines, with no SIP-specific variation. Each role's guide states plainly, step by step, which rule it implements, whether that's a SIP rule or a common one.

> **Note**
>
> This covers the standard journey. For multi-authorizer consents, TPP-side account selection with LFI supplementary information, or a Merchant acting as the user-facing entity, see Handle Journey Variations.

Making a Single Instant Payment | TPP Implementation Guide

This guide is for TPPs building a Single Instant Payment (SIP): a bank service request that initiates a domestic, immediately-executed payment of a fixed amount from a User's payment account at an LFI to a business or personal beneficiary, under a Single Use Consent of type Single Instant Payment Consent. See Single Instant Payment | Description for the full scope (Consumer/SME Debtors and Creditors).

Every step below cites the exact rule it implements. The primary source is Single Instant Payment | Customer Experience | SIP rules, which draws on the Common Rules and Guidelines, Common Components, and Constants and Limits.

1. Meet your standing obligations

Before you build a consent, you carry standing obligations that apply to every journey you run:

- **Carry out KYC and verification of the User**, proportionate to the product. Run full customer due diligence, under Qatar's AML/CFT framework, for any customer-facing product with its own regulatory due diligence requirement. Run at least basic verification (email or mobile) for every product. Where the product supports multiple users on one account, give each user their own permission profile. See TPP Obligations towards Customer Verification.
- **Capture the customer's public IP address** on login. Populate the `x-fapi-customer-ip-address` header on every customer-present request. Never send a masked, default, or TPP-own IP. Never populate the header at all for a customer-not-present request. See Customer IP Address.
- **Use valid ISO 3166-1 country codes and ISO 4217 currency codes** wherever the API schema calls for them. Fall back to the Open Finance user-assigned ZZ, ZZZ, XX, or XXX codes only when the country is genuinely unknown or defunct. See Use of ISO Country and Currency Codes.

2. Collect and review the consent parameters

Implements: Single Instant Payment Consent.

~~Let the User provide and review each of the following before you capture consent:~~

1. **User Payment Account or User LFI.** Offer at least one option: the User's saved Payment Account Identification (an IBAN, for a returning User with a long-term profile), or LFI-only selection, where the User picks their account later at the LFI. Identify the LFI by its Trust Framework Directory trading name and logo. See User Payment Account Selection.
2. **Payment Amount & Currency.** Let the User enter the amount, or pre-populate it. State clearly that the amount is in QAR. Enforce a minimum of 0.01 QAR at a maximum of 2 decimal digits, and cap any Creditor account outside the User's own LFI at the Max Inter-bank Payment Amount (intra-bank payments carry no cap). See Payment Amount & Currency.
3. **Creditor Identification.** Let the User manually enter or select it, or pre-populate it, using at minimum an IBAN plus the Creditor name. Show the full IBAN if the User entered it. Mask it to the last 4 digits if you populated it yourself (for example, a merchant refund). Validate the format and ask the User to re-enter it on failure. Send the Merchant Category Code if the Creditor is a merchant you onboarded. See Creditor Identification.
4. **Debtor Reference** (optional). If you offer one, prefer a Structured Reference over free text: Free Text (22 characters or fewer) plus Bank Name (8 characters or fewer) plus an optional Merchant Name (8 characters or fewer) plus TPP Name, hyphen-separated, up to the Max Payment Reference Length in total. If it overflows, truncate the TPP Name first, then the Merchant Name, then the Bank Name. Never truncate the Free Text Reference itself. Include it on every Payment Initiation Request. See Debtor Reference.
5. **Creditor Reference.** Optionally let the User enter one, or pre-populate and lock it where the use case needs a specific value (card clearing, invoices). Apply the same Structured Reference format as the Debtor Reference, except the Bank Name here is the Debtor LFI. Include it on the Payment Initiation Request. See Creditor Reference.
6. **Purpose of Payment.** Support every category purpose code and description defined in Limits and Constants. Let the User select from a list, or pre-populate it. Validate the format before submission. See Purpose of Payment.

Then set the additional consent controls, which you don't display to the User:

7. **Is Single Authorization flag.** Set or clear it where the Consent needs to be restricted to a single authorizer. See Is Single Authorization flag.
8. **Authorization Expiration DateTime.** Set it where the consent authorization carries specific timing requirements (this matters for multi-authorizer consents too). Bound it by the Max Authorization Time Window. See Authorization Expiration Date Time.
9. **Risk Information Block.** Populate every field the Risk Information Block grid marks as required for your payment scenario: Customer Present or Not Present, Payments to

Merchants you onboarded, P2P/B2P, Payments-to-self, or Payments to non-onboarded Businesses. That page is the definitive, field-by-field source, covering User, Authentication, GeoLocation, Device, Biometric, App, Browser, Behavior, Account-Risk, Transaction, and Merchant indicators. Two standing obligations sit alongside it:

    - Never substitute stale authentication or device data from an unrelated session. Leave the field null if you can't reliably retrieve the original consent-setup context.
    - When onboarding merchants for Pay-by-Account, verify Trading Name and Bank Account details as part of KYC/KYB, and capture the User's Qatar ID before the consent journey for any long-lived consent or data-sharing use case. You don't need to verify the Qatar ID itself.

Finally:

10. **Capture explicit consent** for the SIP to be initiated from the selected payment account, exactly as specified in the parameters above.

3. Stage the consent

Implements: Consent Staging.

- Provide all of the Consent parameters from Step 2 to the OFP before you redirect the User to the LFI.
- If you want asynchronous status updates, register the webhook subscription with the OFP at this same stage. See Step 8 for how you'll use it later.
- The OFP validates format and business-rule compliance, then asks the LFI to validate further. If either check fails, expect a rejection with an appropriate error. See Consent Staging.
- You can query the Consent's status from the OFP at any time.

4. Hand off the User to the LFI

Implements: Hand-off to LFI.

- Show messaging that makes clear the User is about to leave you for their LFI. Suggested wording: "We will securely transfer you to your LFI to authenticate and make the payment." See Hand-off to LFI.
- Trigger Authentication & Authorization at the LFI's authorization endpoint.

- Build the inbound redirection screen per Authentication by LFI | Redirection. Ask the User to identify their LFI first, so the request matches the LFI's capabilities. Use the LFI's Trust Framework logo and trading name. Keep it to a single screen with no extra steps. The exact redirect mechanics (app-to-app deep link, browser-to-browser QR code and call to action, app-to-browser) depend on the User's device and whether the LFI app is installed.
- Show the name and logo of the party the User is being sent to on both redirection screens, inbound and outbound, and keep each on screen long enough for the User to register the transition. See Effective Use of Redirection Screens.
- If the LFI's authorization request turns out malformed, you receive no response at all. The User depends entirely on the LFI to show an error. If the User instead cancels or fails to authenticate at the LFI, you do receive an OIDC error code on redirect back. Use it to show a more specific message. See Error Scenarios During Redirection.

5. Wait for the LFI to authenticate and authorize

The LFI takes over here. It authenticates the User once with MFA. That single authentication covers everything that follows, including any account selection or supplementary-information step. See the General Notes. The LFI then retrieves the staged Consent, presents it, and captures the User's authorization or cancellation. Full detail on what the LFI does at this stage sits in the LFI Implementation Guide. You have nothing to build for this step beyond handling the hand-off back.

6. Receive the hand-off back from the LFI

Implements: Hand-off back to the TPP.

The OFP redirects the User back to you with the required authorization code once the LFI confirms session completion. See Hand-off back to the TPP. From here you resume control and move straight to payment initiation.

7. Submit the payment initiation request

Implements: Payment Initiation (TPP obligations).

1. Submit the payment initiation request to the OFP with exactly the same parameters as the authorized Consent, immediately after you receive the authorization confirmation.

2. Do this within the Max SIP Payment Initiation Time Interval (LIMT-012), currently 10 minutes, or the OFP rejects it outright.
3. Provide a unique value in the mandatory `x-idempotency-key` header on the `POST /payments` request. If the OFP has already seen that key from you within the last 24 hours for the same amount, it short-circuits and replays the original response (the `201 Created`, or the original error) instead of re-triggering the LFI. If the amount differs from the original request under that same key, expect a `409 Conflict` instead. See Hand-off back to the TPP | x-idempotency-key Checks.
4. Display confirmation to the User once a response arrives:
    - **Submitted**: the payment status is `Pending`.
    - **Completed**: the payment status is `AcceptedWithoutPosting`, the final accepted state.
5. Handle rejection. The OFP returns an appropriate error if the request doesn't match the authorized Consent's state or parameters exactly, or if the LFI rejected it for BAU limits or insufficient funds. Surface that error to the User rather than a generic failure.

8. Track status, confirm, and notify

Implements: Payment Status Update, Confirmation to User, and Payment Notifications.

1. **Track status.** Either poll the OFP for the payment status at intervals, subject to fair usage limits, or rely on the webhook you registered in Step 3 to push status changes as they happen. See Payment Status Update for the full three-state Open Finance payment status model: `Pending`, `AcceptedWithoutPosting`, `Rejected`.
2. **Confirm to the User.** Show the outcome of the Service Initiation based on the payment status you receive, using a confirmation screen that fits the customer journey, for every Service Initiation the User is present for. For Long-lived Consents, also show the Consent authorization outcome and any further-authorization requirements. See Confirmation to User.
3. **Notify the beneficiary**, if they're onboarded by you, of the outcome of the Service Initiation. Notify the Debtor a day ahead of any upcoming payment where applicable. In multi-user scenarios, keep every authorizer updated on who has accepted, rejected, or still needs to act. See Payment Notifications.
4. Every customer-facing notification (push, SMS, email, in-app) must clearly identify the beneficiary. Never lead with the beneficiary's bank. Where the beneficiary can't be explicitly

identified (a pooled e-wallet top-up account, for example), include your own name instead, so the User still recognizes the action they approved.

9. Trusted Payee retrieval (where applicable)

Where you also hold Data Sharing access to the User's account, retrieve any Trusted Payee the LFI added during authorization. See LFI guide | Offer to save a Trusted Payee. Call `GET /accounts/{AccountId}/beneficiaries`, check `addedViaOF: true`, and use it to streamline future payments to the same Creditor. See Adding a Trusted Payee (Beneficiary).

10. Account for journey variations

Not every SIP journey follows the path above exactly. Build for these variations too. See Handle Journey Variations for the full detail on each:

- The User selects their account at you, not the LFI, and the LFI still needs to show supplementary information, including the duplicate-payment case.
- The User selects their LFI at you, but a Merchant, not you, is the entity the User actually recognizes.

Making a Single Instant Payment | LFI Implementation Guide

This guide is for LFIs supporting a Single Instant Payment (SIP): the domestic, immediately-executed payment a TPP initiates from a User's payment account, under a Single Use Consent. See Single Instant Payment | Description for the full scope.

Every step below cites the exact rule it implements, drawing on Single Instant Payment | Customer Experience | SIP rules, the Common Rules and Guidelines, Common Components, and Constants and Limits.

1. Support eligible accounts and payment rails

Implements: Supported Accounts & Payment Rails.

Before any SIP can be initiated, your platform needs the following in place:

1. Expose Open Finance APIs for every Retail and SME current account you already offer in your own digital channels.
2. For domestic Bank Service Initiation, support both Debtor and Creditor accounts in QAR only. FX conversion at the Debtor account sits out of scope for this Standard version.
3. Authorize consents only against eligible accounts. Filter dormant and inactive accounts out of every list you show during account selection, consent authorization, and payment initiation. Reject any staged consent or payment request that references one, with a standardized error code back to the TPP through the OFP.
4. Route domestic payment initiation through the Instant Payment Platform (Fawran Core). If the Creditor LFI isn't onboarded to it, the amount is above Fawran's maximum, or it's temporarily unavailable, fail over to Tahweel rather than rejecting the payment. Where routing depends on data elements not yet available for the Debtor (their name or account details), reject with a clear "Rejected" status and error instead of silently rerouting. Re-attempt only once you've captured that information through KYC.
5. Don't apply a cooling-off period, or hold a payment between authorization and submission to the payment rail. Don't apply any transaction limit or other constraint on Open Finance payments more restrictive than the equivalent existing digital channel. See LFI Channel Limits for the specific per-transaction floors (Me-to-Me, Third-Party).

2. Receive the hand-off from the TPP

Implements: Hand-off to LFI and Hand-off to LFI.

1. Don't present any additional inbound redirection screens of your own. The TPP already showed one before redirecting.
2. Request the staged Consent details from the OFP using the unique Consent identifier it received.
3. Initiate User authentication immediately. See Step 3.

3. Authenticate the User

Implements: Authentication.

Authenticate the User with MFA, using the same methods they'd use accessing you directly. This single authentication covers everything through to the end of authorization. You don't repeat it for account selection or a supplementary-information step. See the General Notes. Build to the general principles and the specific redirection model that applies:

- **General principles.** MFA is mandatory for every Service Initiation. Let the User use their normal preferred authentication method. Introduce no unnecessary friction, delay, or promotional content. See Authentication by LFI | Overview.
- **Redirection** (App-to-App, Browser-to-App, Browser-to-Browser, App-to-Browser). Invoke your app directly with no extra screens where it's installed. Otherwise fall back to a mobile-optimized web channel with a QR code and call-to-action hand-off. After authentication, deep-link the User to account confirmation. See Authentication by LFI | Redirection.
- **Decoupled Redirection.** Where the User authenticates on a second device (a kiosk or POS flow, for example), generate a scannable deep-linked code, invoke your app on the second device to perform MFA there, and confirm the outbound result on whichever device continues the journey. See Authentication by LFI | Decoupled Redirection.
- **Redirection screens.** Show the counterparty's name and logo on both the inbound and outbound screens, and keep each visible long enough for the User to register the transition. See Effective Use of Redirection Screens.
- **Error handling.** On a malformed authorization request, stop and show an error. Don't redirect the User back to the TPP. On a User-side error (a cancellation, failed credentials, a mismatched Debtor account), redirect back to the TPP with the appropriate OIDC error code instead. See Error Scenarios During Redirection.

- **SCA principles underpin all of the above**: two-factor MFA from Inherence, Knowledge, and Possession, a uniquely identified authentication operation correlated to the specific consent, and, where possible, a cryptographically verifiable proof-of-authentication built on open standards. See Strong Customer Authentication | Principles.

If authentication fails or the User cancels, redirect back to the TPP with information that the Consent was not authorized, per SIPD-013.

4. Select the payment account

Implements: Payment Account Selection at LFI and Check Is Single Authorization flag.

1. If the staged Consent doesn't already specify a payment account, let the User pick one from their eligible accounts. Show accounts valid for online or mobile payment initiation, with no AML or other restrictions, and show the interim available balance including credit lines, overdrafts and any other financing facility available on the account.
2. Give the User a way to cancel. Hand them back to the TPP with an appropriate error if they do.
3. Check the account's authorization matrix against the "Is Single Authorization" flag the TPP set. If the flag requires single authorization but the account's matrix demands multiple authorizers, show only accounts that support single authorization. If none exist, decline the Consent, cancel the journey, and redirect the User back to the TPP.
4. Reject the Consent, and notify the OFP, if a payment account fixed by the TPP's Consent, or one the User just selected, turns out to have insufficient funds. Never list an insufficient-funds account as selectable in the first place.

5. Present the consent for authorization

Implements: Authorization.

1. Retrieve the staged SIP Consent from the OFP.
2. Reject it outright, notifying the OFP with an error, if the Purpose of Payment violates an existing BAU blocking rule for that channel.
3. Display the TPP's Trading Name. If a non-TPP customer-facing service provider (a Merchant, for example) is involved, show that name alongside the TPP's.
4. Present, at minimum: User Payment Account, Payment Amount & Currency, Creditor Identification (Name, Account, optionally the Creditor LFI), Debtor Note (conditional),

Creditor Reference, and Purpose of Payment.

5. Request the User's authorization.
6. Give the User a way to cancel. Hand back to the TPP with an error, and reject the Consent.
7. Once authorized, move the Consent from `AwaitingAuthorization` to `Authorized`. Do this once every required authorizer has acted (see Step 12), and update the Consent details at the OFP.

Independently, the OFP checks that the Authorization Time Window is still valid before confirming your update.

6. Offer to save a Trusted Payee

Implements: Adding a Trusted Payee (Beneficiary).

On the authorization page (Step 5), for a SIP where the Creditor is an Individual, Merchant, Business, Charity, Government Body, or Other type, and isn't already a Trusted Payee, show a checkbox: "Add [Individual/Merchant/Business/Charity/Government Body/Organization] to my list of Trusted Payees," using the mapping keyed off the Creditor account type. If the Creditor is already trusted, suppress the checkbox, or replace it with "Already Trusted."

7. Populate and apply the Risk Information Block

Implements: Risk Information Block | LFI Obligations.

Use all of the Risk Information Block data the TPP populated, per the field-by-field grid on that page, for your own risk-profiling of the Consent and payment. Apply it in a way that's non-discriminatory against Open Finance payments compared with the equivalent payment initiated through your own online or mobile banking channel.

8. Hand off back to the TPP

Implements: Hand-off back to the TPP and Hand-off back to the TPP.

1. Confirm to the OFP that the User's interaction session is complete.
2. Show messaging that the User is being taken back to their TPP, then redirect them there immediately once the OFP provides the required information, including the authorization code. Don't add your own delay here.

9. Execute the payment

Implements: Payment Initiation (LFI obligations).

1. Trigger payment initiation immediately on receiving the request from the OFP.
2. Retrieve the Creditor Identification details from the encrypted PII block in the original Consent message, and run your normal KYC/KYB checks against them. Reject, and notify the OFP, on failure.
3. Apply your existing BAU account controls and limits exactly as you would on any other channel: single and total transaction value limits, AML checks, and so on. Reject with an error to the OFP on any breach. The LFI Channel Limits page sets the floor for these limits. You may set them higher, never lower.
4. Reject, and notify the OFP, if the selected account has insufficient funds.
5. On success, execute the payment, either through the underlying payment rails or internally as an intra-bank payment, and return the payment's unique Transaction ID to the OFP. For a payment directed to Tahweel, populate every field the Tahweel Direct Credit message requires, convert the purpose code to its Tahweel equivalent, and pass the settlement window Tahweel returns to the OFP. Submit a payment with a future execution date to Fawran on that date, not before. See Supported Accounts & Payment Rails and Purpose of Payment.
6. For intra-bank payments, make sure the Creditor Reference from the SIP appears in the Creditor account information.

10. Update status and notify

Implements: Payment Status Update and Payment Notifications.

1. Update the payment's status throughout its lifecycle, and notify the OFP asynchronously of every change, using the three-state Open Finance status model: `Pending`, `AcceptedWithoutPosting`, `Rejected`. See Payment Status Model for the full ISO 20022 mapping.
2. Notify the User by SMS at minimum, optionally by email or app notification too, on the success or failure of the Service Initiation.
3. Identify the initiating TPP and the payment's beneficiary in every notification. Never lead with the beneficiary's bank as the headline of the message.
4. Align the notification's content with the consent object the User actually authorized, so they recognize the action. Include the TPP's name where the beneficiary can't be explicitly

named, as in a pooled account.

11. Handle exceptions and supplementary information

Implements: Exception Handling and Supplementary Information.

Blocking exceptions stop the journey where it genuinely cannot continue. See the known scenarios table for suggested language: no eligible accounts, an ineligible account, technical failure, authentication error, minors out of scope, an LFI-defined limit exceeded.

1. Determine when an exception applies, per the User Experience Principles.
2. Stop the User continuing the journey they intended.
3. Show a meaningful description of the cause.
4. Offer a redirect back to the TPP, where technically possible.
5. Update the consent object with the appropriate error code.

Where the block is specifically an account or policy issue at the authorization summary screen, grey out and disable the “Pay using QCB” button, explain why, and offer Cancel to return to the TPP.

Non-blocking supplementary information is a warning that does not stop authorization: insufficient balance, entering overdraft, a likely duplicate payment.

1. Determine when it’s needed, per the same User Experience Principles.
2. Show it at the bottom of the authorization screen. It must not prevent completing authorization.
3. Give a meaningful description of the cause and impact.
4. Let the User revise any input, or redirect back to the TPP.
5. Let the User complete authorization despite the warning.
6. Update the consent object with an appropriate error code only if the User elects to cancel.

Recall the parity principle from the General Notes: if your own online channel wouldn’t show this information, don’t show it here either. Never let it function as an extra check on a Consent the User already gave the TPP.

12. Support multi-user authorization

Implements: Authorization | Multi-Authorization Journey Only and Multi-User Authorization Flow.

Where the account's authorization matrix requires more than one authorizer:

1. Tell the User the Consent was received but needs further authorization. The exact messaging is up to you, aligned with your own journeys.
2. Only let a User use payment initiation where it sits within their authority for the account, including shared personal accounts and delegated business authority, resolved against your existing entitlement and authorized-signatory processes.
3. Inform the OFP of the multi-authorization requirement, including the total number of required Authorizers and, per Authorizer, their type (Financial, Manager, Director, Accountant, and so on), the date and time they actioned it, and their status (Pending, Approved, Rejected).
4. Notify the OFP each time an Authorizer in the chain acts.
5. Move the Consent to `Authorized` only once every Authorizer has authorized it.
6. Notify every User about the pending authorization, and notify each other authorizer that a Consent awaits their action, over your existing channels.

13. Account for journey variations

Build for these variations, not just the standard path. See Handle Journey Variations:

- The User already selected their account at the TPP, and you still need to show supplementary information, including a likely-duplicate-payment case.
- The User selected their LFI at the TPP, but a Merchant is the entity the User actually recognizes. Display both names together (Step 5).

Making a Single Instant Payment | OFP Implementation Guide

This guide is for anyone implementing the OFP (Open Finance Platform) role in a Single Instant Payment (SIP): the intermediary that validates and stages the consent, brokers both hand-offs between TPP and LFI, and routes the payment initiation through to execution. See Single Instant Payment | Description for the full scope.

Every step below cites the exact rule it implements, drawing on Single Instant Payment | Customer Experience | SIP rules, the Common Rules and Guidelines, and Constants and Limits.

1. Validate and confirm the staged consent

Implements: Consent Staging and Consent Staging.

1. Receive the Consent parameters the TPP provides before it redirects the User.
2. Check their format and compliance with the business rules for a SIP Consent.
3. Connect to the LFI and ask it to run its own additional parameter validation.
4. Reject the Consent, with an appropriate error to the TPP, if either check fails.
5. Let the TPP retrieve the Consent's status at any point.

2. Broker the hand-off to the LFI

Implements: Hand-off to LFI and Hand-off to LFI.

Once the TPP triggers the LFI's authorization endpoint and the LFI requests the Consent details, provide them based on the unique Consent identifier. That lets the LFI proceed straight to User authentication, with no extra round-trip to the TPP.

3. Check the authorization time window

Implements: Check Authorization Time Window.

While the Consent sits in `AwaitingAuthorization`, reject it and move it to a terminal state if either of these holds:

- The Max Submitter Authorization Time Window (2 hours) has elapsed with no authorizer having acted, and the TPP didn't set its own Authorization Expiration DateTime.
- The TPP's own Authorization Expiration DateTime, where set, has expired without full authorization. That value is itself bounded by the Max Authorization Time Window.

4. Relay multi-user authorization information

Implements: Authorization | Multi-Authorization Journey Only and Multi-User Authorization Flow.

Where the LFI reports that a Consent needs more than one authorizer:

1. Pass all of that information on to the TPP: the total number of Authorizers required, and for each, their Type, when they actioned it, and their status (Pending, Approved, Rejected).
2. Notify the TPP every time an Authorizer in the chain acts, so it can keep the User updated.

5. Broker the hand-off back to the TPP

Implements: Hand-off back to the TPP and Hand-off back to the TPP.

Once the LFI confirms the interaction session is complete, initiate the User's redirection back to the TPP, and provide everything it needs to resume, including the required authorization code.

6. Enforce idempotency on payment initiation

Implements: Hand-off back to the TPP | x-idempotency-key Checks.

On every `POST /payments` request, check whether the mandatory `x-idempotency-key`:

1. **Matches a key already seen from the same TPP, within the last 24 hours, for the same payment amount.** If so, don't trigger a new payment initiation request to the LFI. Instead, replay whatever you returned the first time: the same `201 Created` (with the original payment resource) if that's what happened, or the same error response if it errored.
2. **Matches a key already seen from the same TPP within 24 hours, but for a different amount.** Don't trigger the LFI request. Return a `409 Conflict` instead.

7. Validate and route the payment initiation request

Implements: Payment Initiation (OFP obligations).

1. Accept the TPP's payment initiation request without demanding further MFA or User authorization. The prior Consent authorization already covers it.
2. Check the request maps to a Consent that's genuinely in the `Authorized` state. Reject with the appropriate error or code otherwise.
3. Check every parameter in the request matches the authorized Consent exactly. Reject with an error to the TPP on any mismatch.
4. Forward the request to the LFI, including the authorized Consent identifier, Payment Amount & Currency, Debtor Reference (if provided), Creditor Reference, and Purpose of Payment.
5. Relay whatever the LFI returns back to the TPP:
    - an error, with an appropriate message, if the LFI rejected the payment for a BAU account-control or limit breach
    - an error, with an appropriate message, if the LFI rejected it for insufficient funds
    - otherwise, all available information about the initiated payment, including its unique Transaction ID

8. Propagate payment status

Implements: Payment Status Update.

1. Set up asynchronous notification with the LFI at the same time the Consent is staged, so you learn about status changes without polling.
2. Update the payment's status throughout its lifecycle as the LFI reports progress, using the three-state Open Finance model. See Payment Status Model (`Pending`, `AcceptedWithoutPosting`, `Rejected`) and their ISO 20022 equivalents.
3. Respond to a TPP's status query with the correct code from that model, plus any additional payment-system-specific status or rejection-reason codes.
4. Where a TPP has instead registered for push notifications, at consent-staging time, send them without waiting to be asked.

> **Note**
>
> The status model's cumulative-value and cumulative-count decrement logic on a `Rejected` payment (COMM-150, COMM-151) applies to Variable Recurring Payment consents. It does not apply to a SIP's Single Use Consent.

9. Respect the limits and timers that govern the OFP

Several Limits and Constants bind your own behavior directly, on top of those already referenced above:

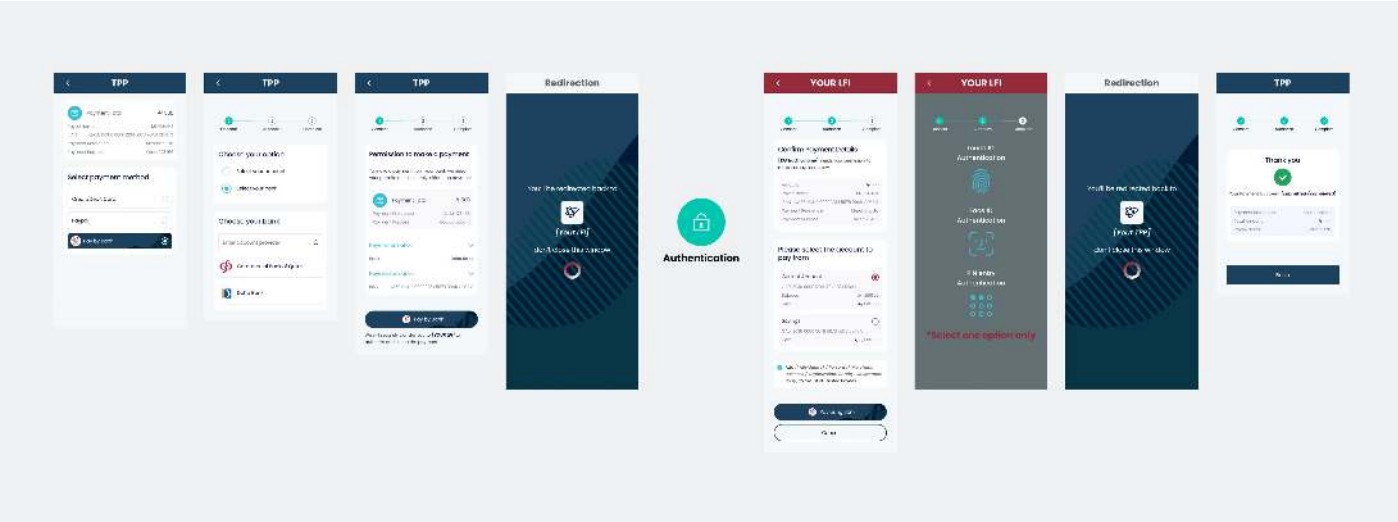
- **Payment Status Update (LIMT-013).** Update the payment resource's status within the internal SLA once a response arrives from the payment system: 500ms total, split 250ms LFI and 250ms OFP processing.
- **Event Notification SLA (LIMT-014).** Send the `POST {WebhookUrl}` event notification to the TPP within 500ms of a Consent or Payment resource status change.

10. Account for journey variations

You have no direct UI role in the variations described in Handle Journey Variations, but the Consent and payment data you relay must stay consistent regardless of which variation the TPP and LFI are running. The supplementary-information and merchant-as-user-facing-entity cases don't change any parameter you validate or forward.

Making a Single Instant Payment | Handle Journey Variations

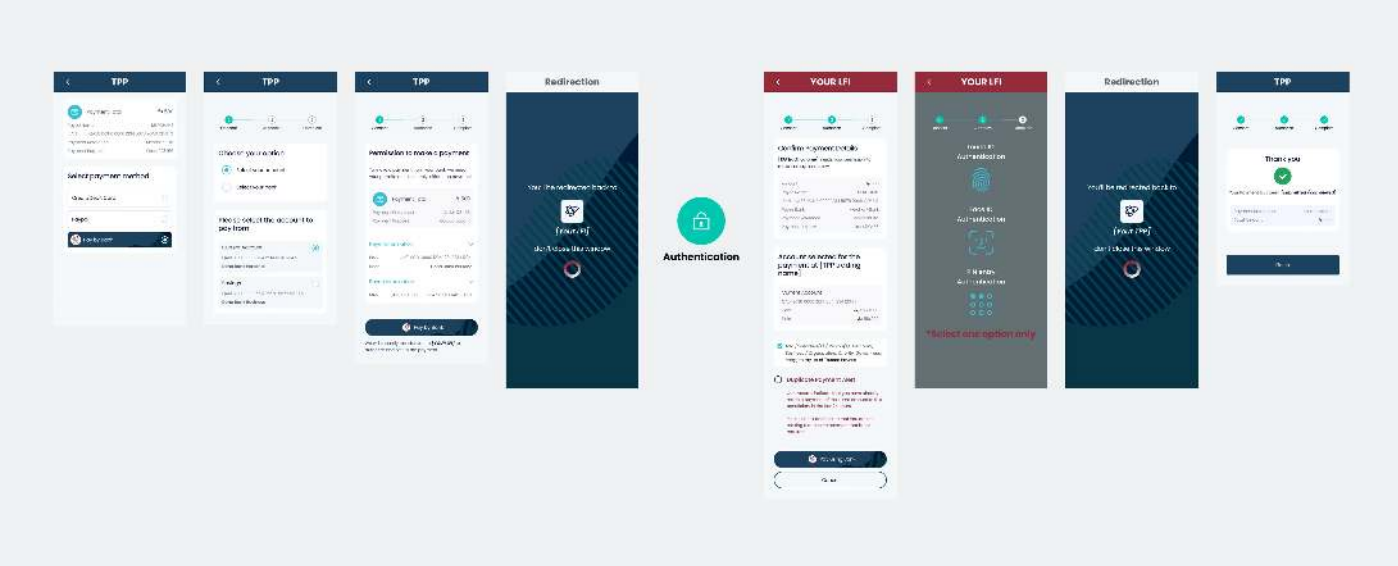
The TPP, LFI, and OFP guides describe the standard journey, where the User selects their LFI at the TPP and their account at the LFI:



This page covers the documented variations on that journey. Two conditions from the General Notes in the Rules & Guidelines shape all of them:

- MFA performed once at authentication remains valid even if the User has to select an account or provide more information afterwards. It never needs repeating.
- Where the payment order is already complete, the LFI may still need to insert a step to show the User supplementary information. See Supplementary Information.

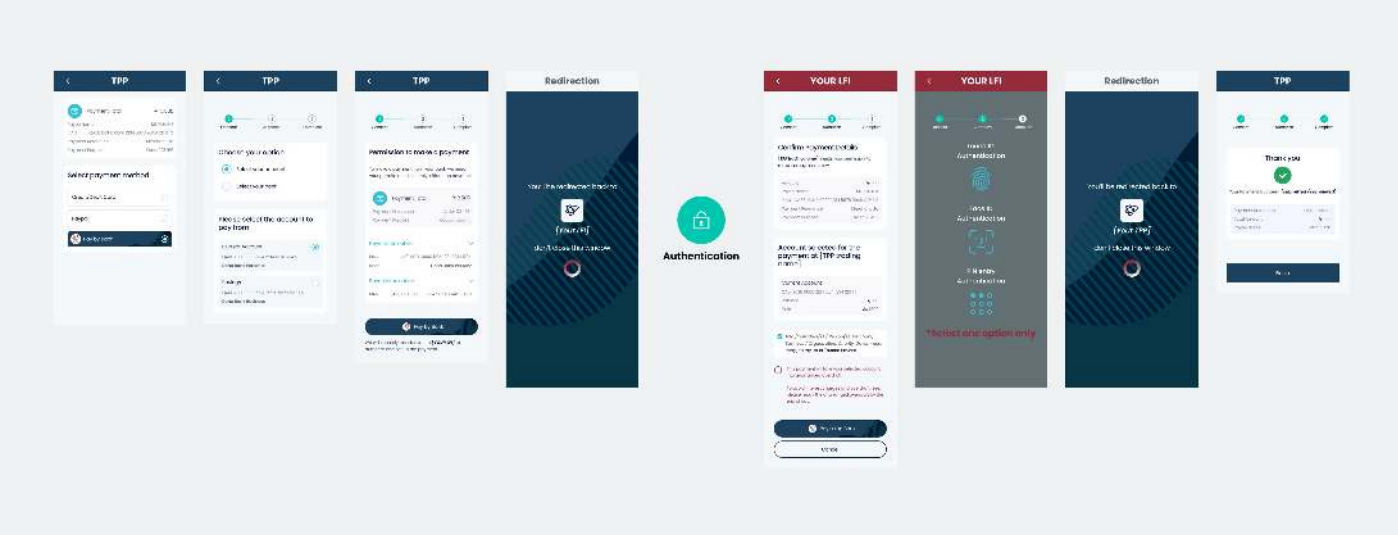
User selects an account at the TPP, and the LFI provides supplementary information (duplicate payment)



Here the User has already selected their payment account at the TPP, so the payment order handed to the LFI is complete. Even so, the LFI detects a condition, a likely duplicate of a recent payment, that it would also flag on its own online channels. It must flag here too, per the parity principle above.

- **TPP:** nothing changes at Step 1 (Collect and review the consent parameters). The account is simply already fixed in the Consent by the time it reaches the LFI.
- **LFI:** insert the supplementary-information screen into Step 5 (Present the consent for authorization). Don't treat it as a blocking exception. It must not add an extra check on the User's consent, per Step 11.

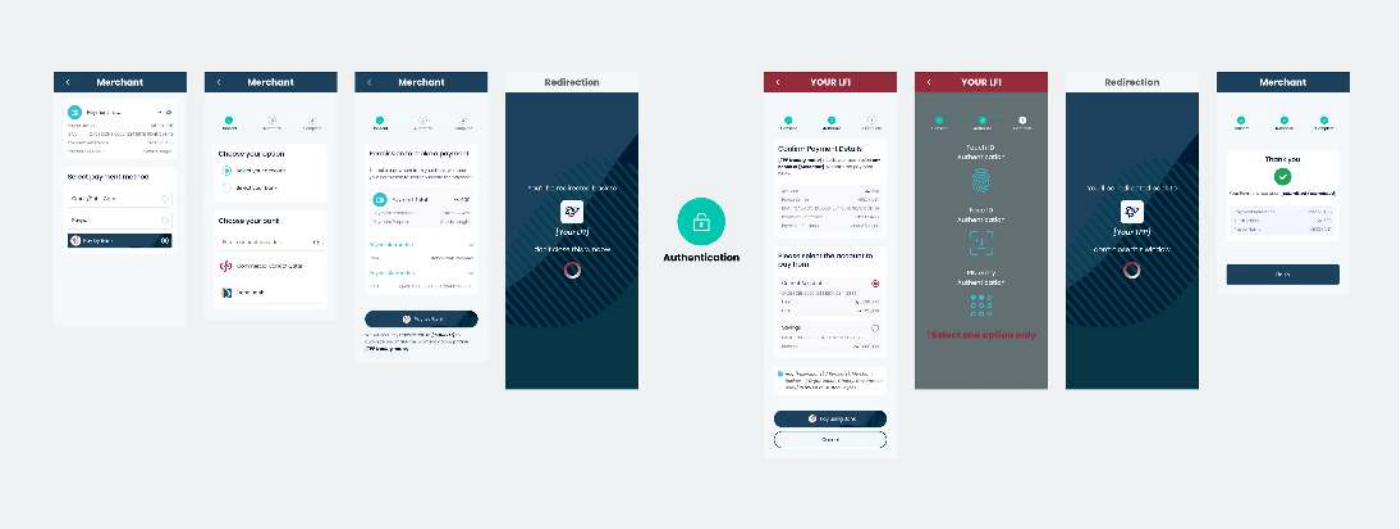
User selects an account at the TPP, and the LFI provides supplementary information



This is the general case of the variation above: any other situation where the LFI needs to show supplementary information during authorization, despite the payment order already being

complete at hand-off. The same rule applies. Insert the screen without blocking the journey or adding an authorization check that wasn't already in the TPP's Consent.

User selects an LFI at the TPP, and a Merchant is the user-facing entity

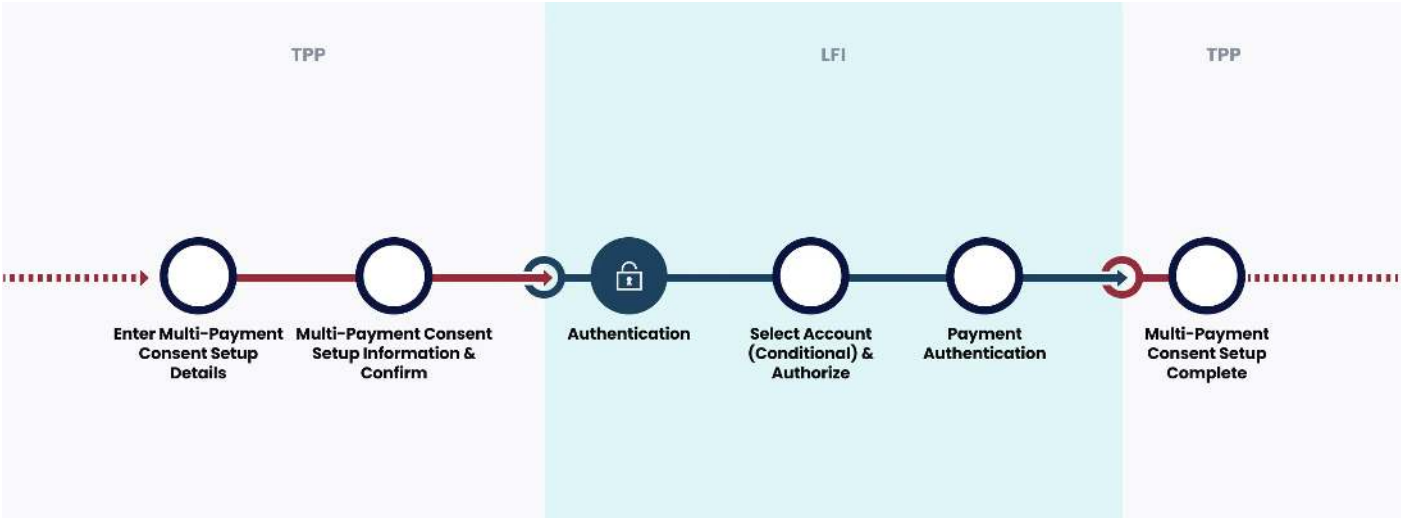


Here the TPP is not the entity the User actually recognizes. A Merchant with a commercial relationship to the TPP is.

- **LFI:** when presenting the consent in Step 5, display both names to the User, the customer-facing Merchant name alongside the underlying TPP's trading name (SIPD-022), so the User can tell who they're really dealing with.
- **TPP:** no change to consent collection or payment submission. The display name pairing is an LFI-side authorization-screen concern.

Making a Variable On-Demand Payment | Overview

A Variable On-Demand (`VariableOnDemand`) payment consent is one of the Multi-Payment types: a long-lived consent that lets a TPP initiate a series of future payments of variable amounts, authorized once by the User at setup, with no further MFA required at the LFI for each individual payment. See What are Multi-Payments and Multi-Payment Types Set-up for the full definition, and the Common Rules and Guidelines for the cross-journey rules this guide builds on.



Three roles, three guides

- **TPP Implementation Guide**: collect and stage the consent, including the Variable On-Demand-specific control parameters, hand the User off to the LFI, submit payment initiations within the consent's limits, track status, and manage consent updates.
- **LFI Implementation Guide**: receive the hand-off, authenticate the User once for the whole consent, present and capture authorization, verify beneficiaries, and process each subsequent payment initiation without further MFA.
- **OFP Implementation Guide**: validate the staged consent, broker both hand-offs, and own the cumulative and per-period control-parameter bookkeeping across the consent's entire lifetime. That last part is genuinely different from a one-off payment.

What makes Variable On-Demand different from a single payment

A Variable On-Demand consent is authorized once, then used for an unknown number of future payments of variable amounts, up to limits set at consent time:

- A maximum individual payment amount, and cumulative value or number caps, either over the consent's lifetime or reset every recurring Control Period (Day, Week, Month, Year).
- One of two Creditor scopes: a single known Creditor, or a Predefined Beneficiary List. Each carries its own extra rules for consent setup and payment initiation.
- An optional Balance Check permission, so the TPP can check funds are available before initiating a payment.

Every step below cites the exact rule it implements, whether that's Multi-Payment-specific (MPGN, MPOD) or a Common Rule.

> **Note**
>
> For the account-scope variations (single, defined-list), see Handle Journey Variations.

Making a Variable On-Demand Payment | TPP Implementation Guide

This guide is for TPPs building a Variable On-Demand (`VariableOnDemand`) Multi-Payment consent: a long-lived consent authorized once by the User, then used to initiate an unknown number of future payments of variable amounts, within limits agreed at setup. See What are Multi-Payments for the full scope.

> **Scope**
>
> This guide covers Variable On-Demand only, not Combined Payments or any other Multi-Payment type.

Every step below cites the exact rule it implements, drawing on the Multi-Payment rules (MPGN, MPOD), the Common Rules and Guidelines, and Constants and Limits.

1. Collect and review the basic consent parameters

Implements: Multi-Payments Consent.

1. **Debtor Account.** Either populate it in the consent's `DebtorAccount` object, if you already hold it, or omit it entirely to signal the LFI must ask the User to select one during authorization. See User Payment Account Selection.
2. **Creditor details**, per the consent's Creditor scope. Decide this before collecting anything else. See Step 3.
    - **Single known Creditor.** Display its details to the User. See Creditor Identification.
    - **Multiple defined Creditors.** Display the details of each Creditor. See Creditor Identification for Variable-defined Beneficiaries.
3. **Debtor Reference** (optional) and **Creditor Reference**. See Debtor Reference and Creditor Reference.
4. **Purpose of Payment**, mandatory. See Purpose of Payment.
5. **Risk Information Block.** See Risk Information Block.
6. **Is Single Authorization flag** and **Authorization Expiration DateTime**, where relevant. See Is Single Authorization flag and Authorization Expiration Date Time.
7. Generate a unique `ConsentId`, and set `BaseConsentId`. Leave it unset only on the very first Multi-Payment consent for a given service. The OFP defaults it to match `ConsentId` in that

case.

2. Set the Variable On-Demand control parameters

Implements: TPP Consent Staging Rules & Guidelines: Variable On-Demand Consent Set-up.

1. **Consent Expiration DateTime.** Let the User pick it, or pre-populate it based on the business case. It's mandatory.
2. **Cumulative control parameters**, both optional, independent of each other and of any per-period caps:
   - `MaximumCumulativeValueOfPayments`: the maximum total value over the consent's lifetime.
   - `MaximumCumulativeNumberOfPayments`: the maximum total count over the consent's lifetime.
3. **Periodic control parameters.** Provide at least one of `MaximumIndividualAmount`, `MaximumCumulativeValueOfPaymentsPerPeriod`, or `MaximumCumulativeNumberOfPaymentsPerPeriod`. Any two, or all three, are allowed. Where both an individual and a per-period value cap are set, the per-period cap must exceed the individual cap. If you set any per-period cap, also provide:
   - `PeriodType`: `Day`, `Week`, `Month`, or `Year`. See Period Type & Start Date for the exact period-boundary rules, including month and year edge cases like the 29th through 31st.
   - `PeriodStartDate`: when the recurring period begins.
4. Respect the Variable On-Demand limits that bound these parameters (LIMT-004 through LIMT-008: Max Variable On-Demand Payment Amount, Max Variable On-Demand Consent Cumulative Value and Payments, and their per-Control-Period equivalents). Agree these between User and TPP within those ceilings.
5. Where the consent uses a Predefined Beneficiary List, respect the Maximum Number of Creditors in a Multi-Creditor Payment Initiation Consent (LIMT-009, between 2 and 10).
6. Provide `ControlParameters.ConsentSchedule` per the steps above.

3. Set up the Multi-Creditors list

Implements: Variable On-Demand Multi-Payments to Multi-Creditors.

Provide the full list of beneficiaries, agreed in advance, with account details, in the PAR's `PersonalIdentifiableInformation` object. The User authorizes these specific Creditors, alongside the control parameters, when approving the consent. No further per-payment authorization is required. See Step 1.

4. Offer the Balance Check permission (optional)

Implements: Balance Check Permission and Balance Check.

If you offer balance checks ahead of a payment, request the `ReadBalances` permission plus one of `ReadAccountsBasic` or `ReadAccountsDetail` in the consent. Use this only to check funds are sufficient immediately before initiating a payment under this same consent, and to optionally show the balance to the User at that point. Never use it as a substitute for a proper Bank Data Sharing consent, for periodic balance refreshes in a Personal Finance Manager, for example.

5. Stage the consent

Implements: Consent Staging.

Capture the User's explicit consent to the parameters presented, then submit the PAR request. See Consent Staging for the general mechanics, identical to the single-payment case.

6. Hand off the User to the LFI

Implements: Hand-off to LFI.

Once the PAR returns `200 OK` with a `request_uri`, tell the User they're being transferred to their LFI to authorize the setup. Suggested wording: "We will securely transfer you to [YOUR LFI] to authorize the payment setup." See Hand-off to LFI for the redirection-screen mechanics, the same as the SIP TPP guide | Step 4.

7. Receive the hand-off back from the LFI

The LFI authenticates the User once, presents the consent, including the Balance Check permission and a Trusted Payee offer where applicable, and captures authorization or cancellation. See the LFI Implementation Guide for the full detail. Resume control once the OFP

redirects the User back with the authorization code, using the same mechanics as Common Rules and Guidelines | Hand-off back to the TPP.

8. Submit payment initiations

Implements: Payment Initiation.

Once authorized, initiate payments at any time, within the consent's terms:

1. Never submit a request above the authorized `MaximumIndividualAmount`.
2. Track cumulative value and number against the lifetime caps, and never submit a request that would breach either.
3. Track cumulative value and number against the current Control Period's caps, and never submit a request that would breach either. The OFP is the actual system of record for these totals. See the OFP guide | Step 6.

If the consent uses a Defined List of Creditors, additionally:

4. Include one of the Predefined Beneficiary List's Creditor Identification entries in the request.

9. Track status, confirm, and notify

Implements: Payment Status Update, Payment Notifications, and Confirmation to User.

Use the same Payment Status Update, Confirmation to User, and Payment Notifications mechanics used for a single payment. See the SIP TPP guide | Step 8. Apply them per individual payment initiated under this consent, not once for the whole consent.

10. Manage consent updates

Implements: Consent Update.

Optionally let the User amend, from a Consent Dashboard: the Debtor Account, Debtor or Creditor Reference, Purpose of Payment, the control parameters, the Control Period and start date, the Consent Expiration DateTime, or, for a Defined List of Creditors, the Predefined Beneficiary List. Whichever parameters you allow, if any:

1. When staging the replacement consent, carry over the original `BaseConsentId` so both consents link as the same ongoing service, and generate a fresh, unique `ConsentId` for the new one.
2. Require the User to re-authenticate with their LFI and authorize the update.
3. Regardless of what you let the User update, they can always revoke the consent outright.

11. Account for journey variations

See Handle Journey Variations for the account-scope variations this guide's standard flow doesn't cover.

Making a Variable On-Demand Payment | LFI Implementation Guide

This guide is for LFIs supporting a Variable On-Demand (`VariableOnDemand`) Multi-Payment consent: authorized once by the User, then used by the TPP to initiate an unknown number of future payments without further MFA. See What are Multi-Payments for the full scope.

> **Scope**
>
> This guide covers Variable On-Demand only, not Combined Payments or any other Multi-Payment type.

Every step below cites the exact rule it implements, drawing on the Multi-Payment rules (MPGN, MPOD) and the Common Rules and Guidelines.

1. Authenticate the User

Implements: Authentication.

Once the User is redirected from the TPP, authenticate them using the same models as for a single payment. See the SIP LFI guide | Step 3 for Redirection and Decoupled Redirection in full. Where your implementation instead uses Centralized/Federated Authentication and Authorization, follow that model's own rules. It isn't covered by the Redirection and Decoupled Redirection pages this guide otherwise draws on.

This single authentication covers the entire consent. Every subsequent payment the TPP initiates under it proceeds without asking the User to authenticate again.

2. Present the consent for authorization

Implements: Authorization.

1. The MFA from Step 1 already covers reviewing and authorizing the consent.
2. Retrieve the staged payment Consent from the OFP by its unique identifier.
3. Don't reject the consent just because its Purpose of Payment would violate an existing BAU blocking rule. The payment-initiation messages that follow may carry a different Purpose of Payment code that doesn't violate anything.

4. If the consent doesn't already specify a payment account, let the User select one. See Payment Account Selection at LFI. Unlike a Single Instant Payment, allow selection of an account with insufficient funds, since the User may intend to fund it before payments actually start, but tell the User it currently has insufficient funds.
5. Don't earmark or block any funds against this consent at authorization time. There's nothing to reserve yet.
6. Check the selected account's authorization status against the TPP's Is Single Authorization flag. See Is Single Authorization flag.
7. Display the TPP's Trading Name, plus any customer-facing service provider's name alongside it, where applicable.
8. Present, at minimum:
    - User Payment Account.
    - Creditor Identification (single Creditor, or a predefined list).
    - Debtor Reference (optional).
    - Creditor Reference.
    - Purpose of Payment.
    - The control parameters (`MaximumIndividualAmount` and any cumulative caps).
    - The Control Period and start date.
    - The Consent Expiration Date & Time.
9. Request the User's authorization:
    - Also request authorization of the Balance Check permission, if the TPP requested it.
    - Where the consent is to a single Creditor, offer to add it to the User's Trusted Payees. See Adding a Trusted Payee (Beneficiary).
10. Let the User cancel. Hand back to the TPP with an error, and reject the consent, if they do.
11. Move the consent from `AwaitingAuthorization` to `Authorized` once every required Authorizer has acted, and update the consent details at the OFP.

The OFP, in turn, checks the Authorization Time Window is valid, confirms your update, and starts tracking the Control Period's totals from the mandatory Control Period Start Date (00:00:00 on the start day through 23:59:59 on the period's last day).

3. Support multi-user authorization

Where more than one authorizer is required, follow Multi User Authorization Flow exactly as for a single payment. See the SIP | FI guide's General Notes reference for how that interacts with the

authorization flow above.

4. Hand off back to the TPP

Implements: Hand-off back to the TPP.

Same mechanics as a single payment. See Hand-off back to the TPP.

5. Provide balance check responses (conditional)

Implements: Balance Check Request Processing (LFI obligations).

When the OFP asks for a balance check under this consent, provide the same balance information defined for the `ReadBalances` permission in Permissions and Data Clusters.

6. Process payment initiation requests

Implements: Processing of Payment Initiation Requests (LFI obligations).

1. Trigger payment initiation immediately on receiving the request from the OFP.
2. Validate the Creditor per the consent's Creditor scope:
    - **Single known Creditor ("Fixed Beneficiaries").** Retrieve the Creditor Identification from the encrypted PII block in the original Consent message and run your normal BAU checks. Reject, and notify the OFP, on failure.
    - **Defined List of Creditors ("Variable-defined Beneficiaries").** Check the request's Creditor Identification exactly matches an entry in the Predefined Beneficiary List. Reject, and notify the OFP, if it doesn't.
3. Accept the request from the OFP without extra MFA or User authorization. This consent already covers it.
4. Apply your existing BAU account controls and limits exactly as on any other channel, including rejecting on a Purpose of Payment that violates existing blocking rules, and notify the OFP with an error on any breach.
5. Reject, and notify the OFP, if the account has insufficient funds.
6. On success, execute the payment, through the payment rails or intra-bank, and return the unique Transaction ID to the OFP.

You don’t need to track the cumulative value and number totals, lifetime or per-period, yourself. The OFP is the system of record for those, and validates them before the request ever reaches you. See the OFP guide | Step 6.

7. Support consent updates

Implements: Consent Update.

Where the TPP allows the User to update the consent, require them to re-authenticate and authorize the update through the same flow as Step 2. The new consent replaces the original, linked through `BaseConsentId`.

Making a Variable On-Demand Payment | OFP Implementation Guide

This guide is for anyone implementing the OFP (Open Finance Platform) role for a Variable On-Demand (`VariableOnDemand`) Multi-Payment consent. Unlike a Single Instant Payment, you here are the system of record for the consent's cumulative control parameters across its entire lifetime. That's the part of this role genuinely specific to Variable On-Demand. See What are Multi-Payments for the full scope.

> **Scope**
>
> This guide covers Variable On-Demand only, not Combined Payments or any other Multi-Payment type.

Every step below cites the exact rule it implements, drawing on the Multi-Payment rules (MPGN, MPOD) and the Common Rules and Guidelines.

1. Validate and confirm the staged consent

Implements: Consent Staging, applying Consent Staging.

Validate the consent's format and business-rule compliance, ask the LFI to run its own additional validation, and reject with an appropriate error if either check fails. These are the same mechanics as the SIP OFP guide | Step 1. Where the consent has no `BaseConsentId` because it's the first Multi-Payment consent for the service, default `BaseConsentId` to match `ConsentId`.

2. Broker the hand-off to the LFI

Provide the Consent details to the LFI, by its unique identifier, once it requests them after the TPP triggers its authorization endpoint. See Hand-off to LFI.

3. Check the authorization time window and start Control Period tracking

Implements: Authorization | 12, 14.

1. Check the Authorization Time Window is still valid before confirming the LFI's authorization update.
2. Where the consent has a Control Period, start tracking its cumulative totals from the mandatory Control Period Start Date. The period runs from 00:00:00 on the start day to 23:59:59 on the period's last day, per Period Type & Start Date.

4. Relay multi-user authorization information

Where more than one authorizer is required, relay that information between LFI and TPP exactly as for a single payment. See Multi User Authorization Flow and the SIP OFP guide | Step 4.

5. Broker the hand-off back to the TPP

Implements: Hand-off back to the TPP.

Same mechanics as a single payment. See Hand-off back to the TPP.

6. Validate payment initiation requests

Implements: Processing of Payment Initiation Requests (OFP obligations).

This is the core of your ongoing role for this consent type:

1. Accept the TPP's request without demanding further MFA or User authorization.
2. Check the request maps to a Consent genuinely in the `Authorized` state. Reject, with an error code and message, any request against a Consent in another state (`Consumed` or `Expired`, for example).
3. Validate the request against the authorized consent:
    - **Execution date** falls on or before the Consent Expiration DateTime.
    - **Payment amount** is at or below the consent's `MaximumIndividualAmount`.
4. Validate against every cumulative control parameter actually defined on the consent:
    - Lifetime total value, including this request, stays at or below `MaximumCumulativeValueOfPayments`.

- Lifetime total count, including this request, stays at or below `MaximumCumulativeNumberOfPayments`.
- Current-period total value, including this request, stays at or below `MaximumCumulativeValueOfPaymentsPerPeriod`.
- Current-period total count, including this request, stays at or below `MaximumCumulativeNumberOfPaymentsPerPeriod`.

> **Note**
>
> You own these running totals. The LFI does not retain or validate them.

5. Once the payment resource is created and sits in `Pending` (not rejected), increment both the lifetime and current-Control-Period totals, value and count, starting from zero for a newly authorized consent, and resetting the per-period totals to zero at 00:00:00 on each new period's first day.

6. Move the consent to the terminal `Consumed` state the moment either lifetime cumulative cap, value or count, is reached exactly.

7. Allow the Creditor Reference on an individual request to differ from the one set in the consent.

8. Reject the request, with an error to the TPP, on any other check failure. When you do, decrement the consent's totals, and the Control Period totals where the payment was made in the current Control Period. See COMM-150 and COMM-151.

9. Forward the accepted request to the LFI: User Payment Account (or identifier), Payment Amount & Currency, the PII object, Debtor Reference (if provided), Creditor Reference, Purpose of Payment.

10. Relay whatever the LFI returns:

    - an error to the TPP, and decrement the totals exactly as in step 8, if the LFI rejected it for a BAU check or limit breach
    - an error to the TPP if rejected for insufficient funds
    - otherwise, the initiated payment's full details, including its Transaction ID

7. Propagate payment status

Implements: Payment Status Update.

Use the same Payment Status Update mechanics as a single payment, applied per individual payment under this consent. See the SIP OFP guide | Step 8.

8. Provide balance check responses (conditional)

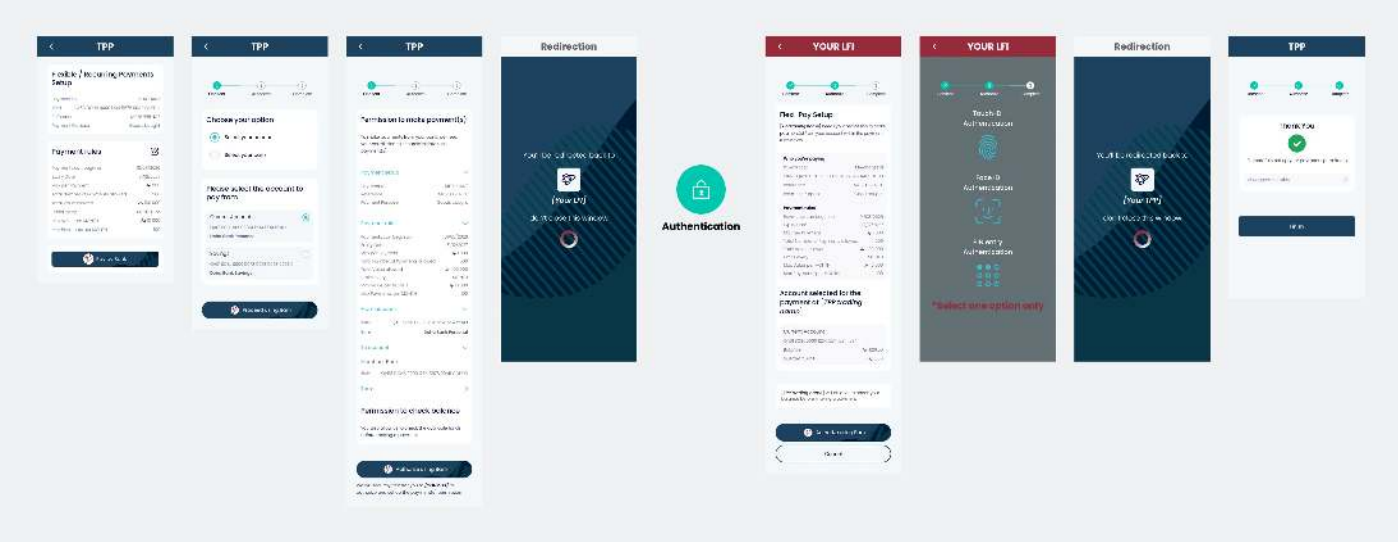
Implements: Balance Check Request Processing (OFP obligations).

Provide the TPP with all available balance information the LFI returns, or an appropriate error if the balance check itself failed.

Making a Variable On-Demand Payment | Handle Journey Variations

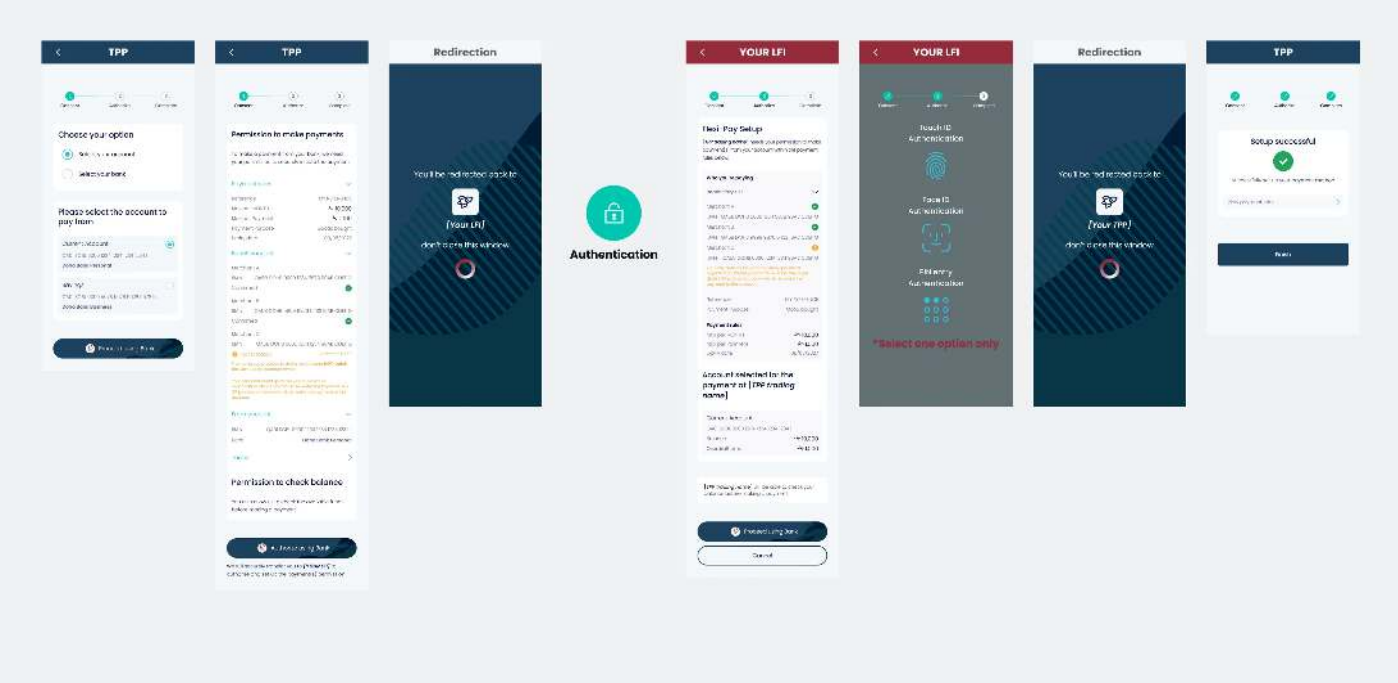
The TPP, LFI, and OFP guides describe the Variable On-Demand flow in general. This page covers the variations within that flow: the two Creditor scopes.

The two Creditor scopes



Implements: Variable On-Demand Multi-Payments to Multi-Creditors.

Multi-Creditors (a defined list)



A predefined list of beneficiaries, agreed in advance and authorized as part of the consent. No further per-payment authorization is required once the consent is authorized.

- **TPP:** provide the full Creditor list up front. See TPP guide | Step 1 and Step 8 (include a matching list entry in every request).
- **LFI:** validate each request's Creditor against the Predefined Beneficiary List exactly. See LFI guide | Step 6.

Single known Creditor

The simplest case: the same as Multi-Creditors but with exactly one beneficiary, referred to as "Fixed Beneficiaries" in the payment-processing rules. Offer to add it as a Trusted Payee at authorization. See LFI guide | Step 2.

Overview

The Platform Guide describes the shared services that support participants in the Qatar Open Finance Framework.

General | Overview

The General section holds the technical reference that applies to every Licensed Financial Institution (LFI) connecting to the QCB Open Finance API Hub, whichever APIs it implements. Its pages describe where each service lives and how each connection is secured. The LFI Integration Guide then describes how the LFI builds and connects its side of the integration.

The pages are drafts for Qatar Central Bank (QCB) review. Each marks the values that QCB or the platform provider has still to confirm as **[TBC – QCB]** or **[TBC]**.

API Hub Domains and DNS

Introduction

Ozone hosts the API Hub in a dedicated QCB environment in Qatar and runs DNS for every API Hub domain. The sections below cover the domain pattern for each environment, the API Hub services the LFI and TPPs call, and the services the LFI hosts.

Every host name here is an example. The base domain, the LFI code and the final host names await a QCB decision [TBC – QCB]. The naming pattern is Proposed, and Ozone will confirm the pattern with the LFI during technical onboarding.

The API Hub has a fourth environment, Internal User Acceptance Testing (UAT), for QCB use only. LFIs and TPPs never connect to Internal UAT, so this page leaves the environment out.

QCB Open Finance Platform Domains

Each LFI gets its own tenant, with its own host names, in Pre-Production and Production. The Sandbox runs one shared model bank.

Environment	Domain (Proposed)	Example DNS name
Production	`*.<lfi-code>.<base-domain>`	`rs1.<lfi-code>.<base-domain>`
Pre-Production	`*.<lfi-code>.preprod.<base-domain>`	`rs1.<lfi-code>.preprod.<base-domain>`
Sandbox	`*.<model-bank-code>.<base-domain>`	`rs1.<model-bank-code>.<base-domain>`

- `<base-domain>`: [TBC – QCB].
- `<lfi-code>`: the LFI's code in host names [TBC – QCB]. A separate tenant per brand means a separate code per brand (Proposed).
- `<model-bank-code>`: the code for the Sandbox model bank [TBC].

Production serves the live APIs across three availability zones. The LFI tests against the API Hub in Pre-Production before go-live. TPPs have no access to Pre-Production (Proposed). TPPs run end-to-end tests in the Sandbox against a model bank with synthetic data.

API Hub Service Examples

The API Hub runs these services in each LFI tenant. Server names are Proposed. Host names follow the pattern in QCB Open Finance Platform Domains.

Server name	Example DNS name	Description	Security	Provider	Consumer
admin	`admin.<lfi-code>.<base-domain>`	Admin Portal. LFI users manage access and view usage, availability and performance reports.	TLS plus user sign-in by one-time password (OTP) or OpenID Connect (method [TBC])	API Hub	LFI
hh	`hh.<lfi-code>.<base-domain>`	Authorization Server API. Called by the LFI's consent journey to process authorization requests.	mTLS, with optional application layer authentication	API Hub	LFI
cm	`cm.<lfi-code>.<base-domain>`	Consent Management API. The LFI reads and updates consents.	mTLS, with optional application layer authentication	API Hub	LFI
auth1	`auth1.<lfi-code>.<base-domain>`	OpenID Connect discovery for the API Hub Authorization Server.	TLS (Proposed)	API Hub	TPP
rs1	`rs1.<lfi-code>.`	Resource Server. Serves the QCB Open Finance APIs to TPPs.	[TBC – QCB], set by the security profile	API Hub	TPP

Server name	Example DNS name	Description	Security	Provider	Consumer
	`<base-domain>`				
as1	`as1.<lfi-code>.<base-domain>`	Authorization Server. Handles TPP authorization and token requests.	[TBC – QCB], set by the security profile	API Hub	TPP

The LFI must use mutual Transport Layer Security (mTLS) on every connection with the API Hub. Application layer authentication is optional (see Application Layer Authentication). The LFI generates its certificates through the Ozone Trust Framework. QCB has not yet chosen the security profile for TPP connections [TBC – QCB]. Ozone recommends FAPI 2.0.

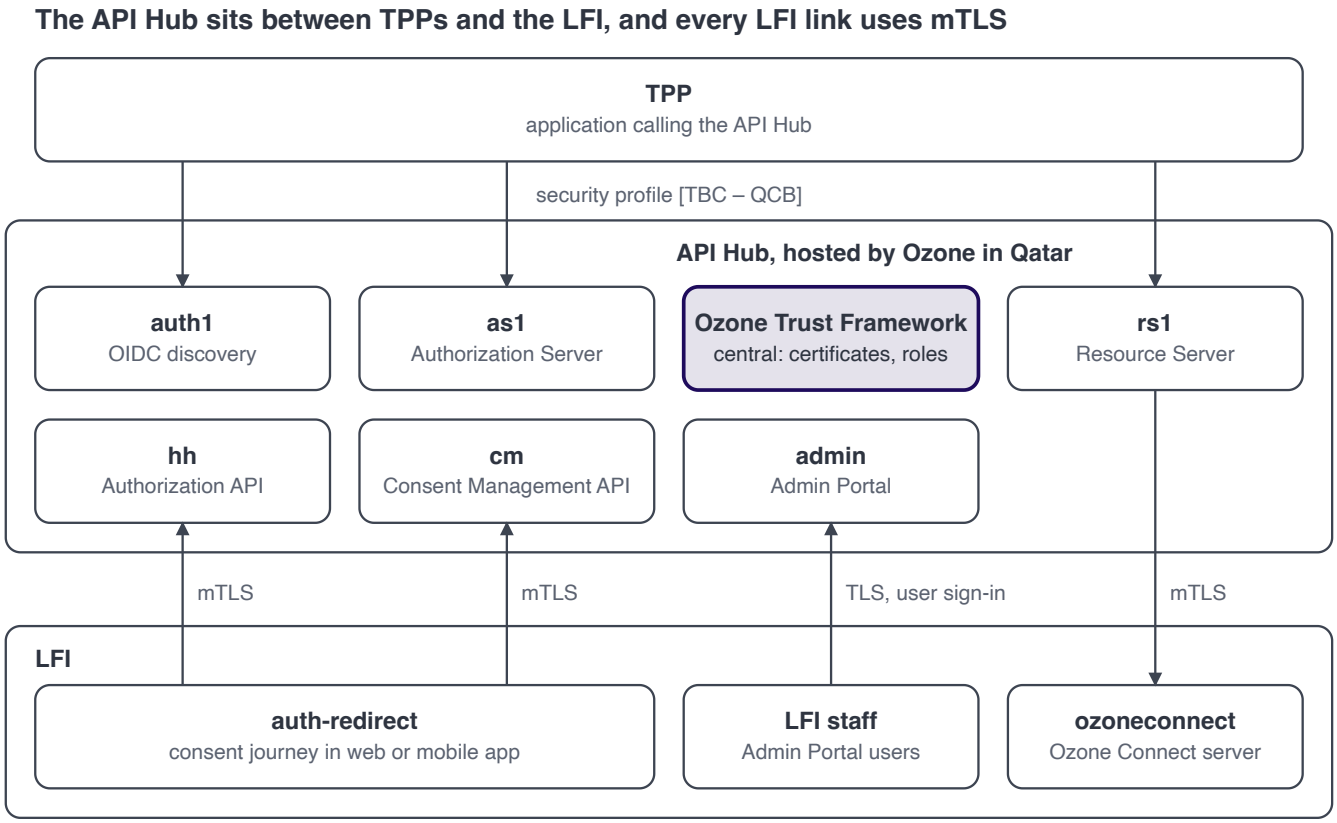
LFI Service Examples

The LFI hosts these services on its own domain, with one set for Pre-Production and one for Production. The LFI submits the host names in its onboarding forms through the service desk (form [TBC]).

Example DNS name	Description	Security	Provider	Consumer
`ozoneconnect.<lfi-domain>`	Ozone Connect server. The API Hub calls it to retrieve account data and initiate payments.	mTLS, with optional application layer authentication	LFI	API Hub
`auth-redirect.<lfi-domain>`	Authorization URL. TPPs send the customer here to start the consent journey in the LFI's web or mobile channel.	[TBC]	LFI	TPP

The network route from the API Hub to the LFI's Ozone Connect server is [TBC]: internet or private connection. The route decides whether the LFI's host names need public DNS entries. Ozone will send the LFI the API Hub's outbound IP addresses for the LFI's allow-list (Proposed).

Network Diagram



The Ozone Trust Framework (shaded) runs as one central service. Every other API Hub service runs inside the LFI's tenant. Each arrow starts at the side making the call.

Keys, CSRs and Certificates

Introduction

This page lists every key and certificate an LFI needs to connect to the QCB Open Finance API Hub. For each one, the tables show who holds the private key and what the LFI must do.

Mutual Transport Layer Security (mTLS) is mandatory on every connection between the API Hub and the LFI. Application layer authentication on top of mTLS is optional. Application Layer Authentication describes the options.

Keys and certificates fall into three groups:

- **Transport** – used for TLS and mTLS connections (see Transport Keys and Certificates).
- **Signing** – used to sign messages (JWS) and published in a JSON Web Key Set (JWKS) (see Signing Keys and Certificates).
- **Encryption** – used to encrypt data (JWE) so that only the key holder can read it (see Encryption Keys and Certificates).

The Ozone Trust Framework does not issue certificates. The certificate authority (CA) for each certificate type, and whether Pre-Production and Production use different CAs or roots, are [TBC – QCB]. Ozone recommends one CA for all three certificate types. Whether signing and encryption keys need certificates at all, or use bare key pairs, is also [TBC – QCB]. The Ozone Trust Framework acts as the directory of participants and hosts the JWKS for each participant.

The Ozone Trust Framework key policy sets the permitted key types, key sizes and algorithms for each key type. The security profile for QCB Open Finance is FAPI 2.0.

C1, S1, Sig1, Enc1 and the other labels on this page are reference names for onboarding discussions. They do not appear in the certificates.

In the labels, C marks a client certificate, the side proving its identity when opening an mTLS connection. S marks a server certificate, on the side receiving the connection. Sig marks a signing key and Enc an encryption key. A C and an S with the same number secure the same connection: 1 for TPP to API Hub, 3 for the LFI to the API Hub Consent Management API and Authorization Server, 4 for the API Hub to the LFI's Ozone Connect endpoint. S2 has no matching C because its endpoints do not use mTLS.

Transport Keys and Certificates

The LFI acts on four of the seven transport certificates: S1, S3, S4 and C3.

Certificate	Description	Issuer	Private key held by	CSR generated by	Certificate generated by	Action require by LFI
C1	Identifies the TPP to the API Hub	Approved CA	TPP	TPP	TPP	None
S2	Identifies the API Hub endpoints that do not use mTLS to the TPP	Approved CA	API Hub	API Hub	API Hub	None
S1	Identifies the API Hub mTLS endpoints to the TPP	Approved CA	API Hub	API Hub; the CSR includes the Subject Alternative Names (SANs) of the endpoints	LFI	Get the CSR provide by the A Hub sig (see S1 and Sig
C4	Identifies the API Hub to the LFI's Ozone Connect endpoint	Approved CA	API Hub	API Hub	API Hub	None
S3	Identifies the API Hub Consent	Approved CA	API Hub	API Hub	LFI	Get the CSR provide

Certificate	Description	Issuer	Private key held by	CSR generated by	Certificate generated by	Action require by LFI
	Management API and Authorization Server endpoints to the LFI					by the A Hub sig (see S1 and Sig
S4	Identifies the LFI's Ozone Connect endpoint to the API Hub	Approved CA	LFI	LFI; the CSR includes a SAN matching the Ozone Connect server hostname	LFI	Genera the key pair and CSR, th get the CSR signed (see S4 and Enc
C3	Identifies the LFI to the API Hub Consent Management API and Authorization Server endpoints	Approved CA	LFI	LFI	LFI, as an application certificate	Registe the applica generat the key pair and CSR, th get the CSR signed (see C3 and Sig

Signing Keys and Certificates

The LFI always acts on Sig2. The LFI acts on Sig4 only when using JWT Auth towards the API Hub. Message signing is a separate, optional part of FAPI 2.0. QCB Open Finance uses message signing for API Hub responses to the TPP (Sig2). Use of message signing for TPP requests (Sig1) is [TBC].

Item	Description	Issuer	Private key held by	CSR generated by	Certificate generated by	Action required by LFI
Sig1	Used by the TPP to sign requests to the API Hub	Approved CA	TPP	TPP	TPP	None
Sig2	Used by the API Hub to sign responses to the TPP	Approved CA	API Hub	API Hub	LFI	Get the CSR provided by the API Hub signed (see S1, S3 and Sig2)
Sig3	Used by the API Hub to sign requests to the LFI. Needed only if the LFI uses JWT Auth on Ozone Connect, or `private_key_jwt`	Approved CA	API Hub	API Hub	API Hub	None

Item	Description	Issuer	Private key held by	CSR generated by	Certificate generated by	Action required by LFI
	client authentication					
Sig4	Used by the LFI to sign requests to the API Hub. Needed only if the LFI uses JWT Auth towards the Consent Management API and Authorization Server	Approved CA	LFI	LFI	LFI, as an application certificate	Generate the key pair and CSR, then get the CSR signed (see C3 and Sig4)

Application Layer Authentication describes JWT Auth and the other options.

Encryption Keys and Certificates

The LFI holds the encryption key for personally identifiable information (PII). The API Hub encrypts stored PII with the LFI's key, on top of platform encryption at rest.

Item	Description	Issuer	Private key held by	CSR generated by	Certificate generated by	Action required by LFI	J
Enc1	Used to encrypt PII so that only the LFI can decrypt it	Approved CA	LFI	LFI	LFI	Generate the key pair and CSR, then get the CSR signed	H th T F

Item	Description	Issuer	Private key held by	CSR generated by	Certificate generated by	Action required by LFI	J
						(see S4 and Enc1)	

Which parties encrypt PII with Enc1, and for which API calls, depends on QCB Open Finance Standard v1 and is [TBC].

Creating Certificates

The LFI needs separate certificates for each API Hub environment. Ozone shares CSRs and other details through the QCB Open Finance service desk.

Environment Considerations

The Ozone Trust Framework has two environments: Pre-Production and Production. Each is a separate directory with its own Users and roles. Ozone Trust Framework Pre-Production serves the API Hub Sandbox and API Hub Pre-Production. Ozone Trust Framework Production serves API Hub Production. The approved CA issues certificates for each API Hub environment.

An LFI that tests as a TPP against the API Hub Sandbox uses its Ozone Trust Framework Pre-Production registration. No separate Sandbox registration is needed.

Certificate Generation for S1, S3 and Sig2 (Private Key Held by the API Hub)

1. Ozone generates the key pairs and CSRs for S1, S3 and Sig2 and sends the CSRs to the LFI through the service desk. The S1 and S3 CSRs include the SANs needed for domain validation.
2. The LFI gets each CSR signed by the approved CA, using the CA's own process.
3. The LFI uploads the signed certificates to a key store in the Ozone Trust Framework, then sends the JWKS URL and key ID (`kid`) for each to Ozone through the service desk.

Certificate Generation for S4 and Enc1 (Private Key Held by the LFI)

1. The LFI generates the key pair and CSR for S4 and for Enc1. The S4 CSR must include a SAN matching the LFI's Ozone Connect server hostname.
2. The LFI gets each CSR signed by the approved CA, using the CA's own process.
3. The LFI uploads the signed certificates to a key store in the Ozone Trust Framework, then sends the JWKS URL and key ID (`kid`) for each to Ozone through the service desk.

The LFI generates keys and CSRs with its own tooling, following the approved CA's requirements. Recommendation: the LFI should keep these private keys inside its own organization and not share them with Ozone, QCB or any other party.

Certificate Generation for C3 and Sig4 (Private Key Held by the LFI)

The LFI needs Sig4 only when using JWT Auth towards the API Hub.

1. The LFI's PTC registers an application in the Ozone Trust Framework for its client of the API Hub Consent Management API and Authorization Server, and links the application to a key store. The key store sets the application's JWKS URL.
2. The LFI generates the key pairs and CSRs for C3 and, if needed, Sig4.
3. The LFI gets each CSR signed as an application certificate by the approved CA, using the CA's own process.
4. The LFI uploads the signed certificates to the key store linked to the application, then sends the JWKS URL and `kid` for each to Ozone through the service desk.

Expiry and Rotation

Certificates are valid for 13 months, as a benchmark for QCB to confirm or change.

- **S1, S3 and Sig2:** Ozone tracks expiry and raises a rotation request through the service desk one month before expiry. The LFI gets the new CSRs signed by the approved CA, as in S1, S3 and Sig2.
- **S4, C3, Sig4 and Enc1:** the LFI holds the private keys, so the LFI tracks expiry and rotates them.

LFI Integration Guide | Overview

> **Document Status**
>
> This Release Candidate has no Qatar Central Bank (QCB) approval yet. It sets out the components a Licensed Financial Institution (LFI) needs to connect to the QCB Open Finance Application Programming Interface (API) Hub, and what each party delivers.
>
> - **Proposed** = the recommended approach, awaiting QCB confirmation.
> - **[TBC – QCB]** = to be confirmed (TBC) by QCB. **[TBC]** = awaiting confirmation from the platform provider. Every value comes from a confirmed source or shows as TBC.

Introduction

This page lists each component of the integration between the API Hub and the LFI's systems. For each component, the table in Key API Hub components shows the provider, the consumer and the usage.

Ozone provides, hosts and operates the API Hub for QCB. QCB approves LFIs for access to the platform and for go-live. QCB also reviews each LFI's test and customer experience (CX) evidence and issues the LFI's certification.

High-Level Consent Authorization Flow

Under QCB Open Finance, a User gives consent at the Third Party Provider (TPP) to share data or initiate a service, then authorizes the consent at the LFI. The User authorizes consent in the LFI's existing internet banking or mobile banking app. The LFI must not provide an alternative interface or app for this.

The sequence of activities is prescribed as follows:

1. The User starts a consent request in a TPP's mobile or web app.
2. The TPP sends the User to the LFI's app or website to authorize the consent.
    1. The LFI updates its app and website to receive the redirect and the request parameters from the TPP.
    2. The LFI verifies the request and its parameters with the API Hub Authorization Server.

3. The LFI authenticates the User with Strong Customer Authentication (SCA). The User reviews the consent, then authorizes or rejects the consent.
    1. The LFI builds the consent authorization screens.
    2. The LFI's screen shows the User the details for the type of consent they are authorizing. For an account information consent, the User sees each property of the data they share with the TPP.
    3. The QCB CX Guidelines set out the user flows and wireframes for each screen [TBC – link].
    4. The LFI records the outcome in the Consent Manager and the Authorization Server.
    5. The User is redirected back to the TPP's app.
4. The API Hub issues the TPP an access token. The TPP then has access to the User's accounts only within the terms of the consent.
    1. The API Hub calls the LFI's Ozone Connect API to retrieve data or initiate the payment.

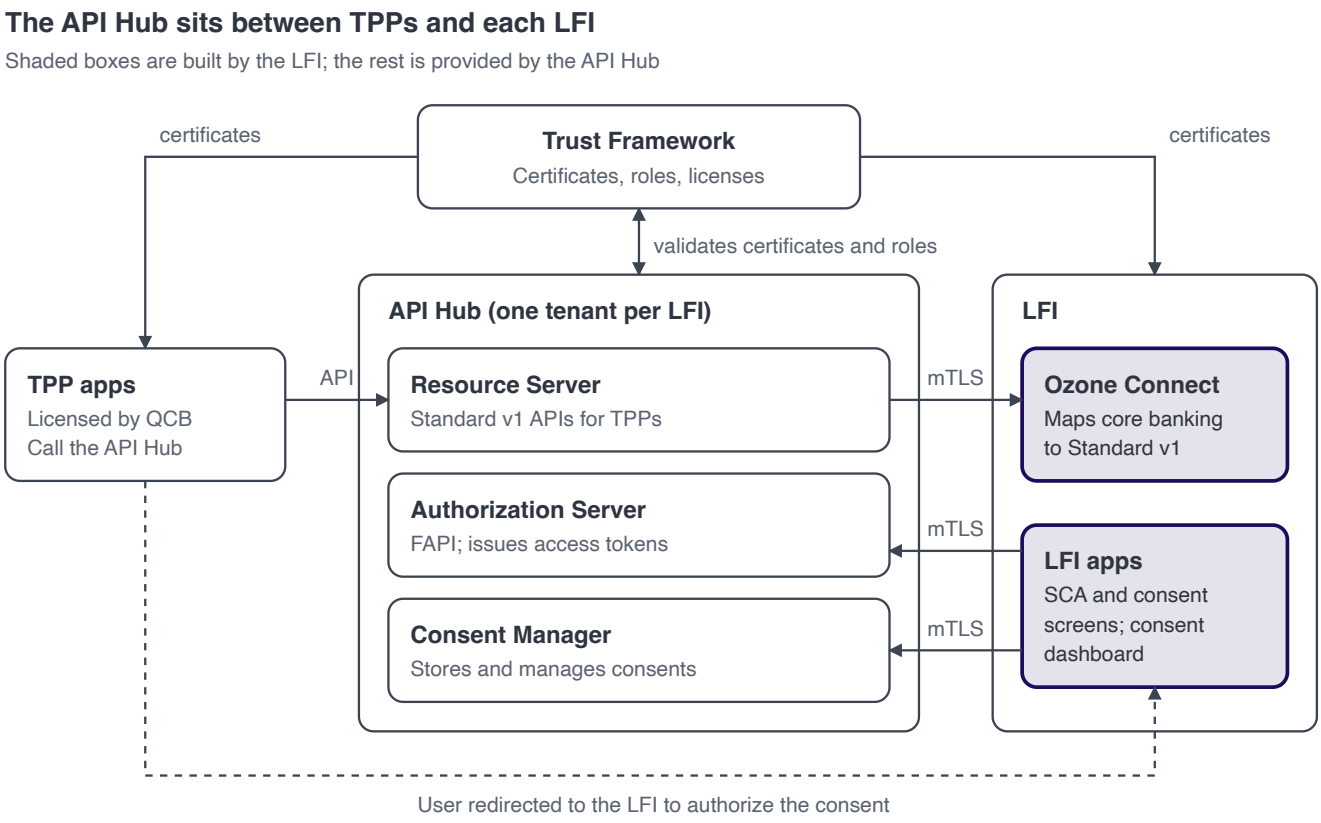
Standard v1 sets the detailed flow for each use case: see the Account Information, Payment Initiation and Authorization Endpoints APIs. QCB has yet to choose the security profile for Standard v1 [TBC – QCB]. The recommended profile is Financial-grade API (FAPI) 2.0.

The API Hub

The API Hub sits between TPPs and the Ozone Connect implementation of each LFI. Each LFI gets a dedicated API Hub host to serve TPP requests. The API Hub handles the TPP-facing APIs, authorization, consent management, TPP verification against the Trust Framework, and reporting. Each LFI must integrate with the Open Finance Framework in three areas: connectivity to the API Hub, its Ozone Connect API, and consent journeys in its own channels.

Integration Architecture

The diagram below shows the API Hub in the context of TPPs and LFI integration requirements.



The diagram shows these flows:

- TPPs call the API Hub.
- The API Hub calls the LFI's Ozone Connect API.
- The LFI's apps use the Authorization Server and the Consent Manager.
- Connections between the API Hub and the LFI use mutual Transport Layer Security (mTLS).
- Every tenant runs in both Pre-Production and Production.

Infrastructure

Each LFI gets two API Hub tenants, Pre-Production and Production, each connected to the LFI's own systems. Production runs across three availability zones for high availability.

Every Hypertext Transfer Protocol Secure (HTTPS) connection between the API Hub and the LFI's environments uses mTLS. Application-layer security on top of mTLS is optional, and the LFI chooses during onboarding (see Application Layer Authentication). The Trust Framework issues the LFI's certificates.

The API Hub also runs a shared Sandbox, a model bank with synthetic data and no LFI systems connected. QCB alone uses a fourth environment, Internal User Acceptance Testing (UAT), to accept new releases. Ozone hosts the API Hub on Google Cloud Platform in Qatar and keeps

production data, backups and audit logs in Qatar. An LFI with separate brands or authorization journeys needs a separate tenant for each brand.

Role of the API Hub

The API Hub performs the following functions in the Open Finance ecosystem:

1. Provides an industry Sandbox: a model bank with lifelike synthetic data and a copy of the consent authorization journeys, plus a Postman collection.
2. Gives each LFI dedicated Pre-Production and Production tenants for TPPs to call.
3. Enforces Standard v1 on the external API, covering the security profile, data model and endpoint operations.
4. Checks certificates, roles and licenses against the Trust Framework in real time. Only TPPs with a QCB license reach the APIs, and only within their licensed roles.
5. Stores and manages User consents. Data flows and payments start only after the User authorizes them, and only within the consent.
6. Supplies QCB with reporting on usage, availability, performance, consents and payments.

Because the API Hub does this, LFIs do not build an external API, manage TPP identities or permissions, or check consent terms on each API request.

Sequence Diagrams

Sequence diagrams show how the User, the TPP, the API Hub and the LFI interact. They cover three pilot use cases:

- Bank data sharing
- Single immediate domestic payment
- (Variable) recurring domestic payments

Key API Hub Components

Five components make up the integration. The API Hub provides two, the LFI builds two, and both sides build the Health Check APIs.

- Ozone builds, deploys and maintains the Authorization Server and the Consent Manager. The LFI consumes both.
- LFIs build and maintain the screens for the consent authorization flow.

- LFIs build and maintain the Ozone Connect API for data sharing and payments.

Component	Provider	Consumer	Interface	Description	Connection	Us
Authorization Server (Heimdall Server)	API Hub	LFI	API	OpenAPI specification. FAPI authorization server. Issues access tokens only when the User has authorized the consent.	mTLS	Au flo
Consent Manager (Consent Management API)	API Hub	LFI	API	OpenAPI specification. Stores consents. The LFI reads and updates consents and payment status logs, and reads audit logs of changes. The LFI builds its consent authorization and consent management screens on this API.	mTLS	Au flo da rej
Ozone Connect	LFI	API Hub	API	OpenAPI specifications	mTLS	Ba sh

Component	Provider	Consumer	Interface	Description	Connection	Us
				for Bank Data Sharing and Bank Service Initiation. The API Hub calls this interface on the LFI's systems. The LFI maps its core banking services to the interface (see Data and Service Mapping). The business rules set which endpoints each LFI must implement [TBC – QCB]. The LFI validates its build with the Ozone Connect Test Suite.		pa
Health Check APIs	LFI and API Hub	LFI and API Hub	API	Each side exposes a health endpoint so the other can confirm the	mTLS	Se av co ch int qu

Component	Provider	Consumer	Interface	Description	Connection	Us
				service is running and mTLS works. Supports integration testing and monitoring. Ozone Connect specification; API Hub specification [TBC].		as (Q
Consent Event and Action APIs	LFI	API Hub	API	OpenAPI specification. Event notifications: the Consent Manager sends the LFI webhook notifications when a consent or payment status changes. Action calls, such as adding information to a consent or running extra checks before	mTLS	Ev no ac

Component	Provider	Consumer	Interface	Description	Connection	Us
				consent creation, are Proposed. QCB has yet to decide whether these are optional for LFIs [TBC – QCB].		

LFI Integration Guide

Introduction

This section has two parts. Integration covers what the LFI builds, with no dependency on onboarding. Deployment (prerequisites and lifecycle) connects the LFI's systems to the API Hub, then runs through testing and go-live.

Integration

The LFI starts building once the Ozone Connect API specification and Standard v1 are available [TBC – date]. Onboarding does not hold up this work. Integration covers three tasks:

1. Build the LFI's Ozone Connect API from the specification, mapping the LFI's services to Standard v1.
2. Validate the build with the Ozone Connect Test Suite. The suite runs in a Docker container on the LFI's own systems.
3. Adapt the LFI's web and mobile apps to:
    1. Accept a redirect from the TPP into the app.
    2. Show consent authorization screens where Users authorize each consent, as set out in the QCB CX Guidelines.
    3. Provide a consent dashboard where Users view and revoke their consents (Proposed, with requirements from the CX Guidelines [TBC – QCB]). The Consent Management Interface guide covers the API side.

Deployment Prerequisites

The LFI needs these before connecting to the API Hub. They do not apply to the integration work.

Pre-Production:

- The LFI's Primary Technical Contact (PTC) has an account on the Pre-Production Trust Framework.
- The PTC has registered on the QCB service desk.
- The PTC raises a service-desk ticket to start API Hub pre-production onboarding.
- The LFI then completes these forms in order, through the service desk:
    - Pre-requisites form
    - Pre-Production configuration form
    - Application Layer Security form
    - Postman Collection form

Production:

- The PTC has an account on the Production Trust Framework.
- The LFI holds pre-production certification.
- The LFI completes the Production configuration form through the service desk.

Deployment Lifecycle

1. Ozone deploys the LFI's Pre-Production and Production tenants. The LFI prepares its own pre-production and production environments.
2. The LFI generates certificates on the Trust Framework.
3. The LFI connects its Pre-Production and Production systems to the API Hub over mTLS.
4. The LFI and Ozone verify mTLS connectivity in both directions:
    1. From the LFI's systems to the API Hub health endpoints.
    2. From the API Hub to the LFI's Ozone Connect server.
5. The LFI deploys its Ozone Connect API and consent journeys to its pre-production environment, then to production.
6. Testing and certification: the LFI submits functional test evidence and CX evidence through the service desk. QCB validates the evidence and issues the LFI's certifications.
7. Go live: the LFI completes live proving with at least one approved TPP from a separate entity, using real friends-and-family accounts. QCB then approves the LFI's go-live.

API Hub User Guide

The document set below is in progress. Links and publication dates for items without a link are [TBC].

1. LFI Integration Guide (this page)
2. Keys, certificate signing requests (CSRs) and certificates
3. Application Layer Security
4. API Hub specifications (OpenAPI)
    1. Bank data sharing — Ozone Connect
    2. Payments (single immediate and variable recurring) — Ozone Connect
    3. Consent Manager
    4. Authorization Server
    5. Health Checks
    6. Consent Events and Actions
5. Sequence diagrams
6. Onboarding forms and configuration guides
7. TPP onboarding
8. User guides
9. API Hub reporting

Security | Overview

The Security section describes how the connections between a Licensed Financial Institution (LFI) and the QCB Open Finance API Hub are secured. It covers calls in both directions: the LFI calling the API Hub's Consent Management API and Authorization Server, and the API Hub calling the LFI's Ozone Connect endpoints. Calls from Third Party Providers (TPPs) to the API Hub are outside its scope.

The pages are drafts for Qatar Central Bank (QCB) review. Each marks the values that QCB or the platform provider has still to confirm as **[TBC – QCB]** or **[TBC]**.

Transport Security

Mutual Transport Layer Security (mTLS) is mandatory on every connection between the API Hub and the LFI, in Pre-Production and Production. The certificates that secure each connection, and how the LFI creates and rotates them, are listed on Keys, CSRs and Certificates.

Application Layer Authentication

Application layer authentication on top of mTLS is optional. The LFI chooses whether to use it, and which option, in the Application Layer Security onboarding form. Application Layer Authentication describes the options.

JWT Authorization describes one of them, JWT Auth: how the LFI signs a JSON Web Token (JWT) when it calls the API Hub, and how it verifies the token the API Hub sends when it calls Ozone Connect.

Application Layer Authentication

Mutual Transport Layer Security (mTLS) is mandatory between the QCB Open Finance API Hub and each LFI.

Application layer authentication provides additional API security, in addition to mTLS.

Introduction

This page covers the application layer authentication options for API calls between LFIs and the API Hub, the configuration that needs to be agreed for each option, and what the LFI implements for each.

Calls run in two directions. The API Hub calls the LFI through Ozone Connect, the LFI's mapping of its data and payment services to the QCB Open Finance standard. The LFI calls the API Hub through the Consent Management API and the Authorization Server APIs, to run consent journeys in its existing web and mobile channels.

The LFI states whether it will implement application layer authentication, and which option, in the Application Layer Security onboarding form, submitted through the QCB Open Finance service desk. The configuration for the chosen option is agreed between the LFI and Ozone during technical onboarding (Proposed).

All options below are Proposed. Availability of each option on the QCB Open Finance API Hub is [TBC]. The overall security profile for QCB Open Finance is [TBC – QCB]. FAPI 2.0 is the recommended profile.

Application Layer Authentication Switched Off

This is the simplest option. API calls carry no application layer authentication, and the integration relies on mTLS alone. Certificates for mTLS are issued through the Ozone Trust Framework (see Keys, CSRs and Certificates).

Because application layer authentication is optional, an LFI may choose this option. The main drawback is that it gives no defense in depth: if the transport layer is compromised, there is no second check on the caller.

API Key

Status: Proposed.

An API key is a secret shared between the LFI and the API Hub. It is a basic form of access token: the API Hub sends the key with each call, and the LFI checks it.

This option applies only to Ozone Connect, that is, calls from the API Hub to the LFI. It is not available for the Consent Management API or the Authorization Server APIs.

The proposed key validity is 12 months or more, with the key rotated once a year. The rotation process and notice period are [TBC].

Client Credentials Grant

Status: Proposed.

The API Hub obtains an access token from an authorization server run by the LFI, using the OpenID Connect client credentials grant. The API Hub acts as a relying party to that server and presents the token on each call.

This option gives a reasonable level of security, which is strengthened if the LFI's authorization server follows a FAPI profile. The drawbacks are that the LFI has to run its own authorization server, and each token request adds a small performance cost.

This option applies only to Ozone Connect. It is not available for the Consent Management API or the Authorization Server APIs.

The proposed validity of any `client_secret` is 12 months or more, with annual rotation. The `scope` values are agreed during technical onboarding.

JWT Auth

Status: Proposed.

JWT Auth is an Ozone mechanism. The caller signs a token with its private key, and the receiver verifies the signature with the caller's public key. It works in both directions.

The main points are:

- Tokens are signed with PS256, an asymmetric algorithm, so no secrets are shared between the LFI and the API Hub.
- No extra infrastructure is needed, such as an authorization server run by the LFI.

- It uses standard formats: JSON Web Signature (JWS) for the token and JSON Web Key Sets (JWKS) for the public keys.
- The Ozone Trust Framework issues and manages signing keys and JWKS for participants. Using these keys for JWT Auth is Proposed.
- The sender controls key rotation and how long each token is valid.
- Claims in the token bind it to the mTLS certificate in use, by checking certificate attributes such as the organizational unit (`OU`) and distinguished name (`DN`). The exact claims are [TBC].
- It is not a published industry standard, so the LFI has to build its own logic to create and validate tokens.

JWT Auth is proposed for Ozone Connect, the Consent Management API and the Authorization Server APIs. JWT Authorization is the detailed implementation guide.

Service Initiation Token

Status: Proposed.

A service initiation token is an extra mechanism for consents that initiate a service, such as payments. The LFI sets an access token on each such consent, and the API Hub sends that token in the `authorization` header of the related calls to the LFI.

The LFI can rotate the token at any time by updating the consent through the Consent Management API.

This option applies only to service initiation calls to Ozone Connect. It can be combined with any of the options above, and overrides them on those calls.

JWT Authorization

This page shows how an LFI builds and verifies the signed JSON Web Token (JWT) for JWT Auth between the LFI's systems and the QCB Open Finance Application Programming Interface (API) Hub. JWT Auth is one of the optional application layer authentication options described on Application Layer Authentication. The LFI must use mutual Transport Layer Security (mTLS) in every case.

Introduction

JWT Auth works in both directions. The sender signs a JWT and puts the token in the `authorization` header. The receiver checks the signature and the claims before processing the request.

The `iss` and `sub` claims tie each token to the sender's client certificate. Both claims must match the subject of the certificate the sender presents at the Transport Layer Security (TLS) handshake. Requirements for the LFI as the Client covers the LFI as the sender. Requirements for the LFI as the Server covers the LFI as the receiver. JWT Auth Claims Reference lists the claims both sides use.

Requirements for the LFI as the Client (Sending a Token)

Follow these steps when the LFI calls the API Hub's Consent Management API or Authorization Server APIs.

Publication of Signing Keys on a JSON Web Key Set (JWKS)

1. In the LFI's organization in the Ozone Trust Framework, the LFI's Primary Technical Contact (PTC) registers an application for its calls to the API Hub and links the application to a key store. The key store sets the application's JWKS Uniform Resource Locator (URL).
2. In that application, the LFI creates a transport certificate for mTLS and a signing key for the JWT (C3 and Sig4 on Keys, CSRs and Certificates). The LFI generates the keys and certificate signing requests (CSRs) with its own tooling, following the approved certificate authority's (CA's) requirements. The approved CA signs the certificates, and the LFI uploads the signed certificates to the key store.

3. Each key has a key identifier (`kid`). The Ozone Trust Framework publishes the `kid` with the public key on the application's JWKS.
4. After the LFI publishes a new key, the LFI must wait at least 10 minutes before signing a token with the key. Receivers cache the JWKS for up to 10 minutes, and the wait lets their caches refresh.

The LFI sets up a key store in each Ozone Trust Framework environment and sends the JWKS URL and `kid` to Ozone through the service desk.

Creating the Authorization Header

1. Synchronize the clock of the signing system with a Network Time Protocol (NTP) time source.
2. Build the JWT header and body with the claims in JWT Auth Claims Reference.
3. Sign the JWT with `PS256`, using the private key of a signing key published as described in Publication of Signing Keys.
4. Send the JWT as a bearer token in the `authorization` Hypertext Transfer Protocol (HTTP) header.
5. Send the request over mTLS with the transport certificate from Publication of Signing Keys. Set `iss` to the Organization (O) value and `sub` to the Organizational Unit (OU) value from the certificate's subject.

Template (decoded header and body):

```
{
  "header": {
    "alg": "PS256",
    "kid": "{kid of the signing key}",
    "typ": "JOSE",
    "cty": "json"
  },
  "payload": {
    "iss": "{O from the transport certificate subject}",
    "sub": "{OU from the transport certificate subject}",
    "aud": "{identifier of the receiving party}",
    "exp": "{expiry, UTC seconds since epoch}",
    "iat": "{issued at, UTC seconds since epoch}",
    "nbf": "{not before, UTC seconds since epoch (optional)}",
    "jti": "{unique token ID}"
  }
}
```

In the signed token, `exp`, `iat` and `nbf` are numbers, not strings.

Requirements for the LFI as the Server (Receiving a Token)

Follow these steps when the API Hub calls the LFI's Ozone Connect endpoints.

Verifying the Authorization Header

1. Synchronize the clock of the verifying system with an NTP time source.
2. Confirm the request arrived over mTLS with a client certificate issued by the approved CA.
3. Extract the JWT from the `authorization` header.
4. Read the `kid` from the JSON Web Signature (JWS) header.
5. Find the public key with the same `kid` on the API Hub's JWKS.
    1. The Ozone Trust Framework publishes each JWKS at a URL built from the organization identifier (ID) and application ID of its owner.
    2. Ozone provides the API Hub's JWKS URL for each environment through the service desk.

3. The Ozone Trust Framework also publishes a revoked-keys list for each JWKS. Recommended: check the list before trusting a cached key.

6. Verify the signature with that public key.
7. Cache the JWKS for no more than 10 minutes.
8. Check every claim against the expected values in JWT Auth Claims Reference. If any check fails, reject the request. The error response returned to the API Hub is [TBC].

JWT Auth Claims Reference

Header

Claim	Expected value	Mandatory	Notes
`alg` (algorithm)	`PS256`	Yes	
`typ` (type)	`JOSE` (JSON Object Signing and Encryption)	Yes	
`cty` (content type)	`json` (JavaScript Object Notation)	Yes	
`kid` (key identifier)	Key identifier of the signing key, as published on the sender's JWKS	Yes	Other key identifiers (`x5c` X.509 certificate chain, `x5u` X.509 URL) are not supported

Body

Claim	Expected value	Mandatory	Notes
`iss` (issuer)	O value from the subject of the sender's transport certificate	Yes	The receiver compares `iss` with the O value of the client certificate on the same connection
`sub` (subject)	OU value from the subject of the	Yes	

Claim	Expected value	Mandatory	Notes
	sender's transport certificate		
`aud` (audience)	Identifier of the receiving party	Yes	Ozone assigns the LFI a provider ID and sends the ID through the service desk. The `aud` value for tokens the LFI sends to the API Hub is [TBC].
`exp` (expiration time)	Expiry time, Coordinated Universal Time (UTC) seconds since epoch	Yes	Recommended token lifetime: 10–30 seconds. Reject the token if the current time is more than 10 seconds after this value.
`iat` (issued at)	Issue time, UTC seconds since epoch	Yes	Reject the token if the current time is more than 10 seconds before this value.
`nbf` (not before)	Not-before time, UTC seconds since epoch	No	When present, reject the token if the current time is more than 10 seconds before this value.
`jti` (JWT ID)	Unique identifier of the token	Yes	Universally Unique Identifier (UUID) version 4 recommended

Ozone Connect | Overview

> **Usage**
>
> - This page provides information on implementing Ozone Connect at LFIs.
> - It provides guidance on implementing the APIs, and especially on error handling in responses to the API Hub.

Introduction

Ozone Connect is an API description provided by Ozone that facilitates integration between the API Hub and LFIs. In practical terms, implementing Ozone Connect means implementing the APIs it defines and ensuring that the API Hub can invoke them successfully:

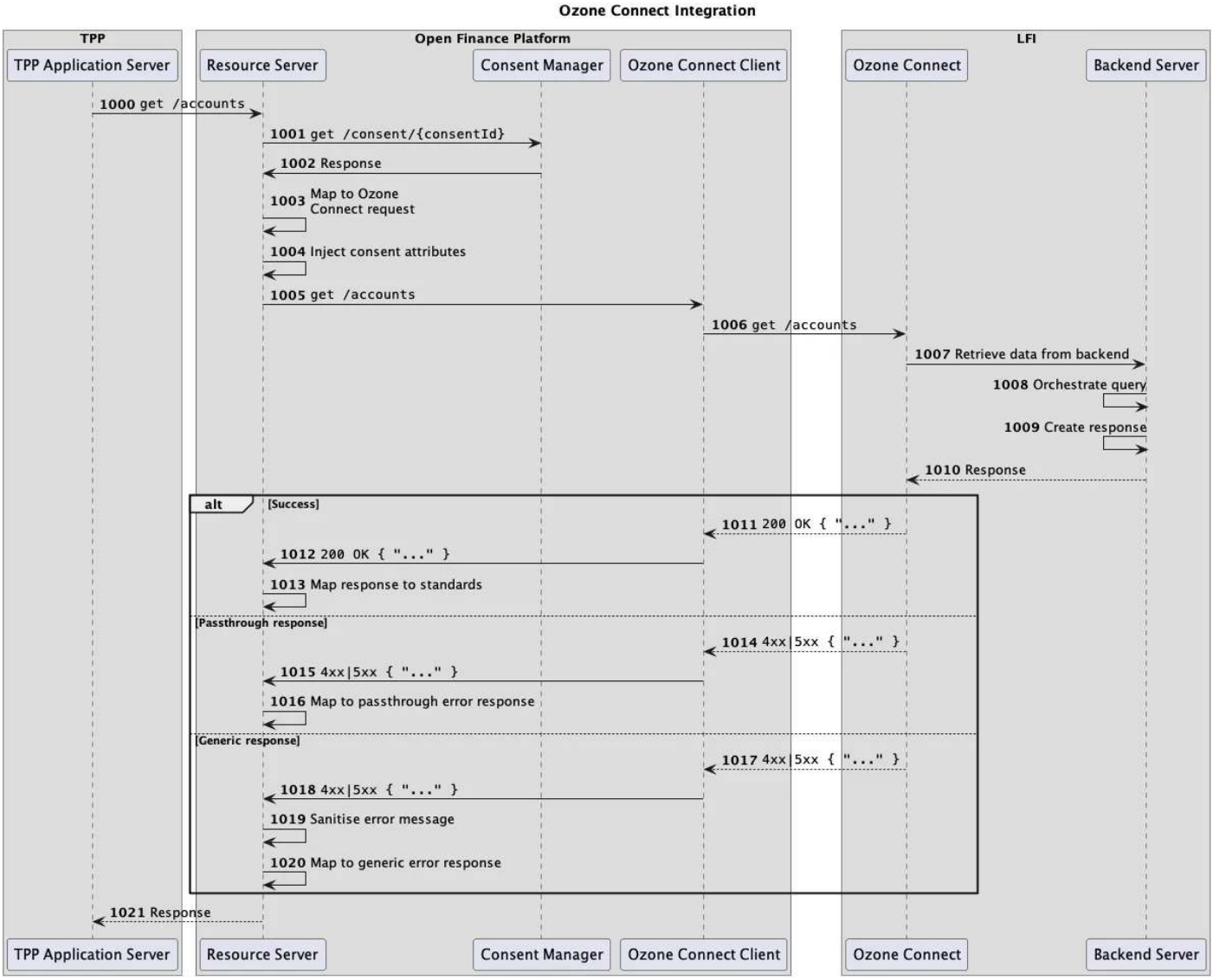
- Bank Data Sharing
- Bank Service Initiation
- Consent Events & Actions
- Health Check

This page provides guidance on the implementation of Ozone Connect from the perspective of ensuring robust and consistent interaction between the API Hub and an LFI.

Context

The integration between the API Hub and Ozone Connect is implemented to provide a consistent mechanism for requesting data or initiating services at a given LFI.

The diagram below puts this in the context of a sequence diagram, showing the interaction between the API Hub and Ozone Connect implemented at the LFI.



To annotate the diagram:

- The TPP makes a call to the Resource Server to request a given resource.
- The consent associated with the Access Token is checked (not shown) and then used to inject parameters that will be used in the call to the Ozone Connect API.
- The API call is made, which is serviced by the LFI based on their implementation.
- The LFI creates the appropriate success or failure response and returns this to the API Hub.
- The API Hub then prepares the appropriate response based on the HTTP status code and the content of the response message.

The high-level flows described above follow accepted paradigms for APIs, save for several core principles that guide how Ozone Connect is used and provide an effective mechanism for communication between the API Hub and Ozone Connect.

The design principles for Ozone Connect are outlined below.

Principle	Rationale	Notes
The *modus operandi* is a successful response.	Provides stability in implementation. Reduces edge cases and error conditions.	Ozone Connect is designed to facilitate consistent success conditions being returned by the implementation. Errors returned by the implementation are therefore only standardised in certain cases, as described below.
Success response payloads reuse definitions from the Standards.	Supports consistency in implementation. Provides a consistent source of truth.	The best approach to supporting Standards adoption is to reuse them. Ozone Connect therefore reuses Standards payloads, overlaying them with request handling as described below to help facilitate a consistent, successful response.
HTTP status codes are the primary indicator for error conditions.	Allows coarse-grained error handling for the majority of errors.	The API Hub will interpret most responses based on the HTTP status code, and introspect the response to relay the correct information back to the TPP. Having a representative HTTP status code is therefore important to allow this to happen correctly.
Error codes are prescribed in exceptional cases.	Allows implementers to use Ozone tooling for testing purposes. Allows reuse of existing error codes for implementers. Avoids close coupling across implementations by different implementers and versions.	Ozone only mandates error codes in exceptional cases as, for the most part, a stable, well-tested connection is expected and error handling is not prescribed. Mandated error codes are described in the Passthrough Errors section below.

The following sections provide further detail on request handling, success responses and error responses.

Request Handling

Parameters

Ozone Connect provides several parameters that give the LFI information about the consent and customer to which the API call relates.

Please refer to the OpenAPI description for each API for details on the parameters supported for each operation.

Resource Identifiers

Resource identifiers are used to retrieve a specific resource. For example, to retrieve an account with the resource identifier `a1f1b06b-8495-4709-95ca-806eac2c95f2` a TPP would call `GET /accounts/{accountId}`.

All resource identifiers are defined by the LFI and returned from Ozone Connect. These are passed through to the TPP **as-is**, without modification or remapping at the API Hub.

Please refer to this knowledge base article on guidance for creating resource identifiers: Using a unique, surrogate identifier instead of a domestic account number or IBAN in APIs <TBC>.

The guidance therein can be generally applied to resource identifiers, especially where the “true” identifier for a given resource is sensitive data that should not be used in a URL.

Please refer to the OpenAPI description for each API for details on the resource identifiers supported for each operation. Resource identifiers are defined as a Parameter Object in the OpenAPI Specification, with `in` set to `path`.

Success Responses

Success responses are described by Ozone Connect, which prescribes what data must be returned by the LFI.

Ozone Connect prescribes the shape of a success response through an OpenAPI Response Object. This comprises the following information:

- The HTTP status code that indicates success (typically, but not exclusively, `200 OK`).
- The media type in which the data must be encoded (typically `application/json`).
- The format of the response, described using a Schema Object.

A separate Response Object is provided for each operation.

LFIs should observe the following guidelines when implementing Ozone Connect.

Guideline	Rationale
Ensure you return all the data visible to your customer in your online channels.	Provides a consistent user experience for consumers across the open finance ecosystem.
Return all other available data in the response to an API operation.	Provides the opportunity for an enriched experience for consumers.
Observe the required HTTP status codes described in the OpenAPI description documents.	Prevents unwarranted failure scenarios being experienced at the API Hub.

Error Responses

Errors are defined in three ways:

- Integration Test Errors: errors that LFIs can use to support integration testing in pre-production.
- Passthrough Errors: errors that **must** be supported by the LFI to ensure successful integration between the API Hub and the LFI.
- Generic Errors: errors that are returned by the LFI and are not prescribed in the Ozone Connect API descriptions.

These are expanded upon in the sections below.

Integration Test Errors

Integration Test Errors are errors that are prescribed by Ozone and can be supported to help with pre-production integration testing of Ozone Connect.

Implementation of Integration Test Errors is optional. Implementers of Ozone Connect must refer to the Testing Tool User Guide <TBC> for the test case error codes, which are detailed under each HTTP response code section.

Passthrough Errors

> ⚠ **Important**
>
> The Standards prescribe error code definitions to encourage uniformity across the ecosystem, by adhering to the existing codes of the domestic payment systems (see Namespaced Error Codes).

Passthrough Errors are errors that are prescribed by either the Standards or by Ozone, and must be supported to ensure the integration between the API Hub and Ozone Connect is correctly implemented.

Passthrough Errors are a composite of:

- The HTTP status code.
- The value of the `errorCode` property in the response payload.

Passthrough Errors are used to drive a specific response behaviour when the response is played back to a client.

LFIs implementing Ozone Connect must implement the Passthrough Errors below to support successful integration with the API Hub.

The `errorMessage` that is provided will be passed on to the TPP, who may subsequently display the error to end users. LFIs must ensure that the error description contains a message that is meaningful to the end user.

Standardized Error Codes

The Ozone Connect API descriptions list the `errorCode` values that are supported. Each `errorCode` value is used in one or more error responses to an API operation.

For clarity, the full list of `errorCode` values is given below. Please refer to the API descriptions as the source of truth for guidance on usage across HTTP status codes.

Error Code	Description	Usage	Pass to TI
`AccessToken.InvalidScope`	The Access Token presented to	Can be used where the preferred API	No

Error Code	Description	Usage	Pass to TI
	the Resource Server has not been granted with the correct scope for the requested resource.	Hub integration pattern is using an OAuth 2.0 Client Credentials grant type. Ignore if this integration pattern is not adopted.	
`Body.InvalidFormat`	The request contains an invalid request body and therefore cannot be parsed.	Can be used where an invalid request is sent from the API Hub.	No
`Consent.AccountTemporarilyBlocked`	The account being accessed to serve a given request is temporarily blocked due to a business rule or BAU process at the LFI.	Can be used to indicate that an API operation cannot be completed, but the condition of the account is transient and access may be restored on retry.	Yes
`Consent.Invalid`	The consent is invalid.	*This is unlikely to be implemented by the LFI, as*	No

Error Code	Description	Usage	Pass to TI
		consent validation is handled by the API Hub, but is included for completeness.	
`Consent.BusinessRuleViolation`	A business rule has been violated at the LFI and the consent operation therefore cannot be performed.	Can be used to indicate where a given action violates a business rule at the LFI, most likely where an LFI applies a BAU process and this blocks the requested operation.	Yes
`Consent.FailsControlParameters`	The requested operation has failed a control parameter specified in the consent.	*This is unlikely to be implemented by the LFI, as control parameters are handled by the API Hub, but is included for completeness.* *This is also referenced below in the [Specific API*	No

Error Code	Description	Usage	Pass to TI
		Operations section, for use with the `validate` operation of the Consent Events & Actions API.	
`Consent.InvalidUserIdentifier`	The user identifier presented by the API Hub does not match the user identifier recorded by the LFI.	Can be used to indicate a mismatch in the value of `psuIdentifier`, but this may also require corrective action on the part of the LFI to patch the related consent with the correct value.	No
`Consent.PermanentAccountAccessFailure`	The account for which consent has been requested is permanently not available.	Can be used to indicate where a consented account has, for example, been closed and therefore API operations can no longer be invoked. *This is also referenced*	Yes

Error Code	Description	Usage	Pass to TI
		below in the Specific API Operations section, for use with the `validate` operation of the Consent Events & Actions API.	
`Consent.TransientAccountAccessFailure`	The account for which consent has been requested is not available due to a transient failure, such as an internal communications issue.	Can be used to indicate where an API operation cannot currently be completed successfully, but a retry may be successful. *This is also referenced below in the Specific API Operations section.*	Yes
`Event.UnexpectedEvent`	An event notification was not expected.	Can be used to indicate unexpected events at the Consent Events & Actions API. Please note that returning this	N/A

Error Code	Description	Usage	Pass to TI
		error **will not** result in a retry.	
`GenericRecoverableError`	An error from which recovery is possible.	Can be used to indicate errors where a retry may be successful.	Yes
`GenericError`	A generic error.	Catch-all error code, with no indication of reason, retry or error handling for the client.	Yes
`JWE.DecryptionError`	A generic JSON Web Encryption (JWE) decryption failure.	Can be used to indicate where decryption of a JWE, used for Personally Identifiable Information (PII), failed due to a general error.	Yes
`JWE.InvalidHeader`	Validation of the header of a JWE failed.	Can be used for more specific error messaging for JWE decryption failures where the header is invalid.	Yes
`JWS.InvalidSignature`	Validation of the signature of a	Can be used for more specific	Yes

Error Code	Description	Usage	Pass to TI
	JWS failed.	error messaging for JWS verification failures where the signature is invalid.	
`JWS.Malformed`	A generic JSON Web Signature (JWS) verification failure.	Can be used where the JWS has an invalid structure that cannot be parsed in order to be verified, especially where JWE decryption has been successful but the encapsulated JWS is malformed.	Yes
`JWS.InvalidClaim`	Validation of one or more claims in a JWS failed.	Can be used to indicate invalid JWS claims.	Yes

Error Code	Description	Usage	Pass to TI
`JWS.InvalidHeader`	Validation of the header of a JWS failed.	Can be used for more specific error messaging for JWS verification failures where the header is invalid.	Yes
`Resource.InvalidFormat`	The expected format of a given resource is invalid.	*This is unlikely to be implemented by the LFI, as the control flow for Ozone Connect does not reflect its use, but is included for completeness.*	No

Namespaced Error Codes

Namespaced Error Codes describe “families” of error codes. Namespaced Error Codes must conform to the pattern prescribed by the relevant API description, and comprise two parts, based on the example `FAWRAN.AM04`, where:

- `FAWRAN` represents the family.
- `AM04` is the error code (in this example, insufficient funds).

Namespaced Error Codes allow error codes to be extended to other platforms, or for codes already prescribed in Qatar to be reused.

Fawran codes are **prescribed** by the Standards and are used to describe error scenarios across Qatar Open Finance. In the given example, where there are insufficient funds in the debtor account, the Fawran code is used to indicate this condition, regardless of whether the payment instruction has been dispatched to Fawran.

Using Fawran error codes as the source of truth in this regard has been prescribed in the Standards to ensure consistency across both open finance and existing payment journeys. However, where an error code has been returned from a payment system, for example Fawran or QATCH, the error code is replayed **exactly** as returned, with no requirement to transpose the error to another Fawran code. In these examples the namespace values `FAWRAN` and `QATCH` can be used as applicable.

LFIs also have the option of implementing their own Namespaced Error Codes, using the prescribed pattern, and are encouraged to do so to provide consistency in the ecosystem. However, LFIs can choose to use the Standardized Error Codes described above.

Please refer to the Fawran Implementation Guide and the QATCH Advanced Link Interface Specifications for the correct error code values.

Specific API Operations

The table below gives guidance that is specific to given operations implemented at Ozone Connect.

> ⚠ **Important**
>
> Please use the API descriptions as the source of truth for the correct HTTP status code and `errorCode` value.

Group	Operation	HTTP Status Code	Error Code
Bank Service Initiation	`POST /payments`	400	`Consent.AccountTemporarilyBl`

Group	Operation	HTTP Status Code	Error Code
Bank Service Initiation	`POST /payments`	400	`Consent.PermanentAccountAcce`
Bank Service Initiation	`POST /payments`	400	`Consent.TransientAccountAcce`
Bank Service Initiation	`POST /payments`	400	`Consent.BusinessRuleViolatic`

Group	Operation	HTTP Status Code	Error Code
Bank Data Sharing	All (other than `GET /accounts`)	400	`Consent.AccountTemporarilyB`
Bank Data Sharing	All (other than `GET /accounts`)	400	`Consent.PermanentAccountAcce`

Group	Operation	HTTP Status Code	Error Code
Bank Data Sharing	All (other than `GET /accounts`)	400	`Consent.TransientAccountAcce`
Consent Events & Actions	`POST /consent/action/validate`	See description	See description

Group	Operation	HTTP Status Code	Error Code

Group	Operation	HTTP Status Code	Error Code

Generic Errors

Generic Errors are errors that are **not** prescribed by the Standards or by Ozone.

Like Passthrough Errors, Generic Errors are a combination of HTTP status code and `errorCode` value. The API Hub will, however, use the `GenericError` label in its response to TPPs and **will not** attempt to transpose any error information in the response. In most cases the API Hub will return a 5xx error, indicating a downstream error to the TPP.

> ⚠ **Important**
>
> The behaviours in this section will be observed if the guidance in the Passthrough Errors section is not adhered to.

LFIs should observe the following guidelines when implementing Generic Errors.

Guideline	Rationale	Notes
Use the `default` Response Object to describe your error response payloads consistently.	The API Hub will readily parse this format, simplifying logging, auditing and debugging.	Please refer to the Ozone Connect API descriptions, which implement the OpenAPI `default` response to indicate the expected format for anything that is not a 2xx response.
Use consistent and deterministic `errorCode` values for failures.	Consistent and deterministic `errorCode` values will simplify debugging error cases.	You should ideally adhere to the guidance set out in the Passthrough Errors section above, although the API Hub is designed to be tolerant of error messages that do not comply exactly with the requirements

Guideline	Rationale	Notes
	Providing consistent error codes will result in better management of the LFI instance on the API Hub.	prescribed in the Ozone Connect API descriptions.
Observe the relevant HTTP status codes for the error condition.	The API Hub will be able to transpose the error in the response to the TPP more effectively.	As with the guidance on success responses, sending a 2xx HTTP status code with the `Error` response will be misleading and cause an incorrect response to the TPP from the API Hub.

Data and Service Mapping | Overview

An LFI returns data to the API Hub in the shape defined by the Ozone Connect API descriptions. The API Hub maps those properties to the properties of the Standards API responses before returning them to the TPP.

The Data and Service Mapping pages show that mapping, property by property, so an LFI can see which Standards property each Ozone Connect property populates, and which properties are mandatory.

Mappings are grouped by Ozone Connect API:

- Account Information Mapping: the mapping between the Bank Data Sharing API and the Account Information API responses.

For payments, Payment PII sets out when an LFI receives the Debtor and Creditor properties of the Personal Identifiable Information in a payment consent and in a payment instruction, for each supported payment type.

Payment Personal Identifiable Information

Introduction

This page provides guidance on how Personal Identifiable Information (PII) is used in the context of different payment journeys.

> **Usage**
>
> - General guidance is provided in this page that sets expectations for LFIs as to when they will receive specific property values.
> - The information **does not** replace any element of the Standards or absolve participants from understanding their obligations in implementing different aspects of the supported payment journeys.

Operations

The Standards currently implement PII in two operations:

- **Creating a payment consent:** Creditor information is supplied through a Rich Authorization Request (RAR), carried in the `authorization_details` of the Pushed Authorization Request (PAR), to bind a consent to either a single payment or to multiple payments under one long-lived consent. The number of Creditors depends on the scenario:
  - For a Single Instant Payment: exactly one Creditor.
  - For a Variable On-Demand payment: either one Creditor, or a defined list of between two and ten Creditors (see Limits).
- **Initiating a payment:** the Creditor of the payment is supplied in the payment initiation request, and must agree with the Creditor information in the consent.

These scenarios are expanded upon in the following section.

Scenarios

The table below provides guidance on the various scenarios that are supported by the Standards.

> **Note**
>
> The properties listed below must be read in conjunction with the following Schema Objects, found in the Consent Manager API description:
>
> - **Payment Consent:** `PaymentInitiation.DomesticPaymentPersonalIdentifiableInformation1Properties`. This Schema Object is found in the Authorization Endpoints API description, which defines the PAR, in the Standards.
> - **Payment Initiation:** `PaymentInitiation.DomesticPaymentInitiationPersonalIdentifiableInformation1Properties`. This Schema Object is found in the Bank Service Initiation API description.
>
> The two Schema Objects above are the source of truth for how the guidelines listed below are interpreted.

In a payment consent, `Initiation.Creditor` is an array, and each of its entries holds the `Creditor`, `CreditorAccount` and `CreditorAgent` of one Creditor. In a payment instruction, `Initiation` holds those three properties for the Creditor of that payment. The `DebtorAccount` is `Initiation.DebtorAccount` in a payment consent.

Instance	Operation	`DebtorAccount` sent by TPP	`DebtorAccount` value	`Creditor` sent by TPP	`Creditor` value
Single Instant Payment Consent	`post /par`	Conditional	When selected by the User at the TPP. If not supplied, the User must select the account at the LFI.	Yes	Single value. Name is required by business rules.
Single Instant Payment Instruction	`post /payments`	No	Not supported in the payment initiation message.	Yes	Must match the value from

Instance	Operation	`DebtorAccount` sent by TPP	`DebtorAccount` value	`Creditor` sent by TPP	`Creditor` value
					the consent.
Variable On-Demand Consent	`post /par`	Conditional	When selected by the User at the TPP. If not supplied, the User must select the account at the LFI.	Yes	Single value.
Variable On-Demand Payment Instruction	`post /payments`	No	Not supported in the payment initiation message.	Yes	Single value. Must match the value from the consent.
Variable On-Demand with Multi-Creditors Consent	`post /par`	Conditional	When selected by the User at the TPP. If not supplied, the User must select the account at the LFI.	Yes	Two to ten values. Defines an authorized list of Creditors.
Variable On-Demand with Multi-	`post /payments`	No	Not supported in the payment initiation message.	Yes	Single value. Must match one

Instance	Operation	`DebtorAccount` sent by TPP	`DebtorAccount` value	`Creditor` sent by TPP	`Creditor` value
Creditors Payment Instruction					value from the consent.

Account Information Mapping | Overview

Context

Bank Data Sharing uses a generic, canonical model for account data that is provided in the Ozone Connect API description.

The properties of the Standards APIs are mapped to Ozone Connect by the API Hub.

Purpose

The purpose of this page is to provide information on the mapping of response payload properties between what is defined in Standards and the Ozone Connect APIs.

Each mapping is provided by API resources in the following pages:

- Accounts
- Balances
- Transactions

The mapping tables within each page indicate whether a given property is mandatory or becomes mandatory when an optional object is returned. While a data field may not be marked as mandatory in the table, if the data is available within the LFI's systems and accessible to the User, it must be returned, subject to the data permissions and the consent of the User as defined in a given Bank Data Sharing consent.

Field paths in the mapping tables are written as JSONPath expressions, rooted at the response body (`$`). `[*]` selects every element of an array, so `$.data[*].id` is the `id` of each object in the `data` array.

Permission Codes

All mapping of data is based on permission codes. Permission codes are agreed between the User and the TPP and describe what data is to be shared.

> The full list of permission codes is available in the Authorization Endpoints API description.

The API Hub acts on the permission codes when a given API operation is invoked, for example `get /accounts/{accountId}/balances`, and performs introspection of the Access Token

again the consent to which it is bound.

The following actions are taken:

1. If the requested API operation is permitted through an appropriate permission code, the request is rejected.
    - For example, to invoke the `get /accounts/{AccountId}/balances` API operation the `ReadBalances` permission code must be granted.
2. If the permission code has been granted the API Hub calls the correct API operation at the Ozone Connect API for the LFI.
    - For example, when `get /accounts/{AccountId}/balances` is called at the API Hub the API Hub maps the operation to `get /accounts/{accountId}/balances` at the Ozone Connect API for the LFI.
3. The LFI will return a response with all available properties based on the Ozone Connect API description.
4. The API Hub will then ***remove*** any properties that are not permitted by the granted permission code.
    - For example, if `get /accounts/{accountId}` is invoked but only `ReadAccountsBasic` is granted by the User all properties relating to `ReadAccountsDetail` will be removed.
    - Permissions are always applied from the top-most property in the JSON payload i.e. if a parent property is excluded based on the permission code granted then all child properties are also removed.

For the avoidance of doubt: LFIs **do not** have to act on the permission codes granted by the User to the TPP, as the API Hub will undertake this action automatically when handling the Ozone Connect response payload.

Accounts
API Operations & Permissions

The Open Finance Framework standards support the following 2 account API operations:

- `get /accounts`
- `get /accounts/{AccountId}`

To access these two API operations the User must have granted one of the following data permissions with the consent:

- `ReadAccountsBasic`
- `ReadAccountsDetail`

API Hub to Standards Response Mapping

Standards API Operation	API Hub API Operation
`get /accounts`	`get /accounts`
`get /accounts/{AccountId}`	`get /accounts/{accountId}`

> **Note**
> The same API Hub data mapping as detailed below is utilised for both Accounts API operations.

Property ID	Standards Property Name	API Hub Property Path	Standards Required	API Requ
	`$.Data`	`$.data[*]`	Yes	Ye
	`$.Data.Account`		Yes	
A01	`$.Data.Account[*].AccountId`	`$.data[*].id`	Yes	Ye
A02		`$.data[*].product`		

Property ID	Standards Property Name	API Hub Property Path	Standards Required	API Requ
A02.01		$.data[*].product.id		
A02.02		$.data[*].product.productname		
A02.03		$.data[*].product.bundleName		
A03		$.data[*].multiAuth		
A04		$.data[*].businessCustomer		
A04.01		$.data[*].businessCustomer.id		Ye
A04.02		$.data[*].businessCustomer.		Ye

Property ID	Standards Property Name	API Hub Property Path	Standards Required	API Requ
		customerName		
A05		$.data[*].customers		
A05.01		$.data[*].customers[*].id		Ye
A05.02		$.data[*].customers[*].customerName		Ye
A06	$.Data.Account[*].AccountHolderName	$.data[*].accountHolderName		
A07	$.Data.Account[*].AccountHolderShortName	$.data[*].accountHolderShortName		
A08	$.Data.Account[*].Status	$.data[*].status		
A09	$.Data.Account[*].StatusUpdateDateTime	$.data[*].statusUpdateDateTime		
A10	$.Data.Account[*].Currency	$.data[*].currency		
A11	$.Data.Account[*].AccountType	$.data[*].accountType		
A12	$.Data.Account[*].AccountSubType	$.data[*].accountSubType		
A13	$.Data.Account[*].Description	$.data[*].description		
A14	$.Data.Account[*].Nickname	$.data[*].nickName		

Property ID	Standards Property Name	API Hub Property Path	Standards Required	API Requ
A15	$.Data.Account[*].OpeningDate	$.data[*].openingDate		
A16	$.Data.Account[*].MaturityDate	$.data[*].maturityDate		
A17	$.Data.Account[*].AccountIdentifiers	$.data[*].accountNumbers		
A17.01	$.Data.Account[*].AccountIdentifiers[*].SchemeName	$.data[*].accountNumbers[*].schemeName	Yes	Ye
A17.02	$.Data.Account[*].AccountIdentifiers[*].Identification	$.data[*].accountNumbers[*].identification	Yes	Ye
A17.03	$.Data.Account[*].AccountIdentifiers[*].Name	$.data[*].accountNumbers[*].name		
A18	$.Data.Account[*].Servicer	$.data[*].servicer		
A18.01	$.Data.Account[*].Servicer.SchemeName	$.data[*].servicer.schemeName	Yes	Ye
A18.02	$.Data.Account[*].Servicer.Identification	$.data[*].servicer.identification	Yes	Ye

Property ID	Standards Property Name	API Hub Property Path	Standards Required	API Requ
A19	`$.Data.Account[*].IsShariaCompliant`	`$.data[*].isShariaCompliant`		

Data Permissions: Basic vs Detail

Based on the permission code granted in Consent the following Account information is provided to the TPP.

> **Note**
>
> Summary: The following properties are not provided in the response payload when `ReadAccountsBasic` data permission has been granted:
>
> - `AccountIdentifiers`
> - `Servicer`

Property ID	Standards Property Name	ReadAccountsBasic	ReadAccountsDetail
	`$.Data`		
	`$.Data.Account`	Yes	Yes
A01	`$.Data.Account[*].AccountId`	Yes	Yes
A06	`$.Data.Account[*].AccountHolderName`	Yes	Yes
A07	`$.Data.Account[*].AccountHolderShortName`	Yes	Yes
A08	`$.Data.Account[*].Status`	Yes	Yes
A09	`$.Data.Account[*].StatusUpdateDateTime`	Yes	Yes

Property ID	Standards Property Name	ReadAccountsBasic	ReadAccountsDetail
A10	$.Data.Account[*].Currency	Yes	Yes
A11	$.Data.Account[*].AccountType	Yes	Yes
A12	$.Data.Account[*].AccountSubType	Yes	Yes
A13	$.Data.Account[*].Description	Yes	Yes
A14	$.Data.Account[*].Nickname	Yes	Yes
A15	$.Data.Account[*].OpeningDate	Yes	Yes
A16	$.Data.Account[*].MaturityDate	Yes	Yes
A17	$.Data.Account[*].AccountIdentifiers	No	Yes
A17.01	$.Data.Account[*].AccountIdentifiers[*].SchemeName	No	Yes
A17.02	$.Data.Account[*].AccountIdentifiers[*].Identification	No	Yes
A17.03	$.Data.Account[*].AccountIdentifiers[*].Name	No	Yes
A18	$.Data.Account[*].Servicer	No	Yes
A18.01	$.Data.Account[*].Servicer.SchemeName	No	Yes

Property ID	Standards Property Name	ReadAccountsBasic	ReadAccountsDetail
A18.02	`$.Data.Account[*].Servicer.Identification`	No	Yes
A19	`$.Data.Account[*].IsShariaCompliant`	Yes	Yes

Balances
API Operations & Permissions

The Open Finance Framework standards support the following Balance API operation:

- `get /accounts/{AccountId}/balances`

To access this API operation the User must have granted the following consent permission:

- `ReadBalances`

API Hub to Standards Response Data Mapping

Standards API Operation	API Hub API Operation
`get /accounts/{AccountId}/balances`	`get /accounts/{accountId}/balances`

Property ID	Standards Property Name	API Hub Property Name	Standards Mandatory	API Hu Mandat
	`$.Data`	`$.data[*]`	Yes	Yes
B01	`$.Data.AccountId`	`$.data[*].accountId`	Yes	Yes
	`$.Data.Balance`		Yes	
B02	`$.Data.Balance[*].CreditDebitIndicator`	`$.data[*].creditDebitIndicator`	Yes	Yes
B03	`$.Data.Balance[*].Type`	`$.data[*].balanceType`	Yes	Yes
B04	`$.Data.Balance[*].DateTime`	`$.data[*].timestamp`	Yes	Yes
B05	`$.Data.Balance[*].Amount`	`$.data[*].amount`	Yes	Yes
B05.01	`$.Data.Balance[*].Amount.Amount`	`$.data[*].amount.amount`	Yes	Yes

Property ID	Standards Property Name	API Hub Property Name	Standards Mandatory	API Hu Mandat
B05.02	$.Data.Balance[*].Amount.Currency	$.data[*].amount.currency	Yes	Yes
B06	$.Data.Balance[*].CreditLine	$.data[*].creditLines		
B06.01	$.Data.Balance[*].CreditLine[*].Included	$.data[*].creditLines[*].included	Yes	Yes
B06.02	$.Data.Balance[*].CreditLine[*].Type	$.data[*].creditLines[*].creditType	Yes	Yes
B06.03	$.Data.Balance[*].CreditLine[*].Amount	$.data[*].creditLines[*].amount	Yes	Yes
B06.03.01	$.Data.Balance[*].CreditLine[*].Amount.Amount	$.data[*].creditLines[*].amount.amount	Yes	Yes

Property ID	Standards Property Name	API Hub Property Name	Standards Mandatory	API Hu Mandat
B06.03.02	`$.Data.Balance[*].CreditLine[*].Amount.Currency`	`$.data[*].creditLines[*].amount.currency`	Yes	Yes
B07	`$.Data.Balance[*].Components`	`$.data[*].components`		
B07.01	`$.Data.Balance[*].Components.BalanceCategory`	`$.data[*].components.BalanceCategory`	Yes	Yes
B07.02	`$.Data.Balance[*].Components.Amount`	`$.data[*].components.Amount`	Yes	Yes
B07.02.01	`$.Data.Balance[*].Components.Amount.Amount`	`$.data[*].components.Amount.Amount`	Yes	Yes
B07.02.02	`$.Data.Balance[*].Components.Amount.Currency`	`$.data[*].components.Amount.Currency`	Yes	Yes
B07.03	`$.Data.Balance[*].Components.BalanceDescription`	`$.data[*].components.BalanceDescription`		

Transactions
API Operations & Permissions

The Open Finance Framework standards support the following transaction API operation:

- `get /accounts/{AccountId}/transactions`

To access this API operation the User must have granted one of the following consent permissions:

- `ReadTransactionsBasic`
- `ReadTransactionsDetail`

API Hub to Standards Response Data Mapping

Standards API Operation	API Hub API Operation
`get /accounts/{AccountId}/transactions`	`get /accounts/{accountId}/transactions`

Property #	Standards Property Name	API Hub Property Name
	`$.Data`	`$.data[*]`
T01	`$.Data.AccountId`	`$.data[*].accountId`
	`$.Data.Transaction`	
T02	`$.Data.Transaction[*].TransactionId`	`$.data[*].transactionId`
T03	`$.Data.Transaction[*].TransactionDateTime`	`$.data[*].transactionDateTime`
T04	`$.Data.Transaction[*].LocalTimeZone`	`$.data[*].localTimeZone`

Property #	Standards Property Name	API Hub Property Name
T05	$.Data.Transaction[*].StatementReference	$.data[*].statementReference
T06	$.Data.Transaction[*].TransactionReference	$.data[*].transactionReference
T07	$.Data.Transaction[*].TransactionType	$.data[*].transactionType
T08	$.Data.Transaction[*].SubTransactionType	$.data[*].subTransactionType
T09	$.Data.Transaction[*].TerminalId	$.data[*].terminalId
T10	$.Data.Transaction[*].Flags	$.data[*].flags
T11	$.Data.Transaction[*].PaymentModes	$.data[*].paymentModes
T12	$.Data.Transaction[*].CreditDebitIndicator	$.data[*].creditDebitIndicator
T13	$.Data.Transaction[*].Status	$.data[*].status
T14	$.Data.Transaction[*].TransactionMutability	$.data[*].transactionMutability
T15	$.Data.Transaction[*].BookingDateTime	$.data[*].bookingDateTime
T16	$.Data.Transaction[*].ValueDateTime	$.data[*].valueDateTime
T17	$.Data.Transaction[*].TransactionInformation	$.data[*].transactionInformation
T18	$.Data.Transaction[*].Amount	$.data[*].amount

Property #	Standards Property Name	API Hub Property Name
T18.01	$.Data.Transaction[*].Amount.Amount	$.data[*].amount.amount
T18.02	$.Data.Transaction[*].Amount.Currency	$.data[*].amount.currency
T19	$.Data.Transaction[*].ChargeAmount	$.data[*].chargeAmount
T19.01	$.Data.Transaction[*].ChargeAmount.Amount	$.data[*].chargeAmount.amount
T19.02	$.Data.Transaction[*].ChargeAmount.Currency	$.data[*].chargeAmount.currency
T19.03	$.Data.Transaction[*].ChargeAmount.ChargeIncluded	$.data[*].chargeAmount.chargeIncluded
T20	$.Data.Transaction[*].ChargeAmountVat	$.data[*].chargeAmountVat
T20.01	$.Data.Transaction[*].ChargeAmountVat.Amount	$.data[*].chargeAmountVat.amount
T20.02	$.Data.Transaction[*].ChargeAmountVat.Currency	$.data[*].chargeAmountVat.currency
T21	$.Data.Transaction[*].CurrencyExchange	$.data[*].currencyExchange
T21.01	$.Data.Transaction[*].CurrencyExchange.SourceCurrency	$.data[*].currencyExchange.sourceCurrency

Property #	Standards Property Name	API Hub Property Name
T21.02	$.Data.Transaction[*].CurrencyExchange.TargetCurrency	$.data[*].currencyExchange.targetCurrency
T21.03	$.Data.Transaction[*].CurrencyExchange.UnitCurrency	$.data[*].currencyExchange.unitCurrency
T21.04	$.Data.Transaction[*].CurrencyExchange.ExchangeRate	$.data[*].currencyExchange.exchangeRate
T21.05	$.Data.Transaction[*].CurrencyExchange.ContractIdentification	$.data[*].currencyExchange.contractIdentification
T21.06	$.Data.Transaction[*].CurrencyExchange.QuotationDate	$.data[*].currencyExchange.quotationDate
T21.07	$.Data.Transaction[*].CurrencyExchange.InstructedAmount	$.data[*].currencyExchange.instructedAmount
T21.07.01	$.Data.Transaction[*].CurrencyExchange.InstructedAmount.Amount	$.data[*].currencyExchange.instructedAmount.amount
T21.07.02	$.Data.Transaction[*].CurrencyExchange.InstructedAmount.Currency	$.data[*].currencyExchange.instructedAmount.currency
T22	$.Data.Transaction[*].BankTransactionCode	$.data[*].bankTransactionCode
T22.01	$.Data.Transaction[*].BankTransactionCode.Domain	$.data[*].bankTransactionCode.domain
T22.02	$.Data.Transaction[*].BankTransactionCode.DomainCode	$.data[*].bankTransactionCode.domainCode

Property #	Standards Property Name	API Hub Property Name
T22.03	$.Data.Transaction[*].BankTransactionCode.Family	$.data[*].bankTransactionCode.family
T22.04	$.Data.Transaction[*].BankTransactionCode.FamilyCode	$.data[*].bankTransactionCode.familyCode
T22.05	$.Data.Transaction[*].BankTransactionCode.SubFamily	$.data[*].bankTransactionCode.subFamily
T22.06	$.Data.Transaction[*].BankTransactionCode.SubFamilyCode	$.data[*].bankTransactionCode.subFamilyCode
T23	$.Data.Transaction[*].ProprietaryBankTransactionCode	$.data[*].proprietaryBankTransactionCode
T23.01	$.Data.Transaction[*].ProprietaryBankTransactionCode.Code	$.data[*].proprietaryBankTransactionCode.code
T23.02	$.Data.Transaction[*].ProprietaryBankTransactionCode.Issuer	$.data[*].proprietaryBankTransactionCode.issuer
T24	$.Data.Transaction[*].Balance	$.data[*].balance
T24.01	$.Data.Transaction[*].Balance.CreditDebitIndicator	$.data[*].balance.creditDebitIndicator
T24.02	$.Data.Transaction[*].Balance.Type	$.data[*].balance.balanceType
T24.03	$.Data.Transaction[*].Balance.Amount	$.data[*].balance.amount

Property #	Standards Property Name	API Hub Property Name
T24.03.01	$.Data.Transaction[*].Balance.Amount.Amount	$.data[*].balance.amount.amount
T24.03.02	$.Data.Transaction[*].Balance.Amount.Currency	$.data[*].balance.amount.currency
T25	$.Data.Transaction[*].MerchantDetails	$.data[*].merchantDetails
T25.01	$.Data.Transaction[*].MerchantDetails.MerchantId	$.data[*].merchantDetails.merchantId
T25.02	$.Data.Transaction[*].MerchantDetails.MerchantName	$.data[*].merchantDetails.merchantName
T25.03	$.Data.Transaction[*].MerchantDetails.MerchantCategoryCode	$.data[*].merchantDetails.merchantCategoryCode
T26	$.Data.Transaction[*].CreditorAgent	$.data[*].creditorAgent
T26.01	$.Data.Transaction[*].CreditorAgent.SchemeName	$.data[*].creditorAgent.schemeName
T26.02	$.Data.Transaction[*].CreditorAgent.Identification	$.data[*].creditorAgent.identification
T26.03	$.Data.Transaction[*].CreditorAgent.Name	$.data[*].creditorAgent.name
T26.04	$.Data.Transaction[*].CreditorAgent.PostalAddress	$.data[*].creditorAgent.postalAddress
T26.04.01	$.Data.Transaction[*].CreditorAgent.PostalAddress.AddressType	$.data[*].creditorAgent.postalAddress.addressType

Property #	Standards Property Name	API Hub Property Name
T26.04.02	$.Data.Transaction[*].CreditorAgent.PostalAddress.AddressLine	$.data[*].creditorAgent.postalAddress.AddressLine
T26.04.03	$.Data.Transaction[*].CreditorAgent.PostalAddress.BuildingNumber	$.data[*].creditorAgent.postalAddress.buildingNumber
T26.04.04	$.Data.Transaction[*].CreditorAgent.PostalAddress.BuildingName	$.data[*].creditorAgent.postalAddress.buildingName
T26.04.05	$.Data.Transaction[*].CreditorAgent.PostalAddress.Floor	$.data[*].creditorAgent.postalAddress.floor
T26.04.06	$.Data.Transaction[*].CreditorAgent.PostalAddress.StreetName	$.data[*].creditorAgent.postalAddress.streetName
T26.04.07	$.Data.Transaction[*].CreditorAgent.PostalAddress.PostBox	$.data[*].creditorAgent.postalAddress.postBox
T26.04.08	$.Data.Transaction[*].CreditorAgent.PostalAddress.DistrictName	$.data[*].creditorAgent.postalAddress.districtName
T26.04.09	$.Data.Transaction[*].CreditorAgent.PostalAddress.CountrySubDivision	$.data[*].creditorAgent.postalAddress.countrySubDivision
T26.04.11	$.Data.Transaction[*].CreditorAgent.PostalAddress.TownName	$.data[*].creditorAgent.postalAddress.townName

Property #	Standards Property Name	API Hub Property Name
T26.04.10	$.Data.Transaction[*].CreditorAgent.PostalAddress.Country	$.data[*].creditorAgent.postalAddress.country
T27	$.Data.Transaction[*].CreditorAccount	$.data[*].creditorAccount
T27.01	$.Data.Transaction[*].CreditorAccount[*].SchemeName	$.data[*].creditorAccount[*].schemeName
T27.02	$.Data.Transaction[*].CreditorAccount[*].Identification	$.data[*].creditorAccount[*].identification
T27.03	$.Data.Transaction[*].CreditorAccount[*].Name	$.data[*].creditorAccount[*].name
T28	$.Data.Transaction[*].DebtorAgent	$.data[*].debtorAgent
T28.01	$.Data.Transaction[*].DebtorAgent.SchemeName	$.data[*].debtorAgent.schemeName
T28.02	$.Data.Transaction[*].DebtorAgent.Identification	$.data[*].debtorAgent.identification
T28.03	$.Data.Transaction[*].DebtorAgent.Name	$.data[*].debtorAgent.name
T28.04	$.Data.Transaction[*].DebtorAgent.PostalAddress	$.data[*].debtorAgent.postalAddress
T28.04.01	$.Data.Transaction[*].DebtorAgent.PostalAddress.AddressType	$.data[*].debtorAgent.postalAddress.addressType

Property #	Standards Property Name	API Hub Property Name
T28.04.02	$.Data.Transaction[*].DebtorAgent.PostalAddress.AddressLine	$.data[*].debtorAgent.postalAddress.addressLine
T28.04.03	$.Data.Transaction[*].DebtorAgent.PostalAddress.BuildingNumber	$.data[*].debtorAgent.postalAddress.buildingNumber
T28.04.04	$.Data.Transaction[*].DebtorAgent.PostalAddress.BuildingName	$.data[*].debtorAgent.postalAddress.buildingName
T28.04.05	$.Data.Transaction[*].DebtorAgent.PostalAddress.Floor	$.data[*].debtorAgent.postalAddress.floor
T28.04.06	$.Data.Transaction[*].DebtorAgent.PostalAddress.StreetName	$.data[*].debtorAgent.postalAddress.streetName
T28.04.07	$.Data.Transaction[*].DebtorAgent.PostalAddress.PostBox	$.data[*].debtorAgent.postalAddress.postBox
T28.04.08	$.Data.Transaction[*].DebtorAgent.PostalAddress.DistrictName	$.data[*].debtorAgent.postalAddress.districtName
T28.04.09	$.Data.Transaction[*].DebtorAgent.PostalAddress.CountrySubDivision	$.data[*].debtorAgent.postalAddress.countrySubDivision
T28.04.10	$.Data.Transaction[*].DebtorAgent.PostalAddress.TownName	$.data[*].debtorAgent.postalAddress.townName

Property #	Standards Property Name	API Hub Property Name
T28.04.11	$.Data.Transaction[*].DebtorAgent.PostalAddress.Country	$.data[*].debtorAgent.postalAddress.country
T29	$.Data.Transaction[*].DebtorAccount	$.data[*].debtorAccount
T29.01	$.Data.Transaction[*].DebtorAccount.SchemeName	$.data[*].debtorAccount.schemeName
T29.02	$.Data.Transaction[*].DebtorAccount.Identification	$.data[*].debtorAccount.identification
T29.03	$.Data.Transaction[*].DebtorAccount.Name	$.data[*].debtorAccount.name
T30	$.Data.Transaction[*].CardInstrument	$.data[*].cardInstrument
T30.01	$.Data.Transaction[*].CardInstrument.CardSchemeName	$.data[*].cardInstrument.cardSchemeName
T30.02	$.Data.Transaction[*].CardInstrument.InstrumentType	$.data[*].cardInstrument.instrumentType
T30.03	$.Data.Transaction[*].CardInstrument.Name	$.data[*].cardInstrument.name
T30.04	$.Data.Transaction[*].CardInstrument.Identification	$.data[*].cardInstrument.identification
T31	$.Data.Transaction[*].SupplementaryData	$.data[*].supplementaryData

Property #	Standards Property Name	API Hub Property Name
T32	$.Data.Transaction[*].BillDetails	$.data[*].billDetails
T32.01	$.Data.Transaction[*].BillDetails.BillerId	$.data[*].billDetails.BillerID
T32.02	$.Data.Transaction[*].BillDetails.BillNumber	$.data[*].billDetails.BillNumber
T32.03	$.Data.Transaction[*].BillDetails.BillPaymentType	$.data[*].billDetails.BillPaymentType
T33	$.Data.Transaction[*].GeoLocation	$.data[*].geoLocation
T33.01	$.Data.Transaction[*].GeoLocation.Latitude	$.data[*].geoLocation.latitude
T33.02	$.Data.Transaction[*].GeoLocation.Longitude	$.data[*].geoLocation.longitude
T34	$.Data.Transaction[*].PaymentPurposeCode	$.data[*].paymentPurposeCode
T35	$.Data.Transaction[*].IsPrimaryInstrument	$.data[*].isPrimaryInstrument
T36	$.Data.Transaction[*].Allocations	$.data[*].allocations
T36.01	$.Data.Transaction[*].Allocations.BalanceCategory	$.data[*].allocations.BalanceCategory
T36.02	$.Data.Transaction[*].Allocations.Amount	$.data[*].allocations.Amount

Property #	Standards Property Name	API Hub Property Name
T36.02.01	`$.Data.Transaction[*].Allocations.Amount.Amount`	`$.data[*].allocations.Amount.Amount`
T36.02.02	`$.Data.Transaction[*].Allocations.Amount.Currency`	`$.data[*].allocations.Amount.Currency`
T36.03	`$.Data.Transaction[*].Allocations.BalanceDescription`	`$.data[*].allocations.BalanceDescription`

Data Permissions: Basic vs Detail

Based on the permission code granted in Consent the following Transactions information is provided to the TPP.

> **Note**
>
> Summary: The following properties are not provided in the response payload when `ReadTransactionsBasic` data permission has been granted:
>
> - `TransactionInformation` (T17)
> - `$.Data.Transaction[*].CurrencyExchange.ContractIdentification`(T21.05) and `$.Data.Transaction[*].CurrencyExchange.QuotationDate` (T21.06) within the `CurrencyExchange` object
> - `BankTransactionCode` (T22) object
> - `ProprietaryBankTransactionCode` (T23) object
> - `CreditorAgent` (T26) object
> - `CreditorAccount` (T27) object
> - `DebtorAgent` (T28) object
> - `$.Data.Transaction[*].DebtorAccount.Name` (T29.03)
> - `SupplementaryData` (T31)
> - `GeoLocation` (T33) object

Property ID	Standards Property Name	ReadTransactionsBasic	ReadTrans
T01	$.Data.AccountId	Yes	
T02	$.Data.Transaction[*].TransactionId	Yes	
T03	$.Data.Transaction[*].TransactionDateTime	Yes	
T04	$.Data.Transaction[*].LocalTimeZone	Yes	
T05	$.Data.Transaction[*].StatementReference	Yes	
T06	$.Data.Transaction[*].TransactionReference	Yes	
T07	$.Data.Transaction[*].TransactionType	Yes	
T08	$.Data.Transaction[*].SubTransactionType	Yes	
T09	$.Data.Transaction[*].TerminalId	Yes	
T10	$.Data.Transaction[*].Flags	Yes	
T11	$.Data.Transaction[*].PaymentModes	Yes	
T12	$.Data.Transaction[*].CreditDebitIndicator	Yes	
T13	$.Data.Transaction[*].Status	Yes	
T14	$.Data.Transaction[*].TransactionMutability	Yes	

Property ID	Standards Property Name	ReadTransactionsBasic	ReadTrans
T15	$.Data.Transaction[*].BookingDateTime	Yes	
T16	$.Data.Transaction[*].ValueDateTime	Yes	
T17	$.Data.Transaction[*].TransactionInformation	No	
T18	$.Data.Transaction[*].Amount	Yes	
T18.01	$.Data.Transaction[*].Amount.Amount	Yes	
T18.02	$.Data.Transaction[*].Amount.Currency	Yes	
T19	$.Data.Transaction[*].ChargeAmount	Yes	
T19.01	$.Data.Transaction[*].ChargeAmount.Amount	Yes	
T19.02	$.Data.Transaction[*].ChargeAmount.Currency	Yes	
T19.03	$.Data.Transaction[*].ChargeAmount.ChargeIncluded	Yes	
T20	$.Data.Transaction[*].ChargeAmountVat	Yes	
T20.01	$.Data.Transaction[*].ChargeAmountVat.Amount	Yes	
T20.02	$.Data.Transaction[*].ChargeAmountVat.Currency	Yes	

Property ID	Standards Property Name	ReadTransactionsBasic	ReadTrans
T21	$.Data.Transaction[*].CurrencyExchange	Limited Fields	
T21.01	$.Data.Transaction[*].CurrencyExchange.SourceCurrency	Yes	
T21.02	$.Data.Transaction[*].CurrencyExchange.TargetCurrency	Yes	
T21.03	$.Data.Transaction[*].CurrencyExchange.UnitCurrency	Yes	
T21.04	$.Data.Transaction[*].CurrencyExchange.ExchangeRate	Yes	
T21.05	$.Data.Transaction[*].CurrencyExchange.ContractIdentification	No	
T21.06	$.Data.Transaction[*].CurrencyExchange.QuotationDate	No	
T21.07	$.Data.Transaction[*].CurrencyExchange.InstructedAmount	Yes	
T21.07.01	$.Data.Transaction[*].CurrencyExchange.InstructedAmount.Amount	Yes	
T21.07.02	$.Data.Transaction[*].CurrencyExchange.InstructedAmount.Currency	Yes	
T22	$.Data.Transaction[*].BankTransactionCode	No	

Property ID	Standards Property Name	ReadTransactionsBasic	ReadTrans
T22.01	$.Data.Transaction[*].BankTransactionCode.Domain	No	
T22.02	$.Data.Transaction[*].BankTransactionCode.DomainCode	No	
T22.03	$.Data.Transaction[*].BankTransactionCode.Family	No	
T22.04	$.Data.Transaction[*].BankTransactionCode.FamilyCode	No	
T22.05	$.Data.Transaction[*].BankTransactionCode.SubFamily	No	
T22.06	$.Data.Transaction[*].BankTransactionCode.SubFamilyCode	No	
T23	$.Data.Transaction[*].ProprietaryBankTransactionCode	No	
T23.01	$.Data.Transaction[*].ProprietaryBankTransactionCode.Code	No	
T23.02	$.Data.Transaction[*].ProprietaryBankTransactionCode.Issuer	No	
T24	$.Data.Transaction[*].Balance	Yes	
T24.01	$.Data.Transaction[*].Balance.CreditDebitIndicator	Yes	
T24.02	$.Data.Transaction[*].Balance.Type	Yes	

Property ID	Standards Property Name	ReadTransactionsBasic	ReadTrans
T24.03	$.Data.Transaction[*].Balance.Amount	Yes	
T24.03.01	$.Data.Transaction[*].Balance.Amount.Amount	Yes	
T24.03.02	$.Data.Transaction[*].Balance.Amount.Currency	Yes	
T25	$.Data.Transaction[*].MerchantDetails	Yes	
T25.01	$.Data.Transaction[*].MerchantDetails.MerchantId	Yes	
T25.02	$.Data.Transaction[*].MerchantDetails.MerchantName	Yes	
T25.03	$.Data.Transaction[*].MerchantDetails.MerchantCategoryCode	Yes	
T26	$.Data.Transaction[*].CreditorAgent	No	
T26.01	$.Data.Transaction[*].CreditorAgent.SchemeName	No	
T26.02	$.Data.Transaction[*].CreditorAgent.Identification	No	
T26.03	$.Data.Transaction[*].CreditorAgent.Name	No	
T26.04	$.Data.Transaction[*].CreditorAgent.PostalAddress	No	
T26.04.01	$.Data.Transaction[*].CreditorAgent.PostalAddress.	No	

Property ID	Standards Property Name	ReadTransactionsBasic	ReadTrans
	AddressType		
T26.04.02	$.Data.Transaction[*].CreditorAgent.PostalAddress.AddressLine	No	
T26.04.03	$.Data.Transaction[*].CreditorAgent.PostalAddress.BuildingNumber	No	
T26.04.04	$.Data.Transaction[*].CreditorAgent.PostalAddress.BuildingName	No	
T26.04.05	$.Data.Transaction[*].CreditorAgent.PostalAddress.Floor	No	
T26.04.06	$.Data.Transaction[*].CreditorAgent.PostalAddress.StreetName	No	
T26.04.07	$.Data.Transaction[*].CreditorAgent.PostalAddress.PostBox	No	
T26.04.08	$.Data.Transaction[*].CreditorAgent.PostalAddress.DistrictName	No	
T26.04.09	$.Data.Transaction[*].CreditorAgent.PostalAddress.CountrySubDivision	No	
T26.04.10	$.Data.Transaction[*].CreditorAgent.PostalAddress.TownName	No	

Property ID	Standards Property Name	ReadTransactionsBasic	ReadTrans
T26.04.11	$.Data.Transaction[*].CreditorAgent.PostalAddress.Country	No	
T27	$.Data.Transaction[*].CreditorAccount	No	
T27.01	$.Data.Transaction[*].CreditorAccount[*].SchemeName	No	
T27.02	$.Data.Transaction[*].CreditorAccount[*].Identification	No	
T27.03	$.Data.Transaction[*].CreditorAccount[*].Name	No	
T28	$.Data.Transaction[*].DebtorAgent	No	
T28.01	$.Data.Transaction[*].DebtorAgent.SchemeName	No	
T28.02	$.Data.Transaction[*].DebtorAgent.Identification	No	
T28.03	$.Data.Transaction[*].DebtorAgent.Name	No	
T28.04	$.Data.Transaction[*].DebtorAgent.PostalAddress	No	
T28.04.01	$.Data.Transaction[*].DebtorAgent.PostalAddress.AddressType	No	
T28.04.02	$.Data.Transaction[*].DebtorAgent.PostalAddress.	No	

Property ID	Standards Property Name	ReadTransactionsBasic	ReadTrans
	AddressLine		
T28.04.03	$.Data.Transaction[*].DebtorAgent.PostalAddress.BuildingNumber	No	
T28.04.04	$.Data.Transaction[*].DebtorAgent.PostalAddress.BuildingName	No	
T28.04.05	$.Data.Transaction[*].DebtorAgent.PostalAddress.Floor	No	
T28.04.06	$.Data.Transaction[*].DebtorAgent.PostalAddress.StreetName	No	
T28.04.07	$.Data.Transaction[*].DebtorAgent.PostalAddress.PostBox	No	
T28.04.08	$.Data.Transaction[*].DebtorAgent.PostalAddress.DistrictName	No	
T28.04.09	$.Data.Transaction[*].DebtorAgent.PostalAddress.CountrySubDivision	No	
T28.04.10	$.Data.Transaction[*].DebtorAgent.PostalAddress.TownName	No	
T28.04.11	$.Data.Transaction[*].DebtorAgent.PostalAddress.Country	No	

Property ID	Standards Property Name	ReadTransactionsBasic	ReadTrans
T29	$.Data.Transaction[*].DebtorAccount	Yes	
T29.01	$.Data.Transaction[*].DebtorAccount.SchemeName	Yes	
T29.02	$.Data.Transaction[*].DebtorAccount.Identification	Yes	
T29.03	$.Data.Transaction[*].DebtorAccount.Name	No	
T30	$.Data.Transaction[*].CardInstrument	Yes	
T30.01	$.Data.Transaction[*].CardInstrument.CardSchemeName	Yes	
T30.02	$.Data.Transaction[*].CardInstrument.InstrumentType	Yes	
T30.03	$.Data.Transaction[*].CardInstrument.Name	Yes	
T30.04	$.Data.Transaction[*].CardInstrument.Identification	Yes	
T31	$.Data.Transaction[*].SupplementaryData	No	
T32	$.Data.Transaction[*].BillDetails	Yes	
T32.01	$.Data.Transaction[*].BillDetails.BillerId	Yes	
T32.02	$.Data.Transaction[*].BillDetails.BillNumber	Yes	

Property ID	Standards Property Name	ReadTransactionsBasic	ReadTrans
T32.03	$.Data.Transaction[*].BillDetails.BillPaymentType	Yes	
T33	$.Data.Transaction[*].GeoLocation	No	
T33.01	$.Data.Transaction[*].GeoLocation.Latitude	No	
T33.02	$.Data.Transaction[*].GeoLocation.Longitude	No	
T34	$.Data.Transaction[*].PaymentPurposeCode	Yes	
T35	$.Data.Transaction[*].IsPrimaryInstrument	Yes	
T36	$.Data.Transaction[*].Allocations		
T36.01	$.Data.Transaction[*].Allocations.BalanceCategory	Yes	
T36.02	$.Data.Transaction[*].Allocations.Amount	Yes	
T36.02.01	$.Data.Transaction[*].Allocations.Amount.Amount	Yes	
T36.02.02	$.Data.Transaction[*].Allocations.Amount.Currency	Yes	
T36.03	$.Data.Transaction[*].Allocations.BalanceDescription	Yes	

Consent Manager | Overview

Introduction

This page provides guidance on the Consent Manager API at the API Hub, in the context of how the API can be invoked at various stages of the consent lifecycle.

> **Note**
>
> The information herein is intended to be informative, but the Consent Manager OpenAPI description remains the source of truth for Consent Manager operations. This page **must** therefore be read in conjunction with the Consent Manager API description.

For information on the consent and payment state model, and the statuses germane to the Consent Manager API, please refer to the Standards.

Context

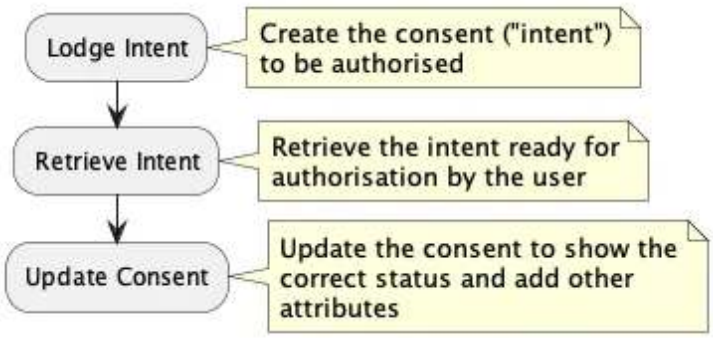
The Consent Manager is a consent management solution created by Ozone, and is part of the Qatar Open Finance platform.

The Consent Manager provides the following features:

- Lodging of unauthorised consents (“intents”) sent by a TPP through a Pushed Authorization Request.
- State management throughout the consent lifecycle, including signalling of consent authorisation.
- Enrichment of consent with, for example, TPP metadata or metadata provided by the LFI.
- Payment state management.

The Consent Manager API can be called at any time when the status of a consent or a payment changes. The most common use of the API is, however, during the Authorization Code flow.

The activity diagram below shows a high-level view of these operations.



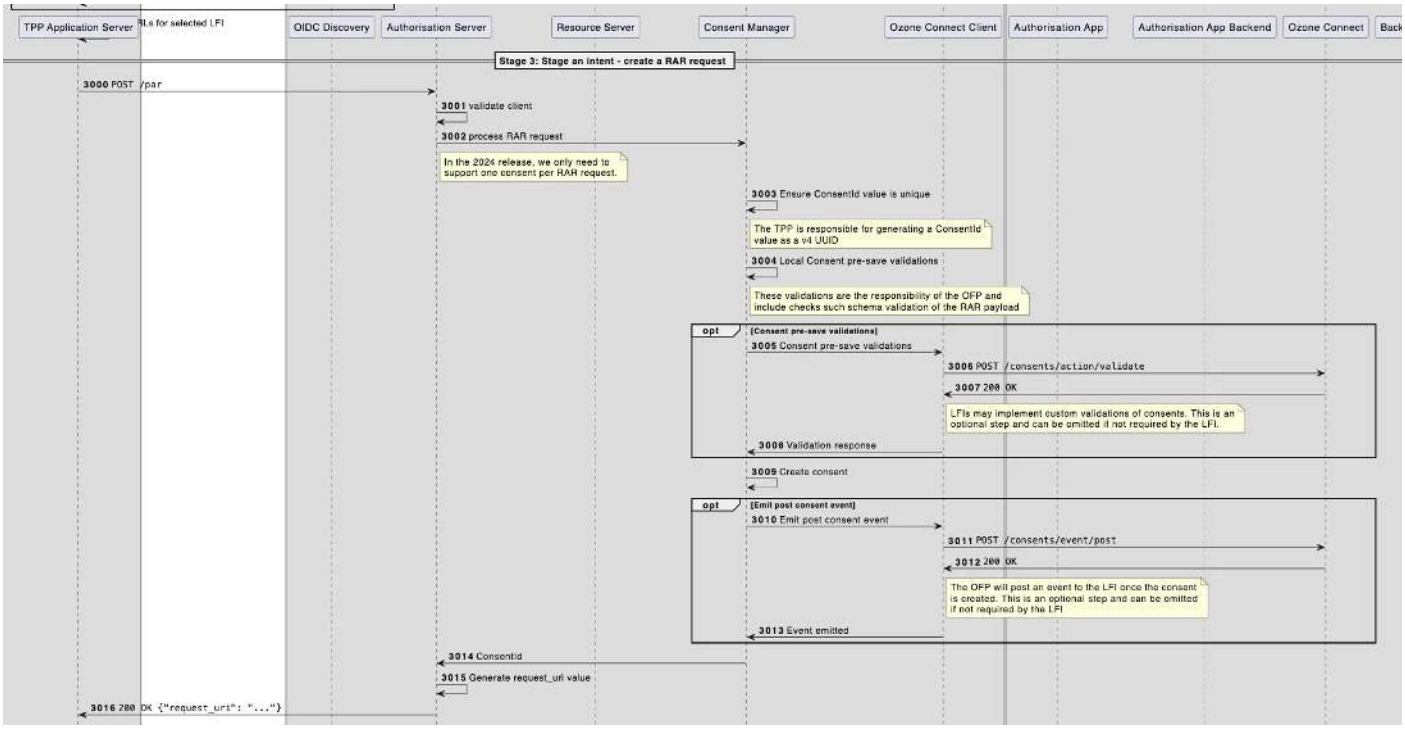
The following sections provide excerpts of the API Hub Sequence Diagrams that describe the end-to-end flow, and show the high-level operations in context to provide clarity on how the Consent Manager API is used.

Lodge Intent

An intent is lodged as a result of a Pushed Authorization Request, and results in a Consent resource being created.

When an intent is authorised by the User it is then termed a consent.

Attribute	Detail
Operation URL and method	`POST /consents`
Initiated When	A TPP submits a Pushed Authorization Request
Invoked By	API Hub
Data Enrichment	TPP metadata
Other Use Cases	No other use cases are currently supported

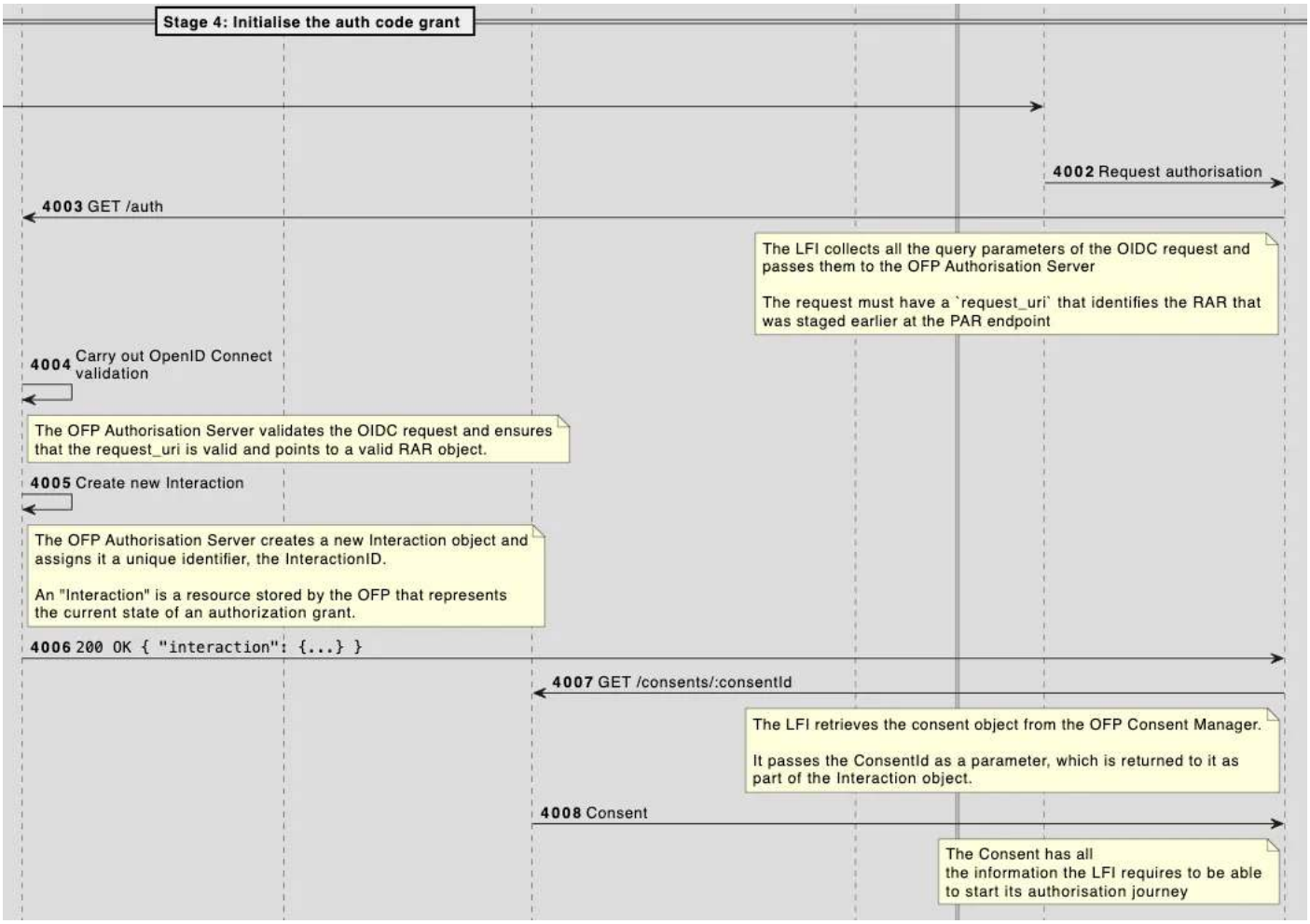


Retrieve Intent

An LFI retrieves a given consent during the Authorization Code flow.

Retrieving the consent allows the LFI to render an authorisation screen, enabling the User to confirm the details of the consent.

Attribute	Detail
Operation URL and method	`GET /consents/{consentId}`
Initiated When	An LFI needs to retrieve the consent ready to authenticate the User, and for the User to authorise the consent
Invoked By	LFI
Data Enrichment	None
Other Use Cases	Consent synchronisation (the LFI retrieves the current state of the consent)

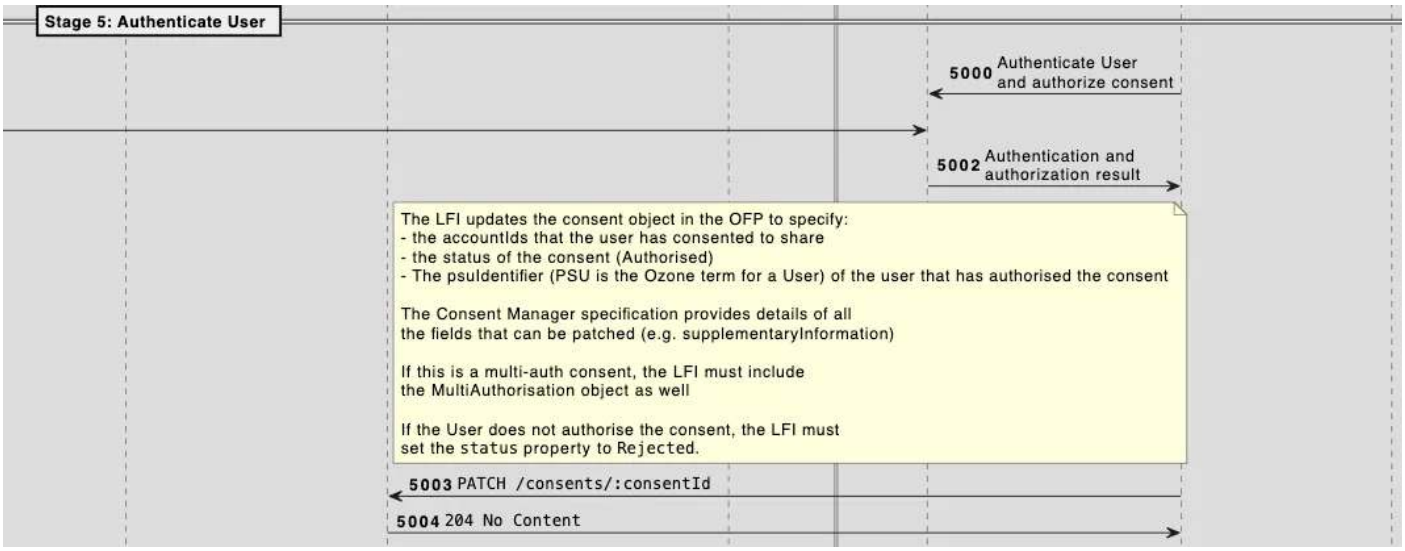


Update Consent

An LFI updates a given consent after the User has completed the authentication and authorisation journey.

The LFI **must** update the consent regardless of whether the consent is authorised or rejected.

Attribute	Detail
Operation URL and method	`PATCH /consents/{consentId}`
Initiated When	An LFI needs to update the consent with properties of the authenticated party, such as User identifiers, status, or multi-authorisation requirements
Invoked By	LFI
Data Enrichment	As described above, and outlined in the OpenAPI description
Other Use Cases	Updating User identifiers for multi-authorisation accounts (for example, corporate accounts with multiple approvers)



Error Handling

The Consent Manager API is a REST API and implements REST semantics, namely:

- Failure scenarios are signalled by an HTTP status code outside the 2xx range.
- An appropriate `errorCode` and `errorMessage` value is returned.

The table below shows the possible `400` errors that can be returned. Please note that variables are shown in the message to indicate where the error message text will vary.

Error Code	Error Message
`general_error`	`default: An error occurre indeterminate error code`
`expected_modification`	`expected_modification: Co patch consent`

Error Code	Error Message
invalid_message_format	invalid_message_format: js does not match schema ${JS string(variable)}
invalid_page_number	invalid_page_number: ensu {variable.query} query pa valid number and greater
invalid_page_size	invalid_page_size: ensure {variable.query} query pa valid number and greater
consent_status_invalid_patch_request	consent_status_invalid_pa Consent status must be on [{variable.allowedConsent
consent_status_invalid	consent_status_invalid: A not be patched if it is a terminal state

Error Code	Error Message
consent_body_missing_revocation	consent_body_missing_revoc consent cannot be revoked revocation data
ensure_consent_deleted	ensure_consent_deleted: fa delete consent
invalid_consentId	invalid_consentId: Ensure is valid
consent_type_invalid	consent_type_invalid: cons {variable.consentType} is
consent_type_not_supported	consent_type_not_supporte type {variable.consentTyp valid
invalid_date_format	invalid_date_format: ensu paymentResponse.statusUpd is in ISO 8601 format

Error Code	Error Message
date_undefined	date_undefined: paymentResponse.statusUpda is required
incorrect_payment_patch_status	incorrect_payment_patch_st status is needed
payment_not_found	payment_not_found: payment found
status_undefined	status_undefined: paymentResponse.status is

Error Code	Error Message
payment_id_not_found	payment_id_not_found: payr found in params
payment_type_not_found	payment_type_not_found: pa not found
failed_to_patch_request	failed_to_patch_request: patch request
not_a_number	not_a_number: not a numbe
consent_type_not_found	consent_type_not_found: co not found
authorization_time_expired	authorization_time_expire authorize the consent sta {variable.lhs} === {variab

Error Code	Error Message
mandatory_field_not_defined	mandatory_field_not_define instruction {variable.msg} {variable.no} not found.
payment_status_invalid_patch_request	payment_status_invalid_pat Consent status must be one [{variable.allowedConsentS
rejected_consent_cannot_be_authorized	consent is rejected and ca authorized
rejected_consent_cannot_have_multiple_authorizers	consent is rejected and ca multiple authorizers

Error Code	Error Message

Please refer to the Consent Manager API description for more details on the structure of the error response payload.

Consent Management Interface

Introduction

LFIs must implement a Consent Management Interface (CMI) prescribed by the Standards to provide a User with information on their consents (see LFI Consent Management Interface).

The Consent Manager API is used to retrieve consent-related data with which to populate the CMI. This guide explains how to use the API to retrieve User consent details, retrieve consent history, and map the elements of the CMI to API properties.

Properties are given as JSONPath expressions, rooted at the response body (`$`).

Retrieving User Consents

To retrieve all consents associated with a User (termed PSU in the Consent Manager) use the `get /psu/{userId}/consents` operation, where `userId` is the LFI unique identifier for the User.

> ⚠ **Caution**
>
> To use this operation the User identifier **must** be patched on the consent using `patch /consents/{consentId}`.

The response will return an array of consent objects containing information about each consent.

CMI for Bank Data Sharing

The table below shows how the response properties map to a given property in the CMI for Bank Data Sharing.

Label	Operation	Property (JSONPath)	Guideline
TPP Trading Name	`get /psu/{userId}/consents`	`$.data[*].request.consent.OnBehalfOf.TradingName` OR `$.data[*].tpp.tppName`	If a trading supplied i consent th should be to the Use otherwise to the TPF

Label	Operation	Property (JSONPath)	Guideline
			from the T Framewor
Last data shared	`get /psu/{userId}/consents`	`$.data[*].consentUsage.lastDataShared`	
Connection Expires	`get /psu/{userId}/consents`	`$.data[*].request.consent.ExpirationDateTime`	
Status	`get /psu/{userId}/consents`	`$.data[*].status`	See Statu Language
Consent ID	`get /psu/{userId}/consents`	`$.data[*].id`	
IBAN	`get /psu/{userId}/consents`	`$.data[*].accountIds`	The surro account ic patched o consent a returned. must be re by the rea the LFI, du restriction the API Hu
Data they get	`get /psu/{userId}/consents`	`$.data[*].request.consent.Permissions`	Permissio be provide the standa data clust language in Permiss Data Clust `Permissi` property p the values to map the

Label	Operation	Property (JSONPath)	Guideline
			permissio the correc language.
First Connected	`get /psu/{userId}/consents` OR `get /consent-groups/{consentGroupId}/consents`	`$.data[*].consentBody.Data.CreationDateTime`	If the cons part of a c group, this the `Creation` of the first in the con group. The property `BaseCons` provides t of the `consentG` parameter consent is of a conse as `BaseCo` is not set, be the `Creation` of the con
Last Updated	`get /psu/{userId}/consents` OR `get /consent-groups/{consentGroupId}/consents`	`$.data[*].consentBody.Data.CreationDateTime` OR `$.data[*].updatedAt`	If the cons part of a c group, this the `Creation` of the late consent ir consent g consent is of a conse

Label	Operation	Property (JSONPath)	Guideline
			as BaseCo is not set, be the upd value of th consent.

CMI for Bank Service Initiation

The table below shows how the response properties map to a given property in the CMI for Bank Service Initiation.

The `PersonalIdentifiableInformation` property of a consent and of a payment initiation request is read in its decrypted form, defined by the `PaymentInitiation.DomesticPaymentPersonalIdentifiableInformation1Properties` and `PaymentInitiation.DomesticPaymentInitiationPersonalIdentifiableInformation1Properties` Schema Objects respectively. In a consent, `Initiation.Creditor` is an array with one entry per Creditor.

Label	Operation	Property (JSONPath)
TPP Trading Name	`get /psu/{userId}/consents`	`$.data[*].request.consentOnBehalfOf.TradingName` OR `data[*].tpp.tppName`
Total paid/Total paid to date	`get /psu/{userId}/consents` OR `get /payment-log?consentId={consentId}`	`$.data[*].consentBody.Dat PaymentConsumption.CumulativeValueOfPayments` `data[*].requestBody.Data.Instruction.Amount.Amount`
Permission Cancelled/Expired/Consumed	`get /psu/{userId}/consents`	`$.data[*].updatedAt`
Last payment made	`get /psu/{userId}/consents`	`$.data[*].consentUsage.lastServiceInitiationAtte`

Label	Operation	Property (JSONPath)
Payment to	get /psu/{userId}/consents OR get /payment-log?consentId={consentId}	$.data[*].request.consentPersonalIdentifiableInfor OR $.data[*].requestBody.PersonalIdentifiableInfor
IBAN	get /psu/{userId}/consents OR get /payment-log?consentId={consentId}	$.data[*].request.consentPersonalIdentifiableInfor OR $.data[*].requestBody.PersonalIdentifiableInfor
Reference	get /payment-log?consentId={consentId}	$.data[*].requestBody.DatCreditorReference
Payment Purpose	get /payment-log?consentId={consentId}	$.data[*].requestBody.DatPaymentPurposeCode
From Account	get /psu/{userId}/consents	$.data[*].accountIds OR $.data[*].request.consent.PersonalIdentifiableInfor
Payment Rules	get /psu/{userId}/consents	$.data[*].request.consentControlParameters
You started this permission	get /psu/{userId}/consents	$.data[*].consentBody.DatCreationDateTime

Label	Operation	Property (JSONPath)
You cancelled this permission	`get /psu/{userId}/consents`	`$.data[*].updatedAt`

Connection History

To provide Connection History information to the User use the `get /consent-groups/{consentGroupId}/consents` operation. This returns all consents in a consent group, which are linked by the value of `BaseConsentId`.

The following guidance is provided on Connection History and the use of `BaseConsentId`:

- Connection History is driven by “revisions” to consents orchestrated by TPPs, where an existing consent is replaced by a new consent.
- When a TPP and User agree a new set of permissions or a change to data access, a new consent can be created and the existing consent moved to an appropriate state (note that the existing consent could also have expired).
- In order to link the existing consent with the new consent, the TPP will set `BaseConsentId` on the new consent to the value of `ConsentId` of the original consent.
- All subsequently created consents that share the same history will use the same `BaseConsentId` value, i.e. that of the first consent agreed between the TPP and the User.
- Retrieving these consents is achieved by using `get /consent-groups/{consentGroupId}/consents`, where `consentGroupId` is the value of `BaseConsentId`.
- The `get /consents/{consentId}/audit` operation does **not** provide the means to retrieve Connection History, as it relates to changes to a single consent, not changes to multiple consents by `BaseConsentId`.

Connection History for Bank Data Sharing

The table below shows how the response properties map to a given property in the CMI for Bank Data Sharing.

Label	Operation	Property (JSONPath)	Guidelines
TPP Trading Name	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].request.consent.OnBehalfOf.TradingName` OR `$.data[*].tpp.tppName`	If a trading name is supplied in the consen then this should be displayed t the User, otherwise f back to the TPP name from the Trust Framework
Consent ID	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].id`	
IBAN	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].accountIds`	The surrogate account identifiers patched on the consen are returne These mus be replace by the real IBAN by the LFI, due to restrictions on PII in the API Hub.

Label	Operation	Property (JSONPath)	Guidelines
Data they get	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].request.consent.Permissions`	Permissions must be provided using the standardise data cluste language provided in Permissions and Data Clusters. Tl `Permissio` property provides th values for LFIs to map the permission code to the correct language.
<Connection date shown as label>	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].consentBody.Data.CreationDateTime`	

Connection History for Bank Service Initiation

Label	Operation	Property (JSONPath)
Payment to	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].request.consent.PersonalIdentifiableInformati`

Label	Operation	Property (JSONPath)
	OR `get /payment-log?consentId={consentId}`	OR `$.data[*].requestBody.Data PersonalIdentifiableInformati`
IBAN	`get /consent-groups/{consentGroupId}/consents` OR `get /payment-log?consentId={consentId}`	`$.data[*].request.consent. PersonalIdentifiableInformati` OR `$.data[*].requestBody.Data PersonalIdentifiableInformati`
Reference	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].request.consent. DebtorReference`
Payment Purpose	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].request.consent. PaymentPurposeCode`
From Account	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].accountIds` OR `$.data[*].request.consent. PersonalIdentifiableInformati`
Payment Rules	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].request.consent. ControlParameters`
<Connection date shown as label>	`get /consent-groups/{consentGroupId}/consents`	`$.data[*].consentBody.Data. CreationDateTime`

Payment History

To provide Payment History information to the User use the `get /payment-log?consentId={consentId}` operation. This returns a summary of all payments for a given consent.

The table below shows how the response properties map to a given property in the Payment History view.

Label	Operation	Property (JSONPath)	Guidelines
TPP Trading Name	`get /payment-log?consentId={consentId}`	`$.data[*].tpp.tppName`	If a trading name is supplied in the consent then this should be displayed to the User, otherwise fall back to the TPP name from the Trust Framework.
Total paid/Total paid to date	`get /payment-log?consentId={consentId}`	`$.data[*].requestBody.Data.Instruction.Amount.Amount`	Sum the values indicated by the JSONPath that are returned by this operation.
<Payment Date>	`get /payment-log?consentId={consentId}`	`$.data[*].paymentResponse.creationDateTime`	
<Payment Reference>	`get /payment-log?consentId={consentId}`	`$.data[*].requestBody.Data.CreditorReference`	
<Amount>	`get /payment-log?consentId={consentId}`	`$.data[*].requestBody.Data.Instruction.Amount.Amount`	

Status Code Language

The Consent Manager API provides consent statuses as codes.

The table below maps a given Consent Manager API status code to the CMI status code that is mandated.

Consent Manager Status Code	CMI Status Code
`Rejected` (where Rejected consents are returned, as the User identifier has been patched to the consent)	Rejected
`AwaitingAuthorization`	Awaiting
`Authorized`	Active
`Suspended`	Suspended
`Revoked`	Cancelled
`Consumed`	Consumed
`Expired`	Expired

If a code does not appear in this list it should be normalised based on the following rules:

- Codes should be split on word boundary.
- Case should be maintained.
- If the status requires truncation for display purposes the whole of the leftmost word should be returned.

Authorization Server | Overview

Introduction

This page provides guidance on error handling and error scenarios for LFIs who are integrating with the Headless Authorization Server (“Heimdall”) API at the API Hub.

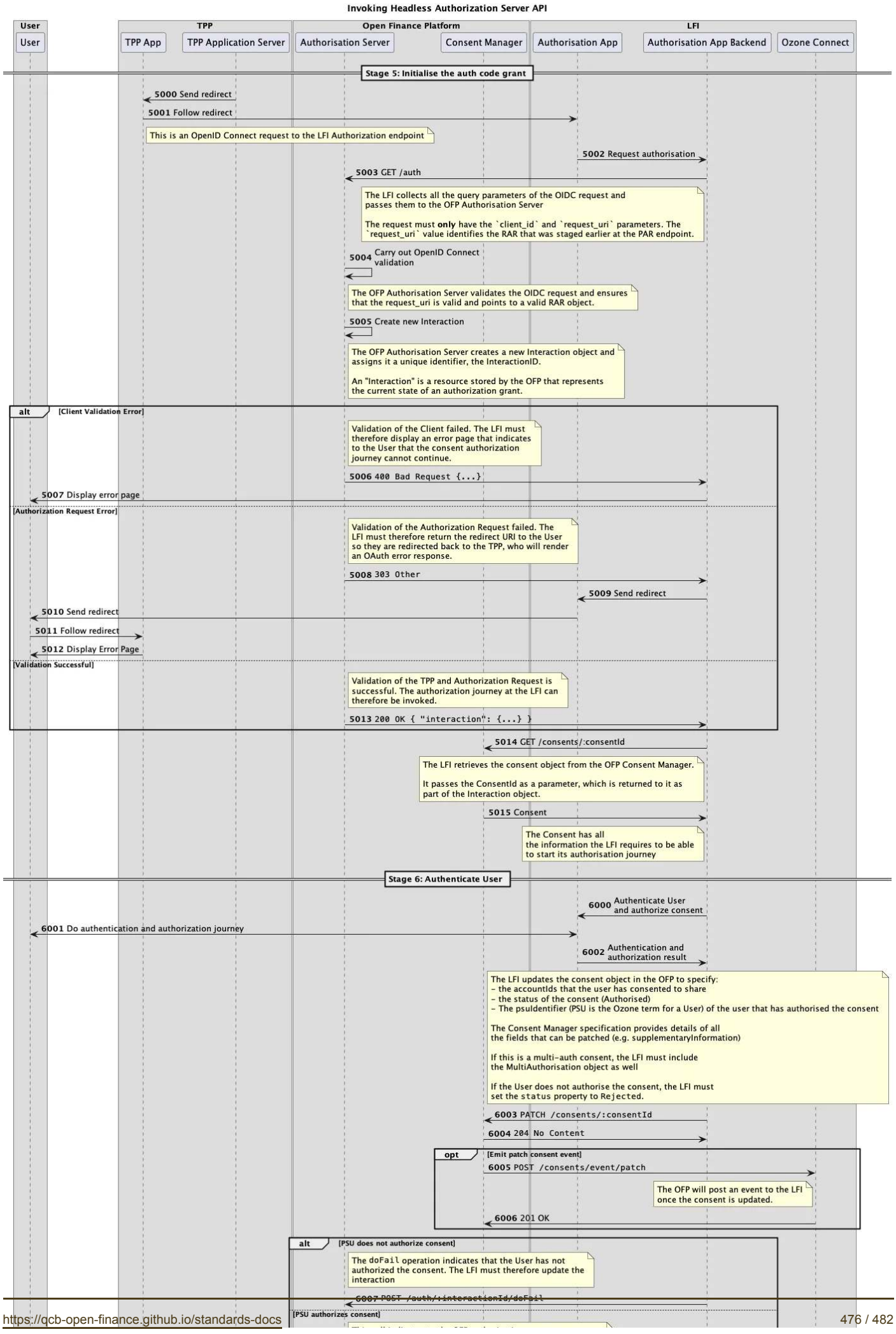
> **Note**
>
> This page **must** be read in conjunction with the Authorization Server API description.

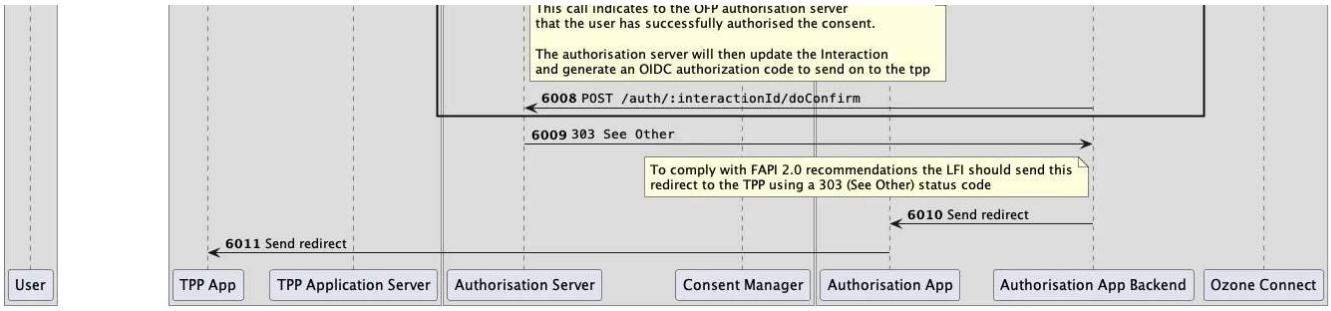
Context

The Ozone Headless Authorization Server (“Heimdall”) provides two primary features for LFIs operating in Qatar Open Finance:

- It validates Authorization Request parameters, including Client and JWT-Secured Authorization Request (JAR) parameters.
- It allows the status of a given User going through authentication and authorization to be tracked, and the appropriate redirect URI to be returned.

The API is typically invoked before and after the User has authenticated at the LFI and authorised the consent. The sequence diagram below provides an excerpt of the API Hub Sequence Diagrams that describe the end-to-end flow, focusing on the Heimdall API.


Invoking Headless Authorization Server API
User
TPP
Open Finance Platform
LFI
User
TPP App
TPP Application Server
Authorisation Server
Consent Manager
Authorisation App
Authorisation App Backend
Ozone Connect
Stage 5: Initialise the auth code grant
5000 Send redirect
5001 Follow redirect
This is an OpenID Connect request to the LFI Authorization endpoint
5002 Request authorisation
5003 GET /auth
The LFI collects all the query parameters of the OIDC request and passes them to the OFP Authorisation Server
The request must only have the `client_id` and `request_uri` parameters. The `request_uri` value identifies the RAR that was staged earlier at the PAR endpoint.
5004 Carry out OpenID Connect validation
The OFP Authorisation Server validates the OIDC request and ensures that the request_uri is valid and points to a valid RAR object.
5005 Create new Interaction
The OFP Authorisation Server creates a new Interaction object and assigns it a unique identifier, the InteractionID.
An "Interaction" is a resource stored by the OFP that represents the current state of an authorization grant.
alt
[Client Validation Error]
Validation of the Client failed. The LFI must therefore display an error page that indicates to the User that the consent authorization journey cannot continue.
5006 400 Bad Request {...}
5007 Display error page
[Authorization Request Error]
Validation of the Authorization Request failed. The LFI must therefore return the redirect URI to the User so they are redirected back to the TPP, who will render an OAuth error response.
5008 303 Other
5009 Send redirect
5010 Send redirect
5011 Follow redirect
5012 Display Error Page
[Validation Successful]
Validation of the TPP and Authorization Request is successful. The authorization journey at the LFI can therefore be invoked.
5013 200 OK { "interaction": {...} }
5014 GET /consents/:consentId
The LFI retrieves the consent object from the OFP Consent Manager.
It passes the ConsentId as a parameter, which is returned to it as part of the Interaction object.
5015 Consent
The Consent has all the information the LFI requires to be able to start its authorisation journey
Stage 6: Authenticate User
6000 Authenticate User and authorize consent
6001 Do authentication and authorization journey
6002 Authentication and authorization result
The LFI updates the consent object in the OFP to specify:
- the accountIds that the user has consented to share
- the status of the consent (Authorised)
- The psuIdentifier (PSU is the Ozone term for a User) of the user that has authorised the consent
The Consent Manager specification provides details of all the fields that can be patched (e.g. supplementaryInformation)
If this is a multi-auth consent, the LFI must include the MultiAuthorisation object as well
If the User does not authorise the consent, the LFI must set the status property to Rejected.
6003 PATCH /consents/:consentId
6004 204 No Content
opt
[Emit patch consent event]
6005 POST /consents/event/patch
The OFP will post an event to the LFI once the consent is updated.
6006 201 OK
alt
[PSU does not authorize consent]
The doFail operation indicates that the User has not authorized the consent. The LFI must therefore update the interaction
6007 POST /auth/:interactionId/doFail
[PSU authorizes consent]



Success Response

When validation of the redirection is successful, the properties of the Pushed Authorization Request and of the TPP who sent it are available in the response. The values returned in the success response, indicated by a `200` HTTP status code, include the `login_hint` parameter, which allows the Qatar ID (QID) or Commercial Registration (CR) number to be sent as a JWE.

Please refer to the Authorization Server API description for further details.

Error Handling

The error scenarios are summarised in the table below. The sequence diagram steps refer to the diagram above.

Scenario	Description	Action	Redirect User to TPP	Sequence Diagram Step
Client Validation Error	Validation of the Client has failed, and the TPP therefore cannot be identified.	An error must be displayed to the User in the LFI app.	No	5006
Authorization Request Error	Validation of the JWT-Secured Authorization Request or Rich Authorization Request has failed.	The User must be redirected to the TPP, who will display an error page.	Yes	5008
User Validation Error	The User has not completed authentication and	The User must be redirected to the TPP, who will display an	Yes	6007 6009

Scenario	Description	Action	Redirect User to TPP	Sequence Diagram Step
	authorised consent, therefore “forcing” an error response to the TPP.	error page reflecting the fact that the User did not complete the authorisation of the requested consent.		

The following sections expand on each error type.

Client Validation Error

Client Validation Errors are returned by Heimdall when the Client belonging to a given TPP cannot be successfully validated.

Readers should note that, with the implementation of Pushed Authorization Requests (PAR) for Qatar Open Finance, the incidence of Client Validation Errors is likely to be low, as the Client and, by extension, the TPP will be validated in advance at the PAR Endpoint.

Client Validation Errors are indicated by a `400 Bad Request` HTTP status code, with an error response payload that describes why validation failed (please refer to the Authorization Server API description for more details). The table below shows the Client Validation Errors returned by the Heimdall API.

Description	Error Code (`error`)	Error Description (`error_description`)
The `client_id` query parameter sent by the Client does not match the value stored with the Authorization Request.	`invalid_client`	The `client_id` value does not match the value stored in the Authorization Request.
The Client, or the TPP who owns the Client, is no longer in a valid state in the Trust Framework, and therefore	`invalid_client`	The status of the Client is no longer valid. The Authorization Request can no longer be completed.

Description	Error Code (`error`)	Error Description (`error_description`)
cannot be successfully validated.		
The Authorization Request is invalid due to a structural, encoding or data error, and validation therefore cannot be completed.	`invalid_request_object`	The error description depends on the error condition experienced at Heimdall.
The request URI is invalid.	`invalid_request_uri`	`request_uri is invalid or expired`
An internal error has stopped validation of the Client. The validation of the Client therefore cannot be completed authoritatively.	`server_error`	The Client cannot be validated due to an internal error.

Authorization Request Error

Authorization Request Errors are returned by Heimdall when the Authorization Request is invalid or cannot be processed.

Authorization Request Errors are indicated by a `303 See Other` HTTP status code, with the `error` and `error_description` parameters contained in the response (please refer to the Authorization Server API description for more details). The table below shows the Authorization Request Errors returned by the Heimdall API.

Description	Error Code (`error`)	Error Description (`error_description`)
The Authorization Request has expired, as the number of seconds given by the `expires_in` value returned by the PAR Endpoint has passed.	`invalid_request`	The Authorization Request has expired.

Description	Error Code (`error`)	Error Description (`error_description`)
An internal error has stopped validation of the Authorization Request. The validation of the Authorization Request therefore cannot be completed.	`server_error`	The error description depends on the error condition experienced at Heimdall.

User Validation Error

User Validation Errors are returned by Heimdall when the LFI signals that User authentication and authorisation of consent has failed.

User Validation Errors are indicated by a `303 See Other` HTTP status code, with the `error` and `error_description` parameters contained in the response (please refer to the Authorization Server API description for more details).

Unlike the other error types, the LFI **must** supply the appropriate `error` and `error_description` parameters when invoking the `doFail` operation. The table below shows an ***example*** list of the errors that could be set by the LFI to indicate that User authentication and authorisation of consent has failed. The LFI is responsible for implementing the correct codes, based on this guidance, their understanding of their User authentication and authorisation flows, and adherence to the appropriate OAuth `error` codes.

Description	Error Code (`error`)	Error Description (`error_description`)
The User does not successfully complete Multi-Factor Authentication.	`access_denied`	The User failed to authenticate at the LFI in order to authorise consent.
The User cannot select one or more accounts for account data sharing, as no in-scope accounts are available. This would only happen where the LFI has not implemented the	`invalid_request`	The User could not authorise consent as no supported accounts are available.

Description	Error Code (`error`)	Error Description (`error_description`)
`validate` operation of the Consent Events & Actions API.		
The User declines to authorise consent of their own volition.	`access_denied`	The User declined to authorise the requested consent.
The LFI experienced an internal error performing User authentication and authorisation, and therefore cannot complete the request.	`server_error`	The error description depends on the error condition experienced at the LFI.
The LFI cannot complete the User authentication and authorisation journey due to high load at their Authorisation App or Backend.	`temporarily_unavailable`	The User could not authenticate or authorise consent due to high load at the LFI.

Please note that if an LFI submits an error code that is not supported by the standards included in the FAPI 2.0 Security Profile, the error code will be overwritten with `invalid_request`.

Trust Framework | Overview

The Trust Framework provides the security framework for the operation of the Qatar Open Finance ecosystem. It is the source of truth for who the participants are, and the API Hub relies on it to enforce security.

The Trust Framework:

- Provides a list of all participants and their associated metadata, including the OpenID Discovery endpoint for each participant.
- Issues the certificates used for communication between the Open Finance Platform and LFIs (see Security Profile).
- Records the names and logos of LFIs, in the Trust Framework Directory, that TPPs use when a User selects their LFI.